

Minerva Access is the Institutional Repository of The University of Melbourne

Author/s:

Chu, Geoffrey G.

Title:

Improving combinatorial optimization

Date:

2011

Citation:

Chu, G. G. (2011). Improving combinatorial optimization. PhD thesis, Engineering, Computer Science and Software Engineering, The University of Melbourne.

Persistent Link:

<https://hdl.handle.net/11343/36679>

Terms and Conditions:

Terms and Conditions: Copyright in works deposited in Minerva Access is retained by the copyright owner. The work may not be altered without permission from the copyright owner. Readers may only download, print and save electronic copies of whole works for their own personal non-commercial use. Any use that exceeds these limits requires permission from the copyright owner. Attribution is essential when quoting or paraphrasing from these works.

Improving Combinatorial Optimization

by

Geoffrey G Chu

Submitted in total fulfillment
of the requirements
of the degree of Doctor of Philosophy

October 2011

Department of Computer Science and Software Engineering
The University of Melbourne

Abstract

Combinatorial Optimization is an important area of computer science that has many theoretical and practical applications. In this thesis, we present important contributions to several different areas of combinatorial optimization, including nogood learning, symmetry breaking, dominance, relaxations and parallelization.

We develop a new nogood learning technique based on constraint projection that allows us to exploit subproblem dominances that arise when two different search paths lead to subproblems which are identical on the remaining unlabeled variables. On appropriate problems, this nogood learning technique provides orders of magnitude speedup compared to a base solver which does not learn nogoods.

We present a new symmetry breaking technique called SBDS-1UIP, which is an extension of Symmetry Breaking During Search (SBDS). SBDS-1UIP uses symmetric versions of the *1UIP nogoods* derived by Lazy Clause Generation solvers to prune symmetric parts of the search space. We show that SBDS-1UIP can exploit at least as many symmetries as SBDS, and that it is strictly more powerful on some problems, allowing us to exploit types of symmetries that no previous general symmetry breaking technique is capable of exploiting.

We present two new general methods for exploiting almost symmetries (symmetries which are broken by a small number of constraints). The first is to treat almost symmetries as conditional symmetries and exploit them via conditional symmetry breaking constraints. The second is to modify SBDS-1UIP to handle almost symmetries. Both techniques are capable of producing exponential speedups on appropriate problems.

We examine three reasonably well known problems: the Minimization of Open

Stacks Problem, the Talent Scheduling Problem (CSPLib prob039), and the Maximum Density Still Life Problem (CSPLib prob032). By applying various powerful techniques such as nogood learning, dynamic programming, dominance and relaxations, we are able to solve these problems many orders of magnitude faster than the previous state of the art.

We identify cache contention as a serious bottleneck that severely limit the amount of speedup achievable when parallelizing SAT and LCG solvers on multi-core systems. We alter the data structures used in the state of the art SAT solver MiniSat [39] to be more cache aware, leading to a sequential SAT solver that is some 80% faster on average and a parallel SAT solver that is some 140% faster on average.

We examine an important issue in parallel search that has not been properly addressed in the literature. Many work stealing schemes used for load balancing completely ignore the branching heuristic, and instead try to maximize the granularity of the work stolen. This can result in many of the processors searching in unfruitful parts of the search tree. We analyze this problem and develop a new work stealing scheme called *confidence based work stealing*, which partitions the work based on how strong the branching heuristic is at each node. The new parallel algorithm produced near linear or super linear speedup on all the problems we tested.

This is to certify that:

- (i) the thesis comprises only my original work towards the PhD except where indicated in the Preface
- (ii) due acknowledgment has been made in the text to all other material used
- (iii) the thesis is fewer than 100,000 words in length, exclusive of tables, maps, bibliographies and appendices

Geoffrey G Chu

Preface

This PhD thesis is based on research conducted at the Department of Computer Science and Software Engineering, University of Melbourne, from January 2008 to July 2011. The research was carried out in collaboration with my supervisor Peter J. Stuckey, and co-authors Maria Garcia de la Banda, Chris Mears, and Christian Schulte. Parts of this thesis has previously appeared in the following publications: [54, 28, 29, 27, 26, 25, 24].

Acknowledgment

I would like to thank my supervisor Peter J. Stuckey, who has given me invaluable advice and support during my candidature. It has been a great privilege to work with someone of such high caliber, and I have thoroughly enjoyed our many interesting discussions. I would also like to thank the members of my advisory committee Mark Wallace and Aaron Harwood, as well as my co-authors Maria Garcia de la Banda, Chris Mears, and Christian Schulte. Most importantly, I would like to thank my family, without which none of this would be possible.

Geoffrey Chu

University of Melbourne

July 2011

Contents

Abstract	iii
Preface	vii
Acknowledgment	ix
1 Introduction	1
1.1 Contributions	6
2 Background	11
2.1 Constraint Satisfaction Problem	12
2.2 Constraint Programming	13
2.3 Boolean Satisfiability	19
I Nogood Learning	23
I.1 Introduction	25
I.2 Deriving Nogoods through Search	25
I.3 Deriving Nogoods through Resolution	26
I.4 Contributions	29
3 Automatic Caching via Constraint Projection	31
3.1 Introduction	32
3.2 Subproblem Dominance and Equivalence	34
3.3 Manual Keys for Caching	36

3.4	Automatic Keys for Caching	41
3.5	Constructing Projection Keys	51
3.6	Examples of Automatic Caching	61
3.7	Related Work	66
3.8	Experiments	72
3.9	Conclusion	76
II	Symmetry Breaking	81
II.1	Definitions	83
II.2	Lexicographical Symmetry Breaking	85
II.3	Dynamic Symmetry Breaking	87
II.4	Contributions	89
4	SBDS and LCG	91
4.1	Introduction	92
4.2	SBDS-1UIP	93
4.3	1UIP vs Decision Nogood	94
4.4	Beyond Complete Methods	95
4.5	Applying Symmetries to 1UIP Nogoods	98
4.6	Experiments	102
4.7	Conclusion	105
5	Exploiting Almost Symmetries	107
5.1	Introduction	108
5.2	Definitions	109
5.3	Almost Symmetry as Conditional Symmetry	110
5.4	Breaking Almost Symmetries with SBDS-1UIP	117
5.5	Related Work	119
5.6	Experiments	120
5.7	Conclusion	123

III	Relaxation and Dominance	125
III.1	Introduction to Relaxations	127
III.2	Definitions	127
III.3	Types of Relaxations	128
III.4	Introduction to Dominance	131
III.5	Contributions	133
6	Minimization of Open Stacks Problem	135
6.1	Introduction	136
6.2	Searching on Customers	138
6.3	Improving the Search	141
6.4	Experiments	151
6.5	Conclusion	156
7	Talent Scheduling Problem	157
7.1	Introduction	158
7.2	Dynamic Programming Formulation	160
7.3	Bounded Dynamic Programming	172
7.4	Double Ended Search	182
7.5	Experiments	189
7.6	Related Work	194
7.7	Conclusion	197
8	Maximum Density Still Life Problem	199
8.1	Introduction	200
8.2	Wastage Reformulation	202
8.3	Solving Small n with Complete Search	207
8.4	Upper Bounds for Large n	210
8.5	Solving the Boundary Relaxation	214
8.6	Lower Bounds for Large n	218
8.7	Constructing Optimal Solutions for Arbitrarily Large n	223
8.8	Conclusion	226

IV	Parallelization	229
IV.1	Contributions	231
9	Cache Conscious Data Structures for SAT	233
9.1	Introduction	234
9.2	Boolean Constraint Propagation	235
9.3	Improving the Cache Behavior	238
9.4	Experiments	248
9.5	Related Work	254
9.6	Conclusion	257
10	Confidence Based Work Stealing	259
10.1	Introduction	260
10.2	Analysis of Work Allocation	263
10.3	Adaptive Work Stealing	266
10.4	Dynamically Updating Solution Density Estimates	266
10.5	Confidence Model	268
10.6	Algorithm	268
10.7	Experiments	273
10.8	Conclusion	278
11	Conclusion	279
	Bibliography	283
A	Appendix	303
1.1	119 industrial SAT instances	303
1.2	24 medium sized industrial instances	303
1.3	22 hard industrial instances	304
1.4	200 industrial instances from SAT Race 2008	305

List of Tables

2.1	Steps taken by $\text{sol}v(C, D)$ algorithm in Example 2.2.1.	16
2.2	Steps taken by $\text{learn}(v, S)$ algorithm in Example 2.3.1.	22
3.1	Experimental results: Knapsack, MOSP and Black Hole	78
3.2	Experimental results: BACP and Radiation	79
4.1	Comparison of three SBDS implementations in CHUFFED, static symmetry breaking in CHUFFED, and LDSB in Eclipse, on the Concert Hall Scheduling problem	104
4.2	Comparison of three SBDS implementations in CHUFFED, static symmetry breaking in CHUFFED, and LDSB in Eclipse, on the Graph Coloring problems	105
5.1	Comparison between ignoring almost symmetries (none), exploiting almost symmetries with conditional symmetry (cond-sym), SBDS-1UIP (sbds-1uip), decomposition using lex (remodel-lex) and decomposition using SBDS-1UIP (remodel-sbds)	122
6.1	Comparison of upper bound heuristic, versus complete search on some difficult problems. Times in milliseconds	146
6.2	Results on the open problems from the Constraint Modeling Challenge 2005, comparing versus the the winner of the challenge [53]. Times in milliseconds.	152
6.3	Comparing customer search versus [53] on harder random instances. Search is to find the best solution found by [53] with node limit 2^{25}	153

6.4	(a) Comparing the effects of each optimization in turn, and (b) comparing the effects of relaxation.	154
7.1	Arithmetic mean solving time (ms) for structured problems of size n , and relative slowdown if the optimization is turned off	190
7.2	Arithmetic mean subproblems solved for structured problems of size n , and relative increase if the optimization is turned off	191
7.3	Arithmetic mean solving time (ms) for random problems with m actors and n scenes.	193
7.4	Arithmetic mean subproblems solved for random problems with m actors and n scenes.	193
7.5	Comparison with the approach of [145] on the examples from that paper.	196
8.1	Possible patterns around dead cells, showing where they donate their tokens and any wastage.	203
8.2	Contributions to the tokens of a live cell from its South neighbor. . .	204
8.3	Possible patterns around a live cell showing token benefactors and any wastage.	205
8.4	Optimum number of live cells in the Maximum Density Still Life Problem found by complete search.	210
8.5	Ratio of wastage to edge cells in an optimal edge for different edge width, and number of patterns that appear in the first three rows. .	213
8.6	Lower bounds on wastage and corresponding upper bounds on live cells as calculated by bounded dynamic programming on a problem relaxation. Instances where the bound on the relaxed problem differed from the original problem are indicated with an asterisk.	218

9.1	Cache conscious optimizations for sequential SAT solving. Comparison of various combinations of optimizations: WL — Watched list modification (9.3.2), PC(n) — Clause modification with length n clause head, n = 16 if labeled as just PC (9.3.3), FL — Store first literal in watched list (9.3.4), BC — Handle binary clauses specially (9.3.5), TC — Handle ternary clauses specially (9.3.6) PA — Pack assigns (9.3.7). The results show the (geometric mean) runtime for the suite, the speedup compared to MiniSat 2.0, I/Loop stands for average number of instructions per inner loop of the propagation engine, L1M/Loop and L2M/Loop stands for the average number of L1 and L2 misses per inner loop of the propagation engine.	249
9.2	Cache conscious optimizations for parallel SAT solving. We compare a parallelized version of MiniSat 2.0, PMiniSat, using the original data structures and the combination of optimizations detailed in Table 9.1. The results are the geometric mean of 20 runs.	252
9.3	Parallel memory stress test.	254
10.1	Experimental results for optimization problems with simple confidence model. The results show: wall clock runtime in seconds (Runtime), speedup relative to the sequential version (Speedup), and runtime efficiency (RunE) which is Speedup/8, CPU utilization (CPU%), communication overhead (Comm%), number of steals (Steals), total number of nodes explored (Nodes), the algorithmic efficiency (AlgE) the total number of nodes explored in the parallel version versus the sequential version, the number of nodes explored to find the optimal solution (Onodes), and the solution finding efficiency (SFE) the total number of nodes explored in the parallel version to find the optimal versus the sequential version. Values for Runtime, CPU%, Comm%, Steals, Nodes, and Onodes are the geometric mean of the instances solved by all 4 versions.	274

10.2 Experimental results for satisfaction problems with simple confidence model	276
10.3 Experimental results showing Nodes and algorithmic efficiency (AlgE) using accurate confidence values, where we follow the confidence value to degree α	277

List of Figures

2.1	A simple graph coloring problem.	13
2.2	Basic search algorithm for propagation based constraint solvers. . . .	15
2.3	Search tree in Example 2.2.1.	16
2.4	Basic branch and bound algorithm for propagation based constraint solvers.	17
2.5	Naive algorithm for the propagation engine.	18
2.6	Implication graph for Example 2.3.1. Decision literals are double boxed.	22
3.1	Search algorithm taking advantage of subproblem dominance.	37
3.2	Two starting sequences for the Black Hole problem that lead to an identical remaining subproblem.	37
3.3	Three examples of partially colored graphs.	38
3.4	Sufficient conditions on propagation to ensure no key is required for the binary constraints above. Note the <i>bool2int()</i> function converts a Boolean <i>b</i> to a 0-1 integer <i>y</i>	52
3.5	Different fixed sub-schedules for <i>cumulative</i> leading to the same key.	58
3.6	Different fixed sub-schedules for <i>cumulative</i> leading to the same key when using decomposition based keys.	61
4.1	A graph coloring problem where we can exploit additional symmetries.	96
5.1	A graph coloring problem where we can exploit additional symmetries.	118

6.1	(a) An example $c(p)$ function: $c_i \in c(p_j)$ if the row for c_i in column p_j has an X. (b) An example schedule: c_i is active when product p_j is scheduled if the row for c_i in column p_j has an X or a $-$	137
6.2	(a) A schedule corresponding to customer order $[c_5, c_4, c_3, c_2, c_1]$. (b) A schedule corresponding to customer order $[c_5, c_1, c_2, c_3, c_4]$. (c) The customer graph (ignoring self edges) with edges labeled by the products that generate them.	139
7.1	(a) An Example $a(s)$ Function: $a_i \in a(s_j)$ if the Row for a_i in Column s_j has an X. (b) An Example Schedule: a_i is on location when scene s_j is Scheduled if the Row for a_i in Column s_j has an X or a $-$	159
7.2	Pseudo-code for best-first call-based dynamic programming algorithm. schedule (Q) returns the minimum cost required for scheduling the set of scenes Q	162
7.3	An optimal order for the problem of Example 7.1.1	163
7.4	The problem of Example 7.1.1 after preprocessing.	166
7.5	An optimal schedule for the scenes $Q = \{s_1, s_2, s_4, s_7, s_8, s_9\}$ Assuming $\{s_3, s_5, s_6, s_{10}, s_{11}, s_{12}\}$ have already been scheduled	166
7.6	Pseudo-code for bounded best-first call-based dynamic programming algorithm. bnd_schedule (Q, U) returns the minimum cost required for scheduling the set of scenes Q if it is less than or equal to U . Otherwise it returns a lower bound on the minimal cost	173
7.7	The schedule defined by the heuristic upper bound algorithm.	175
7.8	An original set of remaining scenes, assuming a_1, a_2, a_3, a_4 are on location.	181
7.9	Pseudo-code for bounded best-first call-based dynamic programming algorithm. bnd_de_schedule (Q, B, E) returns the minimum cost required for scheduling the set of scenes Q if it is less than or equal to U . Otherwise it returns a lower bound on the minimal cost	183
7.10	Two equivalent subproblems.	185
7.11	An optimal schedule for the film “Mob Story”	195
8.1	Optimal solutions for $n = 21$ and $n = 22$	211

8.2	Optimal edge patterns for a width 2 and width 3 boundary relaxation. Wastage is highlighted with a star.	213
8.3	Closed form equations for the wastage lower bound for $n \geq 61$	218
8.4	Closed form equations for the live cell upper bound for $n \geq 61$	218
8.5	The two best ways to fill a corner (up to symmetry). Wastage is highlighted with a star. Note that there are two units of wastage in the cell in the 4th column and 4th row in the second pattern.	219
8.6	Dynamic relaxation lookahead for still life search.	220
8.7	Optimal solution for $n = 100$	222
8.8	A periodic optimal solution where the center sections can be tiled indefinitely to produce optimal solutions for $n = 100 + k * 54$	224
9.1	Boolean constraint propagation of the decision literal d given a current assignment A	237
9.2	Data structure used in MiniSat 2.0	238
9.3	The histogram over the 119 tested instances of the percentage of short watched lists (length ≤ 16) encountered during the inner loop for Boolean constraint propagation.	241
9.4	Watched list data structure	242
9.5	The histogram over the 119 tested instances of the percentage of clauses where $\leq k$ literals were examined in the inner loop for Boolean constraint propagation. The histogram shows the results for $k = 1$ (only first literal examined), $k = 4$ and $k = 8$	243
9.6	Clause data structures	245
9.7	Watched list element data structure	247
9.8	Runtime comparisons on 151 SAT Race instances	250
10.1	Example with reasonably high confidence	270
10.2	Example with low confidence	272

Chapter 1

Introduction

Combinatorial Optimization is an important area of computer science that has many theoretical and practical applications. Many real world problems in the areas of production, scheduling, resource allocation, facility location and layout, etc, can be expressed as combinatorial optimization problems. In a combinatorial optimization problem, we have a finite set of options, and we wish to find the one which is optimal with respect to some objective function, e.g., cost, time. The ability to find good or optimal solutions to real world combinatorial optimization problems can often translate into significant time, resource or monetary savings for industry.

Combinatorial optimization problems are usually modeled in terms of variables, constraints and objective function. A simple example of a combinatorial optimization problem is the 0-1 Knapsack Problem. Suppose we have a knapsack with a certain weight capacity W , and a set of n items, each of which has a particular weight w_i and value v_i , and we wish to maximize the value of the items that can fit into the knapsack. We can model this problem as follows. We define variables $x_i \in \{0, 1\}$, to represent whether the i th item is put into the knapsack ($x_i = 1$ means put in). Since the total weight cannot exceed the capacity of the knapsack, we have the constraint: $\sum_{i=1}^n w_i x_i \leq W$. The objective to be maximized is given by: $\sum_{i=1}^n v_i x_i$.

There are different techniques for solving combinatorial optimization problems, depending on what kind of variables and constraints are involved. Linear Programming (LP) can be used to solve problems which can be modeled using continuous variables,

linear constraints and linear objective function. Such problems can be solved very effectively using the simplex method [110], or the interior point method [81] and belong in the complexity class P. However, in many problems, variables represent quantities which are discrete, e.g., whether an item should be placed in the knapsack or not. Mixed Integer Programming (MIP) can be used to solve linear problems where one or more variables are restricted to be integers. The restriction to integer variables makes the problem NP-hard in general [83]. MIP solvers typically solve such problems using branch and bound. At each node, a linear relaxation of the problem is solved using LP methods in order to get a bound on the objective value. If this bound shows that the subproblem cannot have any solutions better than the best solution found so far, then the subproblem can be pruned. See [34] for more details on MIP solving. However, not all problems can be modeled effectively using only linear constraints. In such cases, other methods may be more appropriate.

Constraint Programming (CP) is a relatively new, and very effective technology for solving combinatorial optimization problems. Constraint programming allows the usage of many kinds of constraints. This expressiveness allows many problems to be modeled much more naturally than in LP. For example, constraints such as disequality (e.g., $x_1 \neq x_2$), multiplication (e.g., $x_1 = x_2 * x_3$), reification (e.g., $b \leftrightarrow x_1 \geq x_2$), Boolean constraints (e.g., $b_1 = b_2 \wedge b_3$), element constraints ($x_1 = a[x_2]$ where $a[i]$ is an array of values), occur commonly in problems but are very difficult to express as linear constraints. Constraint Programming is able to handle such constraints effectively, and excels at handling complex constraints over large sets of finite domain variables (see [10] for a large list of standard CP constraints).

The distinguishing feature of Constraint Programming is the use of inference during search to restrict the search space and to detect the failure of subproblems earlier. At each node, an unfixed variable is chosen and the solver branches upon the value of this variable in order to create several more constrained subproblems. As branching decisions are made, the constraints in the problem are analyzed in order to make further inferences. If some constraint is violated by the partial assignment, then there are no solutions within the subtree and the solver can backtrack. Alternatively, the solver may infer that certain variable/value pairs can no longer be taken without violating

some constraint, in which case these values may be pruned to restrict the search space within that subtree. This kind of inference can dramatically reduce the amount of search that a CP solver has to do compared to a naive enumeration approach, and is one of the core strengths of Constraint Programming. A tremendous amount of work has been done to try to improve the amount of inference we can make for various constraints and to make these inferences as fast as possible (e.g., [127, 128, 149, 122, 152]). Another important feature of Constraint Programming is programmable search. The order in which variables are branched upon, and the order in which their values are tried, can have a dramatic effect on the run time of the algorithm [17, 121, 66, 163]. Many CP systems (e.g., Eclipse, Gecode, CHUFFED) allow standard or custom search strategies to be declared as part of the problem model.

Although tremendous progress has been made in Constraint Programming in the last 20 years, there exists many opportunities for further improvement. In this thesis, we present important contributions to a number of areas in Constraint Programming, including nogood learning, symmetry breaking, dominance, relaxations, and parallelization. The thesis is divided into four parts, each addressing a different area of constraint programming. We give a brief introduction to each of these areas before describing our contributions.

Nogood Learning Nogood learning techniques are used to speedup constraint solving by learning implied constraints called *nogoods*. Such nogoods express sets of conditions under which there can be no solutions to the problem, and can be propagated to prune parts of the search space. There are various ways to derive nogoods. The simplest way is to derive them through exhaustive search. Whenever a subtree reached via a set of decisions has been exhaustively searched and has no solutions, the set of decisions we took to get to the subtree forms a nogood, which we call the decision nogood. Such decision nogoods are easy to derive but are also very weak in terms of pruning strength. It has been suggested that such nogoods can be used in CP solvers that perform restarts in order to prevent them from exploring previously explored parts of the search tree [94]. Another way to derive nogoods is via constraint resolution. Such nogoods are used in SAT solvers [142], and in Lazy Clause

Generation solvers [113, 45]. Whenever the solver finds a conflict, it is possible to find the subset of constraints responsible and to resolve them together to derive a new nogood. Such a nogood expresses the reason for the conflict, and can prevent the solver from making assignments that fail for the same reason again. They are very powerful and can provide orders of magnitude speedup on a wide range of problems.

Symmetry Breaking Symmetry breaking techniques are used to speedup constraint solving in problems with symmetries. Many constraint problems contain large amounts of symmetry. In such problems, examining one assignment is often enough to tell us everything we need to know about its symmetric counterparts (whether it is a solution, what its objective value is, etc). These redundancies in the search can exponentially increase the run time. One way to prevent this is to apply some form of symmetry breaking to prevent the CP solver from exploring symmetric parts of the search space. The most popular symmetry breaking techniques are lexicographical symmetry breaking constraints [118, 32, 98, 120, 50], Symmetry Breaking During Search (SBDS) [62], and Symmetry Breaking by Dominance Detection (SBDD) [43]. Lexicographical symmetry breaking constraints can be statically added to the problem to constrain the solver to look only for solutions which are lexicographically least within its equivalence class. SBDS and SBDD are dynamic symmetry breaking techniques which, after each partial assignment has been explored, adds constraints to disallow all of its symmetric counterparts from being solutions.

Relaxation Relaxation techniques seek to relax a problem P into a related but easier problem P' such that solving P' yields useful information on how to solve P . This information may include candidate solutions or bounds on the objective. There are many ways to relax a problem. The most popular are linear relaxations [137, 156] and Lagrangian relaxations [46, 111, 95]. In linear relaxations, discrete variables are relaxed into continuous variables, and constraints are relaxed into linear constraints. The relaxed problem can be solved very effectively using linear programming techniques such as the simplex method in order to find bounds or candidate solutions for the original problem. In Lagrangian relaxations, the problem is relaxed by removing

certain constraints and adding corresponding “penalty” terms to the objective function which give a non-zero value when the removed constraint is violated. Solving the relaxed problem can yield bounds or candidate solutions for the original problem.

Dominance Dominance is a powerful concept that underlies many of the other major techniques used in CP. Symmetry breaking, relaxations, and many forms of nogood learning are applications of dominance. When we have pairs of objects (valuations, subproblems, problems) where one is known to be better than, or at least as good as, the other with respect to what we are trying to optimize, e.g., one satisfies the constraints while the other does not, one has a better objective value than the other, then it is sufficient to examine the one that is better, and we can prune the worse one without examining it. Finding and exploiting such *dominance relations* [87, 78, 2, 76] can give us exponential speedups on appropriate problems.

Parallelization Parallelization is an obvious way to try to speedup constraint solving, especially in view of the recent proliferation of multi-core systems. There are a number of ways to parallelize CP solvers. The simplest and most popular is search parallelization, where different parts of the search tree are searched by different processors (e.g., [114, 138, 105]). Another possibility is to parallelize the propagation engine by having different processors execute different propagators (e.g., [132, 167]). The basic search algorithm for constraint solving is almost trivially parallelizable. As such, previous work on parallel search has concentrated mainly on load balancing [130, 138], or on developing an appropriate architecture for parallel search [138, 105].

1.1 Contributions

Part I - Nogood Learning In Chapter 3, we present a new nogood learning technique based on constraint projection. In many combinatorial optimization problems, different search paths lead to subproblems which are actually identical on the remaining unlabeled variables. Nogoods derived via constraint projection can be used to exploit such subproblem dominances. On appropriate problems, our new nogood learning technique provides orders of magnitude speedup compared to a base solver which does not learn nogoods. Our technique is competitive with Lazy Clause Generation, which is the current state of the art in nogood learning. The nogoods derived through constraint projection are different in power to the nogoods derived by Lazy Clause Generation through constraint resolution, and can be exponentially stronger on some problems.

Part II - Symmetry Breaking In Chapter 4, we present a new symmetry breaking technique called SBDS-1UIP, which is an extension of SBDS. The original SBDS technique can be thought of as using symmetric versions of decision nogoods to prune symmetric parts of the search space. In our extension SBDS-1UIP, we use symmetric versions of the 1UIP nogoods derived by Lazy Clause Generation solvers instead. We show that SBDS-1UIP can exploit at least as many symmetries as SBDS, and that it is strictly more powerful on some problems. This is a very interesting result, as SBDS is already a complete method which guarantees that at most one solution in each equivalence class is found by the solver. However, SBDS-1UIP can be strictly more powerful than that. We show that SBDS-1UIP can exploit types of symmetries that no previous general symmetry breaking technique is capable of exploiting.

In Chapter 5, we present two new methods for exploiting almost symmetries. Many problems arising from real world applications do not exhibit full problem symmetry, but instead exhibit *almost symmetries*, where the symmetries are slightly broken. Such problems often still have large amounts of symmetries that can be exploited for speedup. However, traditional symmetry breaking methods like lexicographical symmetry breaking constraints, SBDS or SBDD cannot directly handle such partially

broken symmetries. There are several theoretical works in this area [65, 67], but only one general method which has actually been implemented [101]. In this work, the problem is remodeled to separate the subset of constraints which break the symmetry from the rest, so that the traditional symmetry breaking methods can be applied to the ones that satisfy the symmetry. We present two new general methods for exploiting almost symmetries. The first is to treat almost symmetries as conditional symmetries and exploit them by posting conditional symmetry breaking constraints. The second is to modify SDBS-1UIP to handle almost symmetries. Both techniques are capable of producing orders of magnitude speedups on appropriate problems.

Part III - Relaxation and Dominance In Chapter 6, we present a new solver for the Minimization of Open Stacks Problem (MOSP) [162]. The MOSP was the subject of the 2005 Modeling Challenge. It has appeared in many different guises in the literature, e.g., graph pathwidth, gate matrix layout (see [97] for a list of 12 equivalent problems). The MOSP is known to be NP-hard [97]. The previous state of the art in solving the MOSP [53] utilizes a branch and bound algorithm and top down dynamic programming in order to exploit the massive amounts of subproblem equivalences inherent in the problem. This solver was several orders of magnitude faster than all the other entrants in the 2005 Modeling Challenge. Our new solver is many orders of magnitude faster yet again. We identify 4 different dominance relations, which when exploited, provides some 5-6 orders of magnitude speedup. We also find an effective relaxation technique that allows us to speedup the proof of optimality on the hardest instances by a further 3-4 orders of magnitude.

In Chapter 7, we present a new solver for the Talent Scheduling Problem [23]. In the Talent Scheduling Problem, we have a set of actors and a set of scenes. We wish to find an ordering of the scenes such that the cost of hiring the actors are minimized. Each actor is involved in some subset of scenes and must be on location from the first to the last of their scenes, getting paid for each day they are on location. The Talent Scheduling Problem problem has a lot of subproblem equivalences which can be exploited by caching [145], dynamic programming, or similar techniques. We present a new solver based on dynamic programming that is some 2-3 orders of magnitude faster

than the previous state of the art. We show that using bounded dynamic programming gives 1-2 orders of magnitude speedup compared to a naive dynamic programming approach. Exploiting dominance relations give a further ~ 2 times speedup, and utilizing a lower bound calculation based on relaxations gives a further 1 order of magnitude speedup.

In Chapter 8, we completely solve the Maximum Density Still Life Problem [19] for all n . The Maximum Density Still Life Problem is based on Conway’s Game of Life. The aim is to find the maximum number of live cells that can appear in an $n \times n$ region in a *still life* according to the rules of the game. It is an extremely difficult combinatorial optimization problem with a raw search space of $O(2^{n^2})$. Previous search methods using IP [18] and CP [19] could only solve up to $n = 9$, while a CP/IP hybrid method with symmetry breaking [19] could solve up to $n = 15$. An attempt using bucket elimination [90] reduced the time complexity to $O(n^2 2^{3n})$ but increased the space complexity to $O(n 2^{2n})$. This method could solve up to $n = 14$ before it ran out of memory. A subsequent improvement that combined bucket elimination with search [91], used less memory and was able to solve up to $n = 20$. We show that by using a combination of remodeling, relaxations, bounded dynamic programming and customized search guided by deep mathematical insights, we can completely solve the Maximum Density Still Life problem for all n . As a comparison, the previous state of the art method [91] would take in the order of 10^{100} years and 10^{30} bytes of memory to solve $n = 100$. Our new algorithm can solve $n = 100$ in just a few hours using less than 2 Gb of memory.

Part IV - Parallelization In Chapter 9, we identify cache contention as a serious bottleneck that severely limit the amount of speedup achievable by parallel SAT and Lazy Clause Generation solvers on multi-core systems. We alter the data structures used in the state of the art SAT solver MiniSat [39] to be more cache aware, leading to a sequential SAT solver that is some 80% faster on average and a parallel SAT solver that is some 140% faster on average.

In Chapter 10, we examine the interaction between branching heuristics and dynamic load balancing schemes. This important theoretical issue has not been properly addressed in the literature. Many analyses of parallel search and load balancing schemes implicitly or explicitly assume that the total amount of work is fixed (e.g. [89]). This assumption is not true for satisfiable problems or for optimization problems, as finding a (good) solution quickly can dramatically reduce the total amount of work required. Much work in CP is concerned with developing strong branching heuristics which can shape and order the branches of the search tree so that (good) solutions are found early on in the search. However, many commonly used parallelization schemes, e.g., splitting the search tree as close to the root as possible [138], focus on maximizing the granularity of the work partitions and completely ignore the branching heuristic. This can result in many of the processors searching in unfruitful parts of the search tree. We analyze this problem and develop a new work stealing scheme called *confidence based work stealing*, which partitions the work based on how strong the branching heuristic is at each node. The new parallel algorithm produced near linear or super linear speedup on all the problems we tested.

The thesis is organized as follows. In Chapter 2, we introduce the background and notation for the thesis. The remainder of the thesis is divided into 4 parts, each addressing a different area of Constraint Programming. At the beginning of each part, more detailed technical background for that area of CP is given. The four parts cover the areas of: nogood learning (Chapter 3), symmetry breaking (Chapter 4 and 5), relaxation and dominance (Chapter 6, 7 and 8), and parallelization (Chapter 9 and 10). Finally, in Chapter 11, we conclude.

Chapter 2

Background

In this chapter, we provide the basic background and notation required for the remainder of this thesis. We describe Constraint Satisfaction Problems (CSP), Constraint Programming (CP), and Boolean Satisfiability Problems (SAT). More detailed background on nogood learning, symmetry breaking, dominance, relaxations and parallelization techniques will be presented at the beginning of the respective parts of the thesis.

2.1 Constraint Satisfaction Problem

Let $\equiv$ denote syntactical identity, $\Rightarrow$ denote logical implication and $\Leftrightarrow$ denote logical equivalence. A variable v is a mathematical quantity capable of assuming any value from a set of values called the domain of v . A valuation θ over a set of variables V assigns each $v \in V$ to a value in its domain. A constraint c is a logical expression over some set of variables V . If θ assigns the variables in V such that the logical expression c evaluates to *true*, then we say that θ satisfies constraint c . Otherwise, we say that θ violates constraint c .

A *Constraint Satisfaction Problem* (CSP) is a tuple $P \equiv (V, D, C)$, where V is a set of variables, D is a set of domains, and C is a set of constraints. If a valuation θ satisfies all the constraints in C , we call it a *solution* of P . The aim of a constraint satisfaction problem P is typically to find one solution, all solutions, or to prove that none exist. In a *Constraint Optimization Problem*, in addition to V , D , and C , we also have an objective function f which we wish to minimize or maximize. Typically, we wish to find a solution with the minimal or maximal value of f among all the solutions of P .

Example 2.1.1. Suppose we want to model the simple graph coloring problem in Fig 2.1, where we want to color the nodes of the graph using only 3 colors so that nodes sharing an edge are of different colors. We can define variables $V \equiv \{x_1, \dots, x_6\}$ to represent which color each node is colored to, domains $D \equiv \{x_i \in \{1, 2, 3\} \mid i = 1, \dots, 6\}$, and constraints: $C \equiv \{x_1 \neq x_5, x_1 \neq x_6, x_2 \neq x_3, x_2 \neq x_4, x_3 \neq x_4, x_4 \neq x_5, x_4 \neq x_6, x_5 \neq x_6\}$.

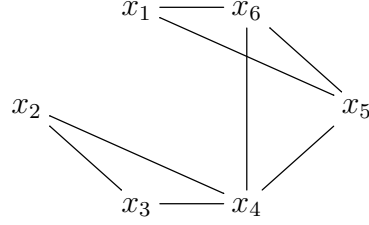


Figure 2.1: A simple graph coloring problem.

It is often useful to consider the domains enforced by a CSP as unary constraints (constraints over a single variable). This allows us to manipulate them as logical expressions, which makes the statement and proofs of many results more succinct. We give further definitions and redefine CSP's slightly.

Each variable v has an initial domain determined by its type, e.g., a Boolean variable v has an initial domain $\{true, false\}$, an integer variable has an initial domain $\mathbb{Z}$, a continuous variable has an initial domain $\mathbb{R}$, etc. A valuation θ assigns each $v \in V$ to a value in its initial domain. We denote the set of all valuations over V as Θ_V . We denote the subset of Θ_V that satisfies c as $solns(c)$. Clearly, a constraint is uniquely determined by $solns(c)$. Given an object O (e.g., a constraint, a valuation), let $vars(O)$ denote the set of variables associated with it.

A constraint satisfaction problem is a tuple $P \equiv (V, D, C)$, where V is a set of variables, D is a set of *unary constraints*, and C is a set of constraints. If a valuation $\theta \in \Theta_V$ satisfies all the constraints in C and D , then we call it a *solution* of P . We denote the set of solutions of P as $solns(P)$. We call a problem a *finite domain problem* if the domain D constrains each variable to take values from a finite set of values.

2.2 Constraint Programming

Constraint Programming (CP) is a very effective method for solving combinatorial problems. A typical CP system consists of two parts: 1) a modeling interface where

a CSP can be stated declaratively by the user, and 2) a solver which solves the CSP. There are many kinds of modeling interfaces, including solver specific modeling languages and API's, as well as more generic modeling languages like Zinc [52] or MiniZinc [112]. In this Thesis, we will model using either direct mathematical notation, or the simple modeling language MiniZinc [112].

Modern CP solvers are typically based on constraint propagation. They solve CSP's by interleaving search with inference. The general algorithm is as follows. We begin with the original problem at the root of the search tree. At each node in the search tree, we propagate the constraints to try to infer variable/value pairs which cannot be taken in any solution to the problem. Such pairs are removed from the current domain. If some variable's domain becomes empty, then the problem has no solution and the solver backtracks. If all the variables are assigned and no constraint is violated, then a solution has been found and the solver can terminate. If inference is unable to detect either of the above two cases, then the solver further divides the problem into a number of more constrained subproblems and searches each of those in turn. We give some additional definitions before giving the pseudo code for this algorithm.

In an abuse of notation, if a symbol A refers to a set of constraints $\{c_1, \dots, c_n\}$, we will often also use the symbol A to refer to the constraint $c_1 \wedge \dots \wedge c_n$. This allows us to avoid repetitive use of conjunction symbols. It should be clear from the context which meaning we take. If we apply set operators to A like $\in, \cup, \cap$, we are treating A as a set of constraints. If we apply logical operators to A like $\wedge, \vee, \Rightarrow$, we are treating it as a constraint. A domain D is a *false domain* iff $D \Rightarrow \text{false}$, i.e., one or more variables are constrained by D to the point that it cannot take any value. Let $fixed(D)$ denote the set of variables fixed by a domain D , i.e., the subset of variables which are constrained by D so that they can only take one value.

The basic algorithm for solving a CSP is shown in Figure 2.2. The function $solv(D, C)$ returns whether the constraint satisfaction problem $P \equiv (V, D, C)$ is satisfiable. $solv$ is implemented with the help of two functions $prop(C, D)$ and $branch(C, D)$. The function $prop(C, D)$ performs the propagation. It takes as input a set of constraints C and a domain D and returns a domain D' s.t. $D' \Rightarrow D$ and

$C \wedge D \Leftrightarrow C \wedge D'$, i.e., it finds and removes forbidden variable/value pairs to produce a smaller domain D' . The *prop* function must propagate strongly enough that if D' fixes all the variables, then either D' gives a solution of C , or D' is a false domain. The function *branch*(C, D) takes as input a set of constraints C and a domain D and returns a set of constraints $B \equiv \{b_1, \dots, b_k\}$ s.t. $C \wedge D \Rightarrow b_1 \vee \dots \vee b_k$. The *branch* function is used to split the problem into several more constrained subproblems and controls the search strategy. We call b_i the *decision constraints*. Typically, the b_i are simple unary constraints like $x = i$, $x \neq i$, $x \geq i$, or $x \leq i$.

```

      solv( $D, C$ )
1       $D' = \text{prop}(C, D)$ 
2      if ( $D' \Rightarrow \text{false}$ ) return false
3      if ( $\text{fixed}(D') = V$ ) return true
4       $B = \text{branch}(C, D')$ 
5      for ( $b \in B$ )
6          if ( $\text{solv}(D', C \cup \{b\})$ ) return true
7      return false

```

Figure 2.2: Basic search algorithm for propagation based constraint solvers.

Example 2.2.1. Consider the simple graph coloring problem above (Figure 2.1). Suppose we branch on the values of the variables in lexicographical order. The steps taken by the search algorithm is shown in Table 2.1. Each line corresponds to a node in the search tree and shows: 1) the depth of the node, 2) the decision that has just been made, 3) the inferences that can be made from that decision, 4) the state of the solver after propagation has been performed (unknown, failure or solution), and 5) the set of decisions to be used to split up the subproblem. Figure 2.3 gives the corresponding search tree. After 9 nodes and 3 failures, the solution $x_1 = 1, x_2 = 2, x_3 = 3, x_4 = 1, x_5 = 2, x_6 = 3$ is found.

It is easy to modify the algorithm to return the first solution or to return all solutions. To return solutions, we simply add some code in line 3 of Figure 2.2 to store/print the solution before returning. To get all solutions, we return *false* in line

Depth	Decision	Inferences	State	$branch(C, D)$
0	-		unknown	$\{x_1 = 1, x_1 = 2, x_1 = 3\}$
1	$x_1 = 1$	$x_5 \neq 1, x_6 \neq 1$	unknown	$\{x_2 = 1, x_2 = 2, x_2 = 3\}$
2	$x_2 = 1$	$x_3 \neq 1, x_4 \neq 1$	unknown	$\{x_3 = 2, x_3 = 3\}$
3	$x_3 = 2$	$x_4 = 3, x_5 = 2, x_6 = 2$	failure	-
3	$x_3 = 3$	$x_4 = 2, x_5 = 3, x_6 = 3$	failure	-
2	$x_2 = 2$	$x_3 \neq 2, x_4 \neq 2$	unknown	$\{x_3 = 1, x_3 = 3\}$
3	$x_3 = 1$	$x_4 = 3, x_5 = 2, x_6 = 2$	failure	-
3	$x_3 = 3$	$x_4 = 1$	unknown	$\{x_5 = 2, x_5 = 3\}$
4	$x_5 = 2$	$x_6 = 3$	solution	-

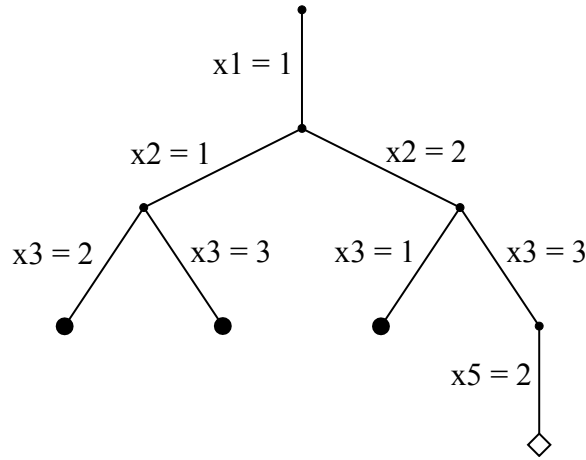
Table 2.1: Steps taken by $solv(C, D)$ algorithm in Example 2.2.1.

Figure 2.3: Search tree in Example 2.2.1.

3 instead of *true* when we find a solution. It is also easy to modify this algorithm to solve optimization problems via the branch and bound algorithm [93]. Suppose we want to minimize some objective function f . We define a global variable *best* which stores the objective value of the best known solution. At every node, in addition to the original constraints C , we also have an additional constraint $f < best$ which constrains the solver to find a solution better than the best found so far. The basic branch and bound algorithm $bab(D, C)$ is shown in Figure 2.4. The optimal value is stored in *best* at the end of the algorithm.

The performance of these algorithms are highly dependent on exactly how the

```

    best =  $\infty$ 
    bab( $D, C$ )
1       $D' = \text{prop}(C \cup \{f < \text{best}\}, D)$ 
2      if ( $D' \Rightarrow \text{false}$ ) return
3      if ( $\text{fixed}(D') = V$ )
4          best =  $f(\text{solution given by } D')$ 
5      return
6       $B = \text{branch}(C, D')$ 
7      for ( $b \in B$ )
8          bab( $D', C \cup \{b\}$ )
9      return

```

Figure 2.4: Basic branch and bound algorithm for propagation based constraint solvers.

functions *prop* and *branch* are implemented, and a vast amount of research in CP is aimed at improving these two functions.

The function *prop*(C, D) is commonly called the propagation engine. There are two primary concerns with regards to the design of the propagation engine: how fast it is, and how much inference it can provide. The first is important because the vast majority of solving time in most CP solvers is spent in the propagation engine. The second is important as it affects how quickly (in terms of search depth) we can determine failure and hence the size of the search tree.

The propagation engine is typically implemented by propagating each constraint individually, until a common fixed point is reached. A propagator p_c for constraint c is a function mapping domains to domains s.t. $c \wedge D \Leftrightarrow c \wedge p_c(D)$, i.e., it performs inference based on c and the current domain D , pruning away any variable/value pairs that cannot be taken given c and D . We say that D is a *fixed point* of propagator p_c if $p_c(D) = D$. A naive way to implement *prop*(C, D) is shown in Figure 2.5.

This naive algorithm for the propagation engine is not very efficient, as there may be many calls to $p_c(D')$ which result in no change to D' . Modern solvers utilize a variety of techniques to reduce the number of times propagators are called. The most effective ones are: watching domain change events to lazily wake up propagators, and


```

prop(C, D)
  D' = D
  do
    changed = false
    for (c ∈ C)
      D'' = pc(D')
      if (D'' ≠ D') changed = true
      D' = D''
  while (changed)
  return D'

```

Figure 2.5: Naive algorithm for the propagation engine.

ordering propagators by priority so that expensive propagators are called less often. See [139] for an in depth analysis of techniques for optimizing propagation engines.

The algorithms for the individual propagators are also of critical importance to the effectiveness of the solver. Enormous amounts of work have been done to try to increase the inference power of propagators and to improve their speed (e.g., [127, 128, 149, 122, 152]). Stronger inference typically costs more time. At one end of the scale, we have the bare minimum amount of inference that a propagator must produce in order for the CP solver to be correct. A propagator p_c for constraint c has to be at least *checking*, i.e., if D fixes all variables in $vars(c)$, then D either satisfies c , or p_c returns a false domain. At the other end of the scale, we have the maximally strong propagators that makes all possible inferences given the current domain and the constraint. A propagator p_c for constraint c is *domain consistent* if: $\forall D, D'$ s.t. $c \wedge D \Rightarrow D'$, $p_c(D) \Rightarrow D'$, i.e., it is at least as strong as any correct propagator for c . Finding the right trade off between inference power and speed is an important and difficult part of solving constraint problems effectively.

The function *branch* determines the search strategy by deciding how each problem is to be subdivided and the order in which the subproblems are to be examined. Many CP systems (Eclipse, Gecode, CHUFFED, etc) allow standard or custom search strategies to be defined as part of the problem model. Search strategies are often divided into two parts: a variable selector which picks the next variable to be branched

on, and a value selector which determines which values to try next. Some common variable selectors include: lexicographical ordering, smallest domain [66], highest degree [37], domain size divided by degree [14], weighted degree [20], domain size divided by weighted degree [20], etc. Some common value selectors include: smallest, largest, middle out, smaller half, larger half, etc. Many problems also benefit from custom search strategies (e.g., [28, 29, 54]). Search Strategies can have a huge effect on the run time of the search algorithm [17, 121, 66, 163]. A good search strategy should make branching decisions which allow failure to be inferred quickly, reducing the depth and size of the search tree. It should also order the branches of the search tree so that branches more likely to lead to (good) solutions are explored first.

2.3 Boolean Satisfiability

The Boolean Satisfiability Problem (SAT) is a special case of the Constraint Satisfaction Problem. While CP allows arbitrary domains and constraints, SAT only allows Boolean variables and clausal constraints. A SAT clause is a constraint of the form: $l_1 \vee \dots \vee l_n$, where each l_i is either a Boolean variable x or its negation $\neg x$. We call x and $\neg x$ *literals*. The specialization to Boolean variables and clauses allows for several powerful optimizations which we describe in this section. Note that $(l_1 \vee \dots \vee l_n) \equiv (\neg l_1 \wedge \dots \wedge \neg l_k \rightarrow l_{k+1} \vee \dots \vee l_n) \equiv (\neg l_1 \wedge \dots \wedge \neg l_n \rightarrow \text{false})$, and we will refer to constraints of any of these forms as clauses.

Most modern SAT solvers are based on the DPLL algorithm [35]. The DPLL algorithm is a special case of the more general CP algorithm given in Figure 2.2, where *prop* enforces domain consistency on all the clauses in the problem, and *branch* picks some unfixed Boolean variable v and returns $\{v, \neg v\}$. Due to the specialization to Boolean clauses, the constraints can be propagated to domain consistency very efficiently using the two watched literal scheme [109]. Many modern SAT solvers also utilize a very powerful dynamic search strategy called Variable State Independent Decaying Sum (VSIDS) [109]. In VSIDS, each Boolean variable has an activity count which is increased each time the variable is involved in a conflict. This activity count decays exponentially with time, and the unfixed variable with the highest activity is

picked to be branched on at each search node.

Modern SAT solvers also implement a powerful technique called clause learning [142]. The aim of clause learning is to derive additional clausal constraints which are implied by the original problem. Such learnt clauses can be added to the constraint store without changing the set of solutions to the problem, and can be used to make additional inferences, hopefully reducing the amount of search required.

In clause learning, the SAT solver constructs a directed, acyclic *implication graph* G as follows. When a literal m becomes true, either through a decision or through propagation, we add a node to the implication graph G representing m . If m was inferred through propagation, then there must be some clause $l_1 \wedge \dots \wedge l_n \rightarrow m$ which produced this inference. We add the directed edges (l_i, m) to the implication graph G .

We call $l_1 \wedge \dots \wedge l_n \rightarrow m$ the *explanation clause* for the inference m . The explanation clause for an inference m is referenced by an array $expl[m]$. Also, there is an array $level[m]$ which returns a pair (l, p) where l is the search depth at which m became true, and p is the number of literals which were set to true at depth l before m was set to true. We define a lexicographical ordering between such pairs so that $(l_1, p_1) < (l_2, p_2)$ iff $(l_1 < l_2 \vee (l_1 = l_2 \wedge p_1 < p_2))$. Before m can be inferred via the explanation clause, we must have $l_1, \dots, l_n$ all true. Hence $level[l_i] < level[m]$ which ensures that the implication graph is acyclic.

A conflict occurs when propagation causes the domain of some variable v to become empty. At this point, the implication graph must have a node representing v and a node representing $\neg v$. We call these two nodes the conflict nodes. Using the implication graph, we can now derive a new learnt clause.

First, we pick a cut in the implication graph with the following properties: all the decision nodes are on one side of the cut (call it the *reason side* R), the conflict nodes are on the other side (call it the *conflict side* S), and all edges in the cut-set are directed from R to S . Many such cuts exist. For example, we can take any literal m made true during the last decision level and define $R = \{l \mid l \in G, level[l] \leq level[m]\}$ and $S = \{l \mid l \in G, level[l] > level[m]\}$.

The resolution rule states that: if $l_1 \wedge \dots \wedge l_n \rightarrow a$ and $a \wedge k_1 \wedge \dots \wedge k_p \rightarrow false$,

then $l_1 \wedge \dots \wedge l_n \wedge k_1 \wedge \dots \wedge k_p \rightarrow \text{false}$. We use $\text{resolve}(c_1, c_2)$ to denote resolution between clauses c_1 and c_2 . Let $\text{lits}(L) = \{l_1, \dots, l_n\}$ if $L \equiv (l_1 \wedge \dots \wedge l_n \rightarrow \text{false})$. $\text{learn}(v, S)$ is an algorithm for deriving a learnt clause.

```

 $\text{learn}(v, S)$ 
   $L = (v \wedge \neg v \rightarrow \text{false})$ 
  while  $(\text{lits}(L) \cap S \neq \emptyset)$ 
     $a = \text{argmax}_{x \in \text{lits}(L) \cap S} (\text{level}[x])$ 
     $L = \text{resolve}(L, \text{expl}[a])$ 
  return  $L$ 

```

We begin with $L = (v \wedge \neg v \rightarrow \text{false})$ which is implied by v 's domain constraint. While L has any literals which are on the conflict side S of the cut, we pick the one with the highest level and eliminate it from L by resolving L with $\text{expl}[a]$. Resolution is possible because $\text{expl}[a]$ is some clause $l_1 \wedge \dots \wedge l_n \rightarrow a$ and L is some clause $a \wedge k_1 \wedge \dots \wedge k_p \rightarrow \text{false}$. The result of resolving two clauses which are implied by the problem is another clause which is implied by the problem. Hence the new L is implied by the problem and by induction, the algorithm is correct. The algorithm always terminates because G is acyclic.

Different choices of the cut give different learnt clauses L , which may have vastly different pruning strengths. A common choice of cut in state of the art SAT solvers is the First Unique Implication Point (1UIP) cut. A literal m is a Unique Implication Point (UIP) if every path from the last decision node to the conflict nodes in G pass through m . The 1UIP is the UIP with the highest $\text{level}[m]$ (first one we meet as we backtrack). We define the 1UIP cut as $R = \{l \mid l \in G, \text{level}[l] \leq \text{level}[m]\}$ and $S = \{l \mid l \in G, \text{level}[l] > \text{level}[m]\}$, where m is the 1UIP.

Example 2.3.1. Suppose we tried to solve our simple graph coloring problem using a SAT solver. We first have to remodel the problem into Boolean variables and clauses. We define Boolean variables $b_{i,k}$, $i = 1, \dots, 6$, $k = 1, 2, 3$ where $b_{i,k} \leftrightarrow x_i = k$. For clarity, we refer to $b_{i,k}$ as $\llbracket x_i = k \rrbracket$ and $\neg b_{i,k}$ as $\llbracket x_i \neq k \rrbracket$. For each x_i , we add $\llbracket x_i = 1 \rrbracket \vee \llbracket x_i = 2 \rrbracket \vee \llbracket x_i = 3 \rrbracket$, $\llbracket x_i \neq 1 \rrbracket \vee \llbracket x_i \neq 2 \rrbracket$, $\llbracket x_i \neq 2 \rrbracket \vee \llbracket x_i \neq 3 \rrbracket$,

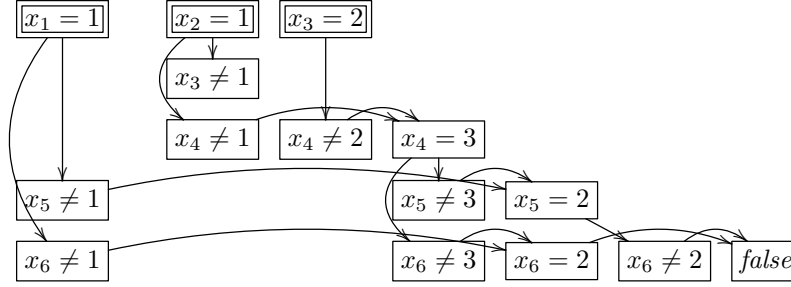


Figure 2.6: Implication graph for Example 2.3.1. Decision literals are double boxed.

L	a	$expl[a]$
$\llbracket x_6 = 2 \rrbracket \wedge \llbracket x_6 \neq 2 \rrbracket \rightarrow false$	$\llbracket x_6 \neq 2 \rrbracket$	$\llbracket x_5 = 2 \rrbracket \rightarrow \llbracket x_6 \neq 2 \rrbracket$
$\llbracket x_5 = 2 \rrbracket \wedge \llbracket x_6 = 2 \rrbracket \rightarrow false$	$\llbracket x_6 = 2 \rrbracket$	$\llbracket x_6 \neq 1 \rrbracket \wedge \llbracket x_6 \neq 3 \rrbracket \rightarrow \llbracket x_6 = 2 \rrbracket$
$\llbracket x_6 \neq 1 \rrbracket \wedge \llbracket x_6 \neq 3 \rrbracket \wedge \llbracket x_5 = 2 \rrbracket \rightarrow false$	$\llbracket x_5 = 2 \rrbracket$	$\llbracket x_5 \neq 1 \rrbracket \wedge \llbracket x_5 \neq 3 \rrbracket \rightarrow \llbracket x_5 = 2 \rrbracket$
$\llbracket x_6 \neq 1 \rrbracket \wedge \llbracket x_6 \neq 3 \rrbracket \wedge \llbracket x_5 \neq 1 \rrbracket \wedge \llbracket x_5 \neq 3 \rrbracket \rightarrow false$	$\llbracket x_6 \neq 3 \rrbracket$	$\llbracket x_4 = 3 \rrbracket \rightarrow \llbracket x_6 \neq 3 \rrbracket$
$\llbracket x_4 = 3 \rrbracket \wedge \llbracket x_6 \neq 1 \rrbracket \wedge \llbracket x_5 \neq 1 \rrbracket \wedge \llbracket x_5 \neq 3 \rrbracket \rightarrow false$	$\llbracket x_5 \neq 3 \rrbracket$	$\llbracket x_4 = 3 \rrbracket \rightarrow \llbracket x_5 \neq 3 \rrbracket$
$\llbracket x_4 = 3 \rrbracket \wedge \llbracket x_6 \neq 1 \rrbracket \wedge \llbracket x_5 \neq 1 \rrbracket \rightarrow false$		

Table 2.2: Steps taken by $learn(v, S)$ algorithm in Example 2.3.1.

$\llbracket x_i \neq 3 \rrbracket \vee \llbracket x_i \neq 1 \rrbracket$. For each CP constraint $x_i \neq x_j$ we had before, we simply have the clauses $\llbracket x_i \neq k \rrbracket \vee \llbracket x_j \neq k \rrbracket$ for $k = 1, 2, 3$. This is sufficient to model the problem as a SAT problem.

Suppose we run the same search as before until the first conflict. Figure 2.6 shows the implication graph at that point. $\llbracket x_6 = 2 \rrbracket$ is the conflict variable. $\llbracket x_4 = 3 \rrbracket$ is the 1UIP, so the 1UIP cut is given by: $S = \{\llbracket x_5 \neq 3 \rrbracket, \llbracket x_6 \neq 3 \rrbracket, \llbracket x_5 = 2 \rrbracket, \llbracket x_6 = 2 \rrbracket, \llbracket x_6 \neq 2 \rrbracket\}$. The algorithm $learn$ runs as shown in Table 2.2. We begin with the conflict clause $\llbracket x_6 = 2 \rrbracket \wedge \llbracket x_6 \neq 2 \rrbracket \rightarrow false$ and proceed to eliminate literals from S using clause resolution until we end up with the 1UIP learnt clause $\llbracket x_4 = 3 \rrbracket \wedge \llbracket x_6 \neq 1 \rrbracket \wedge \llbracket x_5 \neq 1 \rrbracket \rightarrow false$. This learnt clause can provide additional inference. For example, when we try $\llbracket x_1 = 1 \rrbracket$, we can now immediately infer $\llbracket x_4 \neq 3 \rrbracket$. When we then try $\llbracket x_2 = 2 \rrbracket$, we can infer $\llbracket x_4 \neq 2 \rrbracket, \llbracket x_4 = 1 \rrbracket, \llbracket x_3 = 3 \rrbracket$, leading us to a solution in one less conflict.

Part I

Nogood Learning

I.1 Introduction

Nogood learning techniques are used to speed up constraint solving by learning implied constraints called *nogoods* which can be used to prune parts of the search space. The learnt clauses described in Section 2.3 are simply nogoods for the specialized domain of SAT. Nogoods state conditions under which the problem can have no solutions. More formally:

Definition I.1.1. *A nogood n for a problem $P \equiv (V, D, C)$, is a constraint of form: $n \equiv \neg(l_1 \wedge \dots \wedge l_k)$, where l_i are arbitrary constraints, such that $C \wedge D \Rightarrow n$, i.e., the problem constraints imply that the constraints l_i cannot all be satisfied in any solution.*

Once we have derived a nogood, we can use it to prune the search space. If the current domain in a subproblem make all the l_i 's in the nogood true, then that subproblem has no solutions and can be pruned. If the domain make all but one of the l_i 's true, then the remaining l_i must be false for any solution within this subproblem. Typically, the l_i 's are very simple unary constraints such as $x = v$, $x \neq v$, $x \geq v$, $x \leq v$. However, as we shall see in Chapter 3, more complex constraints can also be used as terms in nogoods. We will sometimes write a nogood in the equivalent form $n \equiv l_1 \wedge \dots \wedge l_{k-1} \rightarrow \neg l_k$, to emphasize that we are using $l_1, \dots, l_{k-1}$ true to infer l_k false. There are several ways to derive nogoods, which we describe in the following sections.

I.2 Deriving Nogoods through Search

We can derive nogoods through exhaustive search. Let $P \equiv (V, D, C)$ be a problem. Let $P' \equiv (V, D, C \cup \{d_1, \dots, d_k\})$ be the subproblem of P reached after the set of decisions $d_1, \dots, d_k$. If P' is exhaustively searched and no solutions are found, then clearly, $n \equiv \neg(d_1 \wedge \dots \wedge d_k)$ is a nogood of P . We call n the *decision nogood* of P' .

Decision nogoods are easy to derive, but are also very weak, because they are only capable of pruning valuations that have already been examined. In a simple CP solver that does exhaustive search without any restarts or backjumping, decision nogoods

never prune anything, since we never visit the same valuation twice. However, there are search strategies, e.g., randomized search [63] or dynamic search [126], where the search may occasionally be restarted. It has been suggested that decision nogoods can be learned right before each restart in order to prevent the solver from examining the same parts of the search tree again [94].

Nogoods can be strengthened by removing unnecessary terms from them. For example, given a nogood $n \equiv \neg(d_1 \wedge \dots \wedge d_k)$, if some subset $\{d_{a_1}, \dots, d_{a_m}\}$ of the d_i 's are sufficient for failure, i.e., $C \wedge D \Rightarrow \neg(d_{a_1} \wedge \dots \wedge d_{a_m})$, then $n' \equiv \neg(d_{a_1} \wedge \dots \wedge d_{a_m})$ is strictly stronger than n in terms of pruning strength.

In [94], it is shown that in a solver that uses only binary branching, if the first branch is called the positive decision and the second is called the negative decision, then the negation of the conjunction of the set of positive decisions gives a valid nogood. While this optimization can improve the propagation strength of decision nogoods, it does not increase its theoretical pruning power, as they are still only capable of pruning valuations that have already been examined.

In [36], the author proposes to strengthen the decision nogoods by finding subsets of $\{d_1, \dots, d_{k-1}\}$ which are sufficient to conflict with the last decision d_k , i.e., some subset $\{d_{a_1}, \dots, d_{a_m}\}$ s.t. $C \wedge D \Rightarrow \neg(d_{a_1} \wedge \dots \wedge d_{a_m} \wedge d_k)$. A simple algorithm proposed is to remove from $\{d_1, \dots, d_{k-1}\}$ any d_i where $vars(d_i)$ is not connected to $vars(d_k)$ in the constraint graph, as such decisions cannot possibly be contributing to the inconsistency with d_k . While this is reasonably efficient, the subsets so found are not minimal (in the sense that no smaller subset is sufficient to conflict with d_k), and there are no cheap ways to calculate minimal subsets. However, such strengthening represent a real increase in their pruning power, making them capable of pruning valuations that have never been examined before.

I.3 Deriving Nogoods through Resolution

Conflict directed nogood learning is a class of methods that derive nogoods using constraint resolution. When a conflict is found during search, the set of constraints involved in the conflict are found and are resolved together in some way to derive

a new nogood. The nogood explains why the failure occurred, and can prevent the search from failing again in the same way. Nogoods derived through these methods are typically significantly more powerful than the decision nogoods described in the previous section [165]. While decision nogoods can only prevent the solver from exploring a valuation that has previously been explored, nogoods derived through resolution can prevent the solver from exploring valuations that have never been explored before. They can produce exponential speedup on many problems and can change the asymptotic complexity of the search. We first give some definitions.

Definition I.3.1. *Given current domain D , suppose the propagator for constraint c makes an inference m , i.e., $c \wedge D \Rightarrow m$. An explanation for this inference is a nogood: $\text{expl}(m) \equiv l_1 \wedge \dots \wedge l_k \rightarrow m$ s.t. $c \Rightarrow \text{expl}(m)$ and $D \Rightarrow l_1 \wedge \dots \wedge l_k$.*

The nogood $\text{expl}(m)$ explains why m has to hold given c and the current domain D . We can consider $\text{expl}(m)$ as the fragment of the constraint c from which we inferred that m has to hold.

Example I.3.2. Suppose we have constraint $x \leq y$ and current domain $x \in \{3, 4, 5\}$. A propagator may infer that $y \geq 3$, and a valid explanation can be $x \geq 3 \rightarrow y \geq 3$. Suppose we have constraint $\text{alldiff}(x, y, z)$ and current domain $x, y, z \in \{1, 2\}$. A propagator may infer *false* and a valid explanation can be $x \geq 1 \wedge x \leq 2 \wedge y \geq 1 \wedge y \leq 2 \wedge z \geq 1 \wedge z \leq 2 \rightarrow \text{false}$.

Given two nogoods containing complementary terms, we can resolve them together to derive a new nogood.

Lemma I.3.3. *Given a problem P , if $n_1 \equiv \neg(a \wedge l_1 \wedge \dots \wedge l_p)$ and $n_2 \equiv \neg(\neg a \wedge m_1 \wedge \dots \wedge m_q)$ are both nogoods of P , then so is $n_3 \equiv \neg(l_1 \wedge \dots \wedge l_p \wedge m_1 \wedge \dots \wedge m_q)$.*

Proof. $C \wedge D \Rightarrow n_1 \wedge n_2 \Rightarrow n_3$. $\square$

We call expressions of the form $x = v$ *equality literals*, expressions of the form $x \neq v$ *disequality literals*, and expressions of the form $x \geq v$ or $x \leq v$ *inequality literals*.

We now give a brief history of the development of conflict directed nogood learning.

One of the earliest conflict directed nogood learning schemes is described in [136]. In this system, constraints are only checked for failure w.r.t. the variables that are fixed, i.e., no propagation other than failure is allowed. At each node, the branching heuristic picks a variable and branches on its value. Suppose for current domain D , each variable $x_i \in \text{fixed}(D)$ is set to $x_i = v_i$ for some constant v_i . If the propagator for some constraint c detects failure, and V' is the set of variables in $\text{vars}(c)$ that are fixed, then we can trivially derive the explanation for failure $\bigwedge_{x_i \in V'} x_i = v_i \rightarrow \text{false}$. If $x_k = v_k$ was the last decision we made before this conflict, we store this explanation as $\text{expl}(x_k \neq v_k)$.

At any non-leaf node, when all the values of the branching variable have been tried and failed, it is possible to resolve together the nogoods to derive a new one. Suppose x_k was the branching variable for the non-leaf node and D_k was its domain. From the domain of x_k , we have the nogood $\neg(\bigwedge_{v \in D_k} x_k \neq v)$. We can resolve this nogood with each of $\text{expl}(x_k \neq v), v \in D_k$ to get a new nogood n . n is added to the solver as a normal constraint. n is currently violated. We backtrack to the earliest decision level where n is violated and use n to fail that node. The algorithm then continues. Unfortunately, this rather primitive form of conflict directed nogood learning can only handle the simplest forms of propagations, i.e., failure from fixed variables, and thus is not useful for general CP solving.

A major advance in conflict directed nogood learning was made in the context of SAT solving [142]. The nogood learning scheme for SAT was described in detail in Chapter 2. One major improvement was that due to the restriction to Boolean variables and clauses, explanations can be trivially derived even when the clauses are propagated to domain consistency rather than only checked for failure (the explanation is simply the clause itself). Another important improvement was the development of a different resolution scheme. In SAT, the implication graph formed by the explanation nogoods could be cut in a variety of ways, and the explanation nogoods on one side of the cut is resolved to form a nogood. Different cuts lead to different nogoods of differing strength. In particular, the 1UIP cut, which aims to derive a nogood that is as local to the conflict as possible, has been found to be very effective at reducing search. SAT nogood learning produces orders of magnitude speedup on structured

problems and greatly advanced the industrial use of SAT solving.

More progress was made in CP nogood learning with the introduction of generalized nogoods or g-nogoods [84], where disequality literals were allowed to appear in nogoods. Previous work on nogoods had mostly only used equality literals (called s-nogoods in [84]). G-nogoods are significantly more expressive than s-nogoods, as each g-nogood can potentially represent an exponential number of s-nogoods. This increase in expressiveness allowed arbitrary inferences to be explained, and also increased the pruning power of the nogoods derived. Arbitrary inferences can be explained as follows. Suppose we had initial domain D , current domain D' , and the value v was removed from variable x by constraint c , then we can explain this using: $\bigwedge_{y \in (\text{vars}(c) \setminus \{x\})} \bigwedge_{w \in (D_y \setminus D'_y)} y \neq w \rightarrow x \neq v$. The g-nogoods explaining the propagations form an implication graph just as in the case of SAT, and we can derive a nogood from each conflict in an analogous manner.

Lazy Clause Generation (LCG) [113, 45] represents the current state of the art in CP nogood learning. LCG further extends the types of literals that can be included in nogoods with inequality literals. This extension is particularly important for propagators that use variable bounds for inference. Each LCG nogood can represent a linear number of g-nogoods. This increase in expressiveness makes the LCG nogoods yet more powerful. Modern LCG solvers like CHUFFED can beat state of the art non-learning solvers like Gecode by very significant margins on a wide variety of problems. See [106] for example.

I.4 Contributions

In Chapter 3, we describe a new method for deriving nogoods based on constraint projections. This differs significantly from most previous methods which rely on exhaustive search or constraint resolution. In many combinatorial search problems, it is possible for multiple search paths to lead to subproblems that are actually equivalent, in the sense that every solution of one corresponds to a solution of the other. Such equivalences arise because the projections of the subproblems onto the set of unfixed variables often yield the same problem. We show how we can exploit such

redundancies by using constraint projections to derive nogoods. Such nogoods can give us exponential speedups on appropriate problems. They differ in power and expressiveness compared to LCG nogoods, and can be exponentially stronger on some problems.

Chapter 3

Automatic Caching via Constraint Projection

3.1 Introduction

In many combinatorial search problems, different search paths can lead to subproblems which are actually identical on the remaining unlabeled variables. Consider the simple problem $C \equiv \{alldiff(x_1, x_2, x_3, x_4), x_1 + x_2 + x_3 + x_4 \geq 20, x_1 + 2x_2 \leq 10\}$. Consider the two partial assignments $x_1 = 1, x_2 = 2$, and $x_1 = 2, x_2 = 1$. They both lead to the same problem on x_3, x_4 , i.e., $alldiff(1, 2, x_3, x_4), x_3 + x_4 \geq 17$. Clearly, examining one search path is enough for us to prune the other. Such subproblem equivalences are very common in certain types of problems, leading to a large amount of redundancy in the search. There are a number of methods to prevent this, e.g., caching [145], dynamic programming [11], nogood learning [113, 45], as well as methods that rely on certain problem structures, e.g., decomposition [86, 100] and variable elimination [90]. We briefly describe these methods.

Caching can be used to store results from solving subproblems (e.g., failure, solutions, bounds), so that they can be reused when an equivalent subproblem is encountered later in the search. For caching to be effective, the lookup operation must be efficient. A popular way to implement this operation is to map each subproblem to a *key* which is then used to store and access the information for that subproblem. While caching can be very effective, it can also be very laborious and error prone. It requires the careful manual construction of a key for each model and search strategy, and the modification of the solver to use these keys. The large amount of work required make it quite difficult to use caching as a general method.

Dynamic programming can also exploit subproblem equivalences. However, many constraint problems cannot easily be expressed as dynamic programs, e.g., it may not be recursive, or it may have non-recursive global constraints or side constraints. Also, unlike constraint programming, dynamic programming does not use inference, thus many opportunities for pruning will be lost, leading to a much larger search space.

Nogood learning is another possibility for exploiting subproblem equivalences. Nogood learning techniques like Lazy Clause Generation (LCG) [113, 45] derive nogoods using constraint resolution. Such nogoods explain the reasons why a subproblem failed, and can potentially prune off equivalent subproblems later in the search.

However, there is no guarantee that it will. Whether it does depends on a number of factors such as how propagators explain their propagations, the order in which inferences were made, and which particular cut in the implication graph was used to derive the nogood.

There are certain structural properties of the constraint graph that make a problem more likely to have subproblem equivalences. For example, problems which can be decomposed by fixing a small number of variables will often have subproblem equivalences when one side of the decomposition has been labeled. We can prevent such equivalent subproblems by caching on the variables in the cut [86, 100]. Subproblem equivalences also often arise in problems where the constraint graph has low pathwidth (see [85] for a definition of pathwidth). These can be exploited by methods such as variable elimination [90].

In this chapter we present a new method called automatic caching via constraint projection. Our method differs from the above in that it can automatically detect and exploit caching opportunities in arbitrary optimization problems, and does not rely on problem structure such as low pathwidth or decomposability. The principal insight of our work is to define a key based on constraint projection that can be efficiently computed during search and can be used to exploit a relatively general notion of re-usability which we refer to as subproblem dominance. We cache failures only, thus our method is a form of nogood learning, where the terms of the nogoods consist of constraint projections. We experimentally demonstrate the effectiveness of our approach, which has been implemented in the competitive CP solver CHUFFED. We also provide interesting insights into the relationships between subproblem dominance, dynamic programming, symmetry breaking and nogood learning.

The rest of the chapter is organized as follows. Section 3.2 defines subproblem dominance and subproblem equivalence, and shows how we can make use of these notions to improve search by caching. Section 3.3 gives some examples of manual keys to support caching of specific models. Section 3.4 shows how we can define keys for arbitrary problems on a per constraint basis and taking into account the propagation algorithm of the constraint. Section 3.5 shows how to construct keys and efficient representations of keys for common primitive and global constraints. Section 3.6

illustrates the keys generated by constraint projection on a number of example problems. Section 3.7 discusses earlier work on caching in constraint programming and the relationship with related approaches such as dynamic programming and symmetry elimination. Section 3.8 provides an experimental evaluation showing that automatic caching can be very effective when subproblem equivalence exists. Finally, Section 3.9 provides our conclusions.

3.2 Subproblem Dominance and Equivalence

We restrict our analysis to search algorithms which make decisions on single variables, i.e., decisions of the form $x = v$, $x \neq v$, $x \geq v$, $x \leq v$, or $x \in S$ for some set S . This restriction rules out certain kinds of search, but most commonly used search algorithms are still covered. We express subproblems as $P' \equiv (V, D', C)$ where C are the original constraints, and D' is the current domain at propagation fixpoint. This is possible since all decisions are unary constraints and can be thought of as domain reductions (changing D , but leaving C unchanged).

We use $\pi_U(c)$ to denote the projection of a constraint c onto a subset of its variables U . That is, $\pi_U(c)$ is a constraint over U such that $\theta \in \text{solns}(\pi_U(c))$ iff θ can be extended to a solution of c by some labelling of the variables in $\text{vars}(c) \setminus U$. For example, the projection of $\text{alldiff}([x_1, x_2, x_3, x_4])$ onto the subset of variables $U \equiv \{x_3, x_4\}$ is: $\pi_U(\text{alldiff}([x_1, x_2, x_3, x_4])) \Leftrightarrow \text{alldiff}([x_3, x_4])$. The projection of $(x_1 = 1 \wedge x_1 + x_2 + x_3 \geq 10)$ onto $U \equiv \{x_2, x_3\}$ is: $\pi_U(x_1 = 1 \wedge x_1 + x_2 + x_3 \geq 10) \Leftrightarrow x_2 + x_3 \geq 9$. Typically, we will use projection to project out variables which are already fixed by the constraint (as in the second example above). For domain constraints $D \equiv \{x_i \in S_i \mid x \in V\}$, we use subscripts to denote projection onto $V' \subseteq V$, i.e., $D_{V'} \equiv \{x_i \in S_i \mid x \in V'\}$.

We define subproblem dominance and subproblem equivalence as follows:

Definition 3.2.1. *Let $P \equiv (V, D, C)$ and $P' \equiv (V, D', C)$ be two different subproblems arising during search. Let F be the fixed variables in D , i.e., $F = \text{fixed}(D)$, and U be the unfixed variables in D , i.e., $U = \text{vars}(C) \setminus F$. Let F' be the fixed variables in*

D' , i.e., $F' = \text{fixed}(D')$, and U' be the unfixed variables in D' , i.e., $U' = \text{vars}(C) \setminus F'$. P dominates P' if:

- $F = F'$
- $\pi_U(C \wedge D') \Rightarrow \pi_U(C \wedge D)$

P dominates P' if the same set of variables are fixed in each, and the projection of P' onto U implies the projection of P onto U , i.e., P' constrains the unfixed variables at least as strongly as P . Note that dominance may hold even if the fixed variables in P and P' are fixed to different values. If P dominates P' , then each solution of P' corresponds to a solution of P , as formalized by the following theorem.

Definition 3.2.2. Let $fx(D)$ be a valuation over the set of variables $\text{fixed}(D)$, which maps each variable to the value it is fixed to in D . Given a problem $P \equiv (V, D, C)$, let $fx(P) \equiv fx(D)$.

Theorem 3.2.3. Suppose P dominates P' . Then $\forall \theta \in \text{solns}(P'), (\theta_U \cup fx(P)) \in \text{solns}(P)$. $\square$

Proof. Suppose $\theta \in \text{solns}(P')$. By the definition of projection, $\theta_U \in \text{solns}(\pi_U(C \wedge D'))$. By the definition of subproblem dominance, we have $\theta_U \in \text{solns}(\pi_U(C \wedge D))$. Since D fixes all variables not in U , we then have $\theta_U \cup fx(D) \in \text{solns}(C \wedge D)$, so $\theta_U \cup fx(P) \in \text{solns}(P)$. $\square$

Corollary 3.2.4. If P dominates P' and P has no solutions, then P' has no solutions.

Example 3.2.5. Consider the problem $P_{\text{init}} \equiv (V, D_{\text{init}}, C)$, where $V \equiv \{x_1, \dots, x_5\}$, $D_{\text{init}} \equiv \{x_1 \in \{1, 2, 3\}, x_2 \in \{1, 2, 3, 4\}, x_3 \in \{2, 3, 4\}, x_4 \in \{3, 4, 5\}, x_5 \in \{3, 4, 5\}\}$ and $C \equiv \{x_1 + 2x_2 + x_3 + x_4 + 2x_5 \leq 20\}$. Consider two subproblems $P \equiv (V, D, C)$ and $P' \equiv (V, D', C)$ where $D_F \equiv \{x_1 = 3, x_2 = 1\}$, and $D'_F \equiv \{x_1 = 1, x_2 = 3\}$. Then $U \equiv \{x_3, x_4, x_5\}$ and $D_U \equiv D'_U \equiv \{x_3 \in \{2, 3, 4\}, x_4 \in \{3, 4, 5\}, x_5 \in \{3, 4, 5\}\}$. P dominates P' since $\pi_U(C \wedge D') \Leftrightarrow (x_3 + x_4 + 2x_5 \leq 13 \wedge D'_U) \Rightarrow (x_3 + x_4 + 2x_5 \leq 15 \wedge D_U) \Leftrightarrow \pi_U(C \wedge D)$. $\square$

For the remainder of the chapter we will often make the implicit assumption that for subproblem P , $P \equiv (V, D, C)$ where C is the set of constraints common to all subproblems considered and that, given domain D , $F = \text{fixed}(D)$ and $U = \text{vars}(C) \setminus F$.

Definition 3.2.6. P and P' are equivalent iff P dominates P' and P' dominates P .

Proposition 3.2.7. If P and P' are equivalent then $\forall \theta \in \Theta_U$, $(\theta \cup \text{fx}(P)) \in \text{solns}(P)$ iff $(\theta \cup \text{fx}(P')) \in \text{solns}(P')$. $\square$

Example 3.2.8. Consider the problem $P_{\text{init}} \equiv (V, D_{\text{init}}, C)$, where $V \equiv \{x_1, \dots, x_5\}$, $D_{\text{init}} \equiv \{x_1 \in \{1, 2, 3\}, x_2 \in \{1, 2, 3, 4\}, x_3 \in \{2, 3, 4\}, x_4 \in \{3, 4, 5\}, x_5 \in \{3, 4, 5\}\}$ and $C \equiv \{\text{alldiff}([x_1, x_2, x_3, x_4, x_5])\}$. Consider two subproblems $P \equiv (V, D, C)$ and $P' \equiv (V, D', C)$ where $D_F \equiv \{x_1 = 1, x_2 = 2\}$, and $D'_F \equiv \{x_1 = 2, x_2 = 1\}$. Both subproblems project to $\text{alldiff}([1, 2, x_3, x_4, x_5]) \wedge \{x_3 \in \{2, 3, 4\}, x_4 \in \{3, 4, 5\}, x_5 \in \{3, 4, 5\}\}$ and are therefore equivalent. $\square$

Detecting subproblem dominances allows the search to avoid exploring subproblems known to have no solutions. We alter the basic algorithm for $\text{solv}(V, D, C)$ shown in Figure 2.2 by adding the two lines marked with asterisks in Figure 3.1. The first line checks to see if the current subproblem is dominated by something in the cache N . If so, we can immediately fail it. The second line adds a newly failed subproblem to the cache N .

It is also correct to use this modified algorithm for $\text{solv}(V, D, C)$ in the branch and bound algorithm shown in Figure 2.4. This is because the constraint $f < b$, which constrains the solver to find a solution better than the best solution found so far, strictly increases in strength as the search goes on, and any subproblem that fails under one set of constraints must also fail under a strictly stronger set of constraints.

3.3 Manual Keys for Caching

The principal difficulty in implementing the caching algorithm is to make the lookup and test for subproblem dominance sufficiently efficient. We want a *key* to represent

```

solv(D, C)
  D' = prop(C, D)
  if (D'  $\Rightarrow$  false) return false
  if (fixed(D') = V) return true
  * if ( $\exists P \in \text{Cache}$  where P dominates (D', C)) return false
  B = branch(C, D')
  for (b  $\in$  B)
    if (solv(D', C  $\wedge$  b)) return true
  * N = N  $\cup$  {(D', C)}
  return false

```

Figure 3.1: Search algorithm taking advantage of subproblem dominance.

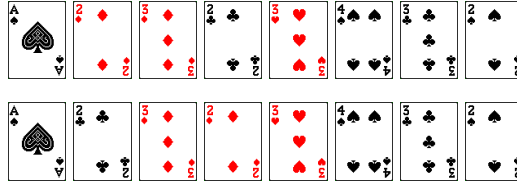


Figure 3.2: Two starting sequences for the Black Hole problem that lead to an identical remaining subproblem.

subproblems which allows dominance to be detected efficiently (preferably in $O(1)$ time). Naively, one may think that D could be used as a key to characterize subproblem $P \equiv (V, D, C)$. However, while D certainly uniquely identifies the subproblem, it is useless as a key since D is different for each subproblem in the search tree and will never produce a match. Thus we need to find a more powerful key; one that can be identical for equivalent subproblems reached through different sets of decisions. We now present hand crafted keys for three problems.

Example 3.3.1. Consider the Black Hole Problem [60], which seeks to find a solution to the Black Hole patience game. In this game the 52 playing cards (of standard 52-card deck) are laid out in 17 piles of 3, with the ace of spades starting in a “black hole”. Each turn, a card at the top of one of the piles can be played into the black hole if it is ± 1 from the card that was played previously, with king wrapping back around to ace. The aim is to play all 52 cards. This was one of two examples used

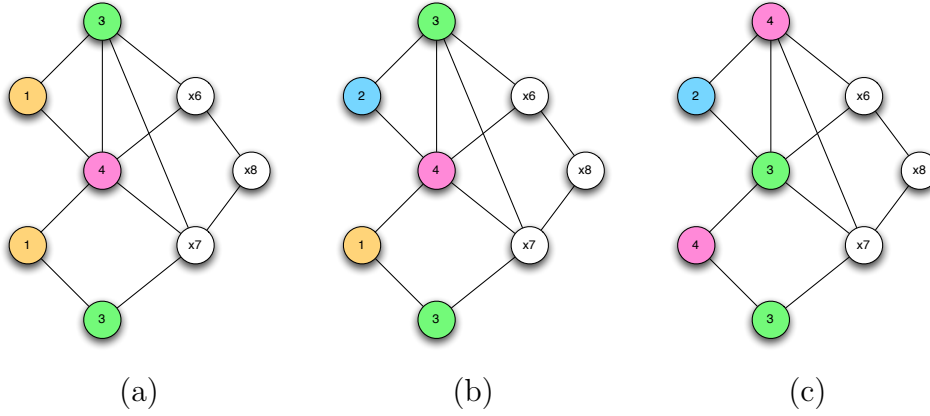


Figure 3.3: Three examples of partially colored graphs.

to illustrate CP with caching in [145]. Suppose the first k cards have been played. The remaining subproblem only depends on the set of unplayed cards, and the value of the last card played. Subproblem equivalences arise because, for any particular assignment, there may be many permutations of the first $k - 1$ assignments which also satisfy the constraints, and all of these will lead to an equivalent subproblem, as exemplified in Figure 3.2. We can exploit this by defining the key as (l, S) , where l is the value of the last card played, and S is the set of unplayed cards. This key allows us to detect the equivalence of all of these subproblems and avoid the redundant search. $\square$

Example 3.3.2. Consider the Graph Coloring Problem, of coloring a graph with some set of colors (represented by integers) so that no two adjacent nodes have the same color. Given a partial (coloring) labeling of the nodes, define the *frontier* as the set of labeled nodes which are adjacent to an unlabeled node. The problem of coloring the remaining uncolored nodes is characterized by the set of labeled nodes, and the values of the labeled nodes on the frontier. It does not depend on the values of the non-frontier labeled nodes, as their value can have no further effect on the unassigned nodes. Subproblem equivalences arise because there could be many different ways to label the non-frontier labeled nodes while producing the same frontier, and all of these lead to equivalent subproblems. Whether this occurs frequently depends on the structure of the graph and the labeling strategy. We can define the key as

$(L, [(x, d_x) \mid x \in F])$, where L is the set of labeled nodes, F is the set of frontier nodes, and d_x is the value assigned to node x . This key allows us to detect the equivalence of these subproblems and avoid the redundant search. The partially colored graph in Figure 3.3(a) has a frontier (the second column) labeled 3, 4, 3, which is identical to the frontier for the graph in Figure 3.3(b). $\square$

Example 3.3.3. The Minimization of Open Stacks Problem (MOSP) [162] can be described as follows. A factory manufactures a number of different products in batches, i.e., all copies of a given product need to be finished before a different product is manufactured, so there are never two batches of the same product. Each customer of the factory places an order requiring one or more different products. Once one product in a customer's order starts being manufactured, a stack is opened for that customer to store all products in the order. Once all the products for a particular customer have been manufactured, the order can be sent and the stack is freed for use by another order. The aim is to determine the sequence in which the products should be manufactured to minimize the maximum number of open stacks, i.e., the maximum number of customers whose orders are simultaneously active. We give a simplified model for the MOSP based on [28], in the modeling language MiniZinc [112].

```

int: n;                                % number of customers

% which pairs of customers share at least 1 product
array[1..n,1..n] of 0..1: W;

array[1..n] of var 1..n: s;             % customer start time
array[1..n] of var 1..n: e;             % customer end time
array[1..n] of var 1..n: x;             % customer end order
var 0..n: objective;                   % number of stacks

constraint inverse(e,x);
constraint forall (i in 1..n) (
    minimum(s[i], [e[j] | j in 1..n where W[i,j]]));

```

```

constraint forall (t in 1..n) (sum (i in 1..n) (
    bool2int((s[i] <= t) /\ (e[i] >= t))) <= objective);

solve :: int_search(x, input_order, indomain_min, complete)
    minimize objective;

```

Each MOSP instance is defined by n , the number of customers, and a Boolean function $W(i, j)$ which tells us whether customers i and j share a product. We have variables, $s[i] \in \{1, \dots, n\}$ representing when each customer's stack is opened, and $e[i] \in \{1, \dots, n\}$ representing when each customer's stack is closed. We channel the customer end times with the customer end order x with *inverse*. Each customer's stack must be opened before the stack of any of the customers with which they share a product closes, because the product they share must have begun production. This gives rise to the *minimum* constraints. Lastly, the number of stacks which are open during each time period is less than or equal to the total number of stacks we need, giving rise to the linear inequality constraints. We label the $x[i]$ in order, i.e., we pick which customer closes first, then which customer closes second, etc.

This problem contains many subproblem equivalences. A solver exploiting these equivalences achieves several orders of magnitude speedup [53]. The subproblem equivalences arise as follows: after labeling $x[1], \dots, x[m]$, if the subproblem has not yet failed, then the remaining subproblem does not depend on what order the first m customer's stacks were closed, but only on the *set* of customers whose stacks were closed.

Let us consider a problem where $n = 6$, customers 1, 3 and 4 want product 1, customers 2 and 6 want product 2, and customers 3 and 5 want product 3. This is represented by the data file:

```

n = 6;
W = [| 1, 0, 1, 1, 0, 0
      | 0, 1, 0, 0, 0, 1
      | 1, 0, 1, 1, 1, 0
      | 1, 0, 1, 1, 0, 0

```

```

| 0, 0, 1, 0, 1, 0
| 0, 1, 0, 0, 0, 1 |];

```

where a 1 in position (i, j) represents that customers i and j have a product in common.

Suppose we are currently looking for a solution with ≤ 4 stacks. Consider the following two subproblems $P \equiv (V, D, C)$ where $D_F \equiv \{x[1] = 4, x[2] = 3, x[3] = 5\}$, and $P' \equiv (V, D', C)$ where $D'_F \equiv \{x[1] = 5, x[2] = 4, x[3] = 3\}$. Neither D nor D' violates the constraints, and they both schedule the same three customers first. The remaining subproblem on $x[4], x[5], x[6]$ is identical for the two subproblems, so P and P' are equivalent. If P has no solutions, then P' also has no solutions. In general, any subproblem which assigned $\{x[1], x[2], x[3]\}$ to the set $\{4, 3, 5\}$ will also have no solutions. This subproblem equivalence can be exploited with a very simple key. In this example, we could use the key $\{x[1], x[2], x[3]\} = \{4, 3, 5\}$, or even more simply, $\{4, 3, 5\}$. Any partial assignment on $\{x[1], x[2], x[3]\}$ which produces the same key necessarily leads to an equivalent subproblem. $\square$

3.4 Automatic Keys for Caching

Manually generating keys is reasonably straightforward for problems that are simple and highly structured. However, it can be very difficult for more complex problems. In this section, we examine how keys can be automatically constructed for arbitrary constraint problems.

By definition, if P dominates P' then the projection of P' onto U logically implies the projection of P onto U . If we could automatically construct keys that characterize the projections of subproblems, we can use such keys for automatic dominance detection. We can do this by exploiting the fact that the piecewise dominance of the projections of all constraints in C is a sufficient condition for subproblem dominance. This is formalized in the following theorem:

Theorem 3.4.1. *Let $P \equiv (V, D, C)$ and $P' \equiv (V, D', C)$ be subproblems arising during search, where $\text{fixed}(D) = \text{fixed}(D') = F$ and $U = \text{vars}(C) \setminus F$. If $\forall c \in$*

$C, \pi_U(c \wedge D'_F) \Rightarrow \pi_U(c \wedge D_F)$ and $D'_U \Rightarrow D_U$, then P dominates P' .

Proof.

$$\begin{aligned}
& \pi_U(C \wedge D') \\
& \Leftrightarrow \pi_U(C \wedge D'_F \wedge D'_U) \\
& \Leftrightarrow \pi_U(\bigwedge_{c \in C} (c \wedge D'_F)) \wedge D'_U \\
(\star) \quad & \Leftrightarrow \bigwedge_{c \in C} (\pi_U(c \wedge D'_F)) \wedge D'_U \\
& \Rightarrow \bigwedge_{c \in C} (\pi_U(c \wedge D_F)) \wedge D_U \\
(\star) \quad & \Leftrightarrow \pi_U(\bigwedge_{c \in C} (c \wedge D_F)) \wedge D_U \\
& \Leftrightarrow \pi_U(C \wedge D_F \wedge D_U) \\
& \Leftrightarrow \pi_U(C \wedge D)
\end{aligned}$$

The third and fifth (marked) equivalences hold because all variables being projected out in every $c \wedge D'_F$ and $c \wedge D_F$ are already fixed. $\square$

The importance of Theorem 3.4.1 lies in the fact that it allows us to treat the projection of each constraint in C separately. Thus, if we can characterize the projection of each constraint in C , we can concatenate these, together with D_U and F , to form a projection key which completely characterizes the subproblem.

3.4.1 Projection Keys

Naively, we can define a function $key(c, D) \equiv \pi_U(c \wedge D_F)$, where $F = fixed(D)$ and $U = vars(D) \setminus F$, to characterize the projection of the constraint c onto the unfixed variables U . There are several problems with this. Firstly, it means that we need one key per constraint, which could make the key for the whole subproblem very large. Secondly, the projection of complex constraints is not always easy to describe or represent, thus $\pi_U(c \wedge D_F)$ may have no reasonably sized representation. We therefore use a more complex definition of $key(c, D_F)$ that allows for more flexibility and several important optimizations.

Definition 3.4.2. Let $w(c, D_F)$ be the weakest domain on U that the propagator p_c for constraint c can return given D_F , i.e., for any D' s.t. $D' \Rightarrow D_F$ and $D' \Rightarrow D_{init}$ we have $p_c(D') \Rightarrow w(c, D_F)$.

Definition 3.4.3. The projection key $key(c, D_F)$ of a constraint c for domain D is a set of constraints over $vars(c)$, such that:

$$\pi_U(c \wedge key(c, D_F)) \wedge w(c, D_F) \Leftrightarrow \pi_U(c \wedge D_F).$$

In general, there are many ways to correctly define $key(c, D_F)$. However, we are interested in defining $key(c, D_F)$ such that we can detect as much dominance as possible. Note that whether a particular definition of $key(c, D_F)$ is correct depends on the consistency level of the propagator used for constraint c . There are two trivially correct definitions for $key(c, D_F)$: $key(c, D_F) \equiv D_{F \cap vars(c)}$ is correct and can always be represented, but it does not allow us to detect any dominances; $key(c, D_F) \equiv \pi_U(c \wedge D_F)$ is correct and allows us to detect many dominances, but it cannot always be represented in a reasonable size.

Example 3.4.4. Let $V \equiv \{x_1, \dots, x_6\}$ and $D_{init} \equiv \{x_i \in \{1, \dots, 6\} \mid i = 1, \dots, 6\}$. We show four definitions of $key(c, D_F)$ for $c \equiv alldiff([x_1, x_2, x_3, x_4, x_5, x_6])$ and show that they do not all detect the same amount of dominance. Suppose we have some D where $D_F \equiv \{x_1 = 3, x_2 = 4, x_3 = 5\}$. Define $key_1(c, D_F) \equiv D_{F \cap vars(c)} \equiv x_1 = 3 \wedge x_2 = 4 \wedge x_3 = 5$. Define $key_2(c, D_F) \equiv \bigwedge_{x \in (F \cap vars(c))} x \in \{v \mid \exists x_i \in F, x_i = v\} \equiv x_1 \in \{3, 4, 5\} \wedge x_2 \in \{3, 4, 5\} \wedge x_3 \in \{3, 4, 5\}$. Define $key_3(c, D_F) \equiv \pi_U(c \wedge D_F) \equiv alldiff([3, 4, 5, x_4, x_5, x_6])$. Define $key_4(c, D) \equiv \bigwedge_{x \in (vars(c) \setminus F)} x \notin \{v \mid \exists x_i \in F, x_i = v\} \equiv x_4 \notin \{3, 4, 5\} \wedge x_5 \notin \{3, 4, 5\} \wedge x_6 \notin \{3, 4, 5\}$. key_1 and key_3 are trivially correct. key_2 is correct because as long as the fixed vars take the same set of values, the constraints on the remaining unfixed vars will be the same. key_4 is correct since it captures the entire effect of D_F on the unfixed variables U .

Now, consider a different subproblem $D'_F \equiv \{x_1 = 4, x_2 = 5, x_3 = 3\}$. $key_1(c, D'_F) \equiv x_1 = 4 \wedge x_2 = 5 \wedge x_3 = 3$ does not match the key of the first subproblem. $key_2(c, D'_F) \equiv x_1 \in \{3, 4, 5\} \wedge x_2 \in \{3, 4, 5\} \wedge x_3 \in \{3, 4, 5\}$ matches and shows that the *alldiff* projects to the same constraint in both subproblems. $key_3(c, D'_F) \equiv alldiff([4, 5, 3, x_4, x_5, x_6])$ does not match syntactically, but is semantically equivalent to $key_3(c, D_F)$. $key_4(c, D'_F) \equiv x_4 \notin \{3, 4, 5\} \wedge x_5 \notin \{3, 4, 5\} \wedge x_6 \notin \{3, 4, 5\}$ also matches. $\square$

We now illustrate how to use projection keys for checking subproblem dominance. Note that dominance checking, and thus projection key calculation, is only ever performed when the solver is at propagation fixpoint.

Theorem 3.4.5. *Let $P \equiv (V, D, C)$ and $P' \equiv (V, D', C)$ be subproblems arising during search from problem $P_{init} \equiv (V, D_{init}, C)$. If $\text{fixed}(D') = \text{fixed}(D)$, and $\forall c \in C, \text{key}(c, D'_F) \Rightarrow \text{key}(c, D_F)$, and $D'_U \Rightarrow D_U$, then P dominates P' .*

Proof.

$$\begin{aligned}
& \pi_U(C \wedge D') \\
& \Leftrightarrow \pi_U(C \wedge D'_F \wedge D'_U) \\
(\star) \quad & \Leftrightarrow \bigwedge_{c \in C} (\pi_U(c \wedge D'_F)) \wedge D'_U \\
& \Leftrightarrow \bigwedge_{c \in C} (\pi_U(c \wedge \text{key}(c, D'_F)) \wedge w(c, D'_F)) \wedge D'_U \\
& \Leftrightarrow \bigwedge_{c \in C} (\pi_U(c \wedge \text{key}(c, D'_F))) \wedge D'_U \\
& \Rightarrow \bigwedge_{c \in C} (\pi_U(c \wedge \text{key}(c, D_F))) \wedge D_U \\
& \Leftrightarrow \bigwedge_{c \in C} (\pi_U(c \wedge \text{key}(c, D_F)) \wedge w(c, D_F)) \wedge D_U \\
& \Leftrightarrow \bigwedge_{c \in C} (\pi_U(c \wedge D_F)) \wedge D_U \\
(\star) \quad & \Leftrightarrow \pi_U(C \wedge D_F \wedge D_U) \\
& \Leftrightarrow \pi_U(C \wedge D)
\end{aligned}$$

The second and eighth (marked) equivalences hold because, again, all variables being projected out in each $c \wedge D'_F$ and $c \wedge D_F$ are already fixed. The fourth and sixth equivalences hold because when we are at propagation fixpoint, $D_U \Rightarrow w(c, D_F)$ for any $c \in C$. $\square$

Definition 3.4.6. *Define the subproblem projection key, $\text{skey}(C, D)$ for subproblem (C, D) as follows: $\text{skey}(C, D) \equiv (F, [\text{key}(c, D_F) \mid c \in C], D_U)$. $\text{skey}(C, D)$ is a tuple of objects, the first of which is a set of variables, and the rest of which are constraints.*

Extend the implication operator $\Rightarrow$ w.r.t. subproblem projection keys as follows: $\text{skey}(C, D) \Rightarrow \text{skey}(C, D')$ iff $F' = F$ and $\forall c \in C, \text{key}(c, D'_F) \Rightarrow \text{key}(c, D_F)$, and $D'_U \Rightarrow D_U$.

Theorem 3.4.5 states that if $\text{skey}(C, D') \Rightarrow \text{skey}(C, D)$, then P dominates P' . Clearly, if $\text{skey}(C, D') \Leftrightarrow \text{skey}(C, D)$, then P and P' are equivalent.

Example 3.4.7. Consider the subproblem $P \equiv (V, D, C)$ where $V \equiv \{x_1, \dots, x_6\}$, $D \equiv \{x_1 = 3, x_2 = 4, x_3 = 5, x_4 \in \{0, 1, 2\}, x_5 \in \{0, 1, 2\}, x_6 \in \{1, 2, 6\}\}$ and $C \equiv \{alldiff([x_1, x_2, x_3, x_4, x_5, x_6]), x_1 + 2x_2 + x_3 + x_4 + 2x_5 \leq 20\}$. Then, $F = \{x_1, x_2, x_3\}$ and $U = \{x_4, x_5, x_6\}$. A projection key for the subproblem is given by: $skey(C, D) \equiv (\{x_1, x_2, x_3\}, [\{x_1, x_2, x_3\} = \{3, 4, 5\}, x_4 + 2x_5 \leq 4], \{x_4 \in \{0, 1, 2\}, x_5 \in \{0, 1, 2\}, x_6 \in \{1, 2, 6\}\})$, if we choose $\pi_U(c \wedge D_F)$ to define the key for the linear constraint $c \equiv x_1 + 2x_2 + x_3 + x_4 + 2x_5 \leq 20$. $\square$

Example 3.4.8. Consider $P \equiv (V, D, C)$ of Example 3.4.7 and another subproblem $P' \equiv (V, D', C)$ where $D' \equiv \{x_1 = 4, x_2 = 5, x_3 = 3, x_4 \in \{0, 1, 2\}, x_5 \in \{0, 1\}, x_6 \in \{1, 2, 6\}\}$. We have that $fixed(D') = fixed(D) = \{x_1, x_2, x_3\}$ and the keys for the *alldiff* are identical. Also, the projection of the linear inequality is $x_4 + 2x_5 \leq 3$. This is stronger than its projection in $key(P)$ as: $x_4 + 2x_5 \leq 3 \Rightarrow x_4 + 2x_5 \leq 4$. Similarly, $D'_U \Rightarrow D_U$. Hence, P dominates P' . $\square$

3.4.2 Special Cases

The more complex definition of $key(c, D_F)$, introduced above, allows us to define $key(c, D_F) \equiv true$ in several very important cases. This is good because the weaker a set of constraints $key(c, D_F)$ is, and the more independent it is of D_F , the better a key it is at detecting dominances. Thus, when applicable, $key(c, D_F) \equiv true$ is essentially the best possible key for dominance detection. $key(c, D_F) \equiv true$ also has the additional benefit that it does not contribute to our key representation (described in the next section), and so we save space as well.

Suppose $F \cap vars(c) = \emptyset$. Then $\pi_U(c \wedge D_F) \equiv \pi_U c$ and therefore, $key(c, D_F) \equiv true$ is trivially correct. This means that for any D_F , if c is inactive (none of its variables are fixed by D_F), we can define $key(c, D_F) \equiv true$.

Suppose $w(c, D_F) \Rightarrow \pi_U(c \wedge D_F)$. Then $key(c, D_F) \equiv true$ is trivially correct. This means that for any D_F , if c 's propagator is strong enough that the propagation from D_F makes c satisfied, we can define $key(c, D_F) \equiv true$. The vast majority of the constraints satisfied by D fall under this category.

Example 3.4.9. Consider the subproblem $P \equiv (V, D, C)$ where $V \equiv \{x_1, \dots, x_4\}$, $D \equiv \{x_1 = 3, x_2 = 4, x_3 \in \{0, 1, 2, 3, 5, 6\}, x_4 \in \{0, 1, 2, 3, 4, 5, 6\}\}$ and $C \equiv \{x_1 \neq x_2, x_2 \neq x_3, x_3 \neq x_4\}$. Then, $F = \{x_1, x_2\}$ and $U = \{x_3, x_4\}$. A correct projection key for this subproblem is given by: $key(C, D) \equiv (\{x_1, x_2\}, [true, true, true], \{x_3 \in \{0, 1, 2, 3, 5, 6\}, x_4 \in \{0, 1, 2, 3, 4, 5, 6\}\})$. The projection key for $x_1 \neq x_2$ is *true* since it is satisfied, similarly the projection key for $x_2 \neq x_3$ is *true* since it is satisfied (even though x_3 is not fixed), and the projection key for $x_3 \neq x_4$ is *true* since none of its variables are fixed. $\square$

3.4.3 Using Projection Keys

In this section, we discuss how we can cache subproblems using projection keys, and how dominance checks can be carried out. Recall that each subproblem $P \equiv (V, D, C)$ is characterized by $key(C, D) \equiv (F, [key(c, D_F) \mid c \in C], D_U)$. We are free to take any low level representation of $key(C, D)$, as long as it is uniquely identified. In particular, we can take advantage of globally known information like the constraint type of each c , their arguments, any constants involved, F , and D_{init} , to dramatically reduce the size of the representation.

We define representations $rep(c, D_F)$ for each $key(c, D_F)$ on a per constraint basis. We require that there be a bijection between tuple $(F, c, rep(c, D_F))$ and the possible values of $key(c, D_F)$. Note that different values of $key(c, D_F)$ can map to the same value of $rep(c, D_F)$, as long as F and c are sufficient to distinguish them. See Section 3.5 for many examples of $key(c, D_F)$ and $rep(c, D_F)$.

We also need to define a representation $rep(D_v)$ for each D_v where $v \in U$. In practice we simply use the existing solver representation of variable domains, e.g., a bit string, a set of ranges, etc. Finally, we need to define a representation $rep(F)$ to represent F , for which we use a bit string.

We can define the representation $rep(C, D)$ of a subproblem in one of two ways:

- The *dense representation* which always includes a representation for each constraint appearing in C , and every unfixed variable U appearing in the problem:

$$rep_d(C, D) \equiv (rep(F), [rep(c, D_F) \mid c \in C], [rep(D_v) \mid v \in U])$$

- the *sparse representation* which omits constraints whose key is *true* and omits unfixed variables whose domain is the same as their initial domain:

$$\begin{aligned} \text{rep}_s(C, D) \equiv & (\text{rep}(F), [(id(c), \text{rep}(c, D_F)) \mid c \in C, \text{key}(c, D_F) \neq \text{true}], \\ & [(id(v), \text{rep}(D(v))) \mid v \in U, D(v) \neq D_{init}(v)]) \end{aligned}$$

where $id(v)$ and $id(c)$ are unique id's for variables and constraints respectively.

Clearly, both representations uniquely identify $(F, [\text{key}(c, D_F) \mid c \in C], D_U)$.

For problems with lots of subproblem equivalences, it is generally the case that $D(v) = D_{init}(v)$ and $\text{key}(c, D_F) \equiv \text{true}$ for a very substantial portion of the variables and constraints. Thus, in the implementation we use $\text{rep}_s(C, D)$ which can take advantage of the sparsity. Note that this means that we only need to define $\text{rep}(c, D_F)$ for D_F where $\text{key}(c, D_F) \neq \text{true}$.

Clearly, if $\text{rep}(C, D) = \text{rep}(C, D')$, then $\text{skey}(C, D) \Leftrightarrow \text{skey}(C, D')$ and P and P' are equivalent. If all we want to do is to detect subproblem equivalence, we can accomplish this very efficiently by using a hash table to check for $\text{rep}(C, D) = \text{rep}(C, D')$, where the projection key is used as both the hash table key and value. We can thus alter the algorithm in Figure 3.1 by replacing line 5 by:

if $((\text{rep}(C, D') \mapsto \text{rep}(C, D')) \in \text{Hashtable})$ return *false*

and replacing line 9 by:

$\text{Hashtable} := \text{Hashtable} \cup (\text{rep}(C, D') \mapsto \text{rep}(C, D'))$

Example 3.4.10. Consider the problem $P \equiv (V, D, C)$ of Example 3.4.7. We can store its projection key $\{x_1, x_2, x_3\} = \{3, 4, 5\} \wedge x_4 + 2x_5 \leq 4 \wedge D_U$ as follows: We store the fixed variables $\{x_1, x_2, x_3\}$ for the subproblem since these must be identical for the equivalence check in any case. We store $\{3, 4, 5\}$ for the *alldiff* constraint, and the fixed value 4 for the linear constraint, which give us enough information given the fixed variables to define the key. The remaining part of the key are domains. Thus, the projection key can be stored as $(\{x_1, x_2, x_3\}, \{3, 4, 5\}, 4, \{0, 1, 2\}, \{0, 1, 2\}, \{1, 2, 6\})$ using the dense representation, and $(\{x_1, x_2, x_3\}, (c_1, \{3, 4, 5\}), (c_2, 4), (x_4, \{0, 1, 2\}), (x_5, \{0, 1, 2\}), (x_6, \{1, 2, 6\}))$ using the sparse representation. $\square$

To exploit subproblem dominances, and not simply equivalences, we need a method to detect when $skey(C, D') \Rightarrow skey(C, D)$. Some constraints naturally produce projections which can dominate one another, while others do not. Consider a linear constraint $x_1 + x_2 + x_3 \leq 5$. If $x_1 = 1$, this projects to $x_2 + x_3 \leq 4$. If $x_1 = 2$, this projects to $x_2 + x_3 \leq 3$, which is clearly a strictly stronger constraint. Thus, we can see that linear constraints tend to produce dominating projections. On the other hand, this is not the case for other kinds of constraints such as *alldiff*. No projection of the *alldiff* constraint ever strictly dominates another. e.g., *alldiff*(1, 2, x_3, x_4, x_5) does not dominate and is not dominated by *alldiff*(1, 2, 3, x_4, x_5). Thus, we can divide up the constraints into two groups: ones which can generate (strictly) dominating projections, and ones which cannot. Note that domain constraints D_v fall in the first group as they can strictly dominate another D'_v .

We can detect subproblem dominances as follows. We first divide the subproblem representation into two parts. The *equivalence part* $rep_e(C, D)$, which consists of $rep(F)$ and $rep(c, D_F)$ for each $c \in C$ that cannot generate dominating projections, and the *dominance part* $rep_d(C, D)$, which consists of $rep(c, D_F)$ for each $c \in C$ that can generate dominating projections, together with $rep(D_U)$.

For dominance detection, we use only the equivalence part $rep_e(c, D_F)$ as the hash table key, and the whole representation $rep(C, D)$ as the value. Now, looking up matches in the hash table returns a set of subproblems with the same $rep_e(C, D)$, but possibly different $rep_d(C, D)$. We then simply compare the dominance parts of each to see if P is dominated by one of these. We alter the algorithm in Figure 3.1 as follows. Line 5 is replaced by:

if $(\exists(rep_e(C, D) \mapsto rep(C, D)) \in HashTable \text{ s.t. } rep_e(C, D) = rep_e(C, D')$
 and $rep_d(C, D)$ dominates $rep_d(C, D'))$ return *false*

and line 9 is replaced by:

$HashTable := HashTable \cup (rep_e(C, D') \mapsto rep(C, D'))$

In the above, we put all the $rep(c, D_F)$ and $rep(D_v)$ that can generate dominating projections into the dominance part. However, we can actually decide on an individual

basis whether to put them in the equivalence or dominance part, as long as we are consistent throughout the search. There is a trade off between speed and dominance detection strength. The more things we include in the equivalence part, the fewer matches we get and the quicker the dominance check, but the less dominance we can detect.

Example 3.4.11. Consider the subproblem $P \equiv (V, D, C)$ from Example 3.4.7. Its projection key is $(\{x_1, x_2, x_3\}, \{3, 4, 5\}, 4, \{0, 1, 2\}, \{0, 1, 2\}, \{1, 2, 6\})$. The key for *alldiff* is in the equivalence part, and we need an exact match to determine dominance. The key for the linear constraint is in the dominance part, and we need to compare the value of the constant term to determine dominance. The keys for the domain constraints are in the dominance part, and we need to check for subset/superset property to determine dominance. For the subproblem P' from Example 3.4.8 we have the projection key $(\{x_1, x_2, x_3\}, \{3, 4, 5\}, 3, \{0, 1, 2\}, \{0, 1\}, \{1, 2, 6\})$. The first two arguments of the tuple are hashed to form the hash table key for P' . It will match the hash table key for P , so we will retrieve the projection key for P . The first two arguments of P and P' 's projection keys match so we are OK. We then compare the third arguments to see that $3 < 4$, so that is also OK. Finally, we check that each of the last three arguments of P' 's key is a subset of the corresponding arguments in P 's key, which is true. Hence, we determine that P dominates P' . $\square$

Note that, for efficiency, our implementation checks $D'_U \Rightarrow D_U$ by using identity ($D'_U \equiv D_U$) so the domains can be part of the hash value. This means that the problem P' of Example 3.4.8 will not be detected as dominated in our implementation, since the domain of x_5 is different.

3.4.4 Branch and Bound Optimization

The presentation so far has concentrated on satisfaction problems. We show that the theory can be extended to branch and bound optimization search. Without loss of generality, let us consider a minimization problem. Branch and bound optimization typically proceeds as follows. We have some objective function f , e.g., $f \equiv \sum_{i=1}^n a_i x_i$ or $f \equiv \max_{i=1}^n x_i$, and one of the constraints in the problem is of the form $c_{obj} \equiv f < b$,

where b is the objective value of the best solution found so far. Suppose that during search, a better solution with $f = b'$ is found, then the search is constrained so that only solutions better than b' can be found, i.e., c_{obj} is strengthened to $c'_{obj} \equiv f < b'$. Search continues until the search space is exhausted, and no better solution exists.

Our previous theorems were proved based on the assumption that C remains the same in P and P' . However, this is not true in branch and bound optimization, as c_{obj} is strengthened to c'_{obj} . We can easily extend Theorem 3.4.5 to the case where all constraints in C monotonically increase in strength as search progresses.

Lemma 3.4.12. *Suppose $c' \Rightarrow c$, $key(c', D'_F) \Rightarrow key(c, D_F)$, and $D'_U \Rightarrow D_U$. Then $\pi_U(c' \wedge D'_F) \wedge D'_U \Rightarrow \pi_U(c \wedge D_F) \wedge D_U$.*

Proof.

$$\begin{aligned}
& \pi_U(c' \wedge D'_F) \wedge D'_U \\
\Leftrightarrow & \pi_U(c' \wedge key(c', D'_F)) \wedge D'_U \\
\Rightarrow & \pi_U(c \wedge key(c, D_F)) \wedge D_U \\
\Leftrightarrow & \pi_U(c \wedge D_F) \wedge D_U
\end{aligned}$$

□

Theorem 3.4.13. *Let $P \equiv (V, D, C)$ and $P' \equiv (V, D', C')$ be subproblems arising during search. Suppose there is a bijective mapping $m : C \rightarrow C'$ s.t. $m(c) \Rightarrow c$, i.e., every constraint $c \in C$ has a corresponding constraint $m(c) \in C'$ which is at least as strong. If $F' = F$, and $\forall c \in C, key(m(c), D'_F) \Rightarrow key(c, D_F)$ and $D'_U \Rightarrow D_U$, then P dominates P' .*

□

The proof is analogous to that of Theorem 3.4.5, except that we apply Lemma 3.4.12 at the appropriate place. Clearly, branch and bound optimization search is covered by Theorem 3.4.13 and we can use projection keys to detect dominance as we did for satisfaction problems.

3.4.5 Redundant constraints

Problems are often modeled with redundant constraints which enhance propagation and reduce the search space. Such redundant constraints can actually be ignored in

the subproblem projection. This is formalized by the following.

Lemma 3.4.14. *Suppose we have $P \equiv (V, D, C)$ and $P' \equiv (V, D, C')$ where $C' \subseteq C$ s.t. $C \Leftrightarrow C'$. Then the projection of P and P' onto U w.r.t. any D is equivalent, i.e., $\pi_U(C \wedge D) \Leftrightarrow \pi_U(C' \wedge D)$.*

Proof. Since $C \Leftrightarrow C'$, $C \wedge D \Leftrightarrow C' \wedge D$, thus $\pi_U(C \wedge D) \Leftrightarrow \pi_U(C' \wedge D)$ for any D . $\square$

Theorem 3.4.15. *Let $P \equiv (V, D, C)$ and $P' \equiv (V, D', C)$ be subproblems arising during search. Suppose $C' \subseteq C$ s.t. $C \Leftrightarrow C'$. If $\text{fixed}(D') = \text{fixed}(D)$, and $\forall c \in C'$, $\text{key}(c, D'_F) \Rightarrow \text{key}(c, D_F)$, and $D'_U \Rightarrow D_U$, then P dominates P' . $\square$*

The proof is analogous to that of Theorem 3.4.5, except that we apply Lemma 3.4.14 at the beginning and end. This means that instead of creating projecting keys from the full set of constraints C , it is sufficient to choose a subset $C' \subseteq C$ which fully define the problem.

3.5 Constructing Projection Keys

In this section we discuss how to create projection keys for many common constraints. Constructing the function $\text{key}(c, D_F)$ is a form of *constraint abduction*, which can be a very complex task. However, we only need to perform constraint abduction on a per constraint basis. Recall that we can always define $\text{key}(c, D_F)$ as $D_{F \cap \text{vars}(c)}$ or $\pi_U(c \wedge D_F)$. However, we want a definition for $\text{key}(c, D_F)$ that detects as much dominance as possible and is representable in a reasonable size. First we try to see if $\text{key}(c, D_F) \equiv \text{true}$ is a valid definition, as this detects the most dominance. Recall that if $F \cap \text{vars}(c) = \emptyset$ or $w(c, D_F) \Rightarrow \pi_U(c \wedge D_F)$, then we can always define $\text{key}(c, D_F) \equiv \text{true}$. We only need to consider more complex definitions of $\text{key}(c, D_F)$ for D_F for which neither of these conditions obviously hold, i.e., we only consider D_F where at least some variables in $\text{vars}(c)$ are fixed, and where D_F is not enough to satisfy c . Next we try $\text{key}(c, D_F) \equiv \pi_U(c \wedge D_F)$, which often detects a fair amount of dominance, but is not always representable. Thirdly, we try to find something weaker but representable. If all else fails, we use $\text{key}(c, D_F) \equiv D_{F \cap \text{vars}(c)}$, which detects no dominance, but is always representable.

Constraint c	Sufficient set of propagation rules
$x = y$	$x = d \leftrightarrow y = d$
$x \neq y$	$x = d \rightarrow y \neq d, y = d \rightarrow x \neq d$
$x \geq y$	$x = d \rightarrow y \leq d, y = d \rightarrow x \geq d$
$x \vee y$	$\neg x \rightarrow y, \neg y \rightarrow x$
$(x = d) \Rightarrow b$	$x = d \rightarrow b, \neg b \rightarrow x \neq d$
$(x = d) \Leftarrow b$	$x \neq d \rightarrow \neg b, b \rightarrow x = d$
$(x \geq d) \Rightarrow b$	$x \geq d \rightarrow b, \neg b \rightarrow x < d$
$(x \geq d) \Leftarrow b$	$x < d \rightarrow \neg b, b \rightarrow x \geq d$
$(x = d') \Rightarrow (y = d)$	$x = d' \rightarrow y = d, y \neq d \rightarrow x \neq d'$
$y = \text{bool2int}(b)$	$b \leftrightarrow y = 1, \neg b \leftrightarrow y = 0$
$y = \text{abs}(x)$	$x = d \rightarrow y = \text{abs}(x), y = d \rightarrow x \in \{d, -d\}$

Figure 3.4: Sufficient conditions on propagation to ensure no key is required for the binary constraints above. Note the $\text{bool2int}()$ function converts a Boolean b to a 0-1 integer y .

3.5.1 Simple Constraints

Here we give the function $\text{key}(c, D_F)$ and the representation $\text{rep}(c, D_F)$ for some simple constraints. Any assumptions about the minimum consistency level of the propagator are stated as necessary.

Binary Constraints Almost all commonly used propagators for binary constraints have the property that once one variable is fixed, propagation on the other variable causes the constraint to become satisfied. For such constraints, either $F \cap \text{vars}(c) = \emptyset$ or $w(c, D_F) \Rightarrow \pi_U(c \wedge D_F)$. Thus, we can define $\text{key}(c, D_F) \equiv \text{true}$ for any D_F which means that such constraints never need to contribute to the projection key. We give a non-exhaustive list of some of these constraints, along with a sufficient set of propagation rules for the above condition to hold in Figure 3.4.

Linear Let c be a linear constraint of the form $c \equiv \sum_{i=1}^n a_i x_i \leq a_0$. Let $S = \{i \mid x_i \in F\}$ and d_i be the value that x_i is fixed to in D_F , for $i \in S$. Let $a'_0 = a_0 - \sum_{i \in S} a_i d_i$. We can define $\text{key}(c, D_F) \equiv \sum_{i \notin S} a_i x_i \leq a'_0$. The variables x_i and the constants a_i are implicitly known from c , and S is implicitly known from F and $\text{vars}(c)$. Thus, the representation only needs to store the single number a'_0 . We can

define $rep(c, D_F) = \{a'_0\}$. The $\geq$ and $=$ case of the linear constraint are analogous.

Reified Linear Let c be a reified linear constraint of the form $c \equiv b \Rightarrow \sum_{i=1}^n a_i x_i \leq a_0$, that is if b is *true* the linear constraint holds and if the linear constraint does not hold then b is *false*. Let $S = \{i \mid x_i \in F\}$, and d_i be the value that x_i is fixed to in D_F , for $i \in S$. Let $a'_0 = a_0 - \sum_{i \in S} a_i d_i$. There are two cases. If $b = \text{true}$, then we can define $key(c, D_F) \equiv \sum_{i \notin S} a_i x_i \leq a'_0$ and $rep(c, D_F) = \{a'_0, \text{true}\}$. If $b = \text{false}$ the constraint is satisfied and $key(c, D_F) = \text{true}$. Otherwise if $b \notin F$, then we can define $key(c, D_F) \equiv b \rightarrow \sum_{i \notin S} a_i x_i \leq a'_0$ and $rep(c, D_F) = \{a'_0, \text{false}\}$. Note that the representation is thus simply the representation of the non-reified linear, plus an additional Boolean value. Other kinds of reified constraints can be handled in a similar manner.

Clause Let c be a clausal constraint of the form $c \equiv \bigvee_{i=1}^n l_i$ where l_i are Boolean literals (either b_i or $\neg b_i$). Let $S = \{i \mid b_i \in F\}$. Suppose $\exists i \in S$ s.t. $l_i = \text{true}$ (that is $l_i \equiv b_i$ and $D(b_i) = \text{true}$ or $l_i \equiv \neg b_i$ and $D(b_i) = \text{false}$). Then $w(c, D_F) \Rightarrow \pi_U(c \wedge D_F)$ and we can define $key(c, D_F) \equiv \text{true}$ and $rep(c, D_F) = \{\text{true}\}$. Suppose $\nexists i \in S$ s.t. $l_i = \text{true}$. Then we can define $key(c, D_F) \equiv \bigvee_{i=1, i \notin S}^n l_i$. Once again, S is implicitly known from F and $vars(c)$, so in fact, we don't need to store any additional information. We can define $rep(c, D_F) = \{\text{true}\}$ in the first case and $rep(c, D_F) = \{\text{false}\}$ in the second.

Arithmetic constraints Most arithmetic constraints like $c \equiv z = x \times y$, $c \equiv z = x \bmod y$, do not project well. Thus, we can only use as the last resort $key(c, D_F) \equiv D_{F \cap var(c)}$ and $rep(c, D_F) = [D_v \mid v \in (F \cap var(c))]$.

3.5.2 Global Constraints

Constraint programming models often make use of global constraints to define important subproblems of the model. On the face of it defining a projection key for a complex global constraint may seem a daunting task. But in many cases it is quite straightforward.

For many global constraints, we can use the following theorem to work out a correct definition of $key(c, D_F)$:

Theorem 3.5.1. *Let $c \Leftrightarrow \bigwedge_{i=1}^n c_i$ be a decomposition of c that introduces no new variables. If $w(c, D_F) \Rightarrow w(c_i, D_F)$ for all i , then we can define $key(c, D_F) \equiv \bigwedge_{i=1}^n key(c_i, D_F)$.*

Proof.

$$\begin{aligned}
& \pi_U(c \wedge key(c, D_F)) \wedge w(c, D_F) \\
\Leftrightarrow & \pi_U(\bigwedge_{i=1}^n (c_i \wedge key(c_i, D_F))) \wedge \bigwedge_{i=1}^n w(c_i, D_F) \wedge w(c, D_F) \\
\Leftrightarrow & \bigwedge_{i=1}^n (\pi_U(c_i \wedge key(c_i, D_F)) \wedge w(c_i, D_F)) \wedge w(c, D_F) \\
\Leftrightarrow & \bigwedge_{i=1}^n \pi_U(c_i \wedge D_F) \wedge w(c, D_F) \\
\Leftrightarrow & \pi_U(\bigwedge_{i=1}^n (c_i \wedge D_F)) \wedge w(c, D_F) \\
\Leftrightarrow & \pi_U(c \wedge D_F)
\end{aligned}$$

The reverse direction of the last equivalence holds because for any correct propagator, we must have $\pi_U(c \wedge D_F) \Rightarrow w(c, D_F)$ if D is a fixpoint of p_c .

Thus, if our global constraint c can be decomposed into simpler constraints for which we already have correctly defined $key(c_i, D_F)$, we can simply take their conjunction as the definition for $key(c, D_F)$. Note that the solver does not need to use the constraint decomposition for propagation. The decomposition is introduced purely to define $key(c, D_F)$. For this reason it is more effective to take a simple, weak decomposition than to take a complicated and strong decomposition with many redundant constraints. Simpler decompositions with fewer constraints produce fewer keys to conjunct, and there is a higher chance that our global propagator has stronger propagation than the decomposition.

Here we give the function $key(c, D_F)$ and the representation $rep(c, D_F)$ for some global constraints. Any assumptions about the minimum consistency level of the propagator are stated as necessary.

Alldiff The *alldiff* constraint is defined by (and decomposes to):

$$c \equiv \text{alldiff}(x_1, \dots, x_n) \Leftrightarrow \bigwedge_{i=1}^n \bigwedge_{j=1, j \neq i}^n x_i \neq x_j.$$

If our propagator enforces at least $x_i = v \rightarrow x_j \neq v$ for all $i \neq j$, then we can apply Theorem 3.5.1 with this decomposition. Since all of the constraints in the decomposition are binary, $\text{key}(c_i, D_F) \equiv \text{true}$ for all of them. So we can define $\text{key}(c, D_F) \equiv \text{true}$ for all D_F for the *alldiff* constraint, which means we don't need to define $\text{rep}(c, D_F)$.

And Let c be a global conjunction constraint of the form $c \equiv y = \bigwedge_{i=1}^n x_i$. We can decompose c into clausal constraints: $c \Leftrightarrow \bigwedge_{i=1}^n (\neg y \vee x_i) \wedge (y \vee \bigvee_{i=1}^n \neg x_i)$. Suppose our *and* propagator is domain consistent. Then we can apply Theorem 3.5.1. So we can define $\text{key}(c, D_F) \equiv \bigwedge_{i=1}^n \text{key}(\neg y \vee x_i, D_F) \wedge \text{key}(y \vee \bigvee_{i=1}^n \neg x_i, D_F)$. Now $\text{key}(c_i, D_F) \equiv \text{true}$ for all the binary clauses in the decomposition, and so this simplifies to $\text{key}(c, D_F) \equiv \text{key}(y \vee \bigvee_{i=1}^n \neg x_i, D_F)$. Thus, the key and representation is exactly the same as that for a clause, which we have already covered in the previous subsection.

Element Let c be an element constraint of the form $c \equiv \text{element}(x, [a_1, \dots, a_n], y)$ where a_i are constants. *element* is defined by (and decomposes to):

$\text{element}(x, [a_1, \dots, a_n], y) \Leftrightarrow \bigwedge_{i=1}^n (x = a_i \rightarrow y = a_i)$. Suppose our propagator enforces at least $x = a_i \rightarrow y = a_i$ and $y \neq a_i \rightarrow x \neq a_i$. Then, we can apply Theorem 3.5.1 with this decomposition. Since all of the constraints in the decomposition are binary, $\text{key}(c_i, D_F) \equiv \text{true}$ for all of them. Hence just as for *alldiff*, we don't need to define $\text{rep}(c, D_F)$.

Inverse The *inverse* constraint is defined by (and decomposes to):

$$c \equiv \text{inverse}(x_1, \dots, x_n, y_1, \dots, y_n) \Leftrightarrow \bigwedge_{i=1}^n \bigwedge_{j=1}^n x_i = j \leftrightarrow y_j = i.$$

If our propagator enforces at least $x_i = j \rightarrow x_j = i$ and $x_i \neq j \rightarrow x_j \neq i$, then we can apply Theorem 3.5.1 with this decomposition. Since all of the constraints in the decomposition are binary, again we don't need to define $rep(c, D_F)$.

Minimum Let c be a minimum constraint of the form $c \equiv y = \min_{i=1}^n x_i$. Let $S = \{i \mid x_i \in F\}$ and let d_i be the value that x_i is fixed to in D_F for $i \in S$. Let $d = \min_{i \in S} d_i$. Assume we have a special constant ∞ , whose value cannot be taken by any integer variable in the system. There are three cases. If $y \notin F$ and $S \neq \emptyset$, then we can define $key(c, D_F) \equiv y = \min(d, \min_{i \notin S} x_i)$ and $rep(c, D_F) = (\infty, d)$. If $y \in F$ and $S \neq \emptyset$, then we can define $key(c, D_F) \equiv e = \min(d, \min_{i \notin S} x_i)$ and $rep(c, D_F) = (e, d)$ where e is the fixed value of y ($y = e \in D_F$). Finally, if $y \in F$ and $S = \emptyset$, then we can define $key(c, D_F) \equiv e = \min_{i=1}^n x_i$ and $rep(c, D_F) = (e, \infty)$ where e is defined as before. The *maximum* constraint is analogous to *minimum*.

Global Cardinality Constraint The global cardinality constraint *gcc* [128] requires that the number of variables in $\{x_1, \dots, x_n\}$ which take the value j be n_j . We consider the case where n_j are constants. It is defined by:

$$c \equiv gcc([x_1, \dots, x_n], [n_1, \dots, n_k]) \Leftrightarrow \bigwedge_{i=1}^k \left(\sum_{j=1}^n bool2int(x_j = i) = n_i \right).$$

Let $S = \{i \mid x_i \in F\}$. Let $n'_j = n_j - \sum_{i \in S} bool2int(x_i = j)$ for $j = 1, \dots, k$. Then we can define $key(c, D_F) \equiv \bigwedge_{j=1}^k (\sum_{i \notin S} bool2int(x_i = j) = n'_j)$. S is known implicitly from $vars(c)$ and F , so we can simply define $rep(c, D_F) = \{n'_1, \dots, n'_k\}$. The *global_cardinality_low_up* constraint which requires the number of values taking value j to be between lower bound l_j and upper bound u_j , defined by

$$gcclu([x_1, \dots, x_n], [l_1, \dots, l_k], [u_1, \dots, u_k]) \Leftrightarrow \bigwedge_{i=1}^k (l_i \leq \sum_{j=1}^n bool2int(x_j = i) \leq u_i),$$

is analogous.

Cumulative Constraint The *cumulative* constraint [152] requires that n tasks with start time s_i , duration d_i and resource utilization r_i are scheduled such that at no time more than b resources are used. It can be defined as:

$$\begin{aligned} c &\equiv \text{cumulative}([s_1, \dots, s_n], [d_1, \dots, d_n], [r_1, \dots, r_n], b) \\ &\equiv \bigwedge_t \sum_{i=1}^n \text{bool2int}((s_i \leq t) \wedge (s_i > t - d_i)) \times r_i \leq b, \end{aligned}$$

where t ranges from $\min_{i=1}^n \min(D_{\text{init}}(s_i))$ to $\max_{i=1}^n \max(D_{\text{init}}(s_i) + d_i)$ which are all the time periods when tasks can be scheduled. We consider only the version where s_i are variables and r_i, d_i and b are constants. We present a handcrafted key $\text{key}(c, D_F)$ which is intuitively obvious, but which is not the most powerful possible. Ideally, we would like to define $\text{key}(c, D) \equiv \pi_U(c \wedge D_F)$. However, $\pi_U(c \wedge D_F)$ cannot really be represented in any simple way.

Intuitively, if we fix the start times of the exact same set of tasks, in such a way that the time profile of the resources they consumed is exactly identical, then, the constraint on the remaining tasks must be the same. We define the time profile of consumed resources as follows: Let $S = [i \mid s_i \in F]$. Let $p(t) = \sum_{i \in S} \text{bool2int}(D_F(s_i) \leq t \wedge D_F(s_i) > t - d_i) \times r_i$ be the amount of resources consumed by the fixed tasks at time t . $p(t)$ is a set of constants determined by D_F . Now we only have to examine the profile for times that unfixed tasks can still actually make use of. Let $t_{\min}$ be the minimum start time of an unfixed tasks in $w(c, D_F)$, and $t_{\max}$ be the maximum end time of an unfixed task in $w(c, D_F)$. Now, we define $\text{key}(c, D_F) \equiv \bigwedge_{t \in t_{\min}..t_{\max}} p(t) = \sum_{i \in S} \text{bool2int}(s_i \leq t \wedge s_i > t - d_i) \times r_i$, which is a constraint over $F \cap \text{vars}(c)$. Clearly, $\pi_U(c \wedge \text{key}(c, D_F)) \wedge w(c, D_F) \Leftrightarrow \pi_U(c \wedge D_F)$. Now, any two D_F and D'_F which has the same set of fixed tasks and produce the same $t_{\min}$ and $t_{\max}$ and $p(t)$ over this range of times will have matching keys.

Example 3.5.2. Consider a cumulative constraint with fixed tasks A, B, C, D, E, F, G, H illustrated in Figure 3.5(a) with task durations $d = [4, 5, 2, 1, 3, 2, 2, 1]$ and resource requirements $r = [1, 3, 2, 2, 1, 1, 1, 1]$ and resource bound $b = 4$. The start times illustrated in Figure 3.5(a) are $s = [0, 0, 5, 5, 6, 6, 8, 7]$. Assume that no other unfixed

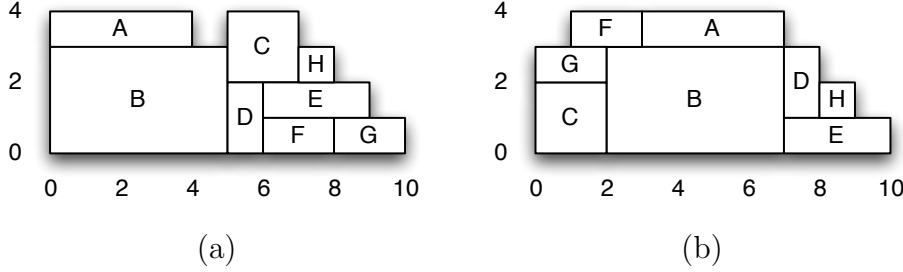


Figure 3.5: Different fixed sub-schedules for *cumulative* leading to the same key.

task (not shown) can start before time 7. Then the key is represented $p(t)$ for $t = 7$ onwards, that is $p(7) = 3, p(8) = 2, p(9) = 1, p(t) = 0, t > 9$. Now consider the cumulative constraint with the same fixed tasks as illustrated in Figure 3.5(b), where again no other task can start before time 7. The key is identical even though the fixed schedules and even profiles of the two subproblems are different. $\square$

3.5.3 More Global Constraints

For some global constraints, the best projection keys can only be expressed as constraints over the *internal variables*, I , of the global propagator. We cannot rigorously define $key(c, D_F)$ as constraints over such internal variables I unless they are explicitly defined. Such internal variables are often defined by the decompositions of the global constraint.

Suppose $c \Leftrightarrow \pi_{vars(c)}(\bigwedge_{i=1}^n c_i)$ is a decomposition that introduces new variables I . We extend $P \equiv (V, D, C)$ to $P' \equiv (V, D', C')$ as follows: Define $D' = D \cup \{v \in D_{init}(v) \mid v \in I\}$. Define c' to be a constraint over $vars(c) \cup I$ s.t. $c' \Leftrightarrow \bigwedge_{i=1}^n c_i$. Define $C' = C \setminus c \cup c'$. Then, P and P' are related as follows: $C \wedge D \Leftrightarrow \pi_{var(C)}(C' \wedge D')$. Clearly, P is satisfiable iff P' is, and there is a surjective mapping from the solutions of P' to P . Thus we can work with the problem P' instead.

Let $F' = F \cup fixed(I)$ and $U' = vars(C') \setminus F'$. Then $D'_{U'} = D_U \cup D'_{U \cap I}$. Now we can apply Theorem 3.5.1 to P' . This tells us that if $w(c', D'_{F'}) \Rightarrow w(c_i, D'_{F'})$ for all i , then we can define $key(c', D'_{F'}) \equiv \bigwedge_{i=1}^n key(c_i, D'_{F'})$. Thus the projection key for P' , as compared to P , changes in the following ways: F becomes $F \cup fixed(I)$, D_U

becomes $D_U \cup D'_{U \cap I}$, and $key(c, D_F)$ which we did not know how to define before, becomes $key(c', D'_{F'}) \equiv \bigwedge_{i=1}^n key(c_i, D'_{F'})$.

Although we had to explicitly define c' and I in order to apply Theorem 3.5.1, c' and I only have to be logically defined. They do not have to be implemented as real constraints or real variables in the system. For example, suppose the global propagator for c has some propagator-specific internal data structures, such that the domains for the variables in I are uniquely determined by them. Then I exist logically in the propagator and the propagator is already implementing a constraint over $vars(c) \cup I$. Also, all extra parts of the key required by the decomposition, i.e., $fixed(I)$, $D'_{U \cap I}$ and $key(c', D'_{F'}) \equiv \bigwedge_{i=1}^n key(c_i, D'_{F'})$, depend only on c_i and I . So if I is kept internally in the propagator, the propagator can produce all of the extra key parts we need. Thus, we can completely encapsulate the decomposition in the propagator without the rest of the system ever needing to know anything about I , or that we extended from P to P' . To express this, we can simply define $key(c, D_F) \equiv \bigwedge_{i=1}^n key(c_i, D'_{F'}) \wedge D'_{U \cap I}$ and $rep(c, D_F) = fixed(I) \uparrow rep(D'_{U \cap I}) + [(id(c_i), rep(c_i, D'_{F'})) \mid key(c_i, D'_{F'}) \neq true]$, and use the normal key construction algorithm as applied to P .

Table The *table* constraint is defined by (and decomposes to):

$$table([x_1, \dots, x_n], [a_{i,j} \mid i = 1..k, j = 1..n]) \Leftrightarrow \pi_{vars(c)}(\bigwedge_{i=1}^k \bigwedge_{j=1}^n r_i \rightarrow x_j = a_{i,j} \wedge \bigvee_{i=1}^k r_i),$$

where r_i are introduced Boolean variables. Suppose our propagator internally keeps track of the domains of the r_i (which tuples are still possible) and enforces at least $r_i \rightarrow x_j = a_{i,j}$, $x_j \neq a_{i,j} \rightarrow \neg r_i$. Then we can extend the problem to include the r_i and apply Theorem 3.5.1. All of the constraints in the decomposition are binary and has $key(c_i, D_F) \equiv true$ except for the clause $\bigvee_{i=1}^k r_i$. Thus we can define $key(c, D_F) \equiv key(\bigvee_{i=1}^k r_i, D_F) \wedge D_{U \cap I}$ and $rep(c, D_F) = fixed(I) \uparrow rep(D_{U \cap I}) \uparrow \{rep(\bigvee_{i=1}^k r_i, D_F)\}$. Now, since the variables in I are all Boolean, $rep(D_{U \cap I})$ is empty. We represent the projection for the clause with just a single Boolean value indicating whether it is satisfied by D_F or not. So $rep(c, D_F) = fixed(I) \uparrow \{true/false\}$.

Cumulative Now we define a projection key for the *cumulative* constraint via decomposition. The *cumulative* constraint can be decomposed in the following way:

$$\begin{aligned}
& \text{cumulative}([s_1, \dots, s_n], [d_1, \dots, d_n], [r_1, \dots, r_n], b) \\
& \Leftrightarrow \bigwedge_{t=1}^n ((so_{i,t} \leftrightarrow s_i \leq t) \wedge (eo_{i,t} \leftrightarrow s_i > t - d_i) \wedge (o_{i,t} \leftrightarrow so_{i,t} \wedge eo_{i,t}) \wedge \\
& \quad \sum_{i=1}^n \text{bool2int}(o_{i,t}) \times r_i \leq b,
\end{aligned}$$

where t is appropriately limited by the ranges of s_i and d_i .

Suppose our propagator internally keeps track of the domains of the introduced variables $I = \{so_{i,t}\} \cup \{eo_{i,t}\} \cup \{o_{i,t}\}$ and enforces the constraints in the decomposition on them. Then we can extend the problem to include I and apply Theorem 3.5.1. All the reified constraints are binary and have $\text{key}(c, D_F) \equiv \text{true}$. However, the Boolean constraints and the linear constraints may be active. We know how to define keys and representations for those from Section 3.5.1. Thus we simply conjunct them and $D_{U \cap I}$ together to define $\text{key}(c, D_F)$ for the cumulative constraint. $\text{rep}(c, D_F)$ consists of the concatenation of $F \cap I$ along with the representations of the active constraints in the decomposition, and the representation of $D_{U \cap I}$.

Now, let us consider what this projection key means semantically. The projection key of linear constraints is $\text{key}(c, D_F) \equiv \sum_{i \in S} a_i x_i \leq a'_0$. Since we have one linear constraint for each time t , these keys tell us exactly how much resource is available at each time t for the set of tasks which can still fit there. This is very similar to the resource profile used in the handcrafted key from Section 3.5.2. However, there is one major difference. It is possible for the linear constraint for some time t to become satisfied through propagation, in which case $\text{key}(c, D_F) \equiv \text{true}$ and that part of the profile no longer matters. In the handcrafted key, we manually took into account such times if they were at the beginning or at the end of the schedule. Here, the projection key does it for all such time t .

Example 3.5.3. Consider the *cumulative* example from Example 3.5.2 once more but this time where B, E, F, G, H are fixed as shown in Figure 3.6(a). The earliest start time for remaining tasks A, C, D is 4 (for task A), so clearly the previously defined key

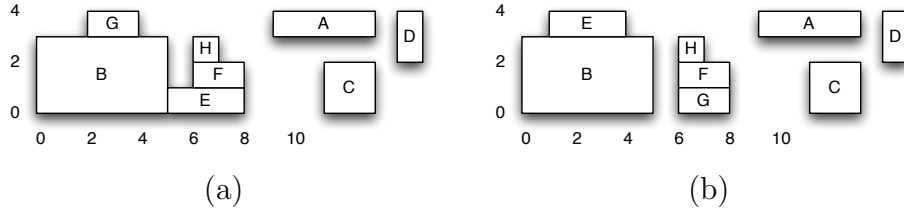


Figure 3.6: Different fixed sub-schedules for *cumulative* leading to the same key when using decomposition based keys.

for this partial schedule is different from that of the state shown in Figure 3.6(b), since the profiles at time 5 are different. In the new key however, in both cases the start times for A , C and D are exactly $4..$, $7..$ and $\{5\} \cup 7..$. The linear constraint at $t = 4$ is satisfied since only $o_{A,4}$ is unfixed, and task A fits in the remaining 1 unit. The linear constraint at $t = 5$ is satisfied since only $o_{A,5}$ and $o_{D,5}$ are unfixed, and both A and D can simultaneously fit in the remaining 3 or 4 units in Figure 3.6(a) and Figure 3.6(b) respectively. The keys for the linear constraint for $t \geq 7$ are clearly equivalent. Hence the decomposition based key will detect these situations as equivalent with respect to *cumulative*. $\square$

As can be seen, using decomposition allows us to define projection keys for even complicated constraints in a straightforward manner. Further, such keys are often fairly simple and powerful. In fact, they are often better at detecting dominance than the intuitively obvious keys that a human being may design.

3.6 Examples of Automatic Caching

In this section, we apply our algorithm to several problems to illustrate what the projection keys look like and what kinds of subproblem dominances they can detect. We also compare our automatically generated keys with handcrafted keys that a human may design. In most cases, our automatically generated projection keys contain significant amounts of redundant information. Thus, a handcrafted key will almost always be smaller in size. However, there are several advantages to automatically generating projection keys. Firstly, subproblem dominances are often very hard to

identify and describe. Secondly, altering a solver to exploit subproblems dominances for a particular problem via handcrafted keys can require a tremendous amount of work. Thirdly, if the problem is ever altered, or side constraints added, etc, the handcrafted key will have to be changed, its correctness will have to be re proven, and the system will have to be altered. All of this is very tedious and error prone. On the other hand, a system which implements caching via projection keys can handle all this automatically. We now go through the three example problems presented previously.

3.6.1 Black Hole

The Black Hole Problem can be modeled as follows:

```
array[1..17, 1..3] of int: layout;      % layout:  pile and layer
array[1..416, 1..2] of int: neighbours; % next card is +/- 1

array[1..52] of var 1..52: x;          % Position of card
array[1..52] of var 1..52: y :: is_output; % Card at position

constraint y[1] == 1;      % Ace of spades starts in blackhole
constraint inverse(x,y);
constraint forall (i in 1..17, j in 1..2) (
    x[layout[i,j]] < x[layout[i,j+1]]);
constraint forall (i in 1..51) (
    table([y[i], y[i+1]], neighbours));

solve satisfy;
```

We take card value v to mean the $(v-1)\%13+1$ th number of the $(v-3)/13+1$ th suit (spades, hearts, clubs, diamonds), e.g., 1 is the ace of spades, 14 is the ace of hearts, 27 is the ace of clubs, 40 is the ace of diamonds. The array *layout* gives the layout of the cards in the 17 piles. The array *neighbours* lists pairs of cards which are ± 1 w.r.t to each other, and is used in the *table* constraints to enforce the ± 1 condition.

For simplicity of explanation we assume that the *table* constraint's propagator is only checking, i.e., when all its variables are fixed, it checks whether that set of values is in its table and fails if it is not.

Consider two subproblems $P \equiv (V, D, C)$ where $D_F \equiv x_1 = 1, x_2 = 41, x_3 = 42, x_4 = 28, x_5 = 16, x_6 = 4, x_7 = 29, x_8 = 2$ and $P' \equiv (V, D', C)$ where $D_F \equiv x_1 = 1, x_2 = 28, x_3 = 42, x_4 = 41, x_5 = 16, x_6 = 4, x_7 = 29, x_8 = 2$, illustrated in Figure 3.2. Assume that both partial assignments satisfy the constraints. According to our manual analysis, since the last cards are the same and the set of unplayed cards are the same, P and P' should lead to equivalent subproblems.

Let S denote the set of played cards, i.e., $\{1, 2, 4, 16, 28, 29, 41, 42\}$, which is the same for both P and P' . In P , the *inverse* constraint will enforce $x_i \neq v$ for $i \geq 9, v \in S$, and $y_i \geq 9$ for $i \notin S$. The *table* constraints perform no propagations. The inequality constraints perform some propagations. For example, if card i is under card j and neither have been played, then since $x_i > x_j, x_j \geq 9 \rightarrow x_i \geq 10$. Now, the *inverse* constraint and the binary inequalities all have $key(c, D_F) \equiv true$. A *table* constraint which is only checking, has $key(c, D_F) \equiv true$ if it is satisfied or inactive, and has $key(c, D_F) \equiv D_{F \cap vars(c)}$ otherwise. The *table* constraints involving only x_i where $i \leq 8$ are all satisfied and has $key(c, D_F) \equiv true$. The ones involving only x_i where $i \geq 9$ are all inactive and has $key(c, D_F) \equiv true$. Thus the only one with a non-trivial key is the one involving x_8 and x_9 , which has $key(c, D_F) \equiv x_8 = 2$. Thus the projection key characterizes the subproblem by F, D_U , and the last card played $x_8 = 2$. Since F and D_U are uniquely determined by the set of unplayed cards, our projection key detects the same amount of equivalence as the handcrafted key. However, it is much larger in size.

3.6.2 Graph Coloring

In Section 3.3, we gave a handcrafted key for the graph coloring problem. It turns out that this key, which is intuitively obvious, is not the strongest key possible for detecting subproblem equivalences. Projection gives a better key. Consider the simple graph coloring problem shown in Figure 3.3.

Consider $P \equiv (V, D, C)$ where $D_F \equiv x_1 = 1, x_2 = 1, x_3 = 3, x_4 = 4, x_5 = 3$ illustrated in Figure 3.3(a). After propagation, we have $F = \{x_1, x_2, x_3, x_4, x_5\}$ and $D_U \equiv \{x_6 \in \{1, 2\}, x_7 \in \{1, 2\}, x_8 \in \{1, 2, 3, 4\}\}$. Since all constraints are binary inequalities, we have $\text{key}(c, D_F) \equiv \text{true}$. Thus, the projection key completely characterizes the subproblem by F and D_U . Consider $P' \equiv (V, D', C)$ where $D'_F \equiv \{x_1 = 2, x_2 = 4, x_3 = 4, x_4 = 3, x_5 = 3\}$ illustrated in Figure 3.3(c). After propagation, we have $F' = \{x_1, x_2, x_3, x_4, x_5\}$ and $D'_{U'} \equiv \{x_6 \in \{1, 2\}, x_7 \in \{1, 2\}, x_8 \in \{1, 2, 3, 4\}\}$. Clearly, $F = F'$ and $D_U = D'_{U'}$ and our projection key shows that these two subproblems are equivalent. However, our handcrafted key would have characterized P with $(\{x_1, x_2, x_3, x_4, x_5\}, [(x_3, 3), (x_4, 4), (x_5, 3)])$ and P' with $(\{x_1, x_2, x_3, x_4, x_5\}, [(x_3, 4), (x_4, 3), (x_5, 3)])$, which are clearly different. Thus, on this problem, the projection key is stronger than the intuitively obvious handcrafted key. The size of the keys are comparable as well.

3.6.3 Minimization of Open Stacks Problem

The model for this problem has been presented in Section 3.1. The linear constraints $\sum_{i=1}^n \text{bool2int}((s[i] \leq t) \wedge (e[i] \geq t)) \leq \text{objective}$ are flattened into primitive constraints before being sent to the solver, since solvers typically only handle primitive constraints natively. Explicitly, those linear inequalities would be replaced by:

```

array[1..n,1..n] of var bool: so;
array[1..n,1..n] of var bool: eo;
array[1..n,1..n] of var bool: o;

constraint forall (t in 1..n) (
  forall (i in 1..n) (
    so[i,t] <-> (s[i] <= t) /\
    eo[i,t] <-> (e[i] >= t) /\
    o[i,t] <-> (so[i,t] /\ eo[i,t])
  ) /\
  sum (i in 1..n) (bool2int(o[i,t]) <= objective)

```

);

Consider the problem data considered in Example 3.3.3. Suppose we are currently solving the problem with *objective* ≤ 3 . Let us consider the following two subproblems $P \equiv (C, D)$ where $D_F \equiv x[1] = 4, x[2] = 3, x[3] = 5$, and $P' \equiv (V, D', C)$ where $D'_F \equiv x[1] = 5, x[2] = 4, x[3] = 3$. We know from manual analysis that these two should produce equivalent subproblems. Let us look at the automatically generated projection keys.

In P , the *inverse* constraint gives $e[4] = 1, e[3] = 2, e[5] = 3$. The *minimum* constraints give $s[1] = 1, s[2] \in \{4, 5, 6\}, s[3] = 1, s[4] = 1, s[5] = 2, s[6] \in \{4, 5, 6\}$. The binary reification constraints and the clausal constraints will now fix many of the *so*, *eo* and *o*. Everything with $t \leq 3$ becomes fixed. Everything related to customers 3, 4, 5 become fixed. All the $so[i, t], t \geq 4$ where customer i 's stack is currently open (i.e., customer 1) become fixed. All the $eo[i, t], t = 4$ becomes fixed. All the $o[i, 4]$ where customer i 's stack is currently open become fixed. All the rest are unfixed. In D_U , $s[2] \in \{4, 5, 6\}, s[6] \in \{4, 5, 6\}, e[1], e[2], e[6] \in \{4, 5, 6\}, x[4], x[5], x[6] \in \{4, 5, 6\}$, and all other unfixed variables are at their initial domains.

Now let's consider the constraint projections. The *inverse*(e, x) constraint and the binary reification constraints $so[i, t] \leftrightarrow (s[i] \leq t)$ and $eo[i, t] \leftrightarrow (e[i] \geq t)$ all have $key(c, D_F) \equiv true$. Of the *minimum* constraints, the ones regarding $s[i], i = 1, 3, 4, 5$ are all satisfied and have $key(c, D_F) \equiv true$. The remaining two have no fixed variables and also have $key(c, D_F) \equiv true$. The linear constraints for $t = 1, 2, 3$ are all satisfied and have $key(c, D_F) \equiv true$. The linear constraint for $t = 4$, projects to $o[2, 4] + o[6, 4] \leq 2$, which is satisfied, so $key(c, D_F) \equiv true$. The linear constraints for $t = 5, 6$ project to $o[1, t] + o[2, t] + o[6, t] \leq 3$ which is also satisfied, and so $key(c, D_F) \equiv true$. Of the clausal constraints $o[i, t] \leftrightarrow (so[i, t] \wedge eo[i, t])$, all the ones with $t = 1, 2, 3$ are satisfied and have $key(c, D_F) \equiv true$. All the ones with $i = 3, 4, 5$ are satisfied and have $key(c, D_F) \equiv true$. The ones with $o[i, t], i = 2, 6, t = 4, 5, 6$ have no fixed variables and have $key(c, D_F) \equiv true$. The one with $o[1, 4]$ is satisfied and has $key(c, D_F) \equiv true$. The ones with $o[i, t], i = 1, t = 5, 6$ are active, and have $key(c, D_F) \equiv \neg eo[i, t] \vee o[i, t]$. As can be seen, almost all the constraints have $key(c, D_F) \equiv true$. This is typical of problems with lots of subproblem equivalences.

Many constraints will either be inactive, or will propagate in such a way that they are no longer relevant for the remaining subproblem.

Now, let us consider P' . Propagation on the *inverse* constraint gives $e[5] = 1, e[4] = 2, e[3] = 3$. The *minimum* constraints give $s[1] = 2, s[2] \in \{4, 5, 6\}, s[3] = 1, s[4] = 2, s[5] = 1, s[6] \in \{4, 5, 6\}$. Once again, the binary reification constraints and the clausal constraints will now fix many of the *so*, *eo* and *o*. We can see, from the previous description, that F , D_U and $key(c, D_F)$ only depends on: the set of closed customers, and the set of currently open customers. These things are identical for P and P' , thus the projection keys of these two subproblems match. The key point here is that although $e[3], e[4], e[5], s[1], s[3], s[4], s[5]$ take on different values in P and P' , their effect on the variables with $t \geq 4$ are identical. And since the remaining non-satisfied constraints in each subproblem only involve variables with $t \geq 4$, the subproblems end up being equivalent.

In general, if the set of closed customers is S , then the set of currently open customers is $[i \mid i \notin S, \exists j \in S. W(i, j)]$, which clearly only depend on S . Thus any two subproblems with the same set of closed customers will have the same projection key. This means that our automatically generated projection key detects exactly the same amount of subproblem equivalence as the handcrafted key presented in Section 3.3. However, the projection key is clearly much larger. On the other hand, this subproblem equivalence is fairly hard to identify manually. The MOSP was the subject of the 2005 Modeling Challenge [146], and of the 13 entrants into the competition, only 3 identified this subproblem equivalence. Thus such subproblem equivalences are hard to identify even for expert problem modelers.

3.7 Related Work

Problem specific approaches to dominance detection/subproblem equivalence are widespread in combinatorial optimization (see e.g., [51, 145]). There is also a significant body of work on caching that rely on problem decomposition by fixing variables (e.g [86, 100]). This work effectively looks for equivalent projected problems, but since they do not take into account the semantics of the constraints, they effectively

use $D_{F \cap \text{vars}(c)}$ for every constraint c as the projection key, which finds strictly fewer equivalent subproblems than our approach to keys. The success of these approaches in finding equivalent subproblems relies on decomposing the projected subproblem into disjoint parts. We could extend our approach to also split the projected problem into connected components but this typically does not occur in the problems of interest to us. Interestingly, [86] uses symmetry detection to detect symmetric (rather than equivalent) subproblems, but the method used does not appear to scale.

3.7.1 Dynamic Programming

Dynamic programming (DP) [11] is a powerful approach for solving optimization problems whose optimal solutions are derivable from the optimal solutions of its subproblems. It relies on formulating an optimization as recursive equations relating the answers to optimization problems of the same form. When applicable, it is often near unbeatable by other optimization approaches.

Constraint programming (CP) with caching is similar to DP, but provides several additional capabilities. For example, arbitrary side constraints not easily expressible as recursions in DP can easily be expressed in CP, and dominance can be expressed and exploited much more naturally in CP.

Consider the 0-1 Knapsack problem, a well known NP-hard problem that is easy to formulate using recursive equations suitable for DP. The DP formulation defines $knp(j, w)$ as the maximum profit achievable using the first j items with a knapsack of size w . The recursive equation is:

$$knp(j, w) = \begin{cases} 0 & j = 0 \vee w \leq 0 \\ \max(knp(j-1, w), knp(j-1, w - w_j) + p_j) & \text{otherwise} \end{cases}$$

The DP solution is $O(nW)$ since values for $knp(j, w)$ are cached and only computed once.

The 0-1 knapsack problem is terrible in its natural CP formulation, it requires $O(2^n)$ search to prove the optimal solution. CP with caching will store as the key: the set of fixed variables F , an integer w representing the remaining weight limit,

and an integer p the remaining profit required. We can see that with a fixed search ordering, F can only take on n values, the representation for the projection key of the weight constraint can only take on W values, and the representation for the projection key of the profit constraint can only take on U values where U is the optimal profit. Thus the maximum number of distinct entries in the hash table, and hence the worse case complexity of the algorithm, is $O(nWU)$, which is much better than $O(2^n)$, but not quite as good as the DP solution.

The solutions are in fact quite different: the DP approach stores the optimal profit for each set of unfixed variables and remaining weight limit, while the CP approach stores the fixed variables and remaining weight plus remaining profit required. The CP approach in fact implements a form of DP with bounding [117]. In particular, the CP approach can detect subproblem dominance. A problem with remaining weight w' and remaining profit required p' is dominated by a problem with remaining weight $w \geq w'$ and remaining profit $p \leq p'$. The DP solution must examine both subproblems since the remaining weights are different.

In practice the number of remaining profits arising for the same set of fixed variables and remaining weight is $O(1)$ and hence the practical number of subproblems visited by the CP approach is $O(nW)$.

Note that while adding a side constraint like $x_3 \geq x_8$ destroys the DP approach (or at least forces it to be carefully reformulated), the CP approach with automatic caching works seamlessly.

3.7.2 Symmetry Breaking

Symmetry breaking aims to speed up execution by not exploring search nodes known to be symmetric to nodes already explored. Once the search is finished, all solutions can be obtained by applying each symmetry to each solution. In particular, Symmetry Breaking by Dominance Detection (SBDD) [43] works by performing a “dominance check” at each search node and, if the node is found to be dominated, not exploring the node.

At first glance, SBDD may appear to be very similar to automatic caching. However, the two techniques actually exploit different kinds of dominances. SBDD detects and exploits *symmetric* subproblems, whereas automatic caching exploits *equivalent* and *dominated* subproblems. In our terminology, we could describe SBDD as follows. Let σ be a mapping from valuations of P to valuations of P . σ is a symmetry if: $\sigma(\theta) \in \text{solns}(P)$ iff $\theta \in \text{solns}(P)$. We can extend σ to a mapping from constraints to constraints as follows: $\theta \in \text{solns}(\sigma(c))$ iff $\sigma(\theta) \in \text{solns}(c)$. We also extend σ to map variables to variables

Definition 3.7.1. Let $P \equiv (V, D, C)$ and $P' \equiv (V, D', C)$ be two different subproblems arising during search. Let $F = \text{fixed}(D)$, $U = \text{vars}(C) \setminus F$, $F' = \text{fixed}(D')$, $U' = \text{vars}(C) \setminus F'$. P symmetrically dominates P' w.r.t. symmetry σ iff

- $\text{fixed}(D) = \text{fixed}(\sigma(D'))$
- $\sigma(\pi_{U'}(C \wedge D')) \Rightarrow \pi_U(C \wedge D)$

It follows from the definition of entailment and symmetry that if P symmetrically dominates P' and P has no solutions, then P' has no solutions either. SBDD works by detecting $\sigma(D'_{F'}) = D_F$ where D_F is a previous partial assignment that led to failure. Since $\sigma(D'_{F'}) = D_F$ implies $\sigma(\pi_{U'}(C \wedge D')) \Rightarrow \pi_U(C \wedge D)$, SBDD can correctly prune P' . Automatic caching on the other hand, looks for subproblem dominance of the form $\pi_U(C \wedge D') \Rightarrow \pi_U(C \wedge D)$, where $D'_F \neq D_F$, which is a different set of dominances.

Note that it is perfectly feasible, and correct, to use both automatic caching and SBDD simultaneously. At a particular node, automatic caching will prune the node if it detects that the subproblem is dominated by a previously failed subproblem, while SBDD will prune the node if it detects that it is dominated by a previously failed symmetric subproblem. We show with the following example that neither method subsumes the other.

Example 3.7.2. Consider $P \equiv (V, D, C)$ where $V \equiv \{x_1, \dots, x_5\}$, $D \equiv \{x_1 \in \{1..3\}, x_2 \in \{1..4\}, x_3 \in \{2..5\}, x_4 \in \{1..5\}, x_5 \in \{1..5\}\}$ and $C \equiv \{\text{alldiff}([x_1, x_2, x_3, x_4, x_5]), x_1 + 4 \leq x_4 + x_5\}$. There is a variable symmetry between x_4 and x_5 .

Consider the subproblems P and P' where $D_F \equiv \{x_1 = 1, x_2 = 2\}$, and $D'_{F'} \equiv \{x_1 = 2, x_2 = 1\}$. In both cases, the *alldiff* propagates to give $x_3 \in \{3, 4, 5\}, x_4 \in \{3, 4, 5\}, x_5 \in \{3, 4, 5\}$. The linear constraint becomes satisfied in both and has $\text{key}(c, D_F) \equiv \text{true}$. Thus their projection keys match and automatic caching detects that P and P' are equivalent. On the other hand, there is no symmetry which maps $x_1 = 1, x_2 = 2$ to $x_1 = 2, x_2 = 1$, so SBDD can do nothing here.

Consider the subproblems P and P' where $D_F \equiv \{x_1 = 2, x_4 = 3\}$, and $D'_{F'} \equiv \{x_1 = 2, x_5 = 3\}$. In P , propagation gives $x_2 \in \{1, 4\}, x_3 \in \{4, 5\}, x_5 \in \{4, 5\}$. In P' , propagation gives $x_2 \in \{1, 4\}, x_3 \in \{4, 5\}, x_4 \in \{4, 5\}$. SBDD would detect that these two subproblems are symmetric, because the partial assignments $x_1 = 2, x_4 = 3$ and $x_1 = 2, x_5 = 3$ map to each other under the symmetry $x_4 \leftrightarrow x_5$. On the other hand, automatic caching determines that they are not equivalent since $F \neq F'$, and can do nothing here.

It is also possible that both techniques detect a subproblem dominance. Consider the subproblems P and P' where $D_F \equiv \{x_4 = 3, x_5 = 4\}$, and $D'_{F'} \equiv \{x_4 = 4, x_5 = 3\}$. Both propagate to $x_1 \in \{1, 2\}, x_2 \in \{1, 2\}, x_3 \in \{2, 5\}$. The linear constraint becomes satisfied in both and has $\text{key}(c, D_F) \equiv \text{true}$. Thus their projection keys match and automatic caching detects that P and P' are equivalent. SBDD detects that they are also symmetric since $x_4 = 3, x_5 = 4$ and $x_4 = 4, x_5 = 3$ map to each other under $x_4 \leftrightarrow x_5$. $\square$

It is possible to extend our caching approach to exploit whole problem symmetries.

Theorem 3.7.3. *Let $P \equiv (V, D, C)$ and $P' \equiv (V, D', C')$ be subproblems arising during search. Let $F = \text{fixed}(D)$, $U = \text{vars}(C) \setminus F$, $F' = \text{fixed}(D')$, $U' = \text{vars}(C') \setminus F'$. Suppose σ is a symmetry s.t. $\forall c \in C, \sigma(c) \in C$. If $\sigma(F') = F$, and $\forall c \in C, \sigma(\text{key}(c, D'_{F'})) \Rightarrow \text{key}(\sigma(c), D_F)$ and $\sigma(D'_{U'}) \Rightarrow D_U$, then P symmetrically dominates P' w.r.t. symmetry σ .* $\square$

Theorem 3.7.3 tells us that once again, we can detect subproblem symmetric dominance by doing a piecewise comparison of constraint projection keys.

Our characterization of subproblem symmetry in terms of projections is strictly more general than the characterization using partial assignments. This is because

$\sigma(D'_{F'}) = D_F$ implies $\sigma(\pi_{U'}(C \wedge D')) \Rightarrow \pi_U(C \wedge D)$, but not vice versa. Therefore it is possible that P and P' are symmetric subproblems even when $\sigma(D'_{F'}) \neq D_F$. For example, this can occur when conditional symmetries exist. Since SBDD only detects $\sigma(D'_{F'}) = D_F$, it only exploits a subset of the possible subproblem symmetries. Automatic caching, which directly works on subproblem projections, has the potential to exploit a strictly wider range of subproblem symmetries than SBDD. Extending automatic caching to efficiently exploit symmetries is a clear avenue of future work.

3.7.3 Nogood learning

Nogood learning approaches in constraint programming attempt to learn nogoods from failures and record these as new constraints in the program. Automatic caching can be considered one such technique, since we record sets of constraint projections which cause failure as nogoods. Another very successful methods is Lazy Clause Generation (LCG) [113, 45]. LCG use nogoods consisting of equality literals ($x = v$), disequality literals ($x \neq v$) and inequality literals ($x \geq v, x \leq v$) to record the reasons for failures, and use SAT techniques to efficiently manage the nogoods. LCG also exploits subproblem equivalences/dominances, and is the main competitor to automatic caching.

In terms of pruning power, neither technique strictly dominates the other. LCG has two advantages. Firstly, in LCG, only constraints which are directly involved in the conflict are used to produce the nogood, whereas in automatic caching, all active and non-satisfied constraints are used. This sometimes allows LCG to produce more powerful nogoods than automatic caching. Secondly, the nogoods derived by LCG can be propagated, rather than merely be checked for failure, so they can prune more. These two factors sometimes allow a much greater reduction in search space. However this extra power comes with a high overhead. In LCG, every nogood that is derived is added as a clausal propagator. This takes $O(n)$ to propagate where n is the number of nogoods kept. In automatic caching however, the failure check is done in $O(1)$.

Automatic caching also has an advantage in terms of the kinds of nogoods they

can express. The language of nogoods used by LCG, i.e., atomic constraints of the form $v = d$ and $v \leq d$, limits the kinds of nogoods that can be expressed, whereas the constraint projections used in automatic caching can sometimes express nogoods which are more powerful. Consider the subproblem $x_1 + 2x_2 + x_3 + x_4 + 2x_5 \leq 20 \wedge C$, with $D \equiv \{x_1 = 1 \wedge x_2 = 2 \wedge x_3 = 3\}$. If this subproblem fails, the projection key stores that $x_4 + 2x_5 \leq 12 \wedge$ other keys leads to failure. LCG would express this as $x_1 \geq 1 \wedge x_2 \geq 2 \wedge x_3 \geq 3 \wedge$ other keys leads to failure, since there are no literals to represent partial sums. The nogood from automatic caching is strictly stronger. For example, $D \equiv \{x_1 = 2 \wedge x_2 = 3 \wedge x_3 = 0\}$ would fire the nogood from automatic caching, but not the one from LCG. This difference is apparent in the experimental results for 0-1 Knapsack.

Another advantage of automatic caching over LCG is in the ease of implementation. Automatic caching can be added to a CP solver by adding to each propagator code to build a representation of the propagator for inclusion in the key, as well as adding a generic caching mechanism to the search. In Section 3.5 we show how to do this for many constraints, and give generic approaches to building efficient representation for global constraints via decomposition. LCG requires much more fundamental changes to a CP solver to record an implication graph of inferences made. In particular each global constraint needs to be extended to explain its inferences, and this is far from straightforward.

3.8 Experiments

We use the state of the art CP solver CHUFFED in our experiments. CHUFFED can be run as a naive CP solver (denoted as CHUFFED), as a CP solver with caching (denoted as CHUFFEDC), and as a Lazy Clause Generation solver (denoted as CHUFFEDL). We also compare against Gecode 3.2.2 [56] – widely recognized as one of the fastest constraint programming systems (to illustrate we are not optimizing a slow system). We use the MurmurHash 2.0 hash function [6]. We use models written in the modeling language MiniZinc [112]. This facilitates a fair comparison between the solvers, as all solvers use the same model and search strategy. Note that caching and lazy clause

generation do not interfere with the search strategies used here, as all they can do is fail subtrees earlier. Thus, CHUFFED with caching or lazy clause generation always finds the same solution as the naive version CHUFFED and Gecode, and any speedup observed comes from a reduced search.

The experiments were conducted on Xeon Pro 2.4GHz processors with a 900 second timeout. Tables 3.1 and 3.2 presents the number of variables and constraints as reported by CHUFFED, the times for each solver in seconds as well as the number of (leaf) fails. For the automatic caching solver we also report the number of cache hits occurring in the search and the average key size (in bytes). We discuss the results for each problem below. All the MiniZinc models and instances are available at www.cs.mu.oz.au/~pjs/autocache/.

3.8.1 Knapsack

0-1 knapsack is ideal for caching. There are two linear constraints: one for the weight and one for the objective, and the domains are all binary. The representation of the key thus simply requires the set of fixed variables and two integers. The non-caching solvers timeout as n increases, as their time complexity is $O(2^n)$. This is a worst case for lazy clause generation since the nogoods generated are not reusable at all.

CHUFFEDC, on the other hand, is easily able to solve much larger instances (see Table 3.1). The node to nW ratio (not shown) stays fairly constant as n increases (varying between 0.86 and 1.06), showing that it indeed has search (node) complexity $O(nW)$. The time to nW ratio grows as $O(n)$ though, since we are using a general CP solver where the linear constraints take $O(n)$ to propagate at each node, while DP requires constant work per node. Hence, automatic caching is not as efficient as pure DP.

3.8.2 MOSP

The MOSP model used in the experiments is similar to that presented in Section 3.3, but has some additional conditional dominance breaking constraints [28] that make

the (non-caching) search much faster. We use random instances from [28]. Automatic caching gives up to two orders of magnitude speedup. The speedup grows exponentially with problem size. LCG also exploits a similar amount of equivalence/dominance and achieves a similar speedup.

3.8.3 Black Hole

The Black Hole model used in the experiments is similar to that presented in Section 3.6.1, but has additional conditional symmetry breaking constraints [61]. We generated random instances and used only the hard ones for this experiment. Automatic caching gives a modest speedup of around 2-3. The speedup is relatively low on this problem because the conditional symmetry breaking constraints have already removed many equivalent subproblems, and the caching is only exploiting the ones which are left. Note that the manual caching reported in [145] achieves speedups in the same range (on hard instances). LCG exploits even more dominance than automatic caching. But this extra dominance exploitation comes at such a high cost that it ends up being much slower overall.

3.8.4 BACP

In the Balanced Academic Curriculum Problem (BACP), we form a curriculum by assigning a set of courses to a set of periods, with certain restrictions on how many courses and how much “course load” can be assigned to each period. We also have prerequisite constraints between courses. The BACP can be viewed as a bin packing problem with a lot of additional side constraints. The subproblem symmetry is the same as that in bin packing. Suppose that a set of courses have been assigned to the first k periods and has “filled” them. The remaining subproblem only depends on the set of unassigned courses, and not on how the earlier courses were assigned to the first k periods. Any permutation of those k period assignments that satisfy the constraints lead to the same subproblem. We use the model of [71], but with some additional redundant constraints that considerably reduce the search required to find an optimal solution for caching and non-caching solvers. Note that by Theorem 3.4.15,

the redundant linear constraints do not have to be considered in the projection key. The 3 instances *curriculum_8/10/12* given in CSPLib can be solved to optimality in just a few milliseconds. We generate random instances with 50 courses, 10 periods, and course credit ranging between 1 and 10. Almost all are solvable in milliseconds so we pick out only the non-trivial ones for the experiment. We also include the 3 standard instances from CSPLib. Automatic caching can give orders of magnitude speedup. Both automatic caching and lazy clause generation are capable of exploiting the subproblem equivalence, giving orders of magnitude speedup. In this case, lazy clause generation exploits more equivalence/dominance and is faster.

3.8.5 Radiation Therapy

In the Radiation Therapy problem [7], the aim is to decompose an integral intensity matrix describing the radiation dose to be delivered to each area, into a set of patterns to be delivered by a radiation source, while minimizing the amount of time the source has to be switched on, as well as the number of patterns used (setup time of machine). The subproblem equivalence arises because there are equivalent methods to obtain the same cell coverages, e.g., radiating one cell with two intensity 1 patterns is the same as radiating it with one intensity 2 pattern, etc. We use random instances generated as in [7]. Once again automatic caching can give orders of magnitude speedup. Both automatic caching and lazy clause generation produce orders of magnitude speedup, though lazy clause generation once again exploits more equivalence/dominance and is faster.

3.8.6 Memory Consumption

The memory consumption of our caching scheme is linear in the number of nodes searched. The size of each key is dependent on the structure of the problem and can range from a few hundred bytes to thousands of bytes. One can read the total cache size from the tables as Key size multiplied by number of fails. On a modern computer, this means we can usually search several hundreds of thousands of nodes before running out of memory. There are simple schemes to reduce the memory usage,

which we plan to investigate in the future. For example, much like in SAT learning, we can keep an “activity” score for each entry to keep track of how often they are used. Inactive entries can then periodically be pruned to free up memory.

3.8.7 Limitations

Automatic caching is not useful for every problem. Although many structured problems do exhibit subproblem dominances, there are also many problems where subproblem dominances do not occur at all. When automatic caching is used on such problems, it will incur a constant factor overhead and produce no benefit whatsoever. The number of cache hits provides a good measure of whether the caching is useful or not. If the proportion of nodes which are being pruned by the caching is low or zero, then the caching is probably not useful. Ideally, the user can simply turn off automatic caching when this occurs. Alternatively, it may be possible to have the solver dynamically check whether the caching is working and to turn it off when it is not. This is an avenue of future research.

3.9 Conclusion

In this chapter, we developed the theory behind subproblem dominance, and showed that they can be detected by comparing projection keys. We described a new technique called automatic caching via constraint projection, where constraint projection is used to derive nogoods from each failed subproblem. These nogoods are propagated using a hash table so that failure can be detected in $O(1)$ time. Our experiments showed that our new technique can produce orders of magnitude speedup compared to the base solver on problems containing subproblem dominances. Comparison with lazy clause generation, which is the current state of the art in nogood learning, shows that the nogoods derived via projection are competitive with those that LCG derives via constraint resolution. The nogoods derived via projection can be exponentially stronger in some problems, e.g., ones with long linear constraints, but can also be

weaker on other problems. The speedups from our technique tends to grow exponentially with problem size.

Automatic caching via constraint projection is quite robust. It can find and exploit subproblem dominances even in models that are not “pure”, e.g., MOSP with dominance and conditional symmetry breaking constraints, Black Hole with conditional symmetry breaking constraints, and BACP which can be seen as bin packing with lots of side constraints and some redundant constraints. It is also easier to extend a CP system to include automatic caching via constraint projection than it is to extend it to implement lazy clause generation. In particular we explained how to easily define projection keys for arbitrarily complex global constraints.

Table 3.1: Experimental results: Knapsack, MOSP and Black Hole

Instance	Vars.	Cons.	Gecode		CHUFFED		CHUFFEDL		CHUFFEDC			
			Time	Fails	Time	Fails	Time	Fails	Time	Fails	Cache hits	Key size
knapsack-20	21	2	0.01	977	0.01	1034	0.02	978	0.01	354	95	54
knapsack-30	31	2	0.38	145183	0.81	145438	3.42	145184	0.02	2160	1305	54
knapsack-40	41	2	17.9	6032500	37.80	6032644	168.48	6032500	0.04	3038	2259	59
knapsack-50	51	2	>900	-	>900	-	>900	-	0.08	6176	4810	60
knapsack-60	61	2	>900	-	>900	-	>900	-	0.10	7802	5993	60
knapsack-100	101	2	>900	-	>900	-	>900	-	0.40	26581	23042	68
knapsack-200	201	2	>900	-	>900	-	>900	-	2.33	110739	103505	80
knapsack-300	301	2	>900	-	>900	-	>900	-	6.50	240581	230470	92
knapsack-400	401	2	>900	-	>900	-	>900	-	13.77	393911	379063	104
knapsack-500	501	2	>900	-	>900	-	>900	-	25.32	595400	576916	116
mosp-30-30-4-1	1021	1861	13.93	34427	2.98	40456	0.49	3061	0.47	5331	225	1650
mosp-30-30-2-1	1021	1861	>900	-	>900	-	4.23	21612	4.26	56012	4896	1656
mosp-40-40-10-1	1761	3281	0.93	766	0.15	814	0.18	655	0.15	703	3	2592
mosp-40-40-8-1	1761	3281	2.72	2652	0.40	2820	0.35	1464	0.27	1590	19	2737
mosp-40-40-6-1	1761	3281	40.85	46630	6.30	49904	1.58	6327	1.26	8632	288	2927
mosp-40-40-4-1	1761	3281	>900	-	328.24	2931342	10.56	32037	7.84	58609	3185	2995
mosp-40-40-2-1	1761	3281	>900	-	>900	-	39.28	91074	36.49	268725	24040	3046
mosp-50-50-10-1	2701	5101	8.62	3513	0.78	3621	0.73	2333	0.59	2498	21	3976
mosp-50-50-8-1	2701	5101	36.88	16820	3.29	18164	1.96	6018	1.56	7327	125	4118
mosp-50-50-6-1	2701	5101	>900	-	234.47	1564331	19.87	41436	12.89	71894	2815	4190
blackhole-1	104	407	101.79	425470	36.85	590628	91.99	111522	17.90	203018	24665	680
blackhole-2	104	411	59.24	273090	21.19	313178	72.28	99595	14.12	161808	11199	650
blackhole-3	104	434	31.00	132406	25.02	541716	58.26	94384	17.63	299079	16704	598
blackhole-4	104	393	68.32	324063	27.77	500890	87.13	118694	15.42	196622	12886	631
blackhole-5	104	429	411.83	2032279	155.28	2654106	146.55	162946	66.41	840335	95166	672
blackhole-6	104	448	157.30	739404	57.60	882158	43.02	75068	24.14	255541	22362	655
blackhole-7	104	407	84.68	482083	32.43	642540	177.17	194795	11.05	125747	12078	632
blackhole-8	104	380	125.83	628861	47.27	777901	112.12	149123	27.35	313000	40262	646
blackhole-9	104	404	87.93	385227	42.69	724076	219.70	264754	23.55	295363	25839	658
blackhole-10	104	364	211.86	1035902	91.85	1567065	391.60	395195	38.09	445726	48605	640

Table 3.2: Experimental results: BACP and Radiation

Instance	Vars.		Gecode		CHUFFED		CHUFFEDL		CHUFFEDC			
	Cons.		Time	Fails	Time	Fails	Time	Fails	Time	Fails	Cache hits	Key size
curriculum_8	838	1942	0.01	0	0.01	0	0.01	0	0.01	0	0	626
curriculum_10	942	2214	0.01	1	0.01	1	0.01	1	0.01	1	0	714
curriculum_12	1733	4121	0.01	0	0.01	0	0.01	1	0.01	0	0	1012
bacp-medium-1	1121	2654	29.47	404779	35.21	404779	5.95	26764	11.59	133456	18505	1045
bacp-medium-2	1122	2650	>900	—	>900	—	0.01	41	9.53	120186	71575	902
bacp-medium-3	1121	2648	462.64	8330028	383.81	8330028	0.03	105	2.40	43886	40973	951
bacp-medium-4	1119	2644	5.76	101650	4.59	101650	0.69	5441	0.61	10084	3178	869
bacp-medium-5	1119	2641	54.17	896859	56.90	896859	0.30	1783	2.41	33884	12688	867
bacp-hard-1	1121	2655	>900	—	>900	—	0.01	5	55.94	727847	479361	924
bacp-hard-2	1118	2651	>900	—	>900	—	0.01	8	191.78	1518859	669450	1039
radiation-6-9-1	877	942	>900	—	>900	—	1.39	4290	11.37	184133	122799	2597
radiation-6-9-2	877	942	>900	—	>900	—	2.68	7835	25.03	431953	285552	2192
radiation-7-8-1	1076	1168	>900	—	>900	—	0.31	1361	0.27	2748	393	3782
radiation-7-8-2	1076	1168	188.12	3940867	177.17	7881662	0.22	808	0.38	4467	2647	3030
radiation-7-9-1	1210	1301	312.09	5518905	283.70	11032846	0.78	3341	1.33	14993	7426	3814
radiation-7-9-2	1210	1301	140.44	2316137	110.82	4631769	3.13	18207	4.43	53683	14491	4068
radiation-8-9-1	1597	1718	>900	—	>900	—	1.89	5232	23.92	227154	152720	5686
radiation-8-9-2	1597	1718	>900	—	>900	—	2.04	6518	9.71	107545	56586	3908
radiation-8-10-1	1774	1894	9.73	101062	8.95	201703	8.63	36612	10.54	88663	10980	7267
radiation-8-10-2	1774	1894	>900	—	>900	—	5.75	10322	45.52	385431	233590	3987

Part II

Symmetry Breaking

Many constraint problems contain large amounts of symmetry. In such problems, examining one valuation is often enough to tell us everything we need to know about its symmetric counterparts (whether it is a solution, what its objective value is, etc). These redundancies in the search can exponentially increase the run time. There are two main ways to deal with this. We can either try to eliminate the symmetry by remodeling the problem (e.g., [61, 7, 143]), or we can apply some form of symmetry breaking which prevents the CP solver from exploring symmetric parts of the search space. In this part of the thesis, we concentrate on symmetry breaking techniques, which can give significant speedups on appropriate problems [48, 118, 47, 124, 119]. The most popular of these are: lexicographical symmetry breaking constraints [118, 32, 98, 120, 50], Symmetry Breaking During Search (SBDS) [62] and Symmetry Breaking by Dominance Detection (SBDD) [43].

II.1 Definitions

Symmetries have been expressed in many different ways depending on which subclass of symmetries the authors are trying to describe [48, 118, 104, 31]. In the most general form, symmetries can be defined as follows:

Definition II.1.1. *A symmetry of a problem $P \equiv (V, D, C)$ is a bijective function $\sigma : \Theta_V \rightarrow \Theta_V$ mapping valuations to valuations such that: $\theta \in \text{solns}(P)$ iff $\sigma(\theta) \in \text{solns}(P)$, i.e., it maps solutions to solutions and non-solutions to non-solutions.*

Definition II.1.2. *A variable symmetry is a symmetry which permutes the set of variables in the problem. Let $\rho : V \rightarrow V$ be a permutation of the variables in the problem. Let σ be a mapping defined by: $\sigma(\theta)(x) = \theta(\rho(x))$. If σ is a symmetry, then it is called a variable symmetry.*

Example II.1.3. Consider the problem $P \equiv (V, D, C)$, $V \equiv \{x_1, x_2, x_3\}$, $D \equiv \{x_i \in \{1, \dots, 5\} \mid i = 1, 2, 3\}$, $C \equiv \{\text{alldiff}(x_1, x_2, x_3), x_1 + x_2 + x_3 \geq 10\}$. x_1 , x_2 and x_3 are fully interchangeable since they have the same domains, and they are interchangeable in all the constraints in the problem. For any permutation ρ of $\{x_1, x_2, x_3\}$, the mapping σ_ρ defined as: $\sigma_\rho(\theta)(x) = \theta(\rho(x))$, is a variable symmetry of P .

Definition II.1.4. A value symmetry is a symmetry which permutes the values among a subset of variables in the problem. Let $V' \subseteq V$ be a subset of the variables with identical domains T . Let $\rho : T \rightarrow T$ be a permutation of the values in T . Let σ be a mapping defined by: $\sigma(\theta)(x) = \rho(\theta(x))$ for $x \in V'$ and $\sigma(\theta)(x) = \theta(x)$ otherwise. If σ is a symmetry, then it is called a value symmetry.

Example II.1.5. Consider the problem $P \equiv (V, D, C)$, $V \equiv \{x_1, x_2, x_3\}$, $D \equiv \{x_i \in \{1, \dots, 5\} \mid i = 1, 2, 3\}$, $C \equiv \{\text{alldiff}(x_1, x_2, x_3), x_1 \neq x_2, x_2 \neq 2, x_3 \neq 4\}$. The values $\{1, 3, 5\}$ on $\{x_1, x_2, x_3\}$ are fully interchangeable, since they are interchangeable in all four of the constraints in the problem. For any permutation ρ of $\{1, 2, 3, 4, 5\}$ such that $\rho(2) = 2$ and $\rho(4) = 4$, the mapping σ_ρ defined as: $\sigma_\rho(\theta)(x) = \rho(\theta(x))$, is a value symmetry of P .

Definition II.1.6. A variable sequence symmetry is a symmetry which permutes sequences of variables in the problem. Let $Q \equiv [[x_{1,1}, \dots, x_{1,m}], \dots, [x_{n,1}, \dots, x_{n,m}]]$ be an $n \times m$ matrix of variables where each variable is distinct. Let ρ be a permutation of $\{1, \dots, n\}$. Let σ be a mapping defined by: $\forall i \in \{1, \dots, n\}, \forall j \in \{1, \dots, m\}, \sigma(\theta)(x_{i,j}) = \theta(x_{\rho(i),j})$ and $\sigma(\theta)(x) = \theta(x)$ for $x \notin Q$. If σ is a symmetry, then it is called a variable sequence symmetry.

Example II.1.7. Consider the problem $P \equiv (V, D, C)$, $V \equiv \{x_{1,1}, x_{1,2}, x_{2,1}, x_{2,2}, x_{3,1}, x_{3,2}\}$, $D \equiv \{x_{i,j} \in \{1, \dots, 5\}\}$, $C \equiv \{x_{1,1} + x_{1,2} = 4, x_{2,1} + x_{2,2} = 4, x_{3,1} + x_{3,2} = 4, x_{1,1} + x_{2,1} + x_{3,1} = 5, x_{1,2} + x_{2,2} + x_{3,2} = 7\}$. We can view the $x_{i,j}$ as a 3×2 matrix. The constraints state that all three rows sum to 4, the first column sums to 5, and the second column sums to 7. The rows are all interchangeable, e.g., simultaneously swapping $x_{1,1}$ with $x_{2,1}$ and $x_{1,2}$ with $x_{2,2}$ preserves solutions. Given any permutation ρ of $\{1, 2, 3\}$, the mapping σ_ρ defined by: $\sigma_\rho(\theta)(x_{i,j}) = \theta(x_{\rho(i),j})$ is a variable sequence symmetry of P .

We are often interested in considering sets of symmetries which form a mathematical group under function composition.

Definition II.1.8. A symmetry group of problem P is a set S of symmetries of P , s.t. S is a group under the operation of function composition, i.e., $\forall \sigma_1, \sigma_2 \in S, \sigma_1 \cdot \sigma_2 \in S, \forall \sigma \in S, \sigma^{-1} \in S$, and the identity mapping I is in S .

Example II.1.9. In Example II.1.3, if R is the set of permutations of $\{x_1, x_2, x_3\}$, then the set of symmetries $\{\sigma_\rho \mid \rho \in R\}$ forms a symmetry group.

Definition II.1.10. *The orbit of a valuation θ under the symmetry group S is the set: $\{\sigma(\theta) \mid \sigma \in S\}$.*

Clearly, each orbit under S consists either entirely of solutions, or entirely of non-solutions.

Definition II.1.11. *A symmetry breaking technique is complete w.r.t. symmetry group S if one and only one valuation within each orbit under S is not pruned by the technique.*

II.2 Lexicographical Symmetry Breaking

Lexicographical symmetry breaking constraints break the symmetry by restricting the search to find only solutions which are lexicographically least within their orbit.

Definition II.2.1. *A lexicographical ordering $<_{lex}$ on the valuations of problem P is defined as follows. We first define a total ordering $<_v$ on the variables in P . Without loss of generality, let this ordering be: $x_1 <_v x_2 <_v \dots <_v x_n$. We then define the total ordering $<_{lex}$ based on $<_v$ as: $\theta_1 <_{lex} \theta_2$ iff $\exists k \in \{1, \dots, n\}$ s.t. $\forall i < k, \theta_1(x_i) = \theta_2(x_i)$ and $\theta_1(x_k) < \theta_2(x_k)$.*

Definition II.2.2. *A lexicographical symmetry breaking constraint for symmetry group S and lexicographical ordering $<_{lex}$ is a constraint $sb(S, <_{lex})$ such that: $\forall \theta \in \Theta_V, \min_{lex}\{\sigma(\theta) \mid \sigma \in S\} \in \text{solns}(sb(S, <_{lex}))$, i.e., within each orbit, the lex-least valuation must not be pruned by $sb(S, <_{lex})$. It is complete if: $\theta \in \text{solns}(sb(S, <_{lex}))$ iff $\theta = \min_{lex}\{\sigma(\theta) \mid \sigma \in S\}$, i.e., within each orbit, only the lex-least valuation is not pruned by $sb(S, <_{lex})$.*

$sb(S, <_{lex})$ can be implemented as one global constraint, or more commonly, decomposed into a set of smaller constraints. For most simple symmetry groups, the complete lexicographical symmetry breaking constraint is easy to find and can be propagated efficiently.

Example II.2.3. Suppose problem P has a variable symmetry on $\{x_1, \dots, x_n\}$. Let S be the symmetry group corresponding to this variable symmetry. Let $<_{lex}$ be the lexicographical ordering based on the variable ordering $x_1 <_v \dots <_v x_n$. Then the complete lexicographical symmetry breaking constraint can be expressed as: $sb(S, <_{lex}) \equiv \bigwedge_{i=1}^{n-1} x_i \leq x_{i+1}$. This constraint can be propagated to domain consistency in $O(n)$.

Suppose problem P has a value symmetry on the values $\{1, \dots, k\}$ over the variables $\{x_1, \dots, x_n\}$. Let S be the symmetry group corresponding to this value symmetry. Let $<_{lex}$ be the lexicographical ordering based on the variable ordering $x_1 <_v \dots <_v x_n$. Suppose we take the minimum of an empty set to be ∞ and take $\infty < \infty$ to be true. Then the complete lexicographical symmetry breaking constraint can be expressed as: $sb(S, <_{lex}) \equiv \bigwedge_{v=1}^{k-1} \min\{i \mid x_i = v\} < \min\{i \mid x_i = v + 1\}$. This constraint can be propagated to domain consistency in $O(n)$.

Suppose problem P has a variable sequence symmetry on the variable sequences $\{[x_{1,1}, \dots, x_{1,n}], \dots, [x_{m,1}, \dots, x_{m,n}]\}$. Let S be the symmetry group corresponding to this variable sequence symmetry. Let $<_{lex}$ be the lexicographical ordering based on the variable ordering $x_{1,1} <_v \dots <_v x_{m,1} <_v \dots <_v x_{1,n} <_v \dots <_v x_{m,n}$. Then the complete lexicographical symmetry breaking constraint can be expressed as: $sb(S, <_{lex}) \equiv \bigwedge_{i=1}^{n-1} lexlesseq([x_{i,1}, \dots, x_{i,n}], [x_{i+1,1}, \dots, x_{i+1,n}])$, where $lexlesseq([a_1, \dots, a_m], [b_1, \dots, b_m]) \equiv (a_1 < b_1) \vee (a_1 = b_1 \wedge a_2 < b_2) \vee \dots \vee (a_1 = b_1 \wedge \dots \wedge a_{m-1} = b_{m-1} \wedge a_m \leq b_m)$. This constraint can be propagated to domain consistency in $O(n^2)$.

For more complicated symmetry groups, the complete lexicographical symmetry breaking constraint may be hard to express and may be very costly to propagate to domain consistency. In such cases, it is typically better to use an incomplete lexicographical symmetry breaking constraint which can be propagated in a reasonable time [50, 47].

Example II.2.4. Suppose problem P had variables $\{x_{i,j} \mid i, j = 1, \dots, n\}$ and has both a row and column symmetry on the $x_{i,j}$. Let S_1 denote the symmetry group consisting of row symmetries, S_2 denote the symmetry group consisting of column

symmetries, and S denote the group generated by S_1 and S_2 . The complete lexicographical symmetry breaking constraint for S has no simple closed form, and is NP-hard to propagate to domain consistency [13]. This is clearly not practical. Instead, we can post the complete lexicographical symmetry breaking constraints for S_1 and S_2 , i.e., break all row symmetries and break all column symmetries individually. This is incomplete for S , but only costs $O(n^2)$ to propagate, and still breaks enough of the symmetry to be worthwhile [47].

Lexicographical symmetry breaking constraints are simple to implement and can work very well under many circumstances. However, they are not always the best choice of symmetry breaking, as the lexicographical ordering may conflict with the search strategy. For example, suppose we chose $<_{lex}$ such that the first solution that the search encounters is not lex-least within its orbit. Then enforcing $sb(S, <_{lex})$ would prune off this solution, forcing the search to find another solution. In the worse case, adding the lexicographical symmetry breaking constraint may actually slow down the search. It is often possible to choose a $<_{lex}$ which is consistent with the search strategy so that there is no conflict. However, if that is not possible, e.g., because we are using a dynamic search strategy, then other symmetry breaking techniques may be more appropriate.

II.3 Dynamic Symmetry Breaking

Dynamic symmetry breaking techniques like SBDS and SBDD never conflict with the search strategy. Rather than preferring the lexicographically least solution in each orbit, dynamic techniques always prefer the first solution found in each orbit. Although SBDS, SBDD, and other similar techniques [8, 102] are often described in other terms, they can all be thought of as using symmetric versions of nogoods to prune symmetric parts of the search space. We first need to define how constraints map under symmetries.

Definition II.3.1. *Given a symmetry σ , we can extend σ to map constraints to constraints as follows: given a constraint c , $\sigma(c)$ is a constraint over V such that: θ*

satisfies $\sigma(c)$ iff $\sigma(\theta)$ satisfies c .

Example II.3.2. Let $P \equiv (V, D, C)$ where $V \equiv \{x_1, x_2, x_3\}$. Let $\sigma : \Theta_V \rightarrow \Theta_V$ be the symmetry which swaps x_1 and x_2 . It is easy to see that for example: $\sigma(x_1 = 1) \equiv x_2 = 1$, $\sigma(x_1 + 2x_2 + 3x_3) \equiv x_2 + 2x_1 + 3x_3$, etc.

It is easy to see that the following identities hold:

- $\sigma(a \wedge b) \Leftrightarrow \sigma(a) \wedge \sigma(b)$
- $\sigma(a \vee b) \Leftrightarrow \sigma(a) \vee \sigma(b)$
- $\sigma(\neg(a)) \Leftrightarrow \neg(\sigma(a))$
- $a \Rightarrow b$ iff $\sigma(a) \Rightarrow \sigma(b)$

In this notation, we can express symmetries as follows:

Lemma II.3.3. *Given a problem $P \equiv (V, D, C)$, a bijective function $\sigma : \Theta_V \rightarrow \Theta_V$ is a symmetry of P iff: $C \wedge D \Leftrightarrow \sigma(C \wedge D)$.*

Recall from Definition I.1.1 that a nogood of $P \equiv (V, D, C)$ is a constraint such that: $C \wedge D \Rightarrow n$. We have the following result:

Theorem II.3.4. *Given a problem $P \equiv (V, D, C)$ and a symmetry σ of P , if n is a nogood of P , then so is $\sigma(n)$.*

Proof. $C \wedge D \Leftrightarrow \sigma(C \wedge D) \Rightarrow \sigma(n)$. So $\sigma(n)$ is also a nogood of P . $\square$

Theorem II.3.4 tells us that if we derive any nogood n , then all the symmetric versions of n are also valid and can be used to prune other parts of the search space.

Example II.3.5. Consider the simple problem $P \equiv (V, D, C)$, $V \equiv \{x_1, x_2, x_3\}$, $D \equiv \{x_i \in \{1, \dots, 5\} \mid i = 1, 2, 3\}$, $C \equiv \{alldiff(x_1, x_2, x_3), x_1 + x_2 + x_3 \geq 10\}$. There is a variable symmetry on $\{x_1, x_2, x_3\}$. The partial assignment $x_1 = 1, x_2 = 2$ fails, giving a nogood $\neg(x_1 = 1 \wedge x_2 = 2)$. Any symmetric version of this nogood, e.g., $\neg(x_2 = 1 \wedge x_1 = 2)$, $\neg(x_2 = 1 \wedge x_3 = 2)$, are also valid nogoods.

SBDS, SBDD, and other similar techniques are all simply applications of Theorem II.3.4. They differ in what kind of nogoods n are used, which σ are applied, and how $\sigma(n)$ is derived and propagated. Both SBDS and SBDD use decision nogoods. When some subproblem $P' \equiv (V, D, C \cup \{d_1, \dots, d_k\})$ has been searched and failed, the decision nogood is $n \equiv \neg(d_1 \wedge \dots \wedge d_n)$. By Theorem II.3.4, for all $\sigma \in S$, $\sigma(n)$ is also a valid nogood for this problem.

SBDS proceeds as follows. Let P'' be the parent subproblem of P' . We check each of the $\sigma(n)$ to see if they are locally satisfied within P'' . If not, then we post the nogood $\sigma(n)$ as a local constraint within P'' . It is proved in [62] that this is sufficiently to break all symmetries completely. Posting $\sigma(n)$ only as a local constraint, even though it holds globally, is an optimization to reduce work. The decision nogood derived when P'' is exhaustively searched is $n' \equiv \neg(d_1 \wedge \dots \wedge d_{k-1})$, and we have $\forall \sigma \in S, \sigma(n') \Rightarrow \sigma(n)$. Hence SBDS will post strictly stronger nogoods $\sigma(n')$ when P'' is finished, and so it is sufficient to post the $\sigma(n)$ locally within P'' .

SBDD is similar, however, instead of posting $\sigma(n)$ as propagators, it stores only n in a nogood store N . Later on, at each other subproblem P''' , the nogood store N is examined to see if $\exists n \in N, \exists \sigma \in S$ s.t. $\sigma(n)$ can propagate in P''' . Another way to think of this is simply that SBDD implements all of $\sigma(n)$ as one giant global propagator whereas SBDS posts them all as separate propagators. SBDD has the advantage that the global propagator may have significantly better space and time complexity than posting the $\sigma(n)$ separately, but it also takes more work to implement.

II.4 Contributions

In Chapter 4, we describe an extension to SBDS called SBDS-1UIP, where we use symmetric versions of the 1UIP nogoods derived by LCG solvers instead of symmetric versions of decision nogoods. We show that this is significantly more powerful than the original SBDS method, allowing us to exploit kinds of redundancies that no previous general technique is capable of exploiting. In particular, SBDS-1UIP's power is beyond complete, as it is capable of using the failure of a subproblem in one orbit to prune a subproblem in a different orbit, and thus does not have to examine at least

one valuation per orbit as previous techniques do. This extra pruning power can give us exponential speedups compared to the traditional symmetry breaking methods on appropriate problems.

In Chapter 5, we consider problems which do not exhibit full problem symmetry, but which exhibit *almost symmetries*, i.e., symmetries which are slightly broken. Such problems often still have large amounts of symmetries that can be exploited for speedup. However, traditional symmetry breaking methods like lexicographical symmetry breaking constraints, SDBS or SBDD cannot directly handle such partially broken symmetries. We present two new general techniques for exploiting almost symmetries. The first is to treat almost symmetries as conditional symmetries and exploit them via conditional symmetry breaking constraints. The second is to modify SDBS-1UIP to handle almost symmetries. Both techniques are capable of producing exponential speedups on appropriate problems.

Chapter 4

Symmetry Breaking During Search and Lazy Clause Generation

4.1 Introduction

As we described in Section II.3, Symmetry Breaking During Search (SBDS) [62] is a technique which utilizes symmetric versions of decision nogoods to prune symmetric parts of the search space. It is an application of Theorem II.3.4. However, Theorem II.3.4 actually applies to any kind of nogood, not just decision nogoods, and there is no reason why we should be restricted to using only decision nogoods. Boolean Satisfiability solvers [109] and Lazy Clause Generation solvers (LCG) [113, 45] are capable of deriving nogoods called 1UIP nogoods, which have empirically been found to produce significantly more pruning than decision nogoods in non-symmetric problems [165]. Clearly, it would be interesting to see if the extra pruning power of 1UIP nogoods carry over to the symmetric case as well. In this chapter, we show how to modify SBDS to use 1UIP nogoods. We call the original version SBDS-DEC, and the modified version SBDS-1UIP.

There are several issues which we have to deal with in order to implement SBDS-1UIP. SBDS-DEC only deals with simple decision nogoods involving equality literals on the decision variables. However, the 1UIP nogoods derived by LCG may have disequality and inequality literals as well, and may also have literals involving intermediate variables that were introduced by the solver. It is not immediately apparent how these more general nogoods map under the symmetries. Our contributions in this chapter are as follows:

- We show that SBDS can be extended to work with more general nogoods, i.e., nogoods with equality, disequality and inequality literals.
- We show how to handle intermediate variables introduced by the solver.
- We prove that SBDS-1UIP is at least as powerful as SBDS-DEC.
- We show that SBDS-1UIP can exploit symmetries that cannot be exploited by SBDS-DEC or by any other previous method.

4.2 SBDS-1UIP

Adapting SBDS to use 1UIP nogoods is simple. When a subproblem P' fails, the LCG solver will derive some 1UIP nogood $n \equiv \neg(l_1 \wedge \dots \wedge l_k)$. Theorem II.3.4 tells us that every symmetric version of this nogood $\sigma(n) \equiv \neg(\sigma(l_1) \wedge \dots \wedge \sigma(l_k))$ is also valid. Upon backtracking to the parent subproblem P'' , we examine $\sigma(n)$ for $\sigma \in S$ where S is the symmetry group for the problem. If some term $\sigma(l_i)$ within the nogood is locally false, then the symmetry σ is locally broken in P'' and we throw away $\sigma(n)$. If none of the $\sigma(l_i)$ are false, then the symmetry σ still holds locally in P'' , and we can post $\sigma(n)$ as a constraint.

In contrast to SBDS-DEC, in SBDS-1UIP, it can be useful to post $\sigma(n)$ as global rather than local constraints. In SBDS-DEC, the decision nogood derived for the parent subproblem P'' always subsumed the nogood derived for subproblem P' so we only had to post $\sigma(n)$ locally. However, the 1UIP nogood at P'' does not generally subsume the one at P' , so posting the $\sigma(n)$ globally can potentially give additional pruning.

Example 4.2.1. Consider a simple problem $P = (V, D, C)$, where $V \equiv \{x_1, \dots, x_5\}$, $D \equiv \{x_i \in \{1, \dots, 8\} | i = 1, \dots, 8\}$, $C \equiv \{alldiff([x_1, x_2, x_3, x_4, x_5]), \sum_{i=1}^5 x_i \leq 12\}$. This problem has a variable symmetry between all 5 variables. Suppose we first try $x_1 = 1$. After propagation, we have $x_2 \geq 2$, $x_3 \geq 2$, $x_4 \geq 2$, $x_5 \geq 2$. Suppose we now try $x_2 = 2$. The *alldiff* constraint forces $x_3 \neq 2$, $x_4 \neq 2$, $x_5 \neq 2$, and the domain constraints force $x_3 \geq 3$, $x_4 \geq 3$, $x_5 \geq 3$. The linear constraint then forces $x_3 \leq 3$, $x_4 \leq 3$, $x_5 \leq 3$, but then x_3, x_4, x_5 are all fixed to 3 and the *alldiff* fails.

On backtracking the LCG solver will infer the 1UIP nogood $x_3 \geq 2 \wedge x_4 \geq 2 \wedge x_5 \geq 2 \rightarrow x_2 \neq 2$, which immediately propagates to prune 2 from the domain of x_2 . We can apply any of the variable symmetries to this nogood to derive another valid nogood. For example, swapping x_2 with x_3 gives us the nogood $x_2 \geq 2 \wedge x_4 \geq 2 \wedge x_5 \geq 2 \rightarrow x_3 \neq 2$. This symmetric nogood can immediately propagate to prune 2 from the domain of x_3 as well. Similarly, we can also infer $x_4 \neq 2$, $x_5 \neq 2$.

In the above simple example, the symmetric 1UIP nogoods perform the same pruning as the symmetric decision nogoods. However, we will show in Section 4.4

that there are problems where they provide different amounts of pruning.

4.3 1UIP vs Decision Nogood

The power of SBDS-1UIP is difficult to analyze, because it depends on how strong the propagators are and how they explain their propagations. However, we can show that under some fairly reasonable assumptions about the solver's propagation strength, SBDS-1UIP is at least as powerful as SBDS-DEC. The properties we need are *monotonicity*, and *propagation symmetry*. Let $prop(C, D)$ characterize the propagation engine of our solver, i.e., $prop$ takes the constraints C and the current domain D and returns a new domain D' .

Definition 4.3.1. *The propagation engine $prop(C, D)$ has monotonicity if: if $D' \Rightarrow D$, then $prop(C, D') \Rightarrow prop(C, D)$, i.e., given a smaller domain as input, it returns a domain at least as small.*

Definition 4.3.2. *The propagation engine $prop(C, D)$ has propagation symmetry w.r.t. symmetry group S if: $\forall \sigma \in S, \sigma(prop(C, D)) \Leftrightarrow prop(C, \sigma(D))$, i.e., given a symmetric domain as input, it returns a symmetric domain as output.*

A sufficient condition for monotonicity is that all propagators are monotonic. A sufficient condition for propagation symmetry is that each symmetry maps propagators to symmetric propagators of the same type/strength. Global symmetric monotonicity is therefore very common, as most propagators are monotonic, and the vast majority of symmetries that are usually exploited are propagator symmetric.

Theorem 4.3.3. *Suppose monotonicity and propagation symmetry hold. Let $n_{dec} \equiv \neg(d_1 \wedge \dots \wedge d_k)$ be the decision nogood derived from the failure of the subproblem $P' \equiv (V, D, C \cup \{d_1, \dots, d_k\})$. Let $n_{1uip} \equiv \neg(l_1 \wedge \dots \wedge l_m)$ be the decision nogood derived from the same subproblem P' . Then for any $\sigma \in S$, if a domain D' that is at fixpoint violates $\sigma(n_{dec})$, then it also violates $\sigma(n_{1uip})$.*

Proof. Suppose we have a domain D' such that D' violates $\sigma(n_{dec})$, i.e., all the $\sigma(d_i)$ are true in D' . Recall that in the subproblem P' , the decisions $d_1, \dots, d_k$ were enough

for the propagation engine to infer each of $l_1, \dots, l_m$. By monotonicity and propagation symmetry, since $\sigma(d_i)$ currently all hold in D' , and D' is at fixpoint, $\sigma(l_i)$ must all hold in D' . Hence, D' also violates $\sigma(n_{1uip})$. $\square$

Theorem 4.3.3 tells us that the symmetric versions of the 1UIP nogoods are at least as powerful as the symmetry versions of the decision nogoods in terms of their ability to fail subproblems. In the next section, we show that SBDS-1UIP can be strictly stronger than SBDS-DEC for some problems.

4.4 Beyond Complete Methods

SBDS-DEC is a complete method which guarantees that once we have visited a partial assignment, we will never visit any symmetric version of it. It may seem surprising that we can do better than this, but it is possible. Fundamentally, SBDS, SBDD and lexicographical constraints all use the failure of a valuation θ to infer the failure of its symmetric counterparts $\sigma(\theta)$. This means that they have to visit at least one member of each orbit. However, this is not true for SBDS-1UIP. The 1UIP nogoods used in SBDS-1UIP are powerful enough that we may not actually need to visit any member of some orbits.

Example 4.4.1. Consider the graph coloring problem shown in Figure 4.1, where we are trying to 5 color the graph. We represent the colors of the nodes with the variables $V \equiv \{x_1, \dots, x_{10}\}$, and domain $D \equiv \{x_i \in \{1, \dots, 5\} | i = 1, \dots, 10\}$. For each edge in the graph between node i and node j , we post the constraint $x_i \neq x_j$. Clearly, all 5 colors are interchangeable, so there is a value symmetry on all the variables in V .

Suppose we make the decisions $x_1 = 1, x_2 = 2, x_3 = 3, x_4 = 4, x_5 = 5$. Propagation produces the domains shown in Figure 4.1. Suppose we try $x_6 = 1$ next. Then propagation gives $x_7 \in \{2, 3\}, x_8 \in \{2, 3\}, x_9 \in \{2, 3\}$. Now, suppose we try $x_7 = 2$. This forces $x_8 = 3, x_9 = 3$, which conflicts. The 1UIP nogood from this conflict is $x_8 \neq 1 \wedge x_8 \neq 4 \wedge x_8 \neq 5 \wedge x_9 \neq 1 \wedge x_9 \neq 4 \wedge x_9 \neq 5 \rightarrow x_7 \neq 2$. After propagating this nogood, we have $x_7 = 3$, which after further propagation, once again conflicts. At this point, we backtrack to before x_6 was labeled and derive the

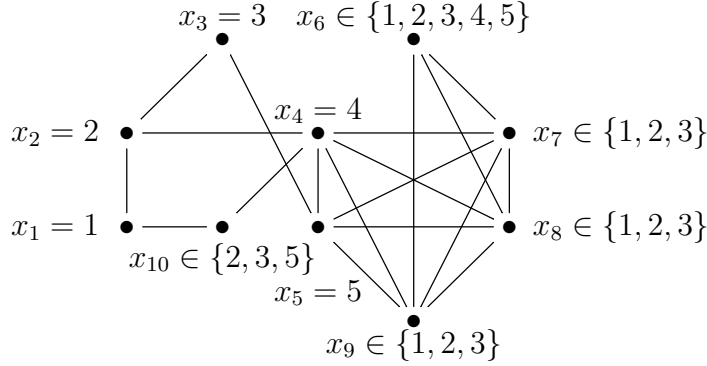


Figure 4.1: A graph coloring problem where we can exploit additional symmetries.

nogood $x_7 \neq 4 \wedge x_7 \neq 5 \wedge x_8 \neq 4 \wedge x_8 \neq 5 \wedge x_9 \neq 4 \wedge x_9 \neq 5 \rightarrow x_6 \neq 1$, which immediately prunes 1 from the domain of x_6 .

Now, let's examine what SBDS-1UIP can do at this point. It is clear that if we swap the values 1 and 2 or swap the values 1 and 3 in this nogood, the LHS remains unchanged while the RHS changes. Therefore, we can post these two symmetric 1UIP nogoods and immediately get the inferences $x_6 \neq 2$ and $x_6 \neq 3$. On the other hand, SBDS-DEC cannot do anything. The decision nogood is $x_1 = 1 \wedge x_2 = 2 \wedge x_3 = 3 \wedge x_4 = 4 \wedge x_5 = 5 \rightarrow x_6 \neq 1$, and it is easy to see since all 5 values are used on the LHS, no matter which value symmetry we use on it, the LHS will have a set of literals incompatible with the current assignment and thus the symmetric nogoods cannot prune anything.

Let us consider whether any of the other symmetry breaking techniques can exploit this symmetry. SBDD is equal in power to SBDS, so it also cannot exploit this symmetry. Lexicographical symmetry breaking constraints cannot exploit this symmetry, since all 5 values have already been used in $x_1, \dots, x_5$ and the constraint is already satisfied. Even conditional symmetry breaking constraints [61] cannot exploit this, as the subproblem does not have value symmetries between 1 and 2 or between

1 and 3 due to the domain of x_{10} .

We note something very interesting. The 1UIP nogood is derived after the failure of subproblem $x_1 = 1, x_2 = 2, x_3 = 3, x_4 = 4, x_5 = 5, x_6 = 1$. The symmetric version of this 1UIP nogood can prune the subproblem $x_1 = 1, x_2 = 2, x_3 = 3, x_4 = 4, x_5 = 5, x_6 = 2$. However, the partial assignments leading to these two subproblems do not actually lie in the same orbit! The valuations in the second subproblem were pruned despite the fact that we have never examined any of their symmetric versions before!

This rather surprising fact is due to the fact that 1UIP nogoods are much more powerful than decision nogoods. Decision nogoods are derived through exhaustive search, and can only prune valuations that we have already examined. Therefore, the symmetric decision nogoods can only prune something for which we have already examined a symmetric version. However, 1UIP nogoods are derived through constraint resolution, and generally prune more valuations than were contained in the failed subproblem from which it was derived. Since 1UIP nogoods can prune valuations that we have never examined before, symmetric versions of 1UIP nogoods can prune valuations for which we have never examined a symmetric version before. Hence, SBDS-1UIP can be strictly stronger than complete methods like SBDS-DEC, SBDD or lexicographical symmetry breaking constraints.

1UIP nogoods are constructed by finding a sufficient set of conditions for the conflict to reoccur, such that the conditions are as “local” to the conflict as possible. This enables SBDS-1UIP to exploit symmetry that exists in the sub-component of a subproblem which is the actual cause of failure. In the subproblem above, the sub-component that caused the failure are the variables x_6, x_7, x_8, x_9 , their current domains in the subproblem, and the constraints linking them. The fact that $x_1, \dots, x_5$ were labeled to values that broke all the symmetries globally did not matter to SBDS-1UIP. It is capable of realizing that symmetries still held locally within x_6, x_7, x_8, x_9 , and can be exploited for additional pruning. It is because LCG solvers give us such precise information about which variables are involved in conflicts that we can exploit this kind of symmetry. We show in our experiments in Section 4.6 that these additional kinds of redundancies can be exploited for more speedup.

4.5 Applying Symmetries to 1UIP Nogoods

SBDS-1UIP is much more powerful than SBDS-DEC. However, having to manipulate 1UIP nogoods raises several problems. In particular, unlike decision nogoods which only involve equality literals on the decision variables, 1UIP nogoods can contain virtually any literal in the problem, i.e., they may include disequality literals, inequality literals, and also literals involving intermediate variables which are introduced by the solver. Intermediate variables can be a rather serious problem, as symmetries are often defined only in terms of the decision variables, and may not properly describe how intermediate variables map to each other.

The original SBDS-DEC method [62] is only capable of handling symmetries which map equality literals to equality literals, i.e., σ such that for any variable x and value v , $\sigma(x = v) \Leftrightarrow x' = v'$ for some variable x' and value v' . This includes most commonly exploited symmetries such as variable symmetries, value symmetries, variable sequence symmetries, value sequence symmetries and the simpler kinds of variable/value symmetries. We extend the SBDS-1UIP method to handle these kinds of symmetries only. We assume that the symmetry declaration gives us enough information to know how equality literals map to each other. We now show how disequality literals, inequality literals and literals on intermediate variables can be handled.

4.5.1 Disequality and Inequality literals

One of the strengths of LCG is the use of disequality and inequality literals in addition to equality literals in explanations and nogoods. This makes many explanations and nogoods much shorter and is essential for explaining bounds propagation. LCG solvers derive 1UIP nogoods of form $n \equiv \neg(l_1 \wedge \dots \wedge l_k)$, where each l_i is an equality, disequality or inequality literal. In order for the symmetric 1UIP nogood $\sigma(n) \equiv \neg(\sigma(l_1) \wedge \dots \wedge \sigma(l_k))$ to be of a form that an LCG solver can handle, we need to express each of the $\sigma(l_k)$ as the conjunction of one or more equality, disequality or inequality literals.

Disequality literals can be handled simply by applying negation to equality literals. If $\sigma(x = v) \Leftrightarrow x' = v'$, then it follows that $\sigma(x \neq v) \Leftrightarrow x' \neq v'$. Inequality literals

are harder to handle. They map simply under variable symmetries but not so simply under value symmetries. For example, let σ be the variable symmetry where we swap x_1 and x_2 . Clearly, $\sigma(x_1 \geq d) \equiv x_2 \geq d$, $\sigma(x_2 \geq d) \equiv x_1 \geq d$, $\sigma(x_1 \leq d) \equiv x_2 \leq d$, $\sigma(x_2 \leq d) \equiv x_1 \leq d$. However, suppose σ is the value symmetry on $x_1, \dots, x_k$ where we swap the values v_1 and v_2 . Without loss of generality, assume $v_1 < v_2$. If $v > v_2$ or $v \leq v_1$, then $x_i \geq v$ simply maps to itself and there is no problem. But if $v_1 < v \leq v_2$, then $\sigma(x_i \geq v)$ is not an inequality literal as it does not allow a contiguous range of values (it allows $x_i = v_1$ and $x_i \geq v$, but disallows $x_i = v_2$).

We can handle this case by converting $x_i \geq v$ into a conjunction of disequality literals before applying the symmetry. If l is a lower bound for x_i 's original domain, then $x_i \geq v \Leftrightarrow x_i \neq l \wedge \dots \wedge x_i \neq v-1$, and $\sigma(x_i \geq v) \equiv \sigma(x_i \neq l) \wedge \dots \wedge \sigma(x_i \neq v-1)$, and we already know how to map disequality literals.

Example 4.5.1. Suppose x has an original domain of $\{1, 2, 3, 4, 5\}$. If σ is a symmetry swapping the values 2 and 4, then $\sigma(x \geq 3) \Leftrightarrow \sigma(x \neq 1) \wedge \sigma(x \neq 2) \Leftrightarrow x \neq 1 \wedge x \neq 4$.

Each inequality literal may potentially map to quite a large number of disequality literals. This can significantly increase the length of $\sigma(n)$ compared to n . However, the impact on performance is not great. Nogoods in LCG solvers are propagated using the two watched literal scheme commonly used in SAT solvers. The cost of this scheme is not linear in the length of the nogood, and increasing the length of the nogood typically does not necessarily increase its propagation cost by much. The only real cost then is the memory to store the longer nogood, but memory is rarely a problem on modern computers.

4.5.2 Intermediate variables

An important problem for manipulating 1UIP nogoods is the fact that intermediate variables may be introduced in the course of converting a high level model to the low level variables and constraints implemented by the solver. For example, a high level model written in the modeling language MiniZinc is first flattened into primitive constraints, with intermediate variables introduced as necessary, and then given to the solver, which may then introduce its own variables, e.g., in global propagators

implemented by decomposition. 1UIP nogoods often contain literals from such intermediate variables. However, if the symmetry declaration was made only in the high level model, it may not specify how literals on such introduced intermediate variables map to each other, and the solver will be unable to derive the symmetric versions of the 1UIP nogoods.

There are two main ways to handle this. Ideally, the user manually extends the symmetry declaration to include any intermediate variables introduced by flattening or by the solver. This is the best solution, but requires the user to do work. Alternatively, we can modify the solver's conflict analysis algorithm so that any literal which is not covered by the symmetry declaration cannot appear in the nogoods it derives.

Example 4.5.2. Consider the Concert Hall problem. In the Concert Hall problem, we have k identical halls, and n orders for renting halls. Each order has a start time, end time and a price. We can choose to accept each order or not. If we accept the order, we must schedule it in one of the k halls. Two accepted orders which overlap in time cannot be scheduled in the same hall. The aim is to maximize the profit we can make. A MiniZinc model for the Concert Hall problem is as follows:

```
int: n;      % number of offers
int: k;      % number of halls

set of int: Offers = 1..n;
array [Offers] of int: start;
array [Offers] of int: end;
array [Offers] of int: price;

% does the pair of offers overlap
array [Offers,Offers] of bool: o = array2d(Offers, Offers,
  [start[i] <= end[j] /\ start[j] <= end[i] | i in Offers, j in Offers]);

array [Offers] of var 1..k+1: x;  % hall to take, k+1 means reject offer
var int: total;                  % total money we can make

constraint forall (i, j in Offers where i < j /\ o[i,j])
  (x[i] = k+1 \/ x[j] = k+1 \/ x[i] != x[j]);
```

```

constraint sum (i in Offers)
  (bool2int(!(x[i] = k+1)) * price[i]) >= total;

solve maximise total;

```

The problem has a value symmetry on all the $x[i]$ variables between values $1, \dots, k$ (but not $k + 1$). Also, if order i and order j has the same start time, end time and price, then there is a variable symmetry between $x[i]$ and $x[j]$.

When this model is flattened into primitive constraints, some intermediate variables will be introduced. The expressions: $x[i] = k + 1$, will be given corresponding variables:

```

array [Offers] of var bool: t;
constraint forall (i in Offers) (t[i] <-> x[i] = k+1);

```

The expressions: $x[i] \neq x[j]$, will be given corresponding variables:

```

array [Offers,Offers] of var bool: r;
constraint forall (i, j in Offers where i < j /\ o[i,j])
  (r[i,j] <-> x[i] != x[j]);

```

The expression: $\text{bool2int}(!(x[i] = k + 1))$, will be given corresponding variables:

```

array [Offers] of var 0..1: b;
constraint forall (i in Offers) (b[i] = bool2int(!t[i]));

```

Suppose we want to extend our symmetry declarations to cover these intermediate variables. Consider the value symmetry between values $1, \dots, k$. Clearly, the $t[i]$ are idempotent under these symmetries, since the value $k + 1$ is unaffected. The $r[i, j]$ are also idempotent under these symmetries although it is less obvious. The $b[i]$ which only depend on the $t[i]$ are similarly idempotent under these symmetries. So in this case, we did not really need to do much work to extend the symmetry. Consider a variable symmetry between $x[i]$ and $x[j]$. Clearly, if we swap $x[i]$ with $x[j]$, then we must also swap $t[i]$ with $t[j]$, and $b[i]$ with $b[j]$. Also, $r[i, k]$ must be swapped with $r[j, k]$ for each $k \neq i, j$. So we have to extend the symmetry declaration to specify these additional mappings.

If getting the user to extend the symmetry declaration is impractical, then we use the second method where we modify the solver to prevent any literals on intermediate variables from appearing in nogoods. We first declare to the solver the set of variables which are actually covered by the symmetry declarations. All other variables are considered intermediate variables. We require that search only make decisions on the variables covered by the symmetry declaration. As described in Chapter 2, the conflict analysis algorithm works as follows. We start with the conflicting nogood. While the nogood has at least two literals from the last decision level, we eliminate the one inferred last by resolving the nogood with the explanation for that literal. We modify this algorithm by adding an additional step. If the nogood contains any literal from an intermediate variable, we also eliminate it by resolving it with the explanation for that literal. This is guaranteed to be possible, because no decisions are made on intermediate variables, so any literal on an intermediate variable cannot be the decision literal and therefore must have an explanation. The conflict analysis will now terminate with a nogood which is free of any literals on the intermediate variables. Since all the literals in the nogood are from variables covered by the symmetry declarations, we know how to calculate the symmetric versions of the nogood.

Example 4.5.3. Suppose the unmodified conflict analysis algorithm derived the nogood $n \equiv \neg(b[1] = 1 \wedge x[2] = 1 \wedge x[3] = 2)$. This nogood has literals on the intermediate variable $b[1]$, so we cannot map it under the symmetries. With the modified conflict analysis algorithm, we would attempt to eliminate the literals on $b[1]$ as well. The explanation for $b[1] = 1$ is: $t[1] = \text{false} \rightarrow b[1] = 1$. The explanation for $t[1] = \text{false}$ is: $x[1] \neq k + 1 \rightarrow t[1] = \text{false}$, so the nogood turns into $n \equiv \neg(x[1] \neq k + 1 \wedge x[2] = 1 \wedge x[3] = 2)$. Now, all literals are covered by the symmetry declaration and we can apply any of our symmetries to it.

4.6 Experiments

We now provide experimental evidence that SBDS-1UIP can be much stronger than SBDS-DEC on some problems, and that symmetry breaking can work well with lazy clause generation. The two problems we will examine are the Concert Hall Scheduling

problem and the Graph Coloring problem. We take the benchmarks used in [102]. The benchmarks are available at <http://www.cmeears.id.au/symmetry/symcache.tar.gz>.

We implemented SBDS in CHUFFED, which is a state of the art lazy clause solver. We run CHUFFED with three different versions of SBDS. The first version is **dec**, where we use symmetric versions of decision nogoods. The second is **1UIP**, where we use symmetric versions of 1UIP nogoods. The third version we call **crippled**, where we use symmetric versions of 1UIP nogoods, but only those nogoods derived from symmetries where **dec** could also exploit the symmetry. We compare against CHUFFED with no symmetry breaking (**none**) and with standard lexicographical symmetry breaking constraints (**static**). Finally, we compare against an implementation of SBDS in [102], which is called Lightweight Dynamic Symmetry Breaking (LDSB). LDSB is implemented on the Eclipse constraint programming platform and was the fastest implementations of dynamic symmetry breaking on the two problems we examine, beating GAP-SBDS [58] and GAP-SBDD [59] by significant margins.

All versions of CHUFFED are run on Xeon Pro 2.4GHz processors. The results for LDSB were run on a Core i7 920 2.67 GHz processor. We group the instances by size, so that the times displayed are the average run times for the instances of each size. A timeout of 600 seconds was used. Instances which timeout are counted as 600 seconds.

The results are shown in Tables 4.1 and 4.2. Eclipse LDSB fails to solve many instances before timeout, and **dec** fails to solve a few instances. **1UIP**, **crippled** and **static** all solve every instance in the benchmarks. In fact, this set of instances, which is of an appropriate size for normal CP solvers, is a bit too easy for lazy clause solvers such as CHUFFED, as is apparent from the run times.

Comparison between **dec** and **1UIP** shows that SBDS-1UIP is superior to SBDS-DEC. Comparison between **crippled** and **1UIP** shows that the additional symmetries that we can only exploit with SBDS-1UIP indeed gives us reduced search and additional speedup. Comparison with **static** shows that dynamic symmetry breaking can be superior to static symmetry breaking on appropriate problems. The comparison with LDSB shows that lazy clause solvers can be much faster than normal CP solvers,

Table 4.1: Comparison of three SBDS implementations in CHUFFED, static symmetry breaking in CHUFFED, and LDSB in Eclipse, on the Concert Hall Scheduling problem

Size	none		1UIP		crippled		dec		static		LDSB	
	Time	Fails	Time	Fails	Time	Fails	Time	Fails	Time	Fails	Time	Nodes
20	259.8	686018	0.04	84	0.05	130	0.07	350	0.05	134	0.29	3283
22	381.5	749462	0.07	181	0.08	299	0.17	1207	0.07	183	0.73	7786
24	576.9	1438509	0.10	275	0.11	316	0.78	3426	0.15	486	2.70	12611
26	483.4	1189930	0.10	282	0.19	677	2.26	5605	0.25	685	2.71	12724
28	530.7	1282797	0.68	1611	1.12	2613	3.64	10530	0.42	1041	9.94	57284
30	581.3	1251980	0.27	761	0.53	2042	19.52	48474	0.52	2300	121.50	722668
32	542.4	936019	0.40	1522	1.01	4845	21.48	65157	1.31	5712	97.90	641071
34	600.0	1039051	1.10	2636	3.22	8761	19.86	48837	1.60	4406	72.73	425718
36	600.0	1223864	1.40	3156	5.02	13606	59.70	131142	2.37	5707	171.14	938439
38	600.0	1027778	1.91	5053	12.56	26556	82.77	178170	3.51	10518	268.05	1211086
40	600.0	1447604	2.96	6648	10.27	27028	102.1	219454	6.40	18169	240.84	1220934

and that they retain this advantage when integrated with symmetry breaking methods. It also show by proxy that SBDS-1UIP is superior to GAP-SBDS or GAP-SBDD on these problems.

The total speed difference between 1UIP and dec is up to 2 orders of magnitude for the Concert Hall problems and up to 4 orders of magnitude for the Graph Coloring problems. Most of this speedup can be explained by the dramatic reduction in search space, which is apparent from the node counts in the results table. The redundancies exploited by lazy clause solvers are different from those redundancies caused by symmetries, and it is very clear here that by exploiting both at the same time with SBDS-1UIP, we get much higher speedups than possible with either of them separately.

It should also be noted that CHUFFED with static symmetry breaking constraints is also reasonably fast. While symmetry breaking constraints cannot exploit the extra redundancies that SBDS-1UIP can, it does have very low overhead and integrates well with lazy clause. Not every problem with symmetry has extra redundancies that SBDS-1UIP can exploit. Often the amount of symmetry actually exploited by the different techniques will be similar, thus SBDS-1UIP will not always gain any additional speedup compared to other techniques.

Table 4.2: Comparison of three SBDS implementations in CHUFFED, static symmetry breaking in CHUFFED, and LDSB in Eclipse, on the Graph Coloring problems

Size	none		1UIP		crippled		dec		static		LDSB	
	Time	Fails	Time	Fails	Time	Fails	Time	Fails	Time	Fails	Time	Nodes
30	140.7	282974	0.00	14	0.06	474	0.26	3049	0.02	277	19.63	56577
32	211.4	390392	0.00	17	0.00	146	0.24	3677	0.00	84	14.11	27178
34	213.9	272772	0.00	25	0.29	1182	3.53	11975	0.03	433	22.06	30127
36	195.9	296358	0.00	36	0.04	467	6.91	23842	0.01	200	35.66	85505
38	224.0	297138	0.00	55	0.04	516	23.55	69480	0.03	526	51.18	107574
40	250.9	423326	0.00	83	0.31	1879	21.07	78918	0.06	878	84.16	185707

Size	none		1UIP		crippled		dec		static		LDSB	
	Time	Fails	Time	Fails	Time	Fails	Time	Fails	Time	Fails	Time	Nodes
20	13.25	39551	0.00	27	0.00	32	0.01	639	0.00	29	0.72	1376
22	11.53	63984	0.00	25	0.00	34	0.02	727	0.00	25	0.16	538
24	66.60	154409	0.00	35	0.00	47	0.07	1992	0.00	32	1.91	2114
26	74.77	277290	0.00	55	0.00	93	0.12	3385	0.00	104	9.56	34210
28	130.5	280649	0.00	62	0.00	84	0.58	6402	0.00	103	9.14	37738
30	267.6	480195	0.00	101	0.01	239	10.48	43835	0.01	359	57.18	215932
32	331.7	600772	0.01	232	0.24	1597	9.98	44216	0.16	1864	110.16	288707
34	219.9	387213	0.20	806	0.40	1946	10.26	47470	0.45	1730	64.14	165943
36	442.6	709888	0.01	317	0.04	857	27.39	113252	0.80	3226	152.62	327472
38	382.5	631403	0.10	798	1.01	5569	31.63	138787	4.12	9413	193.40	508164
40	465.6	531285	0.02	410	0.36	2660	24.68	91847	0.12	2133	196.02	476486

4.7 Conclusion

In this chapter, we showed how SBDS can be modified to use the 1UIP nogoods derived by LCG solvers instead of the usual decision nogoods. We proved that SBDS-1UIP is at least as strong as SBDS-DEC, and showed that it can be strictly stronger on some problems. In particular, it is capable of exploiting symmetries in the subcomponent of a subproblem which actually caused the failure. This kind of redundancy is very difficult to exploit and we are not aware of any other general symmetry breaking method that can exploit it. To implement SBDS-1UIP, we had to solve a number of new issues, such as how to map disequality literals, inequality literals, and literals from intermediate variables under the symmetries. We demonstrated experimentally

that SBDS-1UIP can be orders of magnitude faster than the original SBDS on appropriate problems.

Chapter 5

Exploiting Almost Symmetries

5.1 Introduction

While symmetries occurs frequently in constraint problems, it is more common that constraint problems arising from real world situations contain *almost symmetries*, i.e., symmetries which are broken by a small number of constraints. Such almost symmetries are not well behaved mathematically, and it is difficult to adapt the traditional symmetry breaking techniques like lexicographical constraints, SBDS or SBDD to exploit them.

Almost symmetries are closely related to *conditional symmetries* [61]. In fact, they can be considered two sides of the same coin. There are two kinds of slightly broken symmetry: 1) ones where the symmetry can be restored by *adding* a small number of constraints, and 2) ones where the symmetry can be restored by *removing* a small number of constraints. The first kind is called a conditional symmetry. The second kind is called an almost symmetry. Although conditional symmetries and almost symmetries are conceptually very similar, the methods for exploiting them are significantly different.

Conditional symmetries are relatively well understood and can be exploited very effectively using conditional symmetry breaking constraints [61]. Almost symmetries on the other hand, are not well studied and are much harder to exploit. Previous work on almost symmetries have concentrated on exploiting them by first transforming the original problem P into a related problem P' where the symmetries are restored, and then applying the traditional symmetry breaking techniques to P' [38, 65, 67, 101]. If constraints are removed to restore the symmetry, then the problem P' is less constrained than P and may have a larger search space. A solution of P' may or may not be a solution of P and extra work has to be done to deal with such situations.

We present two new and general ways to exploit almost symmetries. Both methods work natively on P without needing to restore the symmetry. The first method is based on treating almost symmetries as conditional symmetries. We show how to systematically post conditional symmetry breaking constraints to exploit the almost symmetries and discuss their correctness and completeness. The second method is based on modifying SBDS-1UIP, described in the previous chapter, to directly handle

almost symmetries. Both techniques are capable of producing exponential speedups on appropriate problems.

5.2 Definitions

An almost symmetry is a symmetry which holds over most of the constraints in C , but is broken by some small subset $H \subseteq C$. Intuitively, if H is “small” relative to C , then we expect a lot of symmetry left to exploit. Whereas if H is “large” or equal to C , then there is very little symmetry left to exploit. More formally:

Definition 5.2.1. An almost symmetry of a problem $P \equiv (V, D, C)$ is a bijective function $\sigma : \Theta_V \rightarrow \Theta_V$ mapping valuations to valuations such that: $\exists H \subseteq C$ s.t. σ is a symmetry of problem $P' \equiv (V, D, C \setminus H)$, where H is “small” relative to C . We call H a breaking set of σ w.r.t. P .

Definition 5.2.2. An almost symmetry group of problem $P \equiv (V, D, C)$ is a set S of almost symmetries of P such that: $\exists H \subseteq C$, s.t. S is a symmetry group of $P' \equiv (V, D, C \setminus H)$, where H is “small” relative to C . We call H a breaking set of S w.r.t. P .

Example 5.2.3. Consider a simple problem P where $C \equiv x_1 + x_2 + x_3 \geq 3 \wedge x_1 \neq 1 \wedge x_3 \neq 2$. We can consider each of the pairwise variable swaps as almost symmetries. The mapping swapping x_1 and x_2 is an almost symmetry with breaking set $H \equiv \{x_1 \neq 1\}$. The mapping swapping x_1 and x_3 is an almost symmetry with $H \equiv \{x_1 \neq 1, x_3 \neq 2\}$. The mapping swapping x_2 and x_3 is an almost symmetry with $H \equiv \{x_3 \neq 2\}$. The set of all variable permutations forms an almost symmetry group with $H \equiv \{x_1 \neq 1, x_3 \neq 2\}$.

Breaking sets are not necessarily unique. However, there is always a unique, *minimal* breaking set.

Definition 5.2.4. A breaking set H is minimal if no proper subset of it is a breaking set.

Lemma 5.2.5. *Each almost symmetry σ or almost symmetry group S of $P \equiv (V, D, C)$ has a unique minimal breaking set.*

Proof. We prove that the breaking set property is closed under set intersection. Let H_1, H_2 be breaking sets for almost symmetry σ . Let $H_3 \equiv H_1 \cap H_2$. Let $C_i \equiv C \setminus H_i$ for $i = 1, 2, 3$. Then $C_3 \equiv C_1 \wedge C_2$. Since H_1 and H_2 are breaking sets, we have $C_1 \wedge D \Leftrightarrow \sigma(C_1 \wedge D)$, and $C_2 \wedge D \Leftrightarrow \sigma(C_2 \wedge D)$. Then: $(C_3 \wedge D) \Leftrightarrow (C_1 \wedge D) \wedge (C_2 \wedge D) \Leftrightarrow \sigma(C_1 \wedge D) \wedge \sigma(C_2 \wedge D) \Leftrightarrow \sigma(C_3 \wedge D)$. So H_3 is also a breaking set of σ . Since the breaking set property is closed under set intersection, there exists a unique minimal breaking set for σ . The proof for almost symmetry group S is analogous. $\square$

We denote this unique minimal breaking set by $H(\sigma)$ and $H(S)$ for almost symmetry σ and almost symmetry group S respectively. Symmetries are a special case of almost symmetries where $H(\sigma) \equiv \emptyset$ or $H(S) \equiv \emptyset$. All results we derive below apply to symmetries, almost symmetries, or any combination of them in the same problem.

5.3 Almost Symmetry as Conditional Symmetry

Conditional symmetry breaking constraints were first introduced in [61]. The idea is that when certain additional conditions (constraints) are satisfied, the problem may gain additional symmetries which can be exploited.

Definition 5.3.1. *A conditional symmetry of problem $P \equiv (V, D, C)$ is a bijective function $\sigma : \Theta_V \rightarrow \Theta_V$ mapping valuations to valuations such that: $\exists G$, s.t. σ is a symmetry of problem $P' \equiv (V, D, C \cup G)$.*

Definition 5.3.2. *A conditional symmetry group of problem $P \equiv (V, D, C)$, is a set S of bijective mappings such that: $\exists G(S)$, s.t. S is a symmetry group of $P' \equiv (V, D, C \cup G(S))$.*

Example 5.3.3. Consider a simple problem P where $C \equiv x_1 + x_2 + x_3 \geq 3 \wedge x_1 \neq 1 \wedge x_2 \neq 1$. The mapping swapping x_1 and x_2 is an unconditional symmetry. The mapping swapping x_1 and x_3 is a conditional symmetry with $G \equiv \{x_3 \neq 1\}$. The mapping swapping x_2 and x_3 is a conditional symmetry with $G \equiv \{x_3 \neq 1\}$. The set of all variable permutations forms a conditional symmetry group with $H \equiv \{x_3 \neq 1\}$.

Note that the definition of a conditional symmetry and almost symmetry is not mutually exclusive. It's entirely possible that a mapping σ can have its symmetry restored by both adding some constraints and removing some constraints.

Theorem 5.3.4. *Given a conditional symmetry group S , and any lexicographical ordering $<_{lex}$, it is correct to post the conditional symmetry breaking constraint: $csb(S, <_{lex}) \equiv G(S) \rightarrow sb(S, <_{lex})$, where $sb(S, <_{lex})$ is the normal lexicographical symmetry breaking constraint for S .*

Proof. We prove that $csb(S, <_{lex})$ cannot prune all solutions. Suppose $\theta \in solns(P)$ and θ is pruned by $csb(S, <_{lex})$. We show that the lex-least valuation θ' in the orbit of θ under S is also a solution of P and will not be pruned. Let $P' \equiv (V, D, C \cup G(S))$. Since θ was pruned by $csb(S, <_{lex})$, it must satisfy $G(S)$. So $\theta \in solns(P')$. Since S is a symmetry group of P' , $\theta' \in solns(P')$, which means $\theta' \in solns(P)$. Also, θ' is lex-least within its orbit, so it satisfies $sb(S, <_{lex})$ and cannot be pruned by $csb(S, <_{lex})$. $\square$

5.3.1 Breaking a Single Almost Symmetry

We can exploit almost symmetries by considering them as conditional symmetries.

Lemma 5.3.5. *Given an almost symmetry group S with breaking set $H(S)$, S can also be considered a conditional symmetry group with $G(S) \equiv \{\sigma(c) \mid c \in H(S), \sigma \in S\}$.*

Proof. We need to show that S is a symmetry group of $P' \equiv (V, D, C \cup G(S))$. This is true iff: $\forall c \in C \cup G(S), \forall \sigma \in S, C \cup G(S) \Rightarrow \sigma(c)$. We have two cases. Suppose $c \in C \setminus H(S)$. By the definition of an almost symmetry group, $\forall \sigma \in S, C \setminus H(S) \Rightarrow \sigma(c)$, so certainly $\forall \sigma \in S, C \cup G(S) \Rightarrow \sigma(c)$. Suppose $c \notin C \setminus H(S)$, then c must be in $H(S)$ or $G(S)$. However, $H(S) \subseteq G(S)$ by construction, so $c \in G(S)$. Also, $\forall \sigma \in S, \sigma(c) \in G(S)$ since S is a group, so certainly $\forall \sigma \in S, C \cup G(S) \Rightarrow \sigma(c)$. $\square$

Corollary 5.3.6. *Given an almost symmetry group S , it is correct to post the conditional symmetry breaking constraint: $csb(S, <_{lex}) \equiv \bigwedge_{c \in H(S)} \bigwedge_{\sigma \in S} \sigma(c) \rightarrow sb(S, <_{lex})$.*

This $csb(S, <_{lex})$ can be implemented as $|H(S)| \times |S|$ reified constraints, one reified lexicographical symmetry breaking constraint, and a single clausal constraint to link them all. Note that $H(S)$ is generally different for different almost symmetry subgroups S , and we can post a conditional symmetry breaking constraint for each almost symmetry subgroup we wish to exploit.

There are several things we can do to simplify/strengthen this constraint. $csb(S, <_{lex})$ can be rearranged into the form of a nogood: $\neg(a_1 \wedge \dots \wedge a_k)$, where the a_i are either $\sigma(c)$ for some $\sigma \in S, c \in H(S)$, or is $\neg sb(S, <_{lex})$. We can simplify the nogood in the following ways. If $C \Rightarrow \neg a_i$ for some a_i , then the conditional symmetry breaking constraint is redundant and we can throw it away. If $\exists j$ s.t. $C \wedge \bigwedge_{i \neq j} a_i \Rightarrow a_j$, then we can simplify the clause by removing a_j from it. Note that there are some terms we can always simplify away. The identity map I is always in S and $c \Rightarrow I(c)$ for any $c \in C$, so we will remove these terms by default.

Example 5.3.7. Recall the Black Hole Problem, which we described in Example 3.3.1 and gave a MiniZinc model for in Section 3.6.1. We restate the problem for convenience. We seek to find a solution to the Black Hole patience game. In this game the 52 playing cards (of standard 52-card deck) are laid out in 17 piles of 3, with the ace of spades starting in a “black hole”. Each turn, a card at the top of one of the piles can be played into the black hole if it is ± 1 from the card played previously, with king wrapping back around to ace. The aim is to play all 52 cards.

There is an almost symmetry between cards of the same number and different suits. Consider $2\spadesuit$ and $2\heartsuit$. Swapping these two cards ($2\spadesuit \rightleftharpoons 2\heartsuit$) induces a variable swap between $x[2\spadesuit]$ and $x[2\heartsuit]$, and a value swap between $2\spadesuit$ and $2\heartsuit$ on all the y . It can be seen that this is a symmetry of the **inverse** and ± 1 constraints, but is broken by some of the card precedence constraints. Thus we have an almost symmetry group $S \equiv \{I, 2\spadesuit \rightleftharpoons 2\heartsuit\}$.

In an abuse of notation, we will use $2\spadesuit$ to denote $x[2\spadesuit]$, etc. Suppose that in the initial layout, we had piles $5\clubsuit < 2\spadesuit < 7\heartsuit$ and $3\diamondsuit < 2\heartsuit < 9\spadesuit$. Then $H(S) \equiv \{5\clubsuit < 2\spadesuit, 2\spadesuit < 7\heartsuit, 3\diamondsuit < 2\heartsuit, 2\heartsuit < 9\spadesuit\}$ are the only constraints breaking the symmetry. We can construct a conditional symmetry breaking constraint according to Corollary 5.3.6. Suppose we have a lexicographical ordering $<_{lex}$ constructed from

the variable ordering $2\spadesuit <_v 2\heartsuit <_v 2\clubsuit <_v 2\diamondsuit$. Then $sb(S, <_{lex}) \equiv 2\spadesuit < 2\heartsuit$. We now work out $\sigma(c)$ for each $\sigma \in S$, $c \in H(S)$. Recall that all terms with $\sigma = I$ can be simplified away. For $\sigma \equiv 2\spadesuit \Leftrightarrow 2\heartsuit$, $\sigma(5\clubsuit < 2\spadesuit) = 5\clubsuit < 2\heartsuit$, $\sigma(2\spadesuit < 7\heartsuit) = 2\heartsuit < 7\heartsuit$, $\sigma(3\diamondsuit < 2\heartsuit) = 3\diamondsuit < 2\spadesuit$, and $\sigma(2\heartsuit < 9\spadesuit) = 2\spadesuit < 9\spadesuit$. So $csb(S, <_{lex}) \equiv 5\clubsuit < 2\heartsuit \wedge 2\heartsuit < 7\heartsuit \wedge 3\diamondsuit < 2\spadesuit \wedge 2\spadesuit < 9\spadesuit \rightarrow 2\spadesuit < 2\heartsuit$ is a valid conditional symmetry breaking constraint. We can simplify this further by noting that: $3\diamondsuit < 2\heartsuit \wedge 2\heartsuit < 2\spadesuit \rightarrow 3\diamondsuit < 2\spadesuit$, and $2\heartsuit < 2\spadesuit \wedge 2\spadesuit < 7\heartsuit \rightarrow 2\heartsuit < 7\heartsuit$, which means we can remove the second and third condition in the conditional symmetry breaking constraint, giving us: $csb(S, <_{lex}) \equiv 5\clubsuit < 2\heartsuit \wedge 2\spadesuit < 9\spadesuit \rightarrow 2\spadesuit < 2\heartsuit$.

The conditional symmetry breaking constraints we derive here for the Black Hole Problem are identical to those proposed in [61]. However, those authors constructed conditional symmetry breaking constraints for specific problems in an ad-hoc and non-generalizable way, whereas here we describe a systematic way to exploit any almost symmetry with conditional symmetry breaking constraints.

5.3.2 Breaking Multiple Almost Symmetries

It is not always correct to post lexicographical symmetry breaking constraints for multiple symmetries, as they may not be compatible and may end up pruning away all solutions. A sufficient condition for them to be compatible in the full symmetry case is for all of the lexicographical symmetry breaking constraints to use the same lexicographical ordering $<_{lex}$. We show that a similar condition holds for conditional symmetry breaking constraints.

Theorem 5.3.8. *Given constraint problem P with conditional symmetry groups $S_1, \dots, S_k$ and a lexicographical ordering $<_{lex}$, it is correct to post $csb(S_i, <_{lex})$ for each S_i .*

Proof. Suppose θ_0 is a solution of P which is pruned by one of the $csb(S_i)$, say $csb(S_{k_0})$. Then $\theta_0 \notin \text{solns}(sb(S_{k_0}))$, so $\exists \sigma \in S_{k_0}$ s.t. $\sigma(\theta_0)$ is the lex-least solution in the orbit of θ_0 under S_{k_0} . Let $\theta_1 = \sigma(\theta_0)$. We have that $\theta_1 \in \text{solns}(sb(S_{k_0}))$, $\theta_1 \in \text{solns}(C)$ and that $\theta_1 <_{lex} \theta_0$. If θ_1 still violates one of the $csb(S_i)$, say $csb(S_{k_1})$, then we do

the same thing again to get another $\theta_2 = \sigma(\theta_1)$, where $\theta_2 \in \text{solns}(C)$, $\theta_2 <_{\text{lex}} \theta_1$. As long as θ_j violates some $\text{csb}(S_{k_j})$, we can always find another θ_{j+1} which is a solution of P , and which is strictly lex smaller. Since the sequence $\theta_1, \theta_2, \dots$ is monotonically decreasing w.r.t. $<_{\text{lex}}$, and there are only finitely many solutions, it must terminate in a θ_j which is a solution of P and satisfies all of the $\text{csb}(S_i)$. $\square$

From this point on, we will assume that every lexicographical symmetry breaking constraint and conditional symmetry breaking constraint uses the same lexicographical ordering $<_{\text{lex}}$. We will drop the $<_{\text{lex}}$ arguments and write them more simply as $\text{sb}(S)$ and $\text{csb}(S)$ respectively.

5.3.3 Completeness of Conditional Symmetry Breaking Constraints

Recall that a lexicographical symmetry breaking constraint for whole problem symmetry group S is complete iff it prunes all but one solution within each orbit. The situation for almost symmetries is much more complicated. In an almost symmetry group S , the elements of S are not whole problem symmetries. Therefore the orbit of a solution θ under S may contain non-solutions as well as solutions. Such “mixed” orbits are not well behaved mathematically. It is possible to extend the definition of completeness to almost symmetry groups S , i.e., if some orbit O has solutions, then exactly one of these solutions is not pruned. However, this definition of completeness is so strong that achieving completeness probably requires exponential time, largely nullifying any benefit we might gain from symmetry breaking. Instead, we seek incomplete methods which are efficient enough to give us speedup in practice. We show that conditional lexicographical symmetry breaking constraints can achieve a weaker form of completeness which we call *pure orbit completeness*.

Definition 5.3.9. *Given an almost symmetry group S for problem $P \equiv (V, D, C)$, a set of almost symmetry breaking constraints B is pure orbit complete w.r.t. S if: Given any solution $\theta \in \text{solns}(P)$, if $\exists S' \subseteq S$ s.t. 1) $\forall \sigma \in S'$, $\sigma(\theta) \in \text{solns}(P)$, and 2) $\exists \sigma \in S'$ s.t. $\sigma(\theta) <_{\text{lex}} \theta$, then $\theta \notin \text{solns}(B)$, else $\theta \in \text{solns}(B)$.*

Pure orbit completeness requires a subgroup $S' \subseteq S$ under which θ 's orbit is pure (are all solutions) before we prune θ with B . It reduces to the normal definition of completeness if S is a whole problem symmetry group (with $S' = S$).

Lemma 5.3.10. *Given almost symmetry group S for problem $P \equiv (V, D, C)$, if $sb(S)$ is complete w.r.t to S , then $csb(S)$ prunes a solution $\theta \in solns(P)$ iff $\forall \sigma \in S$, $\sigma(\theta) \in solns(P)$, and $\exists \sigma \in S$ s.t. $\sigma(\theta) <_{lex} \theta$.*

Proof. Firstly, $\forall \sigma \in S$, $\sigma(\theta) \in solns(C \wedge D) \Leftrightarrow \forall \sigma \in S$, $\theta \in solns(\sigma(C \wedge D)) \Leftrightarrow \theta \in solns(C \wedge D) \wedge \forall \sigma \in S$, $\forall c \in H(S)$, $\theta \in solns(\sigma(c))$, so θ satisfies the conditional part of $csb(S)$ iff the first condition holds. Secondly, $\exists \sigma \in S$ s.t. $\sigma(\theta) <_{lex} \theta$ iff $\theta \notin sb(S)$, so θ is pruned by the RHS of $csb(S)$ iff the second condition holds. $\square$

Theorem 5.3.11. *Given almost symmetry group S for problem $P \equiv (V, D, C)$, if for every subgroup $S_i \subseteq S$, $sb(S_i)$ is complete w.r.t. S_i , then $B \equiv \{csb(S_i) \mid S_i \subseteq S\}$ is pure orbit complete w.r.t. S , and no stronger.*

Proof. Given any solution $\theta \in solns(P)$, if $\exists S' \subseteq S$ s.t. $\forall \sigma \in S'$, $\sigma(\theta) \in solns(P)$, and $\exists \sigma \in S'$ s.t. $\sigma(\theta) <_{lex} \theta$, then by Lemma 5.3.10, θ will be pruned by $csb(S')$. Conversely, if no such S' exists, then by Lemma 5.3.10, θ cannot be pruned by any of the $csb(S')$. $\square$

Theorem 5.3.11 tells us that the maximum power of this method is exactly pure orbit completeness. Posting $csb(S_i)$ for every almost symmetry subgroup $S_i \subseteq S$ will achieve this. However, there are in general a super-exponential number of such subgroups. We consider whether we can achieve pure orbit completeness by posting only a subset of the $csb(S_i)$.

Theorem 5.3.12. *Given an almost symmetry group S , if S has subgroups $S_1, \dots, S_k$ such that $\bigwedge_{i=1}^k sb(S_i) \Rightarrow sb(S)$, then $\bigwedge_{i=1}^k csb(S_i) \Rightarrow csb(S)$.*

Proof. Since $S_i \subseteq S$, we must have $H(S_i) \subseteq H(S)$ for each S_i .

$$\begin{aligned}
& \bigwedge_{i=1}^k (\bigwedge_{c \in H(S_i)} \bigwedge_{\sigma \in S_i} \sigma(c) \rightarrow sb(S_i)) \\
\Rightarrow & \bigwedge_{i=1}^k (\bigwedge_{c \in H(S)} \bigwedge_{\sigma \in S} \sigma(c) \rightarrow sb(S_i)) \\
\Leftrightarrow & \bigwedge_{c \in H(S)} \bigwedge_{\sigma \in S} \sigma(c) \rightarrow \bigwedge_{i=1}^k sb(S_i) \\
\Rightarrow & \bigwedge_{c \in H(S)} \bigwedge_{\sigma \in S} \sigma(c) \rightarrow sb(S)
\end{aligned}$$

$\square$

Theorem 5.3.12 tells us that some of the $csb(S_i)$ are unnecessary, because they are subsumed by the $csb(S_i)$ of their subgroups. Thus in order to achieve pure orbit completeness w.r.t. S , it is often sufficient to post $csb(S_i)$ for all simple (primitive) symmetry subgroups $S_i \subseteq S$, since the $csb(S_i)$ of composite symmetry subgroups are often subsumed by the $csb(S_i)$ of its simple subgroups. However, the number of simple subgroups is still typically exponential.

Example 5.3.13. Consider the Black Hole Problem from above. Let S_1, S_2, S_3 be the symmetry groups consisting of permutations of $\{2\spadesuit, 2\heartsuit, 2\clubsuit\}$, $\{2\spadesuit, 2\heartsuit\}$ and $\{2\heartsuit, 2\clubsuit\}$ respectively. Let $<_{lex}$ use the variable ordering: $2\spadesuit, 2\heartsuit, 2\clubsuit$. Then Theorem 5.3.12 tells us that $csb(S_1)$ is subsumed by $csb(S_2) \wedge csb(S_3)$.

We try to find an even smaller subset of $csb(S_i)$ which is sufficient for pure orbit completeness.

Theorem 5.3.14. *Given variable almost symmetry group $S \equiv x_1 \rightleftharpoons \dots \rightleftharpoons x_n$, the set of conditional symmetry breaking constraints $B \equiv \bigwedge_{i < j} csb(x_i \rightleftharpoons x_j)$ is pure orbit complete w.r.t. S if: $\forall i < j, \forall c \in H(x_i \rightleftharpoons x_j), |vars(c) \cap vars(S)| = 1$.*

Proof. Let $S' \subseteq S$ be any subgroup of S . Let $orb(x_i) \equiv \{\sigma(x_i) \mid \sigma \in S'\}$ be the orbit of x_i generated by S' . By the properties of a group, if $x_j \in orb(x_i)$ then $orb(x_i) = orb(x_j)$. The orbits partition $\{x_1, \dots, x_n\}$ into disjoint sets. Let $\{x_1, \dots, x_n\} = orb_1 \cup \dots \cup orb_k$. Let $S_{i,j} \equiv x_i \rightleftharpoons x_j$. We show that $\bigwedge_{m=1}^k \bigwedge_{x_i, x_j \in orb_m, x_i \neq x_j} csb(S_{i,j}) \Rightarrow csb(S')$.

Firstly, $\bigwedge_{m=1}^k \bigwedge_{x_i, x_j \in orb_m, x_i \neq x_j} sb(S_{i,j}) \Rightarrow sb(S')$, because any solution to the LHS cannot be improved lexicographically by permutations of variables within each orb_i . Secondly, we show that the conditions for $csb(S_{i,j})$ is a subset of those for $csb(S')$. Consider $c \in H(S_{i,j})$. By assumption, $|vars(c) \cap vars(S)| = 1$. Without loss of generality, suppose $vars(c) \cap vars(S) = \{x_i\}$. Since $x_j \in orb(x_i)$, $\exists \sigma \in S'$ s.t. $\sigma(x_i) = x_j$. Suppose $S_{i,j} \equiv \{I, \sigma'\}$. Then $\sigma'(c) \equiv \sigma(c)$. This is because they both map x_i to x_j , and map every other variable in c maps to themselves since $vars(c) \cap vars(S) = \{x_i\}$. Now, $c \in H(S_{i,j})$, so $C \not\models \sigma'(c) \equiv \sigma(c)$, hence $c \in H(S')$. Thus $\forall c \in H(S_{i,j}), \forall \sigma \in S_{i,j}, \sigma(c)$ is also a condition in $csb(S)$. This, combined with the first result, gives $\bigwedge_{m=1}^k \bigwedge_{x_i, x_j \in orb_m, x_i \neq x_j} csb(S_{i,j}) \Rightarrow csb(S')$. $\square$

Theorem 5.3.14 tells us that under certain circumstances, posting $csb(S_i)$ for all pairwise variable symmetries is sufficient for pure orbit completeness. Similar theorems exist for value symmetries, variable sequence symmetries and value sequence symmetries, as well as for S where S is generated by multiple symmetry groups of the above four types.

Example 5.3.15. Consider the Black Hole Problem from above. Consider the symmetry group S generated by $(2♠, 2♥, 2♣) \rightleftharpoons (2♥, 2♣, 2♠)$, that is simultaneously replacing $2♠$ by $2♥$, $2♥$ by $2♣$, and $2♣$ by $2♠$. This group is simple and Theorem 5.3.12 does not apply. However, if $2♥, 2♣, 2♠$ are in different piles, then the constraints in $H(2♥ \rightleftharpoons 2♣), H(2♣ \rightleftharpoons 2♠), H(2♠ \rightleftharpoons 2♥)$ each involve only one variable in the set $\{2♥, 2♣, 2♠\}$. Thus Theorem 5.3.14 applies and tells us that $csb(S)$ is subsumed by $csb(2♠ \rightleftharpoons 2♥) \wedge csb(2♥ \rightleftharpoons 2♣) \wedge csb(2♣ \rightleftharpoons 2♠)$.

We propose posting only $csb(S)$ for pairwise swaps within each variable, value, variable sequence or value sequence almost symmetry. This is incomplete in general, but in view of Theorem 5.3.14, this may be pure orbit complete or near pure orbit complete for a variety of problems. It also has the benefit that there are typically only a polynomial number of such subgroups, and since $|S|$ is small, they have far fewer conditions than those for larger S and are much more likely to prune something during search.

5.4 Breaking Almost Symmetries with SBDS-1UIP

In the previous chapter, we described a more powerful version of Symmetry Breaking During Search (SBDS) called SBDS-1UIP. Instead of using symmetric versions of the decision nogoods as traditional SBDS does, SBDS-1UIP uses symmetric versions of the 1UIP nogoods derived by Lazy Clause Generation solvers. This allows for the exploitation of new types of symmetries. SBDS-1UIP can be modified to handle almost symmetries relatively easily. In an LCG solver, each nogood n is derived from a certain subset of the constraints $m(n) \subseteq C$, i.e., $m(n) \Rightarrow n$.

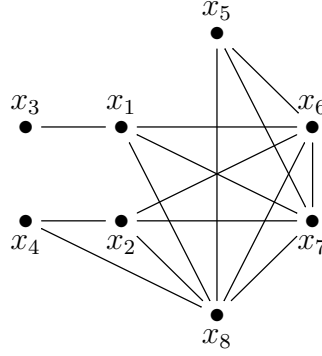


Figure 5.1: A graph coloring problem where we can exploit additional symmetries.

Theorem 5.4.1. *Given almost symmetry σ and nogood n of problem $P \equiv (V, D, C)$, if $m(n)$ and $H(\sigma)$ are disjoint, then $\sigma(n)$ is a valid nogood.*

Proof. $(C \setminus H(\sigma)) \Leftrightarrow \sigma(C \setminus H(\sigma)) \Rightarrow \sigma(m(n)) \Rightarrow \sigma(n) \quad \square$

Theorem 5.4.1 tells us that in order to figure out whether we can apply almost symmetry σ to nogood n to get another valid nogood, we simply need to know $H(\sigma)$ and $m(n)$. $m(n)$ is easy to keep track of. In LCG, new nogoods n are either generated by a constraint c , in which case $m(n) = \{c\}$, or by resolving two nogoods n_1 and n_2 together, in which case $m(n) = m(n_1) \cup m(n_2)$. We use a bit-vector for each nogood to keep track of its $m(n)$. Set union is then implemented as a bit operation. Note that we only need a bit for constraint c if $c \in H(S_i)$ for some S_i we are exploiting. Typically $|H(S_i)|$ is small, as otherwise, the symmetry is too broken for almost symmetry techniques to gain anything anyway. We use the variant of SBDS where we post $\sigma(n)$ iff it can immediately propagate. This is incomplete in general, but is a good trade off between speed and pruning. Completeness for almost symmetries is most likely of exponential complexity in general and is not practical.

Example 5.4.2. Consider the graph coloring problem in Fig 5.1 where we want to 4 color the graph. Normally, if there are two vertices such that they have the exact

same set of neighbors, then they are symmetric (a variable symmetry). However, often the set of neighbors are not identical, but only similar. Consider the variables x_1 and x_2 . They have almost the same set of neighbors, so we can exploit it as an almost symmetry σ where σ swaps x_1 and x_2 , and $H(\sigma) \equiv \{x_1 \neq x_3, x_2 \neq x_4\}$.

Suppose we try $x_5 = 1, x_1 = 2$. Propagation forces $x_6, x_7, x_8 \in \{3, 4\}$ which eventually fails and gives us the 1UIP nogood: $x_6 \neq 1 \wedge x_7 \neq 1 \wedge x_8 \neq 1 \rightarrow x_1 \neq 2$. The set of constraints used to derive this nogood is: $m(n) \equiv \{x_1 \neq x_6, x_1 \neq x_7, x_1 \neq x_8, x_5 \neq x_6, x_5 \neq x_7, x_5 \neq x_8, x_6 \neq x_7, x_6 \neq x_8, x_7 \neq x_8\}$. Clearly, $m(n) \cap H(\sigma) \equiv \emptyset$, thus Theorem 5.4.1 applies and the symmetric nogood $x_6 \neq 1 \wedge x_7 \neq 1 \wedge x_8 \neq 1 \rightarrow x_2 \neq 2$ is also valid, allowing us to immediately prune 2 from x_2 's domain. It is easy to see that $csb(\sigma) \equiv x_1 \neq x_4 \wedge x_2 \neq x_3 \rightarrow x_1 \leq x_2$ would not have made this inference, since the conditions do not hold.

5.5 Related Work

Although there have been several theoretical works on exploiting almost symmetries [65, 67], we are only aware of one working implementation of a general method for exploiting almost symmetries [101]. We describe this method in detail, and compare our new methods against it in the experiments.

One way to exploit almost symmetries is to decompose the problem into two parts P_1 and P_2 , where P_1 contains $C \setminus H(S)$ and is fully symmetric, and P_2 contains $H(S)$. Full symmetry breaking can be applied to P_1 . When a solution of P_1 is found, we try to extend each of its symmetric versions to a solution of P_2 . In [101], the authors propose to implement this by remodeling the problem. They add to the model a “sym var”, whose domain consists of the elements of S . Labeling the sym var chooses which symmetric version of the solution of P_1 to try to extend to P_2 .

More formally, given the global almost symmetry group S for problem $P \equiv (V, D, C)$, we modify P to P' as follows. Let $V_B = vars(C \setminus H(S))$, $V_H = vars(H(S))$ and $V_1 = V_B \setminus V_H$, $V_2 = V_B \cap V_H$ and $V_3 = V_H \setminus V_B$. We add a set of shadow variables V'_2 which is simply a copy of each variable in V_2 . We modify the constraints as follows. For each c in $H(S)$, we replace any occurrence of a variable in V_2 with

its shadow copy in V'_2 . At this point, we have fully decomposed the problem into two parts, $P_1 \equiv (V_1 \cup V_2, D_{V_1 \cup V_2}, C \setminus H(S))$ and $P_2 \equiv (V'_2 \cup V_3, D_{V'_2 \cup V_3}, H(S))$. S is a full symmetry group on P_1 and we can use whatever symmetry breaking approach we desire for S on P_1 .

We now channel P_1 and P_2 into one model as follows. We add a sym var sv , whose domain is S . We post a channeling constraint Q over $V_1 \cup V_2 \cup sv \cup V'_2$ such that: $\theta \in \text{solns}(Q)$ iff if $\theta(sv) = \sigma \in S$, then $\theta_{V'_2} \equiv \sigma(\theta_{V_1 \cup V_2})_{V_2}$, i.e., the channeling constraint forces V'_2 to take on the values that V_2 would take if we had applied σ to this solution of P_1 .

The search proceeds as follows. We first label all the variables in P_1 . If we succeed, we then label the sym var sv , after which the channeling constraint Q forces V'_2 to take on the symmetric values of V_2 . Then we label the remaining variables in P_2 . If we succeed, then the following holds: if $\theta \in \text{solns}(P')$ and $\theta_{sv} = \sigma$, then $\sigma(\theta_{V_1 \cup V_2}) \cup \theta_{V_3} \in \text{solns}(P)$.

Example 5.5.1. Consider a simple graph coloring problem with constraints: $C \equiv \{x_1 \neq x_2, x_2 \neq x_3, x_3 \neq x_1, x_1 \neq 1, x_2 \neq 2\}$. Consider the almost symmetry group $S \equiv x_1 \leftrightarrow x_2 \leftrightarrow x_3$ (saying all three variables are interchangeable). We have $H(S) \equiv \{x_1 \neq 1, x_2 \neq 2\}$. We can construct P' as follows. We add shadow variables x'_1, x'_2 . We modify C to: $C' \equiv \{x_1 \neq x_2, x_2 \neq x_3, x_3 \neq x_1, x'_1 \neq 1, x'_2 \neq 2\}$. We add lexicographical symmetry breaking constraint: $x_1 \leq x_2 \leq x_3$. We decompose the sym var sv into a set of variables and constraints $sv_1, sv_2, sv_3 \in \{1, 2, 3\}, \text{alldiff}([sv_1, sv_2, sv_3])$ so that each solution to sv_i corresponds to a permutation of the variables x_1, x_2, x_3 . We add the channeling constraint $Q \equiv x'_1 = x_{sv_1} \wedge x'_2 = x_{sv_2}$. Consider a solution to P' , e.g., $x_1 = 1, x_2 = 2, x_3 = 3, sv_1 = 3, sv_2 = 1, sv_3 = 2, x'_1 = 3, x'_2 = 1$. Then the corresponding solution to P is $x_1 = 3, x_2 = 1, x_3 = 2$ (by applying the permutation selected by the sv_i).

5.6 Experiments

We compare five methods: 1) ignore all almost symmetries, but exploit full symmetries if any exist using standard lexicographical symmetry breaking constraints (**none**),

2) use conditional symmetry breaking constraints to exploit both symmetries and almost symmetries (**cond-sym**), 3) use SBDS-1UIP to exploit both types of symmetries (**SBDS-1UIP**), 4+5) use the decomposition/remodeling method as described above to exploit both types of symmetries using either 4) lexicographic symmetry breaking (**remodel-lex**) or 5) SBDS-1UIP (**remodel-sbds**). We use the following four problems (all models and instances are available at www.csse.unimelb.edu.au/~pjs/almostsym/).

Black Hole Problem As discussed in Example 5.3.7, the Black hole Problem has a number of variable almost symmetries. We use 100 random instances.

Graph Coloring As discussed in Example 5.4.2, almost variable symmetries arise when pairs of vertices have almost the same set of neighbors. We construct instances as follows. We first randomly partition 50 vertices into 8 sets. For each pair of sets S_i, S_j , we decide with 0.5 probability whether they are densely connected or sparsely connected. If dense, then for each pair of vertices $v_1 \in S_i, v_2 \in S_j$, we add an edge between them with probability 0.99. If sparse, then for each pair of vertices $v_1 \in S_i, v_2 \in S_j$, we add an edge between them with probability 0.01. The end result is that pairs of vertices in the same set $v_1, v_2 \in S_i$ typically have very similar sets of neighbors, with the number of difference varying between about 0 to 3. Roughly 40% of the variable pairs are symmetric (0 difference) while the rest are almost symmetric (> 0 differences), thus the problem contains a mixture of variable symmetries and variable almost symmetries. There is also the standard value symmetry in the problem.

Quasi-Group Completion The Quasi-Group Completion Problem [64] is an instance of the Latin Squares Problem with side constraints of form $x_i = v$. While QCP typically has no symmetries, the Latin Square Problem has many row/column symmetries and value symmetries. We can model these underlying symmetries in QCP as almost symmetries. We try 100 instances of $n = 8$ with 25% of the squares prefilled and find all solutions of each instance.

Concert Hall Problem The Concert Hall Problem [92] normally has value symmetries (representing that all k concert halls are identical). However, more realistically,

Table 5.1: Comparison between ignoring almost symmetries (**none**), exploiting almost symmetries with conditional symmetry (**cond-sym**), SBDS-1UIP (**sbds-1uip**), decomposition using lex (**remodel-lex**) and decomposition using SBDS-1UIP (**remodel-sbds**)

Problem	none		cond-sym		sbds-1uip		remodel-lex		remodel-sbds	
	Time	Fails	Time	Fails	Time	Fails	Time	Fails	Time	Fails
BlackHole	45.8	50427	1.4	4488	21.8	24380	0.1	303	25.7	33481
GraphColor	35.2	73133	28.2	80921	0.5	1488	51.4	179837	129.2	598644
QCP	252.5	4392785	-	-	17.7	279118	300	2497296	300	1137630
ConcertHall	257.1	600190	43.23	86786	224.7	463213	11.9	24310	1.1	1866

halls may vary in size, equipment, location, etc. As a result, it is possible that for each concert only a certain subset of the halls are acceptable, corresponding to side constraints of form $x_i \neq v$. This creates almost value symmetries and almost variable symmetries. We use randomly generated instances following [92], with an additional 50 random side constraints of form $x_i \neq v$.

We use the state of the art solver **CHUFFED**, where SBDS-1UIP has previously been implemented. We modify this implementation to support almost symmetries as described in Section 5.4. To exploit almost symmetries using conditional symmetry breaking constraints, we post the additional constraints $csb(S)$ for each pairwise swap in each variable, value, variable sequence or value sequence symmetry, as proposed in Section 5.3. **CHUFFED** was run on Xeon Pro 2.4GHz processors with a timeout of 300 seconds. Table 5.1 shows average fails and times on each set of instances, with timeouts counting as 300 seconds.

It can be seen from Table 5.1 that all three methods can produce significant speedups on at least some of the problems compared to the baseline of **none**. SBDS-1UIP is fastest on Graph Coloring and QCP, while **remodel-lex** is fastest on Black Hole and **remodel-sbds** is fastest on Concert Hall. The different techniques are effective on different problems. Their effectiveness appears to be very sensitive to the structure of the problem and the types of constraints in them, so that one method that performs well on one problem may perform very poorly on a different problem.

Conditional symmetry breaking constraints tend to be effective if the partial assignments often happen to restore the broken symmetry, i.e., satisfy the symmetric

versions of $H(S)$. It also depends on how high in the search tree the condition becomes satisfied. We get good speedup for Black Hole and Concert Hall, and much less for Graph Coloring. For QCP, the symmetric versions of $H(S)$ for pairwise row swaps, pairwise column swaps, or pairwise value swaps never hold, meaning that during simplification, we would have thrown away all the $csb(S)$ as being redundant, so the method does nothing.

SBDS-1UIP tends to be effective if the $H(S)$ are not often directly involved in the conflict, e.g., because it is a weak constraint (Graph Coloring), or because it becomes satisfied by the partial assignment, or because it has already propagated at a higher decision level (QCP). This is the only method that could get any speedup on the QCP.

The decomposition method is effective if $H(S)$ is weak, so that not too much information is lost in the relaxation $C \setminus H(S)$. This is true for Black Hole and Concert Hall, thus we get very good speedups. On the other hand, if $H(S)$ contains strong constraints that prune many solutions of $C \setminus H(S)$, then the decomposition severely weakens the model, potentially significantly increasing the run time (Graph Coloring, QCP).

5.7 Conclusion

We have defined almost symmetries, and presented two new general techniques for exploiting them. We described how to systematically post conditional symmetry breaking constraints to exploit almost symmetries, and proved several theorems concerning their correctness and completeness. We also described how SBDS-1UIP can be modified to exploit almost symmetries. Our experimental evaluation showed that both techniques can give significant speedup on appropriate problems.

Part III

Relaxation and Dominance

III.1 Introduction to Relaxations

There are a variety of techniques in combinatorial optimization based on *problem relaxations*. Relaxations transform a problem P into a related but easier problem P' , such that every solution in P corresponds to a solution in P' . There are two main ways in which we might want to use relaxations.

In the first situation, we have some problem P which is over-constrained or is simply too hard. We may be willing to settle for some approximate solution of P that is “good enough” according to some criteria. We therefore relax the problem P into an easier to satisfy problem P' and solve P' instead. For example, some of the constraints arising in real life problems may not absolutely have to be satisfied, but may merely express preferences of various strengths. We may be willing to violate some of the less important ones in order to get a solution, rather than to have no solution at all. Techniques like soft constraints [16, 49], valued constraints, etc, can be used to handle such problems.

In the second situation, we have a problem P which we wish to solve exactly. We try to find a relaxed problem P' such that P' can be solved very quickly, and such that solving P' gives us useful information on how to solve P . Such information may include candidate solutions, nogoods, bounds, etc. Techniques of this kind include linear relaxations [137, 156], Lagrangian relaxations [46, 111, 95], and other problem specific relaxations (e.g., [30, 28]). We describe such techniques in further detail below.

III.2 Definitions

We state the definitions and theorems for optimization problems only. The corresponding results for satisfaction problems can be found by setting all objective functions f to $f(\theta) = 0$ for all θ . The types of relaxation we are interested in can be defined as follows:

Definition III.2.1. *Given a problem $P \equiv (V, D, C, f)$ where f is to be minimized, a problem $P' \equiv (V', D', C', f')$ is a relaxation of P under function $\phi : \Theta_V \rightarrow \Theta_{V'}$ if:*

$\forall \theta \in \text{solns}(P), \phi(\theta) \in \text{solns}(P')$ and $f'(\phi(\theta)) \leq f(\theta)$, i.e., ϕ maps each solution of P to a solution of P' that has an objective value at least as good.

The following theorem follows trivially from the above definition.

Theorem III.2.2. *If P' is a relaxation of P , then the following hold:*

- *If P' has no solutions, then P has no solutions*
- *If b is a lower bound on the objective value of P' , then b is a lower bound on the objective value of P*

We describe a relaxation as *tight* or *loose* depending on how different P' is compared to P . For satisfaction problems, a relaxation P' of P is *tight* if for most solutions θ' of P' , there exists a solution θ of P such that $\phi(\theta) = \theta'$, i.e., the relaxation does not introduce many extra solutions. Otherwise, we say that it is *loose*. For optimization problems, a relaxation P' of P is *tight* if $\min\{f(\theta) | \theta \in \text{solns}(P)\} - \min\{f'(\theta) | \theta \in \text{solns}(P')\}$ is small, i.e., the relaxation does not change the optimal value by much. Otherwise, we say that it is *loose*.

The tighter the relaxation is, the more useful the nogoods and bounds derived in P' will be. Also, if the relaxation is tight, then the solutions of P' are very good candidates for solutions of P , and we may be able to find solutions to P much faster by solving P' instead. Much of the difficulty in applying relaxations is to find relaxed problems P' which are tight, but which can also be solved very quickly.

III.3 Types of Relaxations

III.3.1 Simple relaxations

The easiest way to relax a problem is to simply weaken or remove some of its constraints. Given a problem $P \equiv (V, D, C, f)$, we can relax it by replacing C with C' where $C \Rightarrow C'$, or by replacing D with D' where $D \Rightarrow D'$. We can also relax a problem by removing one or more of its variables and projecting the constraints onto the remaining variables $U \subset V$. That is, we define $P' \equiv (U, D_U, C', f')$ where

$C' \equiv \{\bar{\exists}_U.c \mid c \in C\}$, $f'(\theta) = \min\{f(\theta \cup \theta') \mid \theta' \in \Theta_{V \setminus U}\}$ and $\phi(\theta) = \theta_U$. Clearly, if θ is a solution of P , then θ_U is a solution of P' , and by construction, $f'(\theta_U) \leq f(\theta)$. In Chapter 8, we use this type of relaxation to exponentially speedup the optimality proofs for the Maximum Density Still Life Problem.

III.3.2 Lagrangian relaxations

A somewhat more complicated way to relax a problem is the Lagrangian relaxation [46, 111, 95]. In Lagrangian relaxations, we relax the problem $P \equiv (V, D, C, f)$ by moving one or more of the constraints into the objective function as “penalty” terms describing whether the constraint is violated. For each constraint c that we want to move, we define a penalty function $r_c : \Theta_V \rightarrow \mathbb{R}$ mapping valuations to the reals such that if θ satisfies c , then $r_c(\theta) = 0$ (no penalty). If G is the set of constraints to be moved, then the relaxed problem is $P' \equiv (V, D, C \setminus G, f')$, where $f' = f + \sum_{c \in G} r_c$, and ϕ is the identity function. Clearly, if θ is a solution of P , then it is also a solution of P' , and by definition, $\forall c \in G, r_c(\theta) = 0$, so $f(\theta) = f'(\theta)$.

Lagrangian relaxations can be useful when the problem P has a set of “core” constraints which are actually quite easy to solve, but P has been made harder by the presence of some “side” constraints. By moving these side constraints into the objective, we recover the easy to solve core, which can be quickly solved to give us a bound on the objective for the original problem. Lagrangian relaxations have been used effectively in many problems (e.g., [41, 70, 140, 12]).

III.3.3 Linear relaxations

One of the most useful relaxations is the linear relaxation [137, 156]. The problem is relaxed into a Linear Programming (LP) problem, which can then be solved very efficiently using LP methods like the simplex method [110]. In a linear relaxation, we relax the problem $P \equiv (V, D, C)$ by removing the integrality constraints on the variables and linearizing all the constraints. That is, each integer variable $v \in V$ is replaced by a continuous variable v' with the same upper and lower bounds, and each constraint $c \in C$ is replaced by a set of linear constraint $\{l_1, \dots, l_m\}$ such that $c \Rightarrow l_i$

for each l_i .

There are many ways to utilize linear relaxations. For example, Mixed Integer Programming (MIP) solvers [34, 137] use linear relaxations at every subproblem in the search tree to derive bounds which may fail the subproblem. Various LP/CP hybrids have been proposed in order to take advantage of both the strength of CP propagation and the bounds provided by the linear relaxations [129, 69, 80]. Linear relaxations can also be used by individual propagators to make more powerful inferences or to make inferences faster, e.g., in global propagators for bin-packing [21], cumulative [75], all-different [73, 154], element [74, 148] and others.

III.4 Introduction to Dominance

Dominance is a powerful concept that underlies many of the major techniques discussed so far. Symmetry breaking, relaxations, and many forms of nogood learning are applications of dominance. The basic idea behind dominance is to find pairs of objects (valuations, subproblems, problems) where one is better than (or at least as good as) the other with respect to what we are trying to optimize, e.g., one satisfies the constraints while the other does not, one has a better objective value than the other. Finding and exploiting such *dominance relations* [87, 78, 2, 76] can give us exponential speedups on appropriate problems. In this section, we define dominance relations and describe how they can be exploited.

We state the definitions and theorems for optimization problems only. The corresponding results for satisfaction problems can be found by setting all objective functions f to $f(\theta) = 0$ for all θ . We first define dominance relations over valuations.

Definition III.4.1. *Given a problem $P \equiv (V, D, C, f)$ where f is to be minimized, a dominance relation $\preceq$ over Θ_V is a reflexive, transitive binary relation on Θ_V such that: if $\theta_1 \preceq \theta_2$ and $\theta_2 \in \text{solns}(P)$, then $\theta_1 \in \text{solns}(P)$ and $f(\theta_1) \leq f(\theta_2)$.*

Definition III.4.2. *If $\theta_1 \preceq \theta_2$, but $\theta_2 \not\preceq \theta_1$, then we write $\theta_1 \prec \theta_2$.*

If $\theta_1 \preceq \theta_2$, we say that θ_1 *dominates* θ_2 and that θ_2 is *dominated by* θ_1 , where “dominates” means “at least as good as”. If $\theta_1 \prec \theta_2$, we say that θ_1 *strictly dominates* θ_2 , where “strictly dominates” means “strictly better than”. There are two main ways to exploit a dominance relation, depending on whether a dominance is strict or not. We have the following two theorems.

Theorem III.4.3. *Given a problem $P \equiv (V, D, C, f)$, and a dominance relation $\preceq$ over Θ_V . If $\theta_1 \preceq \theta_2$, and we have searched θ_1 , then we can prune θ_2 without changing the satisfiability or optimal value of P .*

Proof. Follows trivially from Definition III.4.1.

Theorem III.4.4. *Given a finite domain problem $P \equiv (V, D, C)$, and a dominance relation $\preceq$ over Θ_V , we can prune all valuations θ such that $\exists \theta' \in \Theta_V$ s.t. $\theta' \prec \theta$, without changing the satisfiability or optimal value of P .*

Proof. Let θ_0 be an optimal solution. If θ_0 is pruned, then there exists some solution θ_1 s.t. $\theta_1 \prec \theta_0$. Then $obj(\theta_1) \leq obj(\theta_0)$, so θ_1 is also an optimal solution. In general, if θ_i is pruned, then there must exist some θ_{i+1} s.t. $\theta_{i+1} \prec \theta_i$ and θ_{i+1} is also an optimal solution. It is impossible for the sequence $\theta_0, \theta_1, \dots$ to repeat, since if $\theta_i \equiv \theta_j$ for some $i < j$, then we must have $\theta_{i+1} \prec \theta_i$ and $\theta_i \prec \theta_{i+1}$, which violates the definition of $\prec$. Since there are finitely many optimal solutions, the sequence $\theta_0, \theta_1, \dots$ must terminate in some θ_i that is an optimal solution and is not pruned.

We will call the pruning allowed by Theorem III.4.3 and Theorem III.4.4 *dominance breaking*. It is easy to see that symmetry breaking is a special case of dominance breaking.

Definition III.4.5. We define the dominance breaking constraint $db(\preceq)$ for dominance relation $\preceq$ as: $solns(db(\preceq)) \equiv \{\theta \mid \theta \in \Theta_V, \nexists \theta' \in \Theta_V \text{ s.t. } \theta' \prec \theta\}$.

Theorem III.4.6. Let S be a symmetry group for problem P . Let $<_{lex}$ be a global lexicographical order on the valuations of P . Let $sb(S, <_{lex})$ be a complete lexicographical symmetry breaking constraint as defined in Definition II.2.2. Let $\preceq$ be a dominance relation defined as follows: $\theta_1 \preceq \theta_2$ iff $\exists \sigma \in S$ s.t. $\sigma(\theta_1) = \theta_2$ and $\theta_1 <_{lex} \theta_2$. Then $db(\preceq) \Leftrightarrow sb(S, <_{lex})$.

Proof. Both $sb(S, <_{lex})$ and $db(\preceq)$ prune exactly those valuations that are not lex-least in their orbits. $\square$

Similarly, dynamic symmetry breaking is an application of Theorem III.4.3, where we define $\theta \preceq \theta'$ iff $\exists \sigma \in S$ s.t. $\sigma(\theta) = \theta'$.

Dominance relations can also be defined over subproblems.

Definition III.4.7. Given a problem $P \equiv (V, D, C, f)$ where f is to be minimized, a dominance relation $\preceq$ over the subproblems of P is a reflexive, transitive binary relation on the subproblems of P such that: if $P_1 \preceq P_2$, then $\min\{f(\theta) \mid \theta \in solns(P_1)\} \leq \min\{f(\theta) \mid \theta \in solns(P_2)\}$.

Theorem III.4.8. If $P_1 \preceq P_2$, and we have examined P_1 , then we can prune P_2 without changing the satisfiability or optimal value of P .

Theorem III.4.9. *Given a finite domain problem $P \equiv (V, D, C)$, and a dominance relation $\preceq$ over a set of disjoint subproblems of P (i.e., no two subproblems share any solutions), we can prune all subproblems P' such that $\exists P''$ s.t. $P'' \prec P'$, without changing the satisfiability or optimal value of P .*

Dominance relations can be found in many problems, allowing us to prune many parts of the search space without having to examine it (e.g., [3, 159, 28, 54, 108, 116]). They can be very powerful in optimization problems, because they provide a completely different and complementary kind of pruning to the branch and bound paradigm. In the branch and bound paradigm, the only way to show that a subproblem P' is suboptimal is to prove a sufficiently strong bound on its objective value. Proving such bounds can be very expensive, especially if the model does not propagate strong bounds on the objective. In the worse case, further search is required, which can take an exponential amount of time. On the other hand, dominance relations can prune a subproblem P' without having to prove any bounds on its objective value at all. In many problems, there are parts of the search space that are obviously suboptimal, and which can be proved to be suboptimal using simple logical arguments. Once dominance relations expressing these are found and proved, the only cost in the algorithm is to check whether a subproblem is dominated, which is typically much lower than that required to prove a sufficiently strong bound to prune the subproblem.

III.5 Contributions

In Chapter 6, we apply relaxation and dominance techniques to the Minimization of Open Stacks Problem [162] (MOSP). We describe 4 different dominance relations, which when exploited, gives around 5-6 orders of magnitude speedup on the hardest instances. We also apply a relaxation technique to the MOSP, where we relax the problem by merging customers together. The relaxation technique gives around 3-4 orders of magnitude speedup on the hardest instances.

In Chapter 7, we apply bounded dynamic programming, relaxation and dominance techniques on the Talent Scheduling Problem (prob039 in CSPLib). Bounded

dynamic programming is a variant of dynamic programming that is much more suitable for optimization problems. Applying it gives some 1-2 orders of magnitude speedup compared to a simple dynamic programming approach. We prove 2 dominance relations, which when exploited, gives a modest ~ 2 times speedup. We also apply relaxation and dominance techniques to derive lower bounds on cost for subproblems. This stronger lower bound significantly reduces the size of the search space, giving a further 1 order of magnitude speedup.

In Chapter 8, we examine the Maximum Density Still Life Problem (prob032 in CSPLib). The Maximum Density Still Life Problem is a very hard combinatorial optimization problem with a raw search space of $O(2^{n^2})$. The previous best approach is only able to solve up to $n = 20$. By using a combination of remodeling, relaxations, bounded dynamic programming and customized search, guided by deep mathematical insights into the problem, we are able to solve the problem for all n . As a comparison, our new approach can solve $n = 100$ in only a matter of hours, whereas the previous best approach would require in the order of 10^{100} years and 10^{30} bytes of memory to solve $n = 100$. It is possible to solve the Still Life problem for all n , because the Still Life Problem becomes well behaved mathematically for sufficiently large n (around $n > 200$), and it is possible to solve all the large cases in constant time.

Chapter 6

Minimization of Open Stacks Problem

6.1 Introduction

In this chapter, we describe a new exact solver for the *Minimization of Open Stacks Problem* (MOSP) [162]. By combining nogood learning and dominance breaking with a branch and bound strategy, our solver is able to solve hard instances of MOSP some 5-6 orders of magnitude faster than the previous state of the art. We also show how relaxation techniques can be used to further speed up the proof of optimality by up to another 3-4 orders of magnitude on the hardest instances.

The MOSP can be described as follows. A factory manufactures a number of different products in batches, i.e., all copies of a given product need to be finished before a different product is manufactured, so there are never two batches of the same product. Each customer of the factory places an order requiring one or more different products. Once one product in a customer's order starts being manufactured, a stack is opened for that customer to store all the products in the order. Once all the products for a particular customer have been manufactured, the order can be sent and the stack is freed for use by another order. The aim is to determine the sequence in which products should be manufactured in order to minimize the maximum number of open stacks, i.e., the maximum number of customers whose orders are simultaneously active. The importance of this problem comes from the variety of real situations in which the problem (or an equivalent version of it) arises, such as cutting, packing, and manufacturing environments, or VLSI design. Indeed the problem appears in many different guises in the literature, including: graph pathwidth and gate matrix layout (see [97] for a list of 12 equivalent problems). The problem is known to be NP-hard [97].

We can formalize the problem as follows. Let P be a set of products, C a set of customers, and $c(p)$ a function that returns the set of customers who have ordered product $p \in P$. Since the products ordered by each customer $c \in C$ are placed in a stack different from that of any other customer, we use c to denote both a client and its associated stack. We say that customer c is active (or that stack c is open) at time k in the manufacturing sequence if there is a product required by c that is manufactured before or at time k , and a product required by c that is manufactured

	p_1	p_2	p_3	p_4	p_5	p_6	p_7
c_1	X	.	.	.	X	.	X
c_2	X	.	.	X	.	.	.
c_3	.	X	.	X	.	X	.
c_4	.	.	X	X	.	X	X
c_5	.	.	X	.	X	.	.

(a)

	p_7	p_6	p_5	p_4	p_3	p_2	p_1
c_1	X	–	X	–	–	–	X
c_2	.	.	.	X	–	–	X
c_3	.	X	–	X	–	X	.
c_4	X	X	–	X	X	.	.
c_5	.	.	X	–	X	.	.

(b)

Figure 6.1: (a) An example $c(p)$ function: $c_i \in c(p_j)$ if the row for c_i in column p_j has an X. (b) An example schedule: c_i is active when product p_j is scheduled if the row for c_i in column p_j has an X or a –.

at or after time k . In other words, c is active from the time the first product ordered by c is manufactured until the last product ordered by c is manufactured. The MOSP aims at finding a schedule for manufacturing the products in P (i.e., a permutation of the products) that minimizes the maximum number of active customers (or of open stacks) at any time. We call a problem with n customers and m products an $n \times m$ problem.

Example 6.1.1. Consider a 5×7 MOSP defined by the set of customers $C \equiv \{c_1, c_2, c_3, c_4, c_5\}$, the set of products $P \equiv \{p_1, p_2, p_3, p_4, p_5, p_6, p_7\}$, and a $c(p)$ function determined by the matrix M shown in Figure 6.1(a), where an X at position M_{ij} indicates that client c_i has ordered product p_j .

Consider the manufacturing schedule given by sequence $[p_7, p_6, p_5, p_4, p_3, p_2, p_1]$ and illustrated by the matrix M shown in Figure 6.1(b), where client c_i is active at position M_{ij} if the position contains either an X (p_j is in the stack) or an – (c_i has an open stack waiting for some product scheduled after p_j). Then, the active customers at time 1 are $\{c_1, c_4\}$, at time 2 $\{c_1, c_3, c_4\}$, at time 3 $\{c_1, c_3, c_4, c_5\}$, at time 4 $\{c_1, c_2, c_3, c_4, c_5\}$, at time 5 $\{c_1, c_2, c_3, c_4, c_5\}$, at time 6 $\{c_1, c_2, c_3\}$, and at time 7 $\{c_1, c_2\}$. The maximum number of open stacks for this particular schedule is thus 5.

The MOSP was chosen as the subject of the first Constraint Modeling Challenge [146] posed in May 2005. Many different techniques were explored in the 13 entries to the challenge. The winning entry by Garcia de la Banda and Stuckey [53] used a dynamic programming approach in a branch and bound framework to solve

the MOSP. The natural search strategy of labeling the products in chronological order leads to many subproblem equivalences. These subproblem equivalences can be exploited using dynamic programming [53] or nogood learning [141]. Also, various lookaheads can be used to calculate upper bounds on the number of open stacks so that suboptimal branches can be pruned earlier. These techniques led to a solver that was orders of magnitude faster than any of the other entries in the 2005 MOSP challenge.

The search strategy used in this winning entry (branching on which product to produce next), is actually far from optimal. As was first discussed in [157] and shown in [158], branching on which customer stack to close next is never worse than branching on which product to produce next, and is usually much better, even when the number of customers is far greater than the number of products. This is the result of a simple dominance relation. In this chapter, we show that combining this search strategy with nogood recording produces a solver that is some 5-6 orders of magnitude faster than the winning entry to the Modeling Challenge on hard instances. We also derive several other dominance rules that provide a further 1-2 orders of magnitude improvement. One rule in particular largely subsumes the effect of the nogood recording. This allows us to reduce the memory usage from a potentially exponential amount to a constant $\sim 2\text{Mb}$ for even the largest solvable instances. We also utilize relaxation techniques to speed up the proof of optimality for the hardest instances by a further 3-4 orders of magnitude. With all the improvements, our solver is able to solve all the open instances from the Modeling Challenge within 10 seconds!

6.2 Searching on Customers

We use a branch and bound search strategy based on branching on which customer to close next, rather than on which product to produce next. This customer based search strategy has been discussed in [157, 155] and is based on the following dominance relation:

Proposition 6.2.1. *Given a product order $U \equiv [p_1, p_2, \dots, p_n]$, define a customer close order $T \equiv [c_1, c_2, \dots, c_m]$ as the order in which customer stacks can close given*

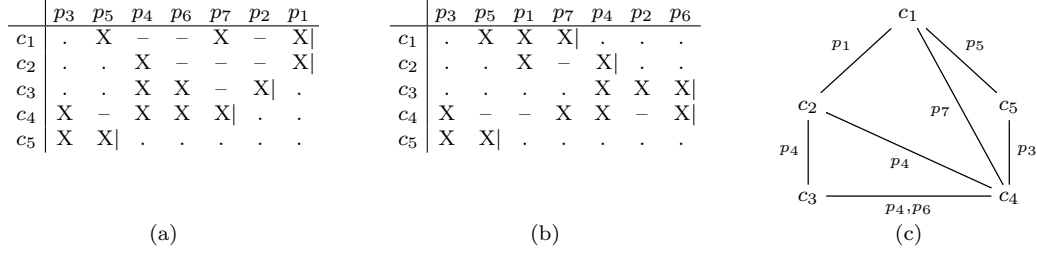


Figure 6.2: (a) A schedule corresponding to customer order $[c_5, c_4, c_3, c_2, c_1]$. (b) A schedule corresponding to customer order $[c_5, c_1, c_2, c_3, c_4]$. (c) The customer graph (ignoring self edges) with edges labeled by the products that generate them.

U. Construct another product ordering U' as follows: schedule the products required by c_1 , then the products required by c_2 , then the products required by c_3 , etc. Then U' uses no more stacks than U .

This dominance relation is proven in [157] and [155], and shows that it is sufficient to examine only product orderings where every product is required for the next customer's stack to close. This dramatically reduces the number of product orderings that have to be examined. We can implement this by branching on which customer's stack to close next, and then scheduling exactly those products which are needed.

Example 6.2.2. Consider the schedule U shown in Figure 6.1(b), the customers are closed in the order $\{c_4, c_5\}$ when p_3 is scheduled, then $\{c_3\}$ when p_2 is scheduled, then $\{c_2, c_1\}$ when p_1 is scheduled. Consider closing the customers in the order $T \equiv [c_5, c_4, c_3, c_2, c_1]$ compatible with U . This leads to a product ordering, for example, of $U' \equiv [p_3, p_5, p_4, p_6, p_7, p_2, p_1]$. The resulting scheduling is shown in Figure 6.2(a). It only requires 4 stacks (and all other schedules with this closing order will use the same maximum number of open stacks).

Rather than thinking in terms of products, it is simpler to think of the MOSP purely in terms of the customers. We define the *customer graph* as follows:

Definition 6.2.3. Let the customer graph $G \equiv (V, E)$ for an instance of the MOSP be: $V = C$ and $E = \{(c_1, c_2) \mid \exists p \in P, \{c_1, c_2\} \subseteq c(p)\}$, i.e., a graph in which nodes

represent customers, and two nodes are adjacent if their customers require at least one product in common.

Note that, by definition, each node is self-adjacent. The customer graph for the problem of Example 6.1.1 is shown in Figure 6.2(c). It is easy to see that any two instances of the MOSP with the same customer graph must have the same minimum number of open stacks. Thus an instance of the MOSP is uniquely specified by its customer graph. The sole effect of the products are to create edges in the customer graph. A MiniZinc model for the MOSP using the customer graph and customer search is given in Example 3.3.3. If a customer close order satisfying these constraints are found, then a corresponding product ordering using the same number of stacks can trivially be constructed. We show that the subproblem equivalences exploited in [53] still apply to the customer search and can be exploited in the same way.

Theorem 6.2.4. *Suppose we have some partial assignment $x[1] = v_1, \dots, x[m] = v_m$, and it does not violate the stack constraints from time 1 to time m . Then there exists solutions mapping the set $\{x[1], \dots, x[m]\}$ to $\{v_1, \dots, v_m\}$ iff there exists solutions extending $x[1] = v_1, \dots, x[m] = v_m$.*

Proof. The “if” direction of the proof is trivial, so we only prove the “only if” direction. Suppose there exists a solution θ' mapping the set $\{x[1], \dots, x[m]\}$ to $\{v_1, \dots, v_m\}$, say $\theta' \equiv x[1] = v'_1, \dots, x[m] = v'_m, x[m+1] = v'_{m+1}, \dots, x[c] = v'_c$, where $v'_1, \dots, v'_m$ is some permutation of $v_1, \dots, v_m$. We show that $\theta \equiv x[1] = v_1, \dots, x[m] = v_m, x[m+1] = v'_{m+1}, \dots, x[c] = v'_c$ is also a solution.

Firstly, by assumption, the partial assignment $x[1] = v_1, \dots, x[m] = v_m$ does not violate the stack constraints from time 1 to time m . It is also easy to see that the values of $x[m+1], \dots, x[c]$ cannot cause the stack constraints from time 1 to time m to be violated, as their only effect is to force some stacks to open at time $m+1$ or later via a $s[i] \leq e[j]$ constraint where $e[j] \geq m+1$. This cannot force any $s[i] \leq t$ where $t \leq m$ to become true, thus the stack constraints from time 1 to time m remain satisfied.

Secondly, the only way that the values of $x[1]$ to $x[m]$ can affect the stack constraints from time $m+1$ to time c is by forcing some stacks to open before time m

via a $s[i] \leq e[j]$ constraint. However, the stack constraints from time $m + 1$ to time c only care about the value of $s[i] \leq t$ where $t \geq m + 1$. So they only care whether the stack was opened before $m + 1$ or not. They do not care exactly when the stack was opened between time 1 and time m . Now, both θ and θ' schedules the same set of customers during time 1 to time m , thus the set of stacks opened before time m is identical and has the same effect on the stack constraints from time $m + 1$ to time c . Then, since the values of $x[m + 1]$ to $x[c]$ are identical in θ and θ' , and by assumption θ' satisfies the stack constraints from time $m + 1$ to time c , θ must also satisfy the stack constraints from time $m + 1$ to time c . Thus θ is also a solution. $\square$

We exploit this result by deriving nogoods of the form $\neg(\{x[1], \dots, x[m]\} = \{v_1, \dots, v_m\})$, and propagating them in $O(1)$ using hash table lookups.

6.3 Improving the Search

In this section we show how to improve the customer search by exploiting several dominance relations. We first define some terminology.

Definition 6.3.1. *Given a customer graph G and a customer c , let $N(c)$ be the set of customers adjacent to c in G .*

In the next three definitions, let S denote the the set of customers whose stacks have been closed at a particular node in the search tree.

Definition 6.3.2. *Let $O(S) \equiv \cup_{c \in S} N(c)$ be the set of stacks which were forced open as a consequence of the closure of the stacks of customers in S .*

Definition 6.3.3. *Given a customer $c \notin S$, let $o(c, S) \equiv N(c) \setminus O(S)$ be the new stacks which will open if c is the next stack to close.*

Definition 6.3.4. *Let $open(c, S) = |o(c, S)|$ and $close(c, S) = |\{d \mid o(d, S) \subseteq o(c, S)\}|$, i.e., the number of new stacks that will open and close respectively if we close c 's stack next.*

Definition 6.3.5. A partial assignment $x[1] = v_1, \dots, x[m] = v_m$ is k -playable if it only needs k stacks during the first m time periods.

We will represent partial assignments $x[1] = v_1, \dots, x[m] = v_m$ as sequences $[v_1, \dots, v_m]$ for easier manipulation. We will also often use the same symbol to denote the sequence and its corresponding set.

6.3.1 Definite Moves

Under certain circumstances, one particular choice for the next customer will become so good that it is at least as good as any other choice. In this case, we can pick that choice and prune all other choices. We call this a *definite move*.

Theorem 6.3.6. Suppose S is some sequence, and $q \notin S$ is a customer such that: $S \uplus [q]$ is k -playable, and $\text{close}(q, S) \geq \text{open}(q, S)$. If S has an extension that uses $\leq k$ stacks, then $S \uplus [q]$ also has an extension that uses $\leq k$ stacks.

Proof. Suppose $U \equiv S \uplus [c_1, c_2, \dots, c_m, q, c_{m+1}, \dots, c_n]$ uses $\leq k$ stacks. We claim that $U' \equiv S \uplus [q, c_1, \dots, c_m, c_{m+1}, \dots, c_n]$ also uses $\leq k$ stacks. The two sequences differ only in the placement of q . The number of stacks which are open at any time before, or any time after the set of customers $\{c_1, c_2, \dots, c_m, q\}$ are played is identical for U and U' , since it only depends on the set of customers closed and not the order. The number of open stacks when q is played in U' is also $\leq k$, since $S \uplus [q]$ is k -playable. Finally, the number of open stacks when $S \uplus [q, c_1, c_2, \dots, c_i]$ is played in U' is always less than or equal to the number of open stacks when $S \uplus [c_1, c_2, \dots, c_i]$ is played in U , because we have at most $\text{open}(q, S)$ extra stacks open, but at least $\text{close}(q, S)$ extra stacks closed. So U' uses $\leq k$ stacks at all times. $\square$

Corollary 6.3.7. Suppose in the branch and bound algorithm, we are currently seeking solutions that use $\leq k$ stacks. If at some search node, S is the current sequence, and there exists $q \notin S$ such that: $S \uplus [q]$ is k -playable and $\text{close}(q, S) \geq \text{open}(q, S)$, then it is correct to prune all choices other than q .

6.3.2 Better Moves

Similarly, there are situations where one choice is at least as good as another, and we can prune the one that is not as good. We call this a *better move*.

Theorem 6.3.8. *Suppose S is some sequence, and $q, r \notin S$ are customers such that: $S \uplus [q]$ and $S \uplus [r, q]$ are both k -playable, and $\text{close}(q, S) \geq \text{open}(q, S \cup \{r\})$. If $S \uplus [r]$ has an extension that uses $\leq k$ stacks, then $S \uplus [q]$ also has an extension that uses $\leq k$ stacks.*

Proof. Suppose $U \equiv S \uplus [r, c_1, c_2, \dots, c_m, q, c_{m+1}, \dots, c_n]$ uses $\leq k$ stacks. We have $\text{close}(q, S \cup \{r\}) \geq \text{close}(q, S) \geq \text{open}(q, S \cup \{r\})$, so after r is played, q becomes a definite move. By the same argument as the previous proof, $U' \equiv S \uplus [r, q, c_1, \dots, c_n]$ uses $\leq k$ stacks. Now, consider $U'' \equiv S \uplus [q, r, c_1, \dots, c_n]$, where we swap the positions of q and r . Consider the stacks which are open when r is played in U'' compared to when r is played in U' . The number of new open stacks is exactly $\text{open}(q, S \cup \{r\})$, and the number of new closed stacks is exactly $\text{close}(q, S)$. So playing r in U'' uses no more stacks than playing r in U' . Also, the number of open stacks when q is played in U'' is $\leq k$ since $S \uplus [q]$ is k -playable. Thus U'' uses $\leq k$ stacks at all times. $\square$

Corollary 6.3.9. *Suppose in the branch and bound algorithm, we are currently seeking solutions that use $\leq k$ stacks. If at some search node, S is the current sequence, and there exists $q, r \notin S$ such that: $S \uplus [q]$ and $S \uplus [r, q]$ are both k -playable, and $\text{close}(q, S) \geq \text{open}(q, S \cup \{r\})$, then it is correct to prune r .*

Although better move seems weaker than definite move, since it only prunes one branch at a time rather than prune all branches but one, it is actually a generalization. It is easy to see from the definitions that any definite move is also a better move with respect to all other moves. Our implementation of better move subsumes definite move so we will simply consider them as one improvement.

6.3.3 Old Move

Sometimes, a subproblem which we are about to explore has actually been fully explored before using an alternative search path. This occurs because of the subproblem

equivalences inherent in the MOSP.

Theorem 6.3.10. *Suppose $S \equiv [s_1, s_2, \dots, s_n]$, and $q \notin S$ is a customer such that: $S' = [s_1, s_2, \dots, s_m, q, s_{m+1}, \dots, s_n]$ is k -playable. If $U \equiv S \uplus [q] \uplus R$ is a solution that uses $\leq k$ stacks, then $U' \equiv S' \uplus R$ is also a solution that uses $\leq k$ stacks.*

Proof. S' is k -playable by assumption so the number of open stacks at any time during S' is $\leq k$. At any point after S' , the number of open stacks are identical for U and U' since it only depends on the set of closed customers and not the order. Hence U' uses $\leq k$ at all times. $\square$

Corollary 6.3.11. *Suppose in the branch and bound algorithm, we are currently seeking solutions that use $\leq k$ stacks. If at some search node, S is the current sequence, and there exists $q \notin S$ such that: $S' = [s_1, s_2, \dots, s_m, q, s_{m+1}, \dots, s_n]$ is k -playable, and we have previously exhaustively searched the subproblem $S'' = [s_1, s_2, \dots, s_m, q]$, then it is correct to prune q .*

This pruning scheme was mentioned in [157], but it was incorrectly stated there. The author of [157] failed to note that the condition that $S' = [s_1, s_2, \dots, s_m, q, s_{m+1}, \dots, s_n]$ is k -playable is in fact crucial, because if it does not hold, then the subproblem rooted at S' would have been pruned via breaking the upper bound on the number of stacks, and none of its extensions would have been explored. Thus pruning q now would be incorrect.

Naively, it would appear to take $O(|C|^3)$ time to check the old move condition at each node. However, it is possible to do so in $O(|C|)$ time. At each node we keep a set $Q(S)$ of all the old moves, i.e., the set of moves q such that we can find $S' = [s_1, s_2, \dots, s_m, q, s_{m+1}, \dots, s_n]$ which is k -playable, and such that move q has already been searched at the node $S'' = [s_1, \dots, s_m]$. Note that by definition, when a move r has been searched at the current node, r will be added to $Q(S)$. It is easy to calculate $Q(S \uplus [s_{n+1}])$ when we first reach that child node. First, $Q(S \uplus [s_{n+1}]) \subseteq Q(S)$, since if $S' = [s_1, s_2, \dots, s_m, q, s_{m+1}, \dots, s_n, s_{n+1}]$ is k -playable then by definition so is $S' = [s_1, s_2, \dots, s_m, q, s_{m+1}, \dots, s_n]$. Second, to check if each $q \in Q(S)$ is also in $Q(S \uplus [s_{n+1}])$, we simply have to check whether the last move

in $S' = [s_1, s_2, \dots, s_m, q, s_{m+1}, \dots, s_n, s_{n+1}]$ is k -playable, as all the previous moves are already known to be k -playable since $q \in Q(S)$. Checking the last move takes constant time so the total complexity is $O(|C|)$. There are some synergies between the better move improvement and the old move improvement. If $q \in Q(S)$ and q is better than move r , then we can add r to $Q(S)$ as well. This allows old move to prune sets that we have never even seen before.

6.3.4 Upper bound heuristic

In this section, we describe an upper bound heuristic which was found to be very effective on our instances. A good heuristic for finding an optimal solution is useful from a practical point of view if no proof of optimality is required. It is also a crucial component for the relaxation techniques described in the next subsection which can give several orders of magnitude speedup on the proof of optimality for hard problems.

In [53] the authors tried multiple branching heuristics in order to compute an upper bound, but only applied them in a greedy fashion, effectively searching only 1 leaf node for each. We can do much better by performing an incomplete search where we are willing to explore a larger number of nodes, but still much fewer than a complete search. Simple ways of doing this using our complete search engine include, sorting the choices according to some criteria, and only trying the first m moves for some m . Or trying all the moves which are no worse than the best by some amount e , etc.

One heuristic that is extremely effective is to only consider the moves where we close a customer stack that is currently open, the intuition being that if a stack is not even open yet, there is no point trying to close it now. Although this seems intuitively reasonable, it is in fact not always optimal. In practice however, an incomplete search using this criteria is very fast, and finds the optimal solution almost all the time, and several orders of magnitude faster than the complete search for some hard instances. The reason for its strength comes from its ability to exploit a not quite perfect dominance relation. Almost all the time, subtrees where we close a stack that is not yet open is dominated by one where we close a currently open stack, and

Table 6.1: Comparison of upper bound heuristic, versus complete search on some difficult problems. Times in milliseconds

Instance	Orig. time	Heur. time	Speedup
100-100-2	4136.3	39.8	104.0
100-100-4	4715.5	43.3	108.8
100-100-6	8.2	12.8	0.6
100-100-8	9.2	6.3	1.5
100-100-10	1.6	1.3	1.3
125-125-2	1159397.3	385.1	3010.3
125-125-4	2593105.1	398.9	6500.1
125-125-6	8975.9	424.9	21.1
125-125-8	187.8	146.1	1.3
125-125-10	22.2	8.3	2.7

thus we can exploit this to prune branches similarly to what we do in Section 6.3. The dominance is not always true however, so using such a pruning rule makes it an incomplete, heuristic search. See Table 6.1 for a brief comparison of the times required to find the optimal solution.

6.3.5 Relaxation

Relaxation techniques have been used on the MOSP before in the context of a local search method [115]. The idea was to try to relax the problem in such a way that the solution density is increased, so that better solutions can be found faster. However, these methods are of no help in proving optimality. In this section we show how relaxations can be used to speed up the proof of optimality.

As is seen in the experimental results in [53], the sparser instances of MOSP are substantially harder than denser instances of MOSP given the same number of customers and products. This can be explained by the fact that in sparser instances, each customer has far fewer neighbors in the customer graph, thus many more moves would fall under the upper bound limit at each node and both the depth and the branching factor of the search tree are dramatically increased compared to a dense instance of the same size.

However, the sparsity of these instances also leads to a potential optimization. Since the instance is sparse and the optimum is low (e.g., 20-50 for a 125×125 problem) it is possible that not all of the constraints are actually required to force the lower bound. It is possible that there is some small “unsatisfiable core” of customers which are producing the lower bound. If such an unsatisfiable core exists and can be identified, we can potentially remove a large number of customers from the problem and make the proof of optimality much quicker. It turns out that this is often possible.

First, we will show how we can relax the MOSP instance. Naively, we can simply delete one or more nodes from the customer graph and remove all edges involving those node. This represents the deletion of some variables and constraints and is of course a valid relaxation. However, we can do much better. We show that given an MOSP instance P corresponding to customer graph G , if G' is a contraction of G , then the MOSP instance P' corresponding to G' is a relaxation of P . Using this, we can get some compensation when we remove nodes, as we can retain some of the edges. This makes P' stronger and allows more nodes to be removed without loosening the lower bound.

Theorem 6.3.12. *Let G' be a contraction of graph G . If P is the MOSP corresponding to G and P' is the MOSP corresponding to G' , then P' is a relaxation of P .*

Proof. We prove the result for the case where G' is derived from G by merging a single pair of adjacent nodes. The general result follows by induction. Suppose G had n nodes $c_1, \dots, c_n$. Without loss of generality, suppose c_1 and c_2 are adjacent and we merge them to create G' . Call the new merged node c_2 . We show that every solution of P that uses k stacks corresponds to a solution of P' that uses $\leq k$ stacks. Let S be a solution of P . Without loss of generality, let it be $S \equiv [v_1, \dots, v_a, 1, v_{a+1}, \dots, v_b, 2, v_{b+1}, \dots, v_{n-2}]$. We claim that $S' \equiv [v_1, \dots, v_a, v_{a+1}, \dots, v_b, 2, v_{b+1}, \dots, v_{n-2}]$ is a solution of P' . We show that for each customer, the number of stacks open when the customer is played in S' for problem P' is no greater than the number of stacks open when the customer is played in S for problem P .

Let us compare the stacks which are open when v_i is played for $i \leq a$. For any $j \geq 3$, c_j 's stack is open in S' iff it is open in S . c_2 's stack is open in S' iff c_1 's or c_2 's stack is open in S . c_1 's stack is never open in S' since c_1 does not exist in G' . In all cases, the number of stacks open in S' is less than or equal to the number of stacks open in S . Let us compare the stacks which are open when v_i is played for $a < i \leq b$. For any $j \geq 3$, c_j 's stack is open in S' iff it is open in S . c_2 's stack may or may not be open in S' , but, c_2 's stack is guaranteed to be open in S because c_1 is adjacent to c_2 , and c_1 had previously been played. So once again, the number of stacks open in S' is less than or equal to the number of stacks open in S . For the remainder of the customers, the number of stacks that are open are identical for S and S' . Hence, S' is a solution that uses $\leq k$ stacks. This shows that P' is a relaxation of P . $\square$

Next we need to identify the nodes which can be removed/merged without loosening the lower bound on the problem. The main idea is that the longer a customer's stack is open in the optimal solutions, the more likely it is that that customer is contributing to the lower bound, since removal of such a customer would mean that there is a high chance that one of the optimal solutions can reduce to one needing one fewer stack. Thus we want to avoid removing such customers. Instead we want to remove or merge customers whose stacks are usually open for a very short time. One naïve heuristic is to greedily remove nodes in the customer graph with the lowest degree. Fewer edges coming out of a node presumably means that the stack is open for a shorter period of time on average.

A much better heuristic comes from the following idea. Suppose there exist a node c such that any neighbor of c is also connected to most of the neighbors of c , then when c is forced open by the closure of one of those neighbors, that neighbor would also have forced most of the neighbors of c to open, and thus c will be able to close soon afterwards and will only be open for a short time. The condition that most neighbors of c are connected to most other neighbors of c is in fact quite common for sparse instances due to the way that the customer graph is generated from the products (each product produces a clique in the graph). To be more precise, in our implementation, the customers are ranked according to:


```

    if ( $lb < ub$ )                                % too relaxed to prove lb
         $G' := \text{unmerge\_one\_pair}(G')$         % unrelax problem
    else return  $ub$                                 %  $lb = ub$ 
return MOSP( $G$ )                                % relaxation failed!

```

As can be seen, the upper bound heuristic is necessary to find a good (optimal) solution quickly. It is also used to detect when we are too relaxed as quickly as possible so that we can unrelax. If the upper bound heuristic is sufficiently good, we will quickly be able to find a relaxation that removes as many customers as possible without being too relaxed. If the upper bound heuristic is weak however, we could waste a lot of time searching in a problem that is in fact too relaxed to ever give us the true lower bound. In practice, we have found that our upper bound heuristic is quite sufficient for the instances we tested it on.

There are a few optimizations we can make to this basic algorithm. Firstly, when an unmerge is performed, we can attempt to extend the last solution found to a solution of this less relaxed problem. If the solution extends, then it is still too relaxed and we need to unmerge again. This saves us having to actually look for a solution to this problem. Secondly, naively, when we perform an unrelax, we can simply unmerge the last pair of nodes that were merged. However, we can do better. One of the weaknesses of the current algorithm is that the nodes to be merged are chosen greedily using equation (6.1). If this happens to choose a bad relaxation that lowers the lower bound early on, then we will not be able to remove any more customers beyond that point. We can fix this to some extent by choosing which pair of nodes to unmerge when we unrelax. We do this by considering each of the problems that we get by unmerging each pair of the current merges. If the last solution found does not extend to a solution for one of these, then we choose that unmerge, as this unrelaxation gives us a chance to prove the true lower bound. If the last solution extends to a solution for all of them, we unmerge the last pair as per usual. This helps to get rid of early mistakes in merging and is useful on several of our instances.

6.4 Experiments

In this section we demonstrate the performance of our algorithm, and the effect of the improvements. The experiments were performed on a Xeon Pro 2.4GHz processor with 2Gb of memory. The code implementing the approaches were compiled using g++ with -O3 optimization.

6.4.1 Modeling Challenge instances

Very stringent correctness tests were performed in view of the large speedups achieved. All versions of our solver were run on the 5000+ instances used in the 2005 model challenge [146], as well as another 100,000 randomly generated instances of size 10×10 to 30×30 and various densities. The answers were compared with the solver submitted to the 2005 model challenge and all answers were identical.

We compare our solver with the previous state of the art MOSP solver, on which our solver is based. The results clearly show that our solver is orders of magnitude faster than the original version. Getting an exact speedup is difficult as almost all of the instances that the original version can solve are solved trivially by our solver in a few milliseconds, whereas instances that our solver finds somewhat challenging are completely unsolvable by the original version.

Our solver was able to solve all the open problems from the Modeling Challenge: SP2, SP3, and SP4. Table 6.2 compares these problems with the best results from the Challenge by [53]. The nodes and times (in milliseconds) for [53] are for finding the best solution they can. The times for our method are for the full solve including proof of optimality (using all improvements).

6.4.2 Harder Random instances

Of the 5000+ instances used in the 2005 challenge, only SP2, SP3 and SP4 take longer than a few milliseconds for our solver to solve. Thus we generate some difficult random instances for this experiment. First we specify the number of customers, number of products and the average number of customers per product. We then calculate a

Table 6.2: Results on the open problems from the Constraint Modeling Challenge 2005, comparing versus the the winner of the challenge [53]. Times in milliseconds.

	Garcia de la Banda and Stuckey [53]			This thesis		
	Best	Nodes	Time	Optimal	Nodes	Time
SP2	19	25785	1650	19	1236	7
SP3	36	949523	~ 3600000	34	84796	410
SP4	56	3447816	~ 14400000	53	1494860	9087

density that will achieve the specified average number of customers per product. The customer vs product table is then randomly generated using the calculated density to determine when to put 1's and 0's. As a post condition, we throw away any instance where the customer graph can be decomposed into separate components. This is done because we want to compare on instances of a certain size, but if the customer graph decomposes, then the instance degenerates into a number of smaller and relatively trivial instances.

We generate instances of sizes 30×30 , 40×40 , 50×50 , 75×75 , 100×100 , 125×125 , 100×50 , 50×100 , and average number of customer per product values of 2, 4, 6, 8, 10. 5 instances for each set of values were generated for a total of 200 instances.

Ideally, we want to measure speedup by comparing total solve time. However, as mentioned before, the instances that our solver finds challenging are totally unsolvable by the original. Table 6.3 is split into two parts. Above the horizontal line are the instances where the original managed to prove optimality. Here, nodes, time (in milliseconds) and speedup are for the total solve. Below the line the original cannot prove optimality. Here, nodes, time and speedup are for the finding a solution that is at least as good as the solver of [53] could find. The column δOpt shows the average distance this solution is from the optimal. Note that our approach finds and proves the optimal in all cases although the statistics for this are not shown in the table. Time to find an equally good solution is not necessarily a good indication of the speedup achievable for the full solve, as other factors like branching heuristics come into play. However, the trend is quite clear. The original solver is run with

Table 6.3: Comparing customer search versus [53] on harder random instances. Search is to find the best solution found by [53] with node limit 2^{25} .

Instance	δOpt	[53]		This paper		
		Nodes	Time(ms)	Nodes	Time(ms)	Speedup
30-30-2	0	14318	480	408	4.4	109
30-30-4	0	48232	1981	158	2.0	979
30-30-6	0	89084	2750	56	0.9	3136
30-30-8	0	83558	2010	18	0.4	5322
30-30-10	0	18662	506	8	0.2	2335
40-40-2	0	669384	192917	1472	14.9	12942
40-40-4	0	3819542	227087	556	6.1	36959
40-40-6	0	11343235	625892	217	2.9	218062
40-40-8	0	8379706	334392	49	0.8	403272
40-40-10	0	3040040	98194	20	0.4	229305
50-50-2	0	12356205	1300311	6344	65.8	19758
50-50-4	0.2	5612259	446409	219	2.7	164728
50-50-6	0.2	7949026	510831	45	0.8	636409
50-50-8	0.2	525741	28337	15	0.4	75274
50-50-10	0	16809	784	7	0.2	3411
75-75-2	0.8	2485310	420935	7030	76.8	5484
75-75-4	2.6	3507703	666784	63	1.4	486669
75-75-6	1.2	4412286	756032	59	1.4	548132
75-75-8	1.2	4121046	519778	19	0.6	841336
75-75-10	0.6	1198282	120087	15	0.5	244128
100-100-2	2.2	3008009	765131	481	9.4	81653
100-100-4	4.8	6777140	2017286	145	3.3	619257
100-100-6	4	1269071	347970	39	1.4	241145
100-100-8	4.4	1686045	414456	31	1.1	363468
100-100-10	1.6	4195494	789039	15	0.7	1097494
125-125-2	1.8	7418402	3276210	36672	436.8	7500
125-125-4	3.8	3412379	1559691	916	20.4	76286
125-125-6	6	6076996	2643707	57	2.3	1144180
125-125-8	6.2	942290	321050	28	1.5	217007
125-125-10	3.4	170852	45798	24	1.3	35647
50-100-2	0.2	90076	9971	97	1.6	6290
50-100-4	1	1973322	139300	23	0.6	220776
50-100-6	0.6	1784	116	13	0.4	301
50-100-8	0	97	9	5	0.2	47
50-100-10	0	99	8	3	0.2	39
100-50-2	0	2393401	438220	11117	133.7	3279
100-50-4	0.4	14211006	3499389	183260	1592.3	2198
100-50-6	1.2	5326088	1395417	1569	21.4	65163
100-50-8	0.6	1522796	408908	3506	45.8	8932
100-50-10	1	3594743	906559	524	10.6	85710

its best options. Our MOSP algorithm is run with “better move”, “old move” and nogood recording turned on (but no relaxation). Both solvers have a node limit of 2^{25} iterations.

Note that because a single move can close multiple stacks, it is possible to completely solve an instance using fewer nodes than there are customers. This occurs frequently in the high density instances. Thus the extremely low node counts shown

Table 6.4: (a) Comparing the effects of each optimization in turn, and (b) comparing the effects of relaxation.

Instance	Better	Old	Nogood	No relax(ms)	Relax(ms)	Removed	Speedup
100-100-2	25.2	63.7	1.21	603120	370	51.2	1630.6
100-100-4	8.94	5.95	1.01	266205	4798	20.8	55.5
100-100-6	1.90	1.71	0.91	10344	3909	8	2.6
100-100-8	1.54	1.30	0.66	551	712	2.4	0.8
100-100-10	2.96	2.75	1.00	46	94	0.6	0.5
125-125-2	—	—	—	59642993	3284	62.2	18161.8
125-125-4	—	—	—	29768563	251634	24.8	118.3
125-125-6	11.9	3.10	0.98	810678	167384	9	4.8
125-125-8	1.65	1.17	0.98	18781	11978	5	1.6
125-125-10	1.27	0.98	0.64	768	1041	3	0.7

(a)

(b)

here are not errors. The speedup is around 2-3 orders of magnitude for the smallest problems (30×30), and around 5-6 orders of magnitude for the hardest problems that the original version can solve (40×40). The speedup appears to grow exponentially with problem size. We cannot get any speedup numbers for the harder instances since the original cannot solve them. However, given the trend in the speedup, it would not be surprising if the speedup for a full solve on the hardest instances solvable by our solver (100×100) is in the realms of 10^{10} or more.

Next we examine the effect of each of our improvements individually by disabling them one at a time. The three improvements we test here are “better move”, “old move” and nogood recording. We use only the instances which are solvable without the improvements and non-trivial, i.e., the 100×100 instances and the easier 125×125 instances. For each improvement, we show the relative slowdown compared to the version with all three optimizations on.

As Table 6.4(a) shows, both “better move” and “old move” can produce up to 1 to 2 orders of magnitude speedup on the harder instances. The lower speedups are from instances that are already fairly easy and solvable in seconds. The results from disabling the nogood recording are very interesting. It is known from previous work, e.g., the DP approach of [53] and the CP approach of [141] that nogood recording or equivalent techniques produce several orders of magnitude speedup. However, these approaches require (in the worst case) exponential memory usage for the nogood table. It appears, however, that once we have the “old move” improvement, we can

actually turn off nogood recording without a significant loss of performance. In fact, some instances run faster. Thus our “old move” improvement largely subsumes the effect of the huge nogood tables used in the DP [53] or CP [141] approaches and reduces the memory usage from an exponential to a linear amount. The solver of [53] uses up all 2Gb of main memory in ~ 5 min with nogood recording. However, our new solver using “old move” pruning uses a constant amount of memory $< 2\text{Mb}$ even for 125×125 problems.

6.4.3 Relaxation

In the following set of experiments, we demonstrate the effectiveness of our relaxation technique. For each of our largest instances, we show in Table 6.4(b) the total runtime (in milliseconds) without relaxation, with relaxation, and the number of customers that was successfully removed without changing the lower bound, as well as the speedup for relaxation. Both versions are run with the customer search strategy, “better move” and “old move” improvements.

As can be seen from the results in Table 6.4(b), relaxation is most effective for sparse instances where we can get up to 3-4 orders of magnitude improvement. There is a slight slowdown for several dense instances but that is because they are trivial to begin with (take $< 1\text{s}$). The sparser the instance, the more customers can be removed without changing the lower bound and the greater the speedup from the relaxation technique. For the hardest instances, 125-125-2, it is often possible to remove some 60-70 of the 125 customers without changing the bound. This reduces the proof of optimality that normally takes 10+ hours into mere seconds. The 125-125-4 instances are now comparatively harder, since we are only able to remove around 25 customers and get a speedup of ~ 100 . Relaxation is largely ineffective for the denser instances like 125-125-8,10. However, dense instances are naturally much easier to solve anyway, so we have speedup where it is needed the most.

Our relaxation techniques are also useful if we only wish to prove a good lower bound rather than the true lower bound. For example, if we only insist on proving a lower bound that is 5 less than the true optimum, then ~ 45 customers can be removed

from the 125-125-4 instances and the bound can be proved in seconds. This is again several orders of magnitude speedup compared to using a normal complete search to prove such a bound. In comparison, although the HAC lower bound heuristic of [9] uses virtually no time, it gives extremely weak lower bounds for the 125-125-4 instances, which are some 30 stacks below the optimum and are of little use.

6.5 Conclusion

In this chapter, we showed how combining nogood learning with dominance breaking can yield an MOSP solver that is some 5-6 orders of magnitude faster than the previous state of the art. One dominance relation led naturally to a customer based reformulation of the problem, yielding some 3-4 orders of magnitude speedup. This customer based search is not new, but has somehow been largely ignored in recent literature. We also proved several new dominance relations, which we called “definite move”, “better move” and “old move”. Exploiting these produced a further 1-2 orders of magnitude speedup. The “old move” improvement in particular is able to subsume the effect of pruning using an extremely large nogood table. This allows us to reduce the memory usage of our solver from an amount exponential in the size of the problem to a constant $\sim 2\text{Mb}$. Finally we showed how relaxation techniques can be used to speed up the proof of optimality of the hardest instances by another 3-4 orders of magnitude.

Chapter 7

Talent Scheduling Problem

7.1 Introduction

The talent scheduling problem [23] can be described as follows. A film producer needs to schedule the scenes of their movie at a given location. Each scene has a duration (the number of days it takes to shoot it) and requires some subset of the cast to be on location. Each cast member is required to be on location from the day the first scene they are in is shot, to the day the last scene they are in is shot, even if they are not required for some of the days in between. Each cast member is paid a certain daily salary for each day they are on location. The problem is to order the scenes in such a way as to minimize the salary cost of the shooting.

We can formalize the problem as follows. Let S be a set of scenes, A a set of actors, and $a(s)$ a function that returns the set of actors involved in scene $s \in S$. Let $d(s)$ be the duration in days of scene $s \in S$, and $c(a)$ be the cost per day for actor $a \in A$. We say that actor $a \in A$ is on location at the time the scene placed in position k , $1 \leq k \leq |S|$ in the schedule is being shot, if there is a scene requiring a scheduled before or at position k , and also there is a scene requiring a scheduled at or after position k . In other words, a is on location from the time the first scene a is in is shot, until the time the last scene a is in is shot. The talent scheduling problem aims at finding a schedule for scenes S (i.e., a permutation of the scenes) that minimizes the total salary cost.

The talent scheduling problem as described in the previous paragraph is certainly an idealized version of the real problem. Real shooting schedules must contend with actor availability, setup costs for scenes, and other constraints ignored in this thesis. In addition, actors can be flown back from location mid shoot to avoid paying their holding costs for extended periods. However, the talent cost, in real situations, is a prominent feature of the movie budget [23]. Hence, concentrating on this core problem is worthwhile. Furthermore, the underlying mathematical problem has many other uses, including archaeological site ordering, concert scheduling, VLSI design and graph layout. See Section 7.6 for more discussion on this.

Example 7.1.1. Consider the talent scheduling problem defined by the set of actors $A = \{a_1, a_2, a_3, a_4, a_5, a_6\}$, the set of scenes $S = \{s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8, s_9, s_{10}, s_{11}\}$,

	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8	s_9	s_{10}	s_{11}	s_{12}	$c(a)$
a_1	X	.	X	.	.	X	.	X	X	X	X	X	20
a_2	X	X	X	X	X	.	X	.	X	.	X	.	5
a_3	.	X	.	.	.	.	X	X	.	.	.	.	4
a_4	X	X	.	.	X	X	.	.	.	.	.	.	10
a_5	.	.	.	X	.	.	.	X	X	.	.	.	4
a_6	.	.	.	.	.	.	.	.	.	X	.	.	7
$d(s)$	1	1	2	1	3	1	1	2	1	2	1	1	

	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8	s_9	s_{10}	s_{11}	s_{12}	$c(a)$
a_1	X	–	X	–	–	X	–	X	X	X	X	X	20
a_2	X	X	X	X	X	–	X	–	X	–	X	.	5
a_3	.	X	–	–	–	–	X	X	.	.	.	.	4
a_4	X	X	–	–	X	X	.	.	.	.	.	.	10
a_5	.	.	.	X	–	–	–	X	X	.	.	.	4
a_6	.	.	.	.	.	.	.	.	.	X	.	.	7
cost	35	39	78	43	129	43	33	66	29	64	25	20	604
extra	0	20	28	34	84	13	24	10	0	10	0	0	223

Figure 7.1: (a) An Example $a(s)$ Function: $a_i \in a(s_j)$ if the Row for a_i in Column s_j has an X. (b) An Example Schedule: a_i is on location when scene s_j is Scheduled if the Row for a_i in Column s_j has an X or a –.

and the $a(s)$ function determined by the matrix M shown in Figure 7.1(a), where an X at position M_{ij} indicates that actor a_i takes part in scene s_j . The daily cost per actor $c(a)$ is shown in the rightmost column, and the duration of each scene $d(s)$ is shown in the last row.

Consider the schedule obtained by shooting the scenes in order $s_1s_2s_3s_4s_5s_6s_7s_8s_9s_{10}s_{11}$. The consequences of this schedule in terms of actor's presence and cost are illustrated by the matrix M shown in Figure 7.1(b), where actor a_i is on location at the j^{th} shot scene if the position M_{ij} contains either an X (a_i is in the scene) or an – (a_i is waiting). The cost of each scene is shown in the second last row, being the sum of the daily costs of all actors on location multiplied by the duration of the scene. The total cost for this schedule is 604. The *extra cost* for each scene is shown in the last row, being the sum of the daily costs of only those actors waiting on location, multiplied by the duration of the scene. The extra cost for this schedule is 223.

The Talent Scheduling problem was introduced in [23]. In its original form each of the scenes is actually a shooting day and, hence, the duration of each of the scenes is 1. A variation of the problem, called concert scheduling [1], considers the case where the cost for each player is identical. The scene scheduling problem is known to be NP-hard [23] even if each actor appears in only two scenes, all actor costs are identical and all durations are identical.

The main contributions of this chapter are:

- We define an effective dynamic programming solution to the problem
- We define and prove correct a number of optimizations for the dynamic programming solution, that increase the size of problems we can feasibly tackle
- We show how using bounded dynamic programming can substantially improve the solving of these problems
- We show how, by considering a more accurate notion of subproblem equivalence, we can substantially improve the solving

The final code can find optimal solutions to problems larger than previous methods.

In Section 7.2 we give our call-based best-first dynamic programming formulation for the talent scheduling problem, and consider ways it can be improved by pre-processing and modifying the search. Section 7.3 examines how to solve a bounded version of the problem, which can substantially improve performance, and how to compute upper and lower bounds for the problem. Section 7.4 investigates a better search strategy where we schedule scenes from both ends of the search and Section 7.5 presents an experimental evaluation of the different approaches. In Section 7.6 we discuss related work and in Section 7.7 we conclude.

7.2 Dynamic Programming Formulation

The talent scheduling problem is naturally expressible in a dynamic programming formulation. To do so we extend the function $a(s)$ which returns the set of actors in scene $s \in S$, to handle a set of scenes $Q \subseteq S$. That is, we define $a(Q) = \cup_{s \in Q} a(s)$

as a function that returns the set of actors appearing in any set of scenes $Q \subseteq S$. Similarly, we extend the cost function $c(a)$ to sets of actors $G \subseteq A$ in the obvious way: $c(G) = \sum_{a \in G} c(a)$.

Let $l(s, Q)$ denote the set of actors on location at the time scene s is scheduled assuming that the set of scenes $Q \subset (S \setminus \{s\})$ is scheduled after s , and the set $S \setminus Q \setminus \{s\}$ is scheduled before s . Then

$$l(s, Q) = a(s) \cup (a(Q) \cap a(S \setminus Q \setminus \{s\})),$$

i.e., the on locations actors are those who appear in scene s , plus those who appear in both a scene scheduled after s and one scheduled before s . The problem is amenable to dynamic programming because $l(s, Q)$ does not depend on any particular order of the scenes in Q or $S \setminus Q \setminus \{s\}$. Let $Q \subseteq S$ denote the set of scenes still to be scheduled, and let $schedule(Q)$ be the minimum cost required to schedule the scenes in Q . Dynamic programming can be used to define $schedule(Q)$ as:

$$schedule(Q) = \begin{cases} 0 & Q = \emptyset \\ \min_{s \in Q} ((d(s) \times c(l(s, Q \setminus \{s\}))) + schedule(Q \setminus \{s\})), & \text{otherwise} \end{cases}$$

which computes, for each scene s , the cost of scheduling the scene s first $d(s) \times c(l(s, Q \setminus \{s\}))$ plus the cost of scheduling the remaining scenes $Q \setminus \{s\}$. Dynamic programming is effective for this problem because it reduces the raw search space from $|S|!$ to $2^{|S|}$, since we only need to investigate costs for each subset of S (rather than for each permutation of S).

7.2.1 Basic Best-first Algorithm

The code in Figure 7.2 illustrates our best-first call-based dynamic programming algorithm, which improves over a naïve formulation by pruning children that cannot yield a smaller cost.

The algorithm starts by checking whether Q is empty, in which case the cost is 0. Otherwise, it checks whether the minimum cost for Q has already been computed


```

schedule( $Q$ )
  if ( $Q = \emptyset$ ) return 0
  if ( $scost[Q]$ ) return  $scost[Q]$ 
   $min := +\infty$ 
   $T := Q$ 
  while ( $T \neq \emptyset$ )
     $s := \text{index } \min_{s \in T} \text{cost}(s, Q \setminus \{s\}) + \text{lower}(Q \setminus \{s\})$ 
     $T := T \setminus \{s\}$ 
    if ( $\text{cost}(s, Q \setminus \{s\}) + \text{lower}(Q \setminus \{s\}) \geq min$ ) break
     $sp := \text{cost}(s, Q \setminus \{s\}) + \text{schedule}(Q \setminus \{s\})$ 
    if ( $sp < min$ )  $min := sp$ 
   $scost[Q] := min$ 
  return  $min$ 

```

Figure 7.2: Pseudo-code for best-first call-based dynamic programming algorithm. $\text{schedule}(Q)$ returns the minimum cost required for scheduling the set of scenes Q

(and stored in $scost[Q]$), in which case it returns the previously stored result (code shown in light gray). We assume the $scost$ array is initialized with zero. If not, the algorithm selects the next scene s to be scheduled (using a simple heuristic that will be discussed later) and computes in sp the value $\text{cost}(s, Q \setminus \{s\}) + \text{schedule}(Q \setminus \{s\})$, where function $\text{cost}(s, B)$ returns the cost of scheduling scene s before any scene in $B \subset S$ (and after any scene in $S \setminus B \setminus \{s\}$), calculated as:

$$\text{cost}(s, B) = d(s) \times c(l(s, B))$$

Note, however, that the algorithm avoids (thanks to the **break**) considering scenes whose lower bound is greater than or equal to the current minimum min , since they cannot improve on the current solution. As a result, the order in which the Q scenes are selected can significantly affect the amount of work performed. In our algorithm, this order is determined by a simple heuristic that selects the scene s with the smallest calculated lower bound if scheduled immediately $\text{cost}(s, Q \setminus \{s\}) + \text{lower}(Q \setminus \{s\})$, where function $\text{lower}(B)$ returns a lower bound on the cost of scheduling the scenes in $B \subseteq S$,

	s_5	s_2	s_7	s_1	s_6	s_8	s_4	s_9	s_3	s_{11}	s_{10}	s_{12}	$c(a)$
a_1	.	.	.	X	X	X	–	X	X	X	X	X	20
a_2	X	X	X	X	–	–	X	X	X	X	.	.	5
a_3	.	X	X	–	–	X	.	.	.	.	.	.	4
a_4	X	X	–	X	X	.	.	.	.	.	.	.	10
a_5	.	.	.	.	.	X	X	X	.	.	.	.	4
a_6	.	.	.	.	.	.	.	.	.	.	X	.	7
cost	45	19	19	39	39	66	29	29	50	25	54	20	434
extra	0	0	10	4	9	10	20	0	0	0	0	0	53

Figure 7.3: An optimal order for the problem of Example 7.1.1

and it is simply calculated as:

$$\text{lower}(B) = \sum_{s \in B} d(s) \times c(a(s))$$

which is the sum of the costs for actors that appear in each scene. The index construct $\text{index}_{s \in Q} e(s)$ returns the s in Q that causes the expression $e(s)$ to take its minimum value.

A call to function `schedule( $S$ )` returns the minimum cost required to schedule the scenes in S . Extracting the optimal schedule found from the array of stored answers `scost[]` is straightforward, and standard for dynamic programming.

Example 7.2.1. Consider the problem of Example 7.1.1. An optimal solution is shown in Figure 7.3. The total cost is 434, and the extra cost 53. $\square$

7.2.2 Preprocessing

We can simplify the problem in the following two ways:

- Eliminating single scene actors: Any actor a' that appears only in one scene s can be removed from s (i.e., we can redefine set $a(s)$ as $a(s) \setminus \{a'\}$) and add its fixed cost $d(s) \times c(a')$ to the overall cost. This is correct because the cost of a' is the same independently of where s is scheduled (since a' will never have to wait while on location).

- Concatenating duplicate scenes: Any two scenes s_1 and s_2 such that $a(s_1) = a(s_2)$ can be replaced by a single scene s with duration $d(s) = d(s_1) + d(s_2)$. This is correct because there is always an optimal schedule in which s_1 and s_2 are scheduled together.

Since each simplification can generate new candidates for the other kind of simplification, we need to repeatedly apply them until no new simplification is possible.

The simplification that concatenates duplicate scenes has been applied before, but not formally proved to be correct. For example, the real scene scheduling data from [23] was used in [145] with this simplification applied.

Lemma 7.2.2. *If there exists s_1 and s_2 in S where $a(s_1) = a(s_2)$, then there is an optimal order with s_1 and s_2 scheduled together.*

Proof. Let Π denote a possibly empty sequence of scenes. In an abuse of notation, and when clear from context, we will sometimes use sequences as if they were sets. Without loss of generality, take the order $\Pi_1 s_1 \Pi_2 s' \Pi_3 s_2 \Pi_4$ of the scenes in S , and consider the actors on location for scene s_1 to be $l(s_1, \Pi_2 s' \Pi_3 s_2 \Pi_4) = A_1$ and for scene s_2 to be $l(s_2, \Pi_4) = A_2$. Now, either $c(A_1) \leq c(A_2)$ (which, since $a(s_1) = a(s_2)$ means that the cost of the actors *waiting* in A_1 is smaller or equal than that of the actors *waiting* in A_2) or $c(A_1) > c(A_2)$. We will show how, in the first case, choosing the new order (a) $\Pi_1 s_1 s_2 \Pi_2 s' \Pi_3 \Pi_4$ can only decrease the cost for each scene. It is symmetric to show that, in the second case, choosing the new order (b) $\Pi_1 \Pi_2 s' \Pi_3 s_1 s_2 \Pi_4$ can only decrease the cost for each scene.

Lets examine the costs of s_1 and s_2 . The cost of s_1 does not change from the original order to that of (a) since the set of scenes before and after s_1 remains unchanged (i.e., since by definition $l(s_1, s_2 \Pi_2 s' \Pi_3 \Pi_4) = l(s_1, \Pi_2 s' \Pi_3 s_2 \Pi_4)$). The cost of s_2 in (a)

is the cost of the actors in

$$\begin{aligned}
l(s_2, \Pi_2 s' \Pi_3 \Pi_4) &= a(s_2) \cup (a(\Pi_2 s' \Pi_3 \Pi_4) \cap a(\Pi_1 s_1)) \\
&\quad \text{By definition of } l(s, Q) \\
&= a(s_1) \cup (a(\Pi_2 s' \Pi_3 \Pi_4) \cap a(\Pi_1 s_1)) \\
&\quad \text{By hypothesis of } a(s_1) = a(s_2) \\
&= a(s_1) \cup (a(\Pi_2 s' \Pi_3 \Pi_4) \cap a(\Pi_1)) \\
&\quad \text{By definition of } a(Q) \\
&= a(s_1) \cup (a(\Pi_2 s' \Pi_3 s_2 \Pi_4) \cap a(\Pi_1)) \\
&\quad \text{By definition of } a(Q) \text{ and by hypothesis of } a(s_1) = a(s_2) \\
&= l(s_1, \Pi_2 s' \Pi_3 s_2 \Pi_4) \\
&\quad \text{By definition of } l(s, Q)
\end{aligned}$$

which is known to be A_1 . Hence, the cost of s_2 can only decrease, since $c(A_1) \leq c(A_2)$.

Lets consider the other scenes. First, it is clear that the products in Π_1 and Π_4 have the same on location actors since the set of scenes before and after remain unchanged. Second, let us consider the changes in the on locations actors for s' , which can be seen as a general representative of scenes scheduled in between s_1 and s_2 in the original order. While in the original order the set of on location actors at the time s' is scheduled is $l(s', \Pi_3 s_2 \Pi_4) = a(s') \cup (a(\Pi_1 s_1 \Pi_2) \cap a(\Pi_3 s_2 \Pi_4))$, in the new order the set of on location actors is $l(s', \Pi_3 \Pi_4) = a(s') \cup (a(\Pi_1 s_1 s_2 \Pi_2) \cap a(\Pi_3 \Pi_4))$. Clearly (a) $a(\Pi_3 \Pi_4) \subseteq a(\Pi_3 s_2 \Pi_4)$ and (b) since $a(s_1) = a(s_2)$, we have that $a(\Pi_1 s_1 s_2 \Pi_2) = a(\Pi_1 s_1 \Pi_2)$. Hence, by (a) and (b) we have that $l(s', \Pi_3 \Pi_4) \subseteq l(s', \Pi_3 s_2 \Pi_4)$, which means the set of on location actors when s' is scheduled can only decrease and, hence, the cost of scheduling it does not increase. $\square$

Example 7.2.3. Consider the scene scheduling problem from Example 7.1.1. Since actor a_6 only appears in one task, we can remove this actor and add a total of $2 \times 7 = 14$ to the cost of the resulting problem to get the cost of the original problem. We also have that $a(s_3) = a(s_{11})$ and, after the simplification above, $a(s_{10}) = a(s_{12})$. Hence, we can replace these pairs by single new scenes of the combined duration. The

	s_1	s_2	s'_3	s_4	s_5	s_6	s_7	s_8	s_9	s'_{10}	$c(a)$
a_1	X	.	X	.	.	X	.	X	X	X	20
a_2	X	X	X	X	X	.	X	.	X	.	5
a_3	.	X	.	.	.	.	X	X	.	.	4
a_4	X	X	.	.	X	X	.	.	.	.	10
a_5	.	.	.	X	.	.	.	X	X	.	4
a_6	.	.	.	.	.	.	.	.	.	.	7
$d(s)$	1	1	3	1	3	1	1	2	1	3	

Figure 7.4: The problem of Example 7.1.1 after preprocessing.

	s_{12}	s_{10}	s_{11}	s_3	s_5	s_6	$o(Q)$	s_1	s_2	s_9	s_8	s_7	s_4	$c(a)$
a_1	X	X	X	X	—	X	—	X	—	X	X	.	.	20
a_2	.	.	X	X	X	—	—	X	X	X	—	X	X	5
a_3	.	.	.	.	.	.	.	.	X	—	X	X	.	4
a_4	.	.	.	.	X	X	—	X	X	.	.	.	.	10
a_5	.	.	.	.	.	.	.	.	.	X	X	—	X	4
a_6	.	X	.	.	.	.	.	.	.	.	.	.	.	7
$d(s)$	1	2	1	2	3	1		1	1	1	1	2	1	

Figure 7.5: An optimal schedule for the scenes $Q = \{s_1, s_2, s_4, s_7, s_8, s_9\}$ Assuming $\{s_3, s_5, s_6, s_{10}, s_{11}, s_{12}\}$ have already been scheduled

resulting preprocessed problem is show in Figure 7.4.□

7.2.3 Scheduling Actor Equivalent Scenes First

Let $o(Q) = a(S \setminus Q) \cap a(Q)$ be the set of on location actors just before an element of Q is scheduled, i.e., those for whom some of their scenes have already been scheduled (appear in $S \setminus Q$), and some have not (appear in Q). We can reduce the amount of search performed by the code shown in Figure 7.2 (and thus improve its efficiency) by noticing that any scene whose actors are exactly the same as those on location now can always be scheduled first without affecting the optimality of the solution. In other words, for every $s \in Q$ for which $a(s) = o(Q)$, there must be an optimal solution to $schedule(Q)$ that starts with s .

Example 7.2.4. Consider the scene scheduling problem of Example 7.1.1. Let us

assume that the set of scenes $Q = \{s_1, s_2, s_4, s_7, s_8, s_9\}$ is scheduled after those in $S \setminus Q = \{s_3, s_5, s_6, s_{10}, s_{11}, s_{12}\}$ have been scheduled. Then, the set of on location actors after $S \setminus Q$ is scheduled is $o(Q) = \{a_1, a_2, a_4\}$ and an optimal schedule can begin with s_1 since $a(s_1) = o(Q)$. An optimal schedule of this form is shown in Figure 7.5. $\square$

Lemma 7.2.5. *If there exists $s \in Q$ where $a(s) = o(Q)$, then there is an optimal order for $\text{schedule}(Q)$ beginning with s .*

Proof. Let Π denote a possibly empty sequence of scenes. As before, we will sometimes use sequences as if they were sets. Without loss of generality, take the order $\Pi_1 \Pi_2 s' \Pi_3 s \Pi_4$ of the scenes in S where $\Pi_2 s' \Pi_3 s \Pi_4$ is the sequence of scenes in Q , and consider altering the order to $\Pi_1 s \Pi_2 s' \Pi_3 \Pi_4$. We show that the cost for each scene in Q can only decrease.

First, it is clear that the scenes in Π_4 have the same on location actors since the set of scenes before and after it remains unchanged. Second, let us consider the changes in the on locations actors for s' , which can be seen as a general representative of scenes scheduled before s in the original order. While in the original order the set of on location actors at the time s' is scheduled is $l(s', \Pi_3 s \Pi_4) = a(s') \cup (a(\Pi_1 \Pi_2) \cap a(\Pi_3 s \Pi_4))$, in the new order the set of on location actors is $l(s', \Pi_3 \Pi_4) = a(s') \cup (a(\Pi_1 s \Pi_2) \cap a(\Pi_3 \Pi_4))$. Now, for every set of scenes $Q'' \subseteq Q'$ we know that $a(Q'') \subseteq a(Q')$, i.e., increasing the number of scenes can only increase the number of actors involved. Thus, we have that (a) $a(\Pi_3 \Pi_4) \subseteq a(\Pi_3 s \Pi_4)$, and (b) since $a(s) = o(Q) = (a(Q) \cap a(\Pi_1))$ we have that $a(s) \subseteq a(\Pi_1)$, and thus that $a(\Pi_1 s \Pi_2) = a(\Pi_1 \Pi_2)$. Hence, by (a) and (b) we have that $l(s', \Pi_3 \Pi_4) \subseteq l(s', \Pi_3 s \Pi_4)$, which means the set of on location actors for s' in the altered schedule can only decrease and, thus, its cost cannot increase. Finally, we also have to examine the cost for s . Since $a(s) = o(Q)$ we have that $l(s, Q \setminus \{s\}) = a(s)$. That means there is no actor waiting if we schedule s now, which is the cheapest possible way to schedule s . Hence, the costs of scheduling this scene here is no more expensive than in the original position. $\square$

We can modify the pseudo code of Figure 7.2 to take advantage of Lemma 7.2.5 by adding the line

if $(\exists s \in Q. a(s) = o(Q))$ **return** $d(s) \times c(l(s, Q \setminus \{s\})) + \text{schedule}(Q \setminus \{s\})$

before the line $\text{min} := +\infty$.

7.2.4 Pairwise Subsumption

When we have two scenes s_1 and s_2 where the actors in one scene (s_1) are a subset of the actors in the other (s_2) and the extra actors $a(s_2) \setminus a(s_1)$ are already on location then we can guarantee a better schedule if we always schedule s_2 before s_1 . Intuitively, this is because if s_1 is shot first the missing actors would be waiting on location for scene s_2 to be shot, while if s_2 is shot first some of those missing actors might not be needed on location anymore.

Lemma 7.2.6. *If there exists $\{s_1, s_2\} \subseteq Q$, such that $a(s_1) \subseteq a(s_2)$, $a(S \setminus Q) \cup a(s_1) \supseteq a(s_2)$, then for any order of Q where s_1 appears before s_2 , there is a permutation of that order where s_2 appears before s_1 with equal or lower cost.*

Proof. Let Π denote a possibly empty sequence of scenes. As before, we will sometimes use sequences as if they were sets. Without loss of generality, take the order $\Pi_1 \Pi_2 s_1 \Pi_3 s' \Pi_4 s_2 \Pi_5$ of scenes in S where $\Pi_2 s_1 \Pi_3 s' \Pi_4 s_2 \Pi_5$ is the sequence of scenes in Q , and consider the actors on location for scene s_1 to be $l(s_1, \Pi_3 s' \Pi_4 s_2 \Pi_5) = A_1$ and for s_2 to be $l(s_2, \Pi_5) = A_2$. Now either $c(A_1) \leq c(A_2)$ or $c(A_1) > c(A_2)$.

Case $c(A_1) \leq c(A_2)$: We show that choosing $\Pi_1 \Pi_2 s_2 s_1 \Pi_3 s' \Pi_4 \Pi_5$ as new order can only decrease the cost for each scene. The cost of s_1 in the original schedule is the cost of the actors in $l(s_1, \Pi_3 s' \Pi_4 s_2 \Pi_5)$ which is computed as $a(s_1) \cup (a(\Pi_3 s' \Pi_4 s_2 \Pi_5) \cap a(\Pi_1 \Pi_2))$, while for the second schedule is the cost of the actors in $l(s_1, \Pi_3 s' \Pi_4 \Pi_5)$ which is computed as $a(s_1) \cup (a(\Pi_3 s' \Pi_4 \Pi_5) \cap a(\Pi_1 \Pi_2 s_2))$. Since by hypothesis $a(\Pi_1) \cup a(s_1) \supseteq a(s_2)$ and by definition $a(\Pi_3 s' \Pi_4 \Pi_5) \subseteq a(\Pi_3 s' \Pi_4 s_2 \Pi_5)$, we have that $l(s_1, \Pi_3 s' \Pi_4 \Pi_5) \subseteq l(s_1, \Pi_3 s' \Pi_4 s_2 \Pi_5)$ and, hence, the cost of s_1 can only decrease.

Regarding s_2 , the set of actors in the new order is

$$\begin{aligned}
l(s_2, s_1 \Pi_3 s' \Pi_4 \Pi_5) &= a(s_2) \cup (a(s_1 \Pi_3 s' \Pi_4 \Pi_5) \cap a(\Pi_1 \Pi_2)) \\
&\text{By definition of } l(s, Q) \\
&= (a(s_2) \cup a(s_1 \Pi_3 s' \Pi_4 \Pi_5)) \cap (a(s_2) \cup a(\Pi_1 \Pi_2)) \\
&\text{Distributing } \cup \text{ over } \cap \\
&= (a(s_1) \cup a(s_2 \Pi_3 s' \Pi_4 \Pi_5)) \cap (a(s_2) \cup a(\Pi_1 \Pi_2)) \\
&\text{By definition of } a(Q) \\
&\subseteq (a(s_1) \cup a(s_2 \Pi_3 s' \Pi_4 \Pi_5)) \cap (a(s_1) \cup a(\Pi_1 \Pi_2)) \\
&\text{By hypothesis of } a(\Pi_1) \cup a(s_1) \supseteq a(s_2) \\
&= l(s_1, \Pi_3 s' \Pi_4 s_2 \Pi_5) \\
&\text{By definition of } l(s, Q)
\end{aligned}$$

which is known to be A_1 . Hence, the cost of s_2 can only decrease in the new schedule.

Lets now consider the other scenes. First, it is clear that the products in Π_1 , Π_2 , and Π_5 have the same on location actors since the set of scenes before and after remain unchanged. Second, let us consider the changes in the on locations actors for s' , which can be seen as a general representative of scenes scheduled in between s_1 and s_2 in the original order. While in the original order the set of on location actors at the time s' is scheduled is $l(s', \Pi_4 s_2 \Pi_5) = a(s') \cup (a(\Pi_1 \Pi_2 s_1 \Pi_3) \cap a(\Pi_4 s_2 \Pi_5))$, in the new order the set of on location actors is $l(s', \Pi_4 \Pi_5) = a(s') \cup (a(\Pi_1 \Pi_2 s_2 s_1 \Pi_3) \cap a(\Pi_4 \Pi_5))$. Clearly (a) $a(\Pi_4 \Pi_5) \subseteq a(\Pi_4 s_2 \Pi_5)$ and (b) since by hypothesis $a(\Pi_1) \cup a(s_1) \supseteq a(s_2)$, we have that $a(\Pi_1 \Pi_2 s_2 s_1 \Pi_3) \subseteq a(\Pi_1 \Pi_2 s_1 \Pi_3)$. Hence, by (a) and (b) we have that $l(s', \Pi_4 \Pi_5) \subseteq l(s', \Pi_4 s_2 \Pi_5)$ and, hence, the cost of scheduling it cannot increase.

Case $c(A_1) > c(A_2)$:

We show that choosing $\Pi_1 \Pi_2 \Pi_3 s' \Pi_4 s_2 s_1 \Pi_5$ as new order can only decrease the cost for each scene.

Regarding s_1 , the set of actors in the new order is

$$\begin{aligned}
l(s_1, \Pi_5) &= a(s_1) \cup (a(\Pi_5) \cap a(\Pi_1 \Pi_2 \Pi_3 s' \Pi_4 s_2)) \\
&\quad \text{By definition of } l(s, Q) \\
&= (a(s_1) \cup a(\Pi_5)) \cap (a(s_1) \cup a(\Pi_1 \Pi_2 \Pi_3 s' \Pi_4 s_2)) \\
&\quad \text{Distributing } \cup \text{ over } \cap \\
&= (a(s_1) \cup a(\Pi_5)) \cap (a(s_2) \cup a(\Pi_1 \Pi_2 s_1 \Pi_3 s' \Pi_4)) \\
&\quad \text{By definition of } a(Q) \\
&\subseteq (a(s_2) \cup a(\Pi_5)) \cap (a(s_2) \cup a(\Pi_1 \Pi_2 s_1 \Pi_3 s' \Pi_4)) \\
&\quad \text{By hypothesis of } a(s_1) \subseteq a(s_2) \\
&= l(s_2, \Pi_5) \\
&\quad \text{By definition of } l(s, Q)
\end{aligned}$$

which is known to be A_2 . Hence, the cost of s_1 can only decrease in the new schedule.

Now since $a(s_1) \subseteq a(s_2)$ we have that $a(s_2 s_1 \Pi_5) = a(s_2 \Pi_5)$ and, since adding scenes can only increase cost, we have that $a(\Pi_1 \Pi_2 \Pi_3 s' \Pi_4) \subseteq a(\Pi_1 s_1 \Pi_2 \Pi_3 s' \Pi_4)$. Thus, $l(s_2, s_1 \Pi_5) \subseteq l(s_2, \Pi_5)$, which means the cost of s_2 can only decrease.

Lets consider the other scenes. As before, it is clear that the products in $\Pi_1 \Pi_2$, and Π_5 have the same on location actors since the set of scenes before and after remain unchanged. Let us then consider the changes in the on locations actors for s' , which can be seen as a general representative of scenes scheduled in between s_1 and s_2 in the original order. While in the original order the set of on location actors at the time s' is scheduled is $l(s', \Pi_4 s_2 \Pi_5) = a(s') \cup (a(\Pi_1 \Pi_2 s_1 \Pi_3) \cap a(\Pi_4 s_2 \Pi_5))$, in the new order the set of on location actors is $l(s', \Pi_4 s_2 s_1 \Pi_5) = a(s') \cup (a(\Pi_1 \Pi_2 \Pi_3) \cap a(\Pi_4 s_2 s_1 \Pi_5))$, Clearly (a) by definition $a(\Pi_1 \Pi_2 s_1 \Pi_3) \supseteq a(\Pi_1 \Pi_2 \Pi_3)$ and (b) by hypothesis of $a(s_1) \subseteq a(s_2)$ we have that $a(\Pi_4 s_2 s_1 \Pi_5) = a(\Pi_4 s_2 \Pi_5)$. Hence, by (a) and (b) we have that $l(s', \Pi_4 s_2 s_1 \Pi_5) \subseteq l(s', \Pi_4 s_2 \Pi_5)$, which means the set of on location actors when s' is scheduled can only decrease and, hence, the cost of scheduling cannot increase. $\square$

Example 7.2.7. Consider the scene scheduling problem of Example 7.1.1. Let us assume that the set of scenes $Q = S \setminus \{s_5\}$ is scheduled after s_5 . Then, the on location

actors after $\{s_5\}$ are $o(Q) = \{a_2, a_4\}$. Consider s_1 and s_6 . Since $a(s_6) \subseteq a(s_1)$ and $o(Q) \cup a(s_6) \supseteq a(s_1)$, s_1 should be scheduled before s_6 . This means we should never consider scheduling s_6 next! $\square$

We can modify the pseudo code of Figure 7.2 to take advantage of Lemma 7.2.6 by adding the line

```

forall ( $s_1 \in T$ )
  if ( $\exists s_2 \in T. a(s_1) \subseteq a(s_2) \wedge a(S \setminus Q) \cup a(s_1) \supseteq a(s_2)$ )  $T := T \setminus \{s_1\}$ 

```

after the line $T := Q$ and before the **while** loop. However, this is too expensive in practice. To make this efficient enough we need to precalculate the pairs P of the form (s, s') where $a(s) \subseteq a(s')$ and just check that $s' \in T$, $s \in T$ and $a(S \setminus Q) \cup a(s) \supseteq a(s')$ for each pair in P .

Pairwise subsumption, was first used in the solution of [145, 144], although restricted to cases where the difference in the sets is one or two elements. Although no formal proof is given, there is an extensive example in [144] explaining the reasoning for the case where the scenes differ by one element.

7.2.5 Optimizing Extra Cost

The *base cost* of a scene scheduling problem is given by $\sum_{s \in S} d(s) \times c(a(s))$. This is the cost for just paying for the time of the actors of the scenes they actually appear in. Instead of minimizing the total cost, we can minimize the *extra cost* which is the total cost minus the base cost (i.e., the cost of paying for actors that are waiting, rather than playing). We can recover the minimal cost by simply adding the base cost to the minimal extra cost.

To do so we simply need to change the **cost** and **lower** functions used in Figure 7.2 as follows:

$$\text{cost}(s, Q) = d(s) \times c(l(s, Q) \setminus a(s))$$

$$\text{lower}(Q) = 0.$$

The main benefit of this optimization is simply that the cost of computing the lower bounds becomes free.

7.3 Bounded Dynamic Programming

We can modify our problem to be a bounded problem. Let $bnd_schedule(Q, U)$ be the minimal cost required to schedule scenes Q if this is less than or equal to the bound U , and otherwise some number k where $U < k \leq schedule(Q)$. We can change the dynamic program to take into account upper bounds U on a solution of interest. The recurrence equation becomes

$$bnd_schedule(Q, U) = \begin{cases} 0 & Q = \emptyset \vee U < 0 \\ \min_{s \in Q} d(s) \times c(a(s, Q \setminus \{s\})) + \\ \quad bnd_schedule(Q \setminus \{s\}, U - d(s) \times c(a(s, Q \setminus \{s\}))), & \text{otherwise} \end{cases}$$

The only complexity here is that the upper bound is reduced in the recursive relation to take into account the cost of scene s .

Using bounding can have two effects, one positive and one negative. On the positive side, we may be able to determine without much search that a subproblem cannot provide a better solution for the original problem, thus restricting the search. On the negative side, it may increase the search space since we have now multiplied the potential number of subproblems by the upper bound U .

7.3.1 Bounded Best-first Algorithm

Some of the potential subproblem explosion of adding bounds can be ameliorated since if $schedule(Q) \leq U$ then $bnd_schedule(Q, U) = schedule(Q)$ and, otherwise, $bnd_schedule(Q, U) \leq schedule(Q)$ (i.e., $bnd_schedule(Q, U)$ is a lower bound for $schedule(Q)$). Therefore, we only need to store one answer in the hash table for problem Q (rather than one per U): either the value $OPT(v)$ indicating we have computed the optimal answer v , or the value $LB(v)$ indicating we have determined a lower bound v on the answer. We assume the hash table is initialized with entries *NONE* indicating no result has been stored. The only time we have to reevaluate a subproblem Q is if the stored lower bound v is less than or equal than the current U .

```

bnd_schedule( $Q, U$ )
  if ( $Q = \emptyset$ ) return 0
  if ( $\text{scost}[Q] = \text{OPT}(v)$ ) return  $v$ 
  if ( $\text{scost}[Q] = \text{LB}(v) \wedge v > U$ ) return  $v$ 
   $\text{min} := +\infty$ 
   $T := Q$ 
  while ( $T \neq \emptyset$ )
     $s := \text{index} \min_{s \in T} \text{cost}(s, Q \setminus \{s\}) + \text{lower}(Q \setminus \{s\})$ 
     $T := T \setminus \{s\}$ 
    if ( $\text{cost}(s, Q \setminus \{s\}) + \text{lower}(Q \setminus \{s\}) \geq U$ ) break
     $\text{sp} := \text{cost}(s, Q \setminus \{s\}) + \text{bnd\_schedule}(Q \setminus \{s\}, U - \text{cost}(s, Q \setminus \{s\}))$ 
    if ( $\text{sp} < \text{min}$ )  $\text{min} := \text{sp}$ 
    if ( $\text{min} \leq U$ )  $U := \text{min}$ 
  if ( $\text{min} \leq U$ )  $\text{scost}[Q] := \text{OPT}(\text{min})$ 
  else  $\text{scost}[Q] := \text{LB}(\text{min})$ 
  return  $\text{min}$ 

```

Figure 7.6: Pseudo-code for bounded best-first call-based dynamic programming algorithm. **bnd_schedule**(Q, U) returns the minimum cost required for scheduling the set of scenes Q if it is less than or equal to U . Otherwise it returns a lower bound on the minimal cost

The code for the bounded dynamic program is shown in Figure 7.6. Note that the hash table handling is slightly more complex, since we can immediately return a lower bound $v > U$ if that is stored in the hash table already.

The key advantage w.r.t. efficiency is that the break in the **while** loop uses the value U rather than min , since clearly no schedule beginning with s will be able to give a schedule costing less than U in this case. This requires us to update the bound U if we find a new minimum. When the search completes we have either discovered the optimal (if it is less than U), in which case we store it as optimal, or we have discovered a lower bound ($> U$) which we store in the hash table.

This means we can prune more subproblems. Note that this kind of addition of bounds can be automated [117].

7.3.2 Upper Bounds

Now that we are running a bounded dynamic program, we need an initial upper bound for the original problem. A trivial upper bound is the maximum possible cost, i.e., if all actors are on location at all times:

$$\left(\sum_{s \in S} d(s)\right) \times \left(\sum_{a \in A} c(a)\right)$$

To generate a better upper bound, we use a heuristic based on the idea that keeping expensive actors waiting around is bad. Thus, it prioritises expensive actors by attempting to keep their scenes together as much as possible (i.e., as long as this does not imply separating scenes of more expensive actors). To do this, the algorithm maintains a sequence of disjoint sets of scenes (each set corresponding to the scenes kept together for some actors) which provides a partial schedule, i.e, the scenes in a set are known to be scheduled after the scenes in any set to the left and before the scenes in any set to the right. The idea is to (a) only partition sets into smaller sets when this benefits the next actor to be processed, and (b) never to insert new scenes into the middle of the schedule (i.e., scenes are never added to an already formed set, and sets are only added at the beginning or the end of the partial schedule).

Initially the schedule is empty. And the remaining actors are $R = A$. Then, we select the remaining actor $a \in R$ with greatest fixed cost $c(a) \times (\sum_{s \in S, a \in a(s)} d(s))$, and we determine the first and last sets in the schedule involving a . If the actor is currently not involved in any set, then we simply add a new set at the end of the schedule with all the scenes in which a appears. If all the scenes involving a are in a single set, we break the set into those involving a and those not, arbitrary placing the second set afterwards. If all the scenes involving a are already scheduled, we split the first set that involves the actor into two: first those not involving the actor, and then those involving the actor. We do the same for the last set involving the actor, except that the set involving the actor goes first. This ensures that the scenes involving a are placed as close as possible without disturbing the scheduling of the previous actors.

If not all the scenes involving a are already scheduled, we first need to decide

	s_{10}	s_{12}	s_8	s_3	s_9	s_{11}	s_1	s_6	s_2	s_5	s_7	s_4	$c(a)$
a_1	X	X	X	X	X	X	X	X	.	.	.	.	20
a_2	.	.	.	X	X	X	X	–	X	X	X	X	5
a_3	.	.	X	–	–	–	–	–	X	–	X	.	4
a_4	.	.	.	.	.	.	X	X	X	X	.	.	10
a_5	.	.	X	–	X	–	–	–	–	–	–	X	4
a_6	X	.	.	.	.	.	.	.	.	.	.	.	7
$d(s)$	1	2	1	2	3	1	1	1	1	1	2	1	

Figure 7.7: The schedule defined by the heuristic upper bound algorithm.

whether to put the set of remaining scenes at the beginning or the end of the current schedule. To do this we calculate the duration for which actor a will be waiting on location if the scenes are scheduled before or after, and place their remaining scenes wherever it leads to the smallest duration. Then, if we place the scenes afterwards, we split the group where the actor a first appears into two: first those not involving the actor, and then those involving the actor. Similarly, if the remaining scenes are scheduled at the beginning we split the last group where a appears into two: first those involving the actor, and those not involving the actor.

This process continues until all actors are considered. We may have some groups which are still not singletons after this process. We order them in any way, since it cannot make a difference to the cost.

Note that this algorithm ensures that the two most expensive actors will never be waiting.

Example 7.3.1. Consider the scene scheduling problem from Example 7.1.1. The fixed cost of the actors $a_1, a_2, a_3, a_4, a_5, a_6$ are respectively 220, 55, 16, 60, 16, 14. Thus, we first schedule all scenes involving a_1 in one group $\{s_1, s_3, s_6, s_8, s_9, s_{10}, s_{11}, s_{12}\}$. We next consider a_4 , which has some scenes scheduled (s_1 and s_6) and some not (s_2 and s_5). Thus, we first need to decide whether to place the set $\{s_2, s_5\}$ after or before the current schedule. Since the duration for which a_4 will be waiting on location is 0 in both cases, we follow the default (place it after) and split the already scheduled group into those not involving a_4 and those involving a_4 , resulting in partial schedule $\{s_3, s_8, s_9, s_{10}, s_{11}, s_{12}\} \{s_1, s_6\} \{s_2, s_5\}$. The total durations of the groups are 9, 2,

and 4 respectively

We next consider a_2 , whose scenes $\{s_4, s_7\}$ are not scheduled. The duration for a_2 placing these at the beginning is $9 + 2 = 11$, while placing them at the end is $2 + 4 = 6$. Thus, again we place them at the end, and split the first group, obtaining the partial schedule $\{s_8, s_{10}, s_{12}\} \{s_3, s_9, s_{11}\} \{s_1, s_6\} \{s_2, s_5\} \{s_4, s_7\}$.

We next consider a_3 , whose scenes are all scheduled and some appear in the first and the last group. We thus split these two groups to obtain $\{s_{10}, s_{12}\} \{s_8\} \{s_3, s_9, s_{11}\} \{s_1, s_6\} \{s_2, s_5\} \{s_7\} \{s_4\}$. Then we consider a_5 , whose scenes are also all scheduled and appear first in the second group and last in the last group. Splitting these groups has no effect since a_5 appears in all scenes in the group so the partial schedule is unchanged. Similarly a_6 only appears in one group (the first) so this is split into those containing a_6 and those not to obtain $\{s_{10}\} \{s_{12}\} \{s_8\} \{s_3, s_9, s_{11}\} \{s_1, s_6\} \{s_2, s_5\} \{s_7\} \{s_4\}$. The final resulting schedule is shown in Figure 7.7. $\square$

Note that we can easily improve a heuristic solution of a scene scheduling problem by considering swapping the positions of any two pairs of scenes, and making the swap if it lowers the total cost. This heuristic method is explored in [23]. We also tried a heuristic that attempted to build the schedule from the middle by first choosing the most expensive scene and then choosing the next scene that minimizes cost to left or right. However, our experiments indicate that the upper bounds provided by any heuristic have very little effect on the overall computation of the optimal order, probably because the `bnd_schedule` function overwrites the upper bound as soon as it finds a better solution. Hence, we did not explore many options for better heuristic solutions. Instead, we focused on devising better search strategies.

7.3.3 Looking Ahead

We can further reduce the search performed in `bnd_schedule` (and `schedule`) by looking ahead. That is, we examine each of the subproblems we are about to visit, and if we have already calculated an optimal value or correct bound for them, we can use this to get a better estimate of the lower bound cost. Furthermore, we can use this opportunity to change the lower bound function so that it memos any lower bound

calculated in the hash table *scost*. The only modification required is to change the definition of the **lower** function to:

```

lower(Q)
  if (scost[Q] = OPT(v)) return v
  if (scost[Q] = LB(v)) return v
  lb :=  $\sum_{s \in Q} d(s) \times c(a(s))$  %% if we are using normal costs
  lb := 0                               %% if we are using extra costs
  scost[Q] := LB(lb)
  return lb

```

This has the effect of giving a much better lower bound estimate and, hence, reducing search.

Lookahead is somewhat related to the lower bounding technique used in Russian Doll Search [150], but in that case all smaller problems are forced to be solved before the larger problem is tackled, while lookahead is opportunistic, just using results that are already there.

7.3.4 Better Lower Bounds

If we are storing the lower bound computations, as described in the previous subsection, it may be worthwhile spending more time to derive a better lower bound. Here we describe a rather complex lower bound which is strong enough to reduce the number of subproblems examined by 1-2 orders of magnitude. We use the following straightforward result

Lemma 7.3.2. *Let $a_i, b_i, i = 1, \dots, n$ be positive real numbers. Let π be a permutation of the indices. Define $f(\pi) = \sum_{i=1}^n [a_{\pi(i)} \times \sum_{j=1}^i b_{\pi(j)}]$. The permutation π which minimizes $f(\pi)$ satisfies $b_{\pi(1)}/a_{\pi(1)} \leq b_{\pi(2)}/a_{\pi(2)} \leq \dots \leq b_{\pi(n)}/a_{\pi(n)}$. $\square$*

This lemma allows us to solve certain special cases of the Talent Scheduling Problem with a simple sort. Consider the following special case. We have a set of actors

$a_1, \dots, a_n$ already on location, and a set of scenes $s_1, \dots, s_n$ where s_i only involves the actor a_i for each i . Then given a schedule $s_{\pi(1)} s_{\pi(2)} \dots s_{\pi(n)}$ where π is some permutation, the cost is given by $f(\pi) = \sum_{i=1}^n c(a_{\pi(i)}) \times \sum_{j=1}^i d(s_{\pi(j)})$. This is of the form required for Lemma 7.3.2, and we can find the optimal scene permutation π_{opt} simply by sorting the numbers $d(s_i)/c(a_i)$ in ascending order. The minimum cost can then be calculated by a simple summation. Unfortunately, in general, the subproblems for which we wish to calculate lower bounds do not fall under the special case, as scenes generally involve multiple actors. To take advantage of the lemma then, we need to do much more work. The following theorem shows how we can take advantage of Lemma 7.3.2 by relaxing the general case of the Talent Scheduling Problem into the special form required.

Theorem 7.3.3. *Let Q be a set of scenes remaining to be scheduled. Let $A' = o(Q)$, the actors currently on location. Without loss of generality, let $A' = \{a_1, \dots, a_n\}$. Let $Q' \subseteq Q$ be the set of unscheduled scenes that involve at least one actor from A' . Let $sc(s) = \sum_{a \in A' \cap a(s)} c(a)$. Let $x(a, s) = 1$ if $a \in a(s)$, and 0 otherwise. Let $w(a, s) = x(a, s) \times c(a)/sc(s)$. Let $e(a) = \sum_{s \in Q'} w(a, s) \times d(s)$. Let $f(\pi) = \sum_{k=1}^n c(a_{\pi(k)}) \times \sum_{i=1}^k e(a_{\pi(i)})$. A correct lower bound on the extra cost for actors A' for scenes Q' is given by $f(\pi_{opt}) - \sum_{s \in Q'} d(s) \times [sc(s) + \sum_{a \in A' \cap a(s)} c(a)^2/sc(s)]/2$, where π_{opt} is the permutation of the indices given by sorting $r(a_i) = e(a_i)/c(a_i)$ in ascending order.*

Proof. First we describe what each of the defined quantities mean. $sc(s)$ gives the sum of the cost of the actors for scene s , but only counting the actors which are currently on location. $w(a, s)$ is a measure of how much actor a is contributing to the cost of scene s . We have $0 \leq w(a, s) \leq 1$, and $\sum_{a \in A' \cap a(s)} w(a, s) = 1$. $e(a)$ is a weighted sum of the duration of the scenes that a is involved in, weighted by $w(a, s)$. $f(\pi)$ is constructed so that it follows the form required for Lemma 7.3.2 to apply, which we will take advantage of. The actual lower bound is given by the minimum value of $f(\pi)$, minus a certain constant.

Given any complete schedule that extends the current partial schedule, there is an order in which the on location actors $a_1, \dots, a_n$ may finish. Without loss of generality,

label the actors so that they finish in the order $a_1, a_2, \dots, a_n$ (break ties randomly). We have the following inequalities for the cost of the remaining schedule $t(a)$ for each of these actors:

$$\begin{aligned} t(a_k) &\geq c(a_k) \times \sum_{\{s \in Q' \mid \exists i, i \leq k, a_i \in a(s)\}} d(s) \\ &\geq c(a_k) \times [\sum_{i=1}^k e(a_i) + \sum_{\{s \in Q' \mid a_k \in a(s)\}} [d(s) \times (1 - \sum_{i=1}^k w(a_i, s))]] \end{aligned}$$

These inequalities hold for the following reasons. Consider a_k . Any scene which involves any of $a_1, \dots, a_k$ must be scheduled before a_k can leave, since by definition $a_1, \dots, a_{k-1}$ leave no later than a_k . So for such scenes s , we must pay $c(a_k) \times d(s)$ for actor a_k , which gives rise to the first inequality. Now, in the second line, the scene durations from the first line are split up and summed together in a different way, with some terms thrown away. The second line consists of two sums within the outer set of square brackets. A scene which does not involve any of $a_1, \dots, a_k$ will not be counted in any $e(a_i)$ in the first sum, and is not counted by the second sum which only counts scenes involving a_k . So as required, such durations do not appear in the second line. A scene which involves some of $a_1, \dots, a_k$ will have part of its duration counted in the first sum. To be exact, a proportion $\sum_{i=1}^k w(a_i, s) \leq 1$ of it is counted in the first sum. The second sum counts the bits that were not counted in the first sum for scenes that involve a_k . Since the second line never counts more than $d(s)$ for any scene appearing in the first line, the inequality is valid.

Now, we split the last line of the inequality into its two parts and sum over the actors. Define U and V as follows:

$$\begin{aligned} U &= \sum_{k=1}^n c(a_k) \times \sum_{i=1}^k e(a_i) \\ V &= \sum_{k=1}^n c(a_k) \times \sum_{\{s \in Q' \mid a_k \in a(s)\}} [d(s) \times (1 - \sum_{i=1}^k w(a_i, s))] \end{aligned}$$

Then $U + V$ is a lower bound on the cost for the actor finish order $a_1, \dots, a_n$. As can be seen, U corresponds to $f(\pi)$ in the theorem. Different permutations of actor finish order π will give rise to different values of U equal to $f(\pi)$. By applying Lemma 7.3.2, we can quickly find a lower bound on U over all possible actor finish orders. That is, for each actor a , we calculate $r(a) = e(a)/c(a)$. We then sort the actors based on $r(a)$ from smallest to largest and label them from a'_1 to a'_n . We then calculate U using finish order $a'_1, \dots, a'_n$, which will give us a lower bound on U over

all possible actor finish orders.

V , on the other hand, although it looks like it depends on the actor finish order, actually evaluates to a constant.

$$\begin{aligned}
V &= \sum_{k=1}^n c(a_k) \times \sum_{\{s \in Q' \mid a_k \in a(s)\}} [d(s) \times (1 - \sum_{i=1}^k w(a_i, s))] \\
&= \sum_{k=1}^n \sum_{\{s \in Q' \mid a_k \in a(s)\}} c(a_k) \times [d(s) \times (1 - \sum_{i=1}^k w(a_i, s))] \\
&= \sum_{s \in Q'} \sum_{k=1, a_k \in a(s)}^n c(a_k) \times [d(s) \times (1 - \sum_{i=1}^k w(a_i, s))] \\
&= \sum_{s \in Q'} \sum_{k=1, a_k \in a(s)}^n c(a_k) \times d(s) - \sum_{s \in Q'} \sum_{k=1, a_k \in a(s)}^n c(a_k) \times d(s) \times \sum_{i=1}^k w(a_i, s) \\
&= \sum_{s \in Q'} \sum_{k=1, a_k \in a(s)}^n c(a_k) \times d(s) - \sum_{s \in Q'} \sum_{k=1, a_k \in a(s)}^n c(a_k) \times d(s) \times \sum_{i=1, a_i \in a(s)}^k c(a_i) / sc(s) \\
&= \sum_{s \in Q'} \sum_{k=1, a_k \in a(s)}^n c(a_k) \times d(s) - \sum_{s \in Q'} d(s) / sc(s) \times \sum_{k=1, a_k \in a(s)}^n \sum_{i=1, a_i \in a(s)}^k c(a_k) \times c(a_i)
\end{aligned}$$

The first double sum is simply the base cost needed to pay each actor for each scene they appear in, and is clearly a constant. Of the second term, only the innermost double sum may be dependent on the actor finish order. Let $W(s) = \sum_{k=1, a_k \in a(s)}^n \sum_{i=1, a_i \in a(s)}^k c(a_k) \times c(a_i)$.

$$\begin{aligned}
2 \times W(s) &= 2 \times \sum_{k=1, a_k \in a(s)}^n \sum_{i=1, a_i \in a(s)}^k c(a_k) \times c(a_i) \\
&= \sum_{k=1, a_k \in a(s)}^n \sum_{i=1, a_i \in a(s)}^k c(a_k) \times c(a_i) + \sum_{i=1, a_i \in a(s)}^n \sum_{k=i, a_k \in a(s)}^n c(a_k) \times c(a_i) \\
&= \sum_{k=1, a_k \in a(s)}^n \sum_{i=1, a_i \in a(s)}^k c(a_k) \times c(a_i) + \sum_{k=1, a_k \in a(s)}^n \sum_{i=k, a_i \in a(s)}^n c(a_i) \times c(a_k) \\
&= \sum_{k=1, a_k \in a(s)}^n \sum_{i=1, a_i \in a(s)}^n c(a_k) \times c(a_i) + \sum_{k=1, a_k \in a(s)}^n c(a_i) \times c(a_i) \\
&= sc(s)^2 + \sum_{k=1, a_k \in a(s)}^n c(a_k)^2 \\
W(s) &= [sc(s)^2 + \sum_{k=1, a_k \in a(s)}^n c(a_k)^2] / 2
\end{aligned}$$

which is constant. Now, $U + V$ gives a lower bound for the total cost. A lower bound for the extra cost is simply $U + V$ minus the base cost of the actors A' for the scenes Q' . Luckily, this term already appears as the first term in V . Thus the lower bound for the extra cost is $f(\pi_{opt}) - \sum_{s \in Q'} d(s) / sc(s) \times W(s) = f(\pi_{opt}) - \sum_{s \in Q'} d(s) \times [sc(s) + \sum_{a \in A' \cap a(s)} c(a)^2 / sc(s)] / 2$ as claimed. $\square$

Example 7.3.4. Consider the scenes shown in Figure 7.8, where the cost/duration of each actor/scene is 1 for simplicity. To calculate $f(\pi_{opt})$, we need to calculate $r(a)$ and sort them. Since the costs are all 1, we have $r(a_1) = e(a_1) = 11/6$, $r(a_2) = e(a_2) = 2$, $r(a_3) = e(a_3) = 11/6$, $r(a_4) = e(a_4) = 4/3$. So we reorder the actors as $a'_1 = a_4$, $a'_2 = a_1$, $a'_3 = a_3$, $a'_4 = a_2$ and calculate $f(\pi)$ using finish order $a'_1, \dots, a'_4$, which gives $f(\pi_{opt}) = 1 \times 4/3 + 1 \times (4/3 + 11/6) + 1 \times (4/3 + 11/6 + 11/6) + 1 \times (4/3 + 11/6 + 11/6 + 2) =$

	s_1	s_2	s_3	s_4	s_5	s_6	s_7	
a_1	X	.	.	.	X	.	X	1
a_2	.	X	.	.	X	X	.	1
a_3	.	.	X	.	.	X	X	1
a_4	.	.	.	X	.	.	X	1
	1	1	1	1	1	1	1	

Figure 7.8: An original set of remaining scenes, assuming a_1, a_2, a_3, a_4 are on location.

16.5, which is a lower bound on U over all actor finish orders. Next, we calculate $\sum_{s \in Q} d(s) \times [sc(s) + \sum_{a \in A' \cap a(s)} c(a)^2 / sc(s)] / 2 = 1 + 1 + 1 + 1 + 3/2 + 3/2 + 2 = 9$. Thus the lower bound for the extra cost at this node is $16.5 - 9 = 7.5$. $\square$

If we are optimizing the extra cost (Section 7.2.5), then to implement this lower bound, we simply need to add the following code into the code for **lower** before the saving of the lower bound in *scost*.

```

 $A' := o(Q)$ 
for ( $a \in A'$ )
     $r[a] := 0$ 
for ( $s \in Q'$ )
     $a'(s) = a(s) \cap A'$ 
     $total\_cost := \sum_{i \in a'(s)} c(i)$ 
     $total\_cost\_sq := \sum_{i \in a'(s)} c(i)^2$ 
    for ( $a \in a'(s)$ )
         $r[a] = r[a] + d(s) / total\_cost$ 
         $lb = lb - d(s) \times (total\_cost + total\_cost\_sq / total\_cost) / 2$ 
Sort  $A'$  based on  $r[a]$  in ascending order
 $c := \sum_{i \in A'} c(i)$ 
for ( $a \in A'$ )
     $lb = lb + c \times r[a] \times c(a)$ 
     $c = c - c(a)$ 

```

Clearly this is quite an expensive calculation.

7.4 Double Ended Search

We will say an actor is *fixed* if we know the first and last scene where the actor appears. Knowing that an actor is fixed is useful, because the cost for that actor is fixed (thus the name) regardless of the schedule of the remaining intervening scenes, if any. For this reason it is beneficial to search for a solution by alternatively placing the next scene in the first remaining unfilled slot and the last remaining unfilled slot, since this will increase the number of fixed actors. Let B denote the set of scenes scheduled at the beginning, and E the set of scenes scheduled at the end. We know the cost of any actor appearing both in scenes of B and scenes of E , since we know the duration of the remaining set of scenes $Q = S \setminus B \setminus E$. This strategy was used in the branch-and-bound solution of [23]. A priori this might appear to be a bad strategy since the search space has increased: there are more subproblems of the form “schedule remaining scenes Q given scenes in B are scheduled before and scenes in E are scheduled after (where $B \cup Q \cup E = S$)”, than there are “schedule remaining scenes Q given scenes in $S \setminus Q$ are scheduled before”. However, as we will see in the experiments, this is compensated by the fact that we will get much more accurate estimates on the cost of the remaining schedule.

The change in search strategy causes considerable changes to the algorithm. The sub problems are now defined by B the set of scenes scheduled at the beginning, and E the set of scenes scheduled at the end. The search tries to schedule each remaining scene s at the beginning of the remaining scenes, just after B , and then swaps the role of B and E to continue building the schedule. We can thus modify the cost function to ignore the cost of actors already fixed by B and E (i.e., those in $a(B) \cap a(E)$), and take only into account the cost of actors *newly* fixed by the scene. This can be done as follows:

$$\begin{aligned} \text{cost}(s, B, E) = & d(s) \times c(l(s, S \setminus B \setminus E \setminus \{s\}) \setminus (a(B) \cap a(E))) \\ & + \sum_{a \in ((a(s) \setminus a(B)) \cap a(E))} d(S \setminus B \setminus E \setminus \{s\}) \times c(a) \end{aligned}$$

```

bnd_de_schedule( $B, E, U$ )
   $Q := S \setminus B \setminus E$ 
  if ( $a(Q) \subseteq a(B) \cap a(E)$ ) return 0
   $hv := \text{hash\_lookup}(B, E)$ 
  if ( $hv = OPT(v)$ ) return  $v$ 
  if ( $hv = LB(v) \wedge v > U$ ) return  $v$ 
   $min := +\infty$ 
   $T := Q$ 
  while ( $T \neq \emptyset$ )
     $s := \text{index}_{s \in T} \min \text{cost}(s, B, E) + \text{lower}(B \cup \{s\}, E)$ 
     $T := T \setminus \{s\}$ 
    if ( $\text{cost}(s, B, E) + \text{lower}(B \cup \{s\}, E) \geq U$ ) break
     $sp := \text{cost}(s, B, E) + \text{bnd\_de\_schedule}(E, B \cup \{s\}, U - \text{cost}(s, B, E))$ 
    if ( $sp < min$ )  $min := sp$ 
    if ( $min \leq U$ )  $U := min$ 
  if ( $min \leq U$ )  $\text{hash\_set}(B, E, OPT(min))$ 
  else  $\text{hash\_set}(B, E, LB(min))$ 
  return  $min$ 

```

Figure 7.9: Pseudo-code for bounded best-first call-based dynamic programming algorithm. $\text{bnd_de_schedule}(Q, B, E)$ returns the minimum cost required for scheduling the set of scenes Q if it is less than or equal to U . Otherwise it returns a lower bound on the minimal cost

where the first part adds the cost for scheduling scene s excluding the fixed actors ($a(B) \cap a(E)$), and the second part adds the cost of each actor a which is newly scheduled by s (appears in $(a(s) \setminus a(B))$) and already scheduled at the end (appears in $a(E)$).

The lower bound cost function also has to change to ignore the actors fixed by B and E :

$$\text{lower}(B, E) = \sum_{s \in S \setminus B \setminus E} d(s) \times c(a(s) \setminus (a(B) \cap a(E)))$$

The code for the new algorithm is shown in Figure 7.9. The algorithm first tests whether there are any remaining actors to be scheduled: If $a(Q) \subseteq a(B) \cap a(E)$

then all actors playing in scenes of Q are fixed (must be on location for the entire period regardless of Q schedule), and we simply return 0 (since their cost has already been taken into account). Otherwise, the algorithm checks the hash table to find whether the subproblem has been examined before. Note that we replaced the array of subproblems $scost[Q]$ by two functions `hash_lookup( $B, E$ )`, which returns the value stored for subproblem B, E , and `hash_lookup( $B, E, ov$ )`, which sets the stored value to ov . The remainder of the code is effectively identical to `bnd_schedule` using the new definitions of `cost` and `lower`. The only important thing to note is that the recursive call swaps the positions of beginning and end sets, thus forcing the next scene to be scheduled at the other end.

Note that any solution to the scene scheduling problem has an equivalent solution where the order of the scenes is reversed (a fact that has been noticed by many authors). We are implicitly using this fact in the definition of `bnd_de_schedule` when we reverse the order of the B and E arguments to make the search double ended, since we treat the problem starting with B and ending in E as equivalent to the problem starting with E and ending in B . We can also take advantage of this symmetry when detecting equivalent subproblems (i.e. when looking up whether we have seen the problem before). A simple way of achieving this is to store and lookup problems assuming that $B \leq E$ (that is, in lexicographic order).

```

hash_lookup( $B, E$ )  =  if ( $B \leq E$ )  $scost[B, E]$  else  $scost[E, B]$ 
hash_set( $B, E, ov$ ) =  if ( $B \leq E$ )  $scost[B, E] := ov$  else  $scost[E, B] := ov$ 

```

7.4.1 Better Equivalent Subproblem Detection

While taking into account symmetries helps, we can further help the detection of equivalent subproblems by noticing that the cost of scheduling the scenes in $Q = S \setminus B \setminus E$ does not really depend on B and E . Rather, it depends on $o(B)$ and $o(E)$, i.e., on the set of actors that will always be on location at the beginning and at the end of Q , respectively.

Example 7.4.1. Consider the partial schedule of the problem of Example 7.1.1 where

	s_{12}	s_1	s_9	$o(B)$	s_4	s_8	s_2	s_7	s_{10}	$o(E)$	s_5	s_6	s_{11}	s_3	$c(a)$
a_1	X	X	X	—	—	X	—	—	X	—	—	X	X	X	20
a_2	.	X	X	—	X	—	X	X	—	—	X	—	X	X	5
a_3	.	.	.	.	.	X	X	X	.	.	.	.	.	.	4
a_4	.	X	—	—	—	—	X	—	—	—	X	X	.	.	10
a_5	.	.	X	—	X	X	.	.	.	.	.	.	.	.	4
a_6	.	.	.	.	.	.	.	.	X	.	.	.	.	.	7
$d(s)$	1	1	1		1	1	1	2	2		3	1	1	2	
	s_3	s_{11}	s_5	s_1	$o(B')$	s_{10}	s_7	s_2	s_8	s_4	$o(E')$	s_9	s_6	s_{12}	$c(a)$
a_1	X	X	—	X	—	X	—	—	X	—	—	X	X	X	20
a_2	X	X	X	X	—	—	X	X	—	X	—	X	.	.	5
a_3	.	.	.	.	.	.	X	X	X	.	.	.	.	.	4
a_4	.	.	X	X	—	—	—	X	—	—	—	—	X	.	10
a_5	.	.	.	.	.	.	.	.	X	X	—	X	.	.	4
a_6	.	.	.	.	.	X	.	.	.	.	.	.	.	.	7
$d(s)$	2	1	3	1		2	2	1	1	1		1	1	1	

Figure 7.10: Two equivalent subproblems.

$B = \{s_1, s_9, s_{12}\}$ and $E = \{s_3, s_5, s_6, s_{11}\}$. The remaining scenes to schedule are $Q = \{s_2, s_4, s_7, s_8, s_9, s_{10}\}$. An optimal schedule of Q (given B and E) is shown at the top of in Figure 7.10. The total cost ignoring the fixed actors a_1 , a_2 and a_4 is $16 + 8 + 14 = 48$.

Consider the subproblem where $B' = \{s_3, s_{11}, s_5, s_1\}$ and $E' = \{s_9, s_6, s_{12}\}$. The remaining scenes to schedule are still $Q = \{s_2, s_4, s_7, s_8, s_9, s_{10}\}$. Now $o(B') = o(E)$ and $o(E') = o(B)$ and hence any optimal order for the first subproblem can provide an optimal schedule for this subproblem, by reversing the order of the schedule. This is illustrated at the bottom of Figure 7.10. $\square$

We can modify the hash function to take advantage of these subproblem equivalences. We will store the subproblem value on $o(B)$, $o(E)$, and Q under the assumption that $o(B) \leq o(E)$.

$\text{hash_lookup}(B, E)$

$Q := S \setminus B \setminus E$


```

if ( $o(E) < o(B)$ ) return  $scost[o(E), o(B), Q]$ 
return  $scost[o(B), o(E), Q]$ 

```

```

hash_set( $B, E, ov$ )

```

```

 $Q := S \setminus B \setminus E$ 
if ( $o(E) < o(B)$ )  $scost[o(E), o(B), Q] := ov$ 
else  $scost[o(B), o(E), Q] := ov$ 

```

In order to prove the correctness of the equivalence we need the following intermediate result.

Lemma 7.4.2. *For every $Q, Q' \subseteq S$ such that $Q \cap Q' = \emptyset$ (which is the same as saying $Q' \subseteq S \setminus Q$), we have that $a(Q) \cap a(Q') = o(Q) \cap a(Q') = a(Q') \cap o(Q') = o(Q) \cap o(Q')$.*

Proof. Let us first prove that $o(Q) \cap a(Q') = a(Q) \cap a(Q')$. We have that:

$$\begin{aligned}
 o(Q) \cap a(Q') &= (a(Q) \cap a(S \setminus Q)) \cap a(Q') \\
 &\quad \text{By definition of } o(Q) \\
 &= a(Q) \cap (a(S \setminus Q) \cap a(Q')) \\
 &\quad \text{By associativity of } \cap \\
 &= a(Q) \cap a(Q') \\
 &\quad \text{By hypothesis of } Q' \subseteq S \setminus Q
 \end{aligned}$$

A symmetric reasoning can be done to prove that $a(Q) \cap o(Q') = a(Q) \cap a(Q')$. To prove that $o(Q) \cap o(Q') = a(Q) \cap a(Q')$ we follow a similar reasoning:

$$\begin{aligned}
 o(Q) \cap o(Q') &= (a(Q) \cap a(S \setminus Q)) \cap (a(Q') \cap a(S \setminus Q')) \\
 &\quad \text{By definition of } o(Q) \\
 &= (a(Q) \cap a(S \setminus Q')) \cap (a(S \setminus Q) \cap a(Q')) \\
 &\quad \text{By associativity of } \cap \\
 &= a(Q) \cap a(Q') \\
 &\quad \text{By hypothesis of } Q' \subseteq S \setminus Q \text{ and } Q \subseteq S \setminus Q'
 \end{aligned}$$

□

Given the above result, one could decide to hash on $a(B)$ and $a(E)$ (rather than on $o(B)$ and $o(E)$). This is also correct but it would miss some equivalences since: while $o(B) \cap o(E) = a(B) \cap a(E)$, $a(B)$ might contain more actors than $o(B)$, those who start and finish within B and will thus never be on location during the scenes in Q . Therefore, these actors are not relevant for Q . The same can be said for $a(E)$ and $o(E)$.

Theorem 7.4.3. *Let $\Pi_1\Pi_2\Pi_3$ and $\Pi_4\Pi_2\Pi_5$ be two permutations of S such that $o(\Pi_4) = o(\Pi_1)$, $o(\Pi_5) = o(\Pi_3)$. Then, the cost of every scene of Π_2 is the same in $\Pi_1\Pi_2\Pi_3$ as in $\Pi_4\Pi_2\Pi_5$.*

Proof. Without loss of generality, let Π_2 be of the form $\Pi'_2 s \Pi''_2$. We will show that the cost of s is the same in $\Pi_1\Pi_2\Pi_3$ and $\Pi_4\Pi_2\Pi_5$. Now

$$\begin{aligned}
l(s, \Pi''_2 \Pi_3) &= a(s) \cup (a(\Pi''_2 \Pi_3) \cap a(\Pi_1 \Pi'_2)) \\
&\quad \text{By definition of } l(s, Q) \\
&= a(s) \cup ((a(\Pi''_2) \cup a(\Pi_3)) \cap (a(\Pi_1) \cup a(\Pi'_2))) \\
&\quad \text{By definition of } a(Q) \\
&= a(s) \cup ((a(\Pi''_2) \cap a(\Pi_1)) \cup (a(\Pi''_2) \cap a(\Pi'_2)) \cup (a(\Pi_3) \cap a(\Pi_1)) \cup (a(\Pi_3) \cup a(\Pi'_2))) \\
&\quad \text{Distributing } \cap \text{ over } \cup \\
&= a(s) \cup ((a(\Pi''_2) \cap o(\Pi_1)) \cup (a(\Pi''_2) \cap a(\Pi'_2)) \cup (o(\Pi_3) \cap o(\Pi_1)) \cup (o(\Pi_3) \cup a(\Pi'_2))) \\
&\quad \text{by the Lemma 7.4.2} \\
&= a(s) \cup ((a(\Pi''_2) \cap o(\Pi_4)) \cup (a(\Pi''_2) \cap a(\Pi'_2)) \cup (o(\Pi_5) \cap o(\Pi_4)) \cup (o(\Pi_5) \cup a(\Pi'_2))) \\
&\quad \text{Since } o(\Pi_4) = o(\Pi_1) \text{ and } o(\Pi_5) = o(\Pi_3) \\
&= a(s) \cup ((a(\Pi''_2) \cap a(\Pi_4)) \cup (a(\Pi''_2) \cap a(\Pi'_2)) \cup (a(\Pi_5) \cap a(\Pi_4)) \cup (a(\Pi_5) \cup a(\Pi'_2))) \\
&= a(s) \cup ((a(\Pi''_2) \cup a(\Pi_5)) \cap (a(\Pi_4) \cup a(\Pi'_2))) \\
&= l(s, \Pi''_2 \Pi_5)
\end{aligned}$$

7.4.2 Revisiting the Previous Optimizations

Once we are performing double ended search, we introduce fixed actors which are no longer of any importance to the remaining subproblem since their cost is fixed. We

may be able to improve the previous optimizations by ignoring fixed actors whenever performing a double ended search.

Preprocessing

The second preprocessing step (concatenating duplicate scenes) can now be applied during search. This is because, given fixed actors $F = a(B) \cap a(E)$, we can apply Lemma 7.2.2 if $a(s_1) \cup F = a(s_2) \cup F$, since the cost of the fixed actors is irrelevant. This means we should concatenate any scenes in $Q = S \setminus B \setminus E$ where $a(s_1) \cup F = a(s_2) \cup F$. We can modify the search strategy in `bnd_de_schedule` to break the scenes in Q into equivalent classes $Q_1, \dots, Q_n$ where $\forall s_1, s_2 \in Q_i. a(s_1) \cup F = a(s_2) \cup F$, and then consider scheduling each equivalence class. In many cases the equivalence class will be of size one!

Scheduling Actor Equivalent Scenes First

Lemma 7.2.5 can be extended so that we can always schedule a scene s first where $o(B) = a(s) \cup F$ since the on location actors will include the fixed actors and the extra cost for them will be paid for scene s wherever it is scheduled.

Pairwise Subsumption

The extension of Lemma 7.2.6 also holds if $a(s_1) \cup F \subseteq a(s_2) \cup F$ and $a(B) \cup a(s_1) \supseteq a(s_2)$ (since $F \subseteq a(B)$). But this means we need to do a full pairwise comparison of all scenes in $Q = S \setminus B \setminus E$, for each subproblem considered. We did implement this, and although it did cut down search substantially, the overhead of the extra comparison did not pay off. This is the only optimization not used in the experimental evaluation.

Optimizing Extra Cost

This is clearly applicable in the doubled ended case, but it complicates the computation of $\text{cost}(s, B, E)$ since we now have to determine exactly which scenes a newly fixed actor occurs in, rather than just adding the cost of the actor for the entire duration of the remaining scenes.

Looking Ahead

This is applicable as before. Note that `lower` takes the same arguments (B, E) (excluding the upper bound) as `bnd_de_schedule`. We have to modify the definition of `lower(B, E)` to make use of `hash_lookup` and `hash_set`.

Better Lower Bounds

The same reasoning on better lower bounds can be applied to the set of actors $o(B) \setminus F$, since the actors in F will always be on location in the remaining subproblem.

Indeed, we sum the results of the better lower bounds calculated from both ends for $o(B) \setminus F$ and $o(E) \setminus F$, since the actors in these sets cannot overlap (by the definition of F).

Better Equivalent Subproblem Detection

We could improve equivalent subproblem detection by noticing that the fixed actors play no part in determining the schedule of the remaining scenes $Q = S \setminus B \setminus E$. We could thus build a hash function based on the form of the remaining scenes after eliminating the fixed actors $F = a(B) \cap a(E)$. But the cost of determining this reduced form seems substantial since, in effect, we have to generate new scenes and hash on sets of them. We have not attempted to implement this approach.

7.5 Experiments

We tested our approach on the two sets of problem instances detailed below. All experiments were run on Xeon Pro 2.4GHz processors with 2GB RAM running Red Hat Linux 5.2. The dynamic programming code is written in C, with no great tuning or clever data structures, and many runtime flags to allow us to compare the different versions easily. The dynamic programming software was compiled with gcc 4.1.2 using `-O3`. Timings are calculated as the sum of user and system time given by `getrusage`, since it accords well with wall-clock times for these CPU-intensive programs. For

Table 7.1: Arithmetic mean solving time (ms) for structured problems of size n , and relative slowdown if the optimization is turned off

n	Time (ms)	7.2.3	7.2.4	7.3.3	7.4.2	7.3.2	7.3.4	7.2.5	7.4.1	7.4	7.3
18	78	1.09	1.29	1.24	1.04	1.01	2.75	1.21	0.98	0.44	31.94
19	157	1.15	1.32	1.23	1.04	0.99	3.06	1.20	1.05	0.53	33.18
20	190	1.12	1.39	1.18	1.03	0.99	3.29	1.19	1.02	0.45	47.05
21	317	1.17	1.35	1.17	1.04	1.02	4.13	1.21	1.06	0.51	61.12
22	702	1.18	1.37	1.24	1.06	0.99	3.75	1.19	1.14	0.69	52.16
23	870	1.19	1.48	1.23	1.05	1.01	4.30	1.20	1.09	0.63	—
24	1269	1.23	1.47	1.23	1.11	1.02	5.08	1.19	1.16	0.74	—
25	1701	1.26	1.55	1.25	1.08	1.00	5.32	1.20	1.20	0.86	—
26	2934	1.29	1.63	1.33	1.12	1.01	6.15	1.22	1.25	0.98	—
27	3699	1.31	1.79	1.36	1.14	1.02	7.12	1.23	1.25	1.07	—
28	5172	1.35	1.92	1.38	1.15	1.00	7.83	1.24	1.26	1.17	—

the problems that take significant time we observed around 10% variation in timings across different runs of the same benchmark.

7.5.1 Structured Benchmarks

The first set of benchmarks are structured problems based on the realistic talent scheduling of Mob Story first used in [23]. We use these problems to illustrate the effectiveness of the different optimizations.

We first extended the benchmarks film103, film105, film114, film116, film118, film119 used in [145], adding three new actors to each problem to bring it to 11, and bringing the number of scenes to 28 (the original problems each involve 8 actors and either 18 or 19 scenes). This gave us 6 *base* problems of size 11×28 . These base problems were constructed in such a way that preprocessing did not simplify them (so that the number of “important” actors and scenes was known).

Then, from each base problem we generated smaller problems by removing in turn newly added scenes. In particular, for each base problem we obtained 10 problems ranging from 11×27 to 8×18 , where each problem in the sequence is a subproblem of the larger ones, and the original problem from [145] was included.

Table 7.2: Arithmetic mean subproblems solved for structured problems of size n , and relative increase if the optimization is turned off

n	Subproblems	7.2.3	7.2.4	7.3.3	7.4.2	7.3.2	7.3.4	7.2.5	7.4.1	7.4	7.3
18	5091	1.08	1.18	1.02	1.06	1.06	10.21	1.00	1.07	0.93	73.29
19	9699	1.12	1.22	1.03	1.09	1.01	11.18	0.99	1.11	1.43	75.43
20	10467	1.11	1.26	1.02	1.07	1.01	12.36	1.00	1.09	0.95	114.63
21	16154	1.15	1.21	1.02	1.07	1.03	15.88	1.00	1.14	1.24	152.42
22	36531	1.17	1.23	1.03	1.12	1.00	13.38	0.99	1.21	1.97	121.31
23	42224	1.17	1.32	1.03	1.10	1.02	15.73	1.00	1.13	1.52	—
24	59349	1.22	1.33	1.02	1.21	1.03	18.50	0.98	1.16	1.98	—
25	77766	1.25	1.37	1.03	1.17	1.00	19.00	0.98	1.19	2.08	—
26	136778	1.28	1.40	1.03	1.23	1.01	20.49	1.00	1.20	2.51	—
27	167232	1.28	1.50	1.03	1.25	1.04	23.63	0.99	1.19	2.70	—
28	233328	1.31	1.56	1.03	1.27	1.01	25.04	0.99	1.19	2.88	—

From each base problem we also generated smaller problems by randomly removing k scenes where k varied from 1 to 10. In particular, for each base problem we obtained 10 problems ranging from 11×27 to 11×18 , where the sets of removed scenes in differently sized problem are unrelated (as opposed to have a subset/superset relationship).

In total this created 126 core problems.

From each *core* problem we generated three new variants: **equal duration** where all durations are set to 1, **equal cost** where the cost of all actors are set to 1, and **equal cost and duration** where all durations and costs are set to 1.

We compared the executions of running the dynamic program with all optimizations enabled, and individually turning off each optimization. The optimizations are: scheduling actor equivalent scenes first (Section 7.2.3), pairwise subsumption (Section 7.2.4), looking ahead (Section 7.3.3), concatenating duplicate scenes (Section 7.4.2), upper bounds (Section 7.3.2), better lower bounds (Section 7.3.4), optimizing extra cost (Section 7.2.5), better equivalent subproblem detection (Section 7.4.1), double-ended search (Section 7.4), and bounded dynamic programming (Section 7.3). The average times in milliseconds obtained by running the dynamic program with all optimizations enabled for each size n , are shown in the second column of Table 7.1.

The remaining columns show the relative average time when each of the optimizations is individually turned off. For the last column without bounding, only the problems up to size 22 are shown. Table 7.2 shows the same results in terms of the number of subproblems solved (that is, the number of pairs (B, E) appearing in calls to `bnd_de_schedule` or Q in earlier variants).

The tables clearly show that bounded dynamic programming (Section 7.3) is indispensable for solving these problems. Better lower bounding is clearly the next most important optimization, massively reducing the number of subproblems visited. Doubled-ended search (Section 7.4) is also very important except for the fact that better lower bounding (Section 7.3.4) improves the single-ended search much more than it does the double-ended search, so only on the larger examples does it begin to win. Without better lower bounding it completely dominates single-ended search. The next most effective optimization is pairwise subsumption (Section 7.2.4). Looking ahead (Section 7.3.3) and scheduling actor equivalent scenes first (Section 7.2.3) are quite beneficial, as are optimizing extra cost (Section 7.2.5) and better equivalent subproblem detection (Section 7.4.1). The upper bounds optimization (Section 7.3.2) is clearly unimportant, only reducing the number of problems slightly. Note that while some optimizations given more or less constant improvements with increasing size, most are better as size increases.

If we look at the different variants individually (in results not shown) we find that the **equal duration** variants are slightly (around 7-10%) harder than the core problems, while the **equal cost** and **equal cost and duration** variants are 3–4 times harder than the core problems, indicating that cost is very important for pruning.

7.5.2 Random Benchmarks

The second set of benchmarks is composed of randomly generated benchmarks. We use these problems to show the effect of number of actors and number of scenes on problem difficulty.

The problems were generated in a manner almost identical to that used in [23]: for a given combination of m actors and n scenes we generate for each actor $i \in \{1, \dots, m\}$

Table 7.3: Arithmetic mean solving time (ms) for random problems with m actors and n scenes.

m	Number of Scenes n												
	16	18	20	22	24	26	28	30	32	34	36	38	40
8	7	20	39	94	141	323	362	685	1403	2291	2977	2408	7101
10	11	33	85	165	441	650	1981	2531	3179	8901	10690	13426	20907
12	21	47	149	319	829	2056	3830	6674	10082	13155	20903	—	—
14	25	75	255	759	1519	3700	8862	12705	17602	—	—	—	—
16	41	129	357	1012	2602	6284	14130	23270	—	—	—	—	—
18	53	221	533	1463	3708	11546	18797	—	—	—	—	—	—
20	87	248	757	2745	6680	15414	21194	—	—	—	—	—	—
22	119	338	997	2855	11090	18672	—	—	—	—	—	—	—
m	42	44	46	48	50	52	54	56	58	60	62	64	
8	7697	15669	19004	21703	23939	25891	49547	42433	49406	61351	62089	—	
10	25903	—	—	—	—	—	—	—	—	—	—	—	

Table 7.4: Arithmetic mean subproblems solved for random problems with m actors and n scenes.

m	Number of Scenes n												
	16	18	20	22	24	26	28	30	32	34	36	38	40
8	569	1477	2431	5440	6905	17825	20020	37803	81388	124579	153515	113402	296798
10	784	2377	5408	8747	23898	33692	108048	128041	149676	387515	420769	484846	663511
12	1780	3032	9261	15757	41955	113971	184998	309110	410699	510775	668273	—	—
14	1846	4880	15388	43122	74523	194726	403624	521340	626265	—	—	—	—
16	3071	8153	20366	50527	128969	301001	623235	889799	—	—	—	—	—
18	3218	16317	29785	71885	175373	510349	742264	—	—	—	—	—	—
20	4911	14612	41608	153560	340470	668144	768588	—	—	—	—	—	—
22	4929	17559	52531	138078	547756	782389	—	—	—	—	—	—	—
m	42	44	46	48	50	52	54	56	58	60	62	64	
8	312200	575387	610651	501283	585939	578558	747825	788145	832748	924486	869846	—	
10	736360	—	—	—	—	—	—	—	—	—	—	—	

(a) a random number¹ $n_i \in \{2, \dots, n\}$ indicating the number of scenes actor i is in, (b) n_i different random numbers between 1 and n indicating the set of scenes actor i is in, and (c) a random number between 1 and 100 indicating the cost of actor i . For each combination of actors $m \in \{8, 10, 12, 14, 16, 18, 20, 22\}$ and scenes $n \in \{16, \dots, 64\}$, we generate 100 problems. Note that, given the above method, a scene might contain no actors while an actor must be involved in at least two scenes (and at most all).

We ensured preprocessing could not simplify any instance.

We ran the instances with a memory bound of 2Gb.

¹In [23] they generate a number from $\{1, \dots, n\}$ but actors appearing in only 1 scene are uninteresting (see Section 7.2.2).

Table 7.3 shows the average time in milliseconds obtained for finding an optimal schedule for all random instances of each size which did not run out of memory, while Table 7.4 shows the average number of subproblems solved. The entries — show where less than 80 of the 100 instances solved without running out of memory. The schedules were computed using all optimizations. The results show that while the number of scenes is clearly the most important factor in the difficulty of the problem, if the number of actors is small then the problem difficulty is limited. While increasing the number of actors increases difficulty, as it grows larger than the number of scenes, the incremental difficulty decreases. Note also that the random problems are significantly easier than the structured problems.

While we should be careful when reading these tables, since the difficulty of each 100 random benchmarks considered in each cell can vary remarkably (the standard deviation is usually larger than the average shown), the trend is clear enough.

7.6 Related Work

The talent scheduling problem (which appears as prob039 in CSPLIB [33] where it is called the rehearsal problem) was introduced in [23]. They consider the problem in terms of shooting days instead of scenes so, in effect, all scenes have the same duration. Note, however, that once we make use of Lemma 7.2.2 the requirement for different durations arises in any case. They give one example of a real scene scheduling problem, arising from the film *Mob Story*, containing 8 actors and 28 scenes. They show that the problem is NP-hard even in the very restricted case of each actor appearing in exactly two scenes and all costs and durations being one, by reduction to the optimal linear arrangement (OLA) problem [55]

In their paper they consider two methods to solve the scene scheduling problem. The first method is a branch and bound search, where they search for a schedule by filling in scenes from both ends in an alternating fashion (double ended search). They optimize on extra cost, and the lower bounds they use are simply the result of fixed costs (so equivalent to the definition of **lower** in Section 7.4 minus the fixed costs). They do not store equivalent solutions and, hence, are very limited in the size of the

	25	26	24	27	22	23	19	20	21	5	28	8	11	9	6	7	9	2	16	17	18	3	15	13	14	1	12	3		
Luce	.	.	.	.	.	.	.	.	.	.	.	.	.	X	X	X	X	X	-	-	-	X	X	X	X	X	X	X	X	10
Tom	.	.	.	.	.	X	X	X	X	-	X	X	-	X	-	-	-	X	-	X	X	X	-	X	X	X	X	.	.	4
Mindy	.	.	.	.	.	.	X	X	X	X	-	X	-	X	-	-	-	X	-	X	X	X	.	.	.	.	.	.	.	5
Maria	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	X	X	X	-	X	X	X	.	.	.	.	5
Gianni	.	X	X	X	-	-	X	-	-	-	X	X	-	X	-	-	-	X	X	.	.	.	.	.	.	.	.	.	.	5
Dolores	.	.	X	X	X	X	X	X	X	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	40
Lance	.	.	.	.	.	.	.	X	X	X	-	X	-	X	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	4
Sam	.	.	.	.	.	.	.	.	.	.	.	.	X	X	X	X	X	.	.	.	.	.	.	.	.	.	.	.	.	20
$d(s)$	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	

Figure 7.11: An optimal schedule for the film “Mob Story”

problem they can tackle. Their experiments go up to 14 scenes and 14 actors.

The second method is a simple greedy hill climbing search. Given a starting schedule they consider all possible swaps of pairs of scenes, and move to the schedule after a swap if the resulting cost is less. They continue doing this until they reach a local minimum. On their randomly generated problems the heuristic approach give answers around 10-12% off optimal regardless of size. They use this algorithm to re-schedule Mob Story with an extra cost of \$16,100 as opposed to the hand solution of \$36,400. This solution required 1.05 second on their AMDAHL mainframe. In comparison, our best algorithm finds an optimal answer with extra cost \$14,600 in 0.1 seconds on a Xeon Pro 2.4GHz processor (which is admittedly very much more powerful). The search only considers 6,605 different subproblems. Note that after preprocessing, it only involves 20 scenes. The optimal solution found is shown in Figure 7.11 (costs are divided by 100).

Adelson et al. [1] define a restricted version of the talent scheduling problem for rehearsal scheduling where the costs of all actors are uniform, and also note how it can be used for an application in archeology. They give a dynamic programming formulation as a recurrence relation, more or less identical to that shown at the beginning of Section 7.2. They report solving an instance (from a real archaeological problem) with 26 “actors” and 16 “scenes” in 84 seconds on a CDC 7600 computer. We were not able to locate this benchmark.

Smith [145, 144] uses the talent scheduling problem as an example of a permutation problem. These papers solved the problem using constraint programming search with

Table 7.5: Comparison with the approach of [145] on the examples from that paper.

Problem	Size		Smith		this chapter	
	Actors	Scenes	Time	Cached states	Time	Subproblems
MobStory	8	20	64.71s	136,765	108ms	6,605
film105	8	18	16.07s	40,511	20ms	1,108
film116	8	19	125.8s	225,314	156ms	13,576
film119	8	18	70.80s	144,226	84ms	7,105
film118	8	19	93.10s	205,190	40ms	1,980
film114	8	19	127.0s	267,526	84ms	4,957
film103	8	19	76.69s	180,133	64ms	4,103
film117	8	19	76.86s	174,100	96ms	7,227

caching of search states, which is very similar to dynamic programming with bounds. This paper considers both scheduling from one end, or from both ends. This paper was the first to use a form of pairwise subsumption, restricted to the case where the scenes differ by at most 2 actors. It also used the preprocessing of merging identical scenes (without proof). This was the first approach (we are aware of) to calculate the optimal solution to the Mob Story problem.

A comparison of the approaches is shown in Table 7.5. The table shows the sizes (after preprocessing). Note that the timing results for [145] are for a 1.7GHz Pentium M PC running ILOG Solver 6.0, whereas our results are for Xeon Pro 2.4GHz processors running gcc on Red Hat Linux. However, note also that there is around 3 orders of magnitude difference between our times and those of [145]. Also the number of cached states in the approach of [145] is around two orders of magnitude bigger than the number of subproblems (which is the equivalent measure). This is probably a combination of our better lower bounds, better detection of equivalent states and better search strategy. A web page with the problems and solutions can be found at www.csse.unimelb.edu.au/~pjs/talent/.

The talent scheduling problem is a generalization of the Optimal Linear Arrangement (OLA) problem (see [23]). This is a very well investigated graph problem, with applications including VLSI design [77], computational biology [82], and linear algebra [131]. The OLA is known to be very hard to solve, it has no polynomial time

approximation scheme unless NP-complete problems can be solved in randomized sub-exponential time [5]. Unfortunately, the problem size in this domain is in the thousands, which means methods that find exact linear arrangements (as dynamic programming does), cannot be applied. Interestingly, there are heuristic methods [88] that use exact methods as part of the entire process, and our algorithm could potentially be applied here.

The talent scheduling problem is highly related to the problem of minimizing the maximum number of open stacks. In this problem there are no durations, or costs and the aim is to minimize the maximum number of actors on location at any time. The problem has applications in cutting, packing and VLSI design problems. Compared to the talent scheduling problem, the open stacks problem has been well studied (see e.g.[160, 161, 162, 157, 42, 9]). The best current solution is our dynamic programming approach [53], but surprisingly almost none of the methods used there to improve the base dynamic programming approach are applicable to the talent scheduling problem. In the end the solutions are quite different, probably because the open stacks problem, while also NP-hard, is fixed parameter tractable [162], as opposed to the talent scheduling problem.

7.7 Conclusion

The talent scheduling problem is a very challenging combinatorial problem, because it is very hard to compute accurate bounds estimates from partial schedules. In this chapter we have shown how to construct an efficient dynamic programming solution by carefully reasoning about the problem to reduce search, as well as adding bounding and searching in the right manner. The resulting algorithm is orders of magnitude faster than other complete algorithms for this problem, and solves significantly larger problems than other methods.

There is still scope to improve the dynamic programming solution, by determining better heuristic orders in which to try scheduling scenes, and possibly determining better dynamic lower bounds by reasoning on the graph of actors that share scenes. One very surprising thing for us, was how much harder the talent scheduling problem

is than the highly related problem of minimizing the maximum number of open stacks.

Chapter 8

Maximum Density Still Life Problem

8.1 Introduction

The *Game of Life* was invented by John Horton Conway and is played on an infinite board made up of square cells. The game takes place through discrete time steps. Each cell c in the board is either alive or dead during each time period. The live/dead state of cell c at time $t+1$, denoted as $state(c, t+1)$, can be obtained from the number l of live neighbors of c at time t and from $state(c, t)$ as follows:

$$state(c, t+1) = \begin{cases} l < 2 & \text{dead} & [\text{Death by isolation}] \\ l = 2 & state(c, t) & [\text{Stable condition}] \\ l = 3 & \text{alive} & [\text{Birth condition}] \\ l > 3 & \text{dead} & [\text{Death by overcrowding}] \end{cases}$$

The board is said to be a *still life* at time t if it is stable under these rules, i.e., it is identical at $t+1$. For example, an empty board is a still life. Given a finite $n \times n$ region where all cells outside must be dead, the *Maximum Density Still Life Problem* is to compute the maximum density of live cells that can appear in the $n \times n$ region in a still life, or equivalently, the maximum number of live cells that can appear in the $n \times n$ region.

The raw search space of the Still Life Problem has size $O(2^{n^2})$. Thus, it is extremely difficult even for small values of n . Previous search methods using IP [18] and CP [19] could only solve up to $n = 9$, while a CP/IP hybrid method with symmetry breaking [19] could solve up to $n = 15$. An attempt using bucket elimination [90] reduced the time complexity to $O(n^2 2^{3n})$ but increased the space complexity to $O(n 2^{2n})$. This method could solve up to $n = 14$ before it ran out of memory. A subsequent improvement that combined bucket elimination with search [91] used less memory and was able to solve up to $n = 20$.

In this chapter, we combine deep mathematical insights into the Still Life Problem with several powerful search techniques to completely solve the problem for all n . This is an incredible improvement. For example, our new approach can solve $n = 100$ in only a matter of hours, whereas the previous best approach would require in the order of 10^{100} years and 10^{30} bytes of memory to solve $n = 100$. We are in fact able to solve the Still Life problem for all n . This is because the Still Life Problem becomes well

behaved mathematically for sufficiently large n (around $n > 200$), and it is possible to solve all the large cases in constant time.

The overall solution plan has four parts: 1) use complete search with a very strong model to solve the problem for all “small” n ($n \leq 50$), 2) use bounded dynamic programming on a relaxation of the problem to prove upper bounds on live cells for all medium and large n ($n > 50$) in constant time, 3) use a custom search to look for special form solutions to prove lower bounds on live cells for medium n (around $50 < n \leq 200$), 4) look for special form periodic solutions that can be tiled to construct arbitrarily large solutions to prove lower bounds on live cells for large n (around $n > 200$) in constant time. The lower and upper bounds proved in parts 2, 3 and 4 coincide, thus they are the optimums for those n . Each of these parts require deep mathematical insights into the problem as well as clever application of CP techniques. We give a brief overview of them here.

Part 1 We give a new insightful proof that the maximum density of live cells in the infinite case ($n = \infty$) is $\frac{1}{2}$. The proof is based on counting “wastage”. Wastage is calculated by looking at each 3×3 pattern and seeing how much space we have “wasted” by not fitting in enough live cells into the local area. This proof allows us to reformulate the Maximum Density Still Life Problem into one of minimizing wastage rather than maximizing the number of live cells. The new model gives very tight lower bounds on wastage that dramatically increases the pruning strength of the model. This model, coupled with a simple lookahead, allows a Lazy Clause Generation solver to solve the problem up to around $n = 50$ using complete search.

Part 2 We conjecture that for sufficiently large n , all wastage which is forced to occur by the still life constraints are forced by only the constraints near the edge of the $n \times n$ region. That is, only the boundary conditions cause suboptimality compared to the optimal density of $\frac{1}{2}$ in the infinite case. If this conjecture holds, then it is possible to get a very good or optimal lower bound on the wastage (and thus upper bound on live cells) simply by relaxing the Still Life Problem onto its boundary and solving it, i.e. ignore all constraints other than those within the first k rows of

the edge of the $n \times n$ region for some small k . This relaxed problem has the interesting property that the pathwidth of its constraint graph is $O(k)$ instead of the $O(n)$ of the original. There exists various techniques for solving such low pathwidth problems which can reduce the complexity from $O(2^{nk})$ to only $O(n2^{2k})$, e.g., caching [145], nogood learning [113, 45], dynamic programming [11], variable elimination [90]. In this case, we use bounded dynamic programming [117]. For fixed and small k , these relaxed problems can be solved in $O(n)$ time. We will see however that due to the translational symmetry in the problem, the optimal solutions become periodic and we can solve the edge relaxation for all n in $O(1)$, giving us very tight upper bounds on the number of live cells.

Part 3 We conjecture that for sufficiently large n , there always exists optimal solutions of the following form: wastage only exists at the four 4×4 corners of the board, or in the one row beyond the edge of the board. Based on this conjecture, we search for these special form solutions using a variant of limited discrepancy search with dynamic relaxations as a lookahead. Such a search can find optimal solutions for up to $n = 200$ or so. We know that the solution is optimal if the number of live cells in the solution coincides with the upper bound on live cells proved in part 2.

Part 4 The Still Life Problem becomes mathematically well behaved for sufficiently large n . This raises the possibility that optimal solutions can be constructed in a systematic way. We find optimal solutions for $n \sim 200$ which are periodic, and which satisfy certain other constraints. If such solutions are found, then they can be tiled indefinitely to produce arbitrarily large, provably optimal solutions.

8.2 Wastage Reformulation

The maximum density of live cells in a still life on an infinite board is known to be $\frac{1}{2}$ [40]. However, this proof is quite complex and only applies to the infinite case. In this section we provide a much simpler proof that can easily be extended to the finite case and gives much better insight into the possible sub-patterns that can occur in

Pattern:						
Beneficiaries	$\{\}$	$\{S\}$	$\{S, W\}$	$\{E, W\}$	$\{E, W\}$	$\{\}$
Wastage	2	1	0	0	0	2

Table 8.1: Possible patterns around dead cells, showing where they donate their tokens and any wastage.

an optimal solution.

Theorem 8.2.1. *The maximum density of live cells in a still life on an infinite board is $\frac{1}{2}$.*

Proof. We initially assign 2 tokens to each cell in the board. We will redistribute the tokens of each dead cell among its live neighbors according to a particular set of rules. It is clear that if we can prove that after the redistribution, all live cells on the board end up with at least 4 tokens, then there must have been at least as many dead cells as live cells, and thus the density of live cells is $\leq \frac{1}{2}$.

The rules for the token redistribution is as follows. The tokens of a dead cell are redistributed only to its orthogonal neighboring live cells, i.e. those that share an edge with the dead cell. Table 8.1 shows all possible patterns of orthogonal neighbors (up to symmetries) around a dead cell. Live cells are marked with a black dot, dead cells are unmarked, and cells whose state is irrelevant for our purposes are marked with a “?”. Each pattern indicates the beneficiaries, i.e., the North, East, South or West neighbors that receive 1 token from the center dead cell. We describe as “wastage” the tokens from a dead cell that does not need to be redistributed to any live cell for the proof to work (i.e., it is not needed to reach our target of 4 tokens for each live cell).

As it can be seen from the table, a dead cell gives 1 token to each of its live orthogonal neighbors if it has ≤ 2 live orthogonal neighbors, 1 token to the two opposing live orthogonal neighbors if it has 3, and gives no tokens if it has 0 or 4 orthogonal neighbors. Wastage occurs whenever the dead cell has 0, 1 or 4 live

	<table><tr><td>?</td><td>?</td><td>?</td></tr><tr><td>?</td><td>●</td><td>?</td></tr><tr><td></td><td></td><td></td></tr></table>	?	?	?	?	●	?				<table><tr><td>?</td><td>?</td><td>?</td></tr><tr><td>?</td><td>●</td><td>?</td></tr><tr><td>●</td><td></td><td></td></tr></table>	?	?	?	?	●	?	●			<table><tr><td>?</td><td>?</td><td>?</td></tr><tr><td>?</td><td>●</td><td>?</td></tr><tr><td>●</td><td></td><td>●</td></tr></table>	?	?	?	?	●	?	●		●
?	?	?																												
?	●	?																												
?	?	?																												
?	●	?																												
●																														
?	?	?																												
?	●	?																												
●		●																												
Pattern:																														
token received	1	1	0																											

Table 8.2: Contributions to the tokens of a live cell from its South neighbor.

orthogonal neighbors.

Given this set of redistribution rules, it is sufficient to examine the 3 bordering cells on each side of a live cell to determine how many tokens are obtained from the orthogonal neighbor on that side. For example, the token obtained by the central live cell from its South neighbor is illustrated in Table 8.2.

The tokens obtained by a live cell can therefore be computed by simply adding up the tokens obtained from its four orthogonal neighbors. Since each live cell starts off with 2 tokens, it must receive at least 2 extra tokens to end up with ≥ 4 tokens. Let us then look at all possible patterns around a live cell and see where the cell will receive tokens from. Table 8.3 shows all possible neighborhoods of a live cell (up to symmetries). For each pattern, it shows the benefactors, i.e., the North, East, South or West neighbors that give 1 token to the live cell, and the resulting amount of wastage, which occurs whenever a live cell receives more than 2 tokens.

Note that the last pattern does not receive sufficient extra tokens, just 1 from the South neighbor. However, the last two patterns always occur together in unique pairs due to the still life constraints (each of the central live cells has 3 neighbors so the row above the last pattern, and the row below the second last pattern must only consist of dead cells). So, as a final redistribution step, within each pair of these two patterns, we transfer the extra token from the second last pattern to the last.

Clearly, after all transfers are done, all live cells end up with ≥ 4 tokens, and this completes our proof that the maximum density on an infinite board is $\frac{1}{2}$.

The above proof is not only much simpler than that of [40], it also provides us with good insight into how to compute useful bounds for the case in which the board is finite. In particular, it allows us to know exactly how much we have lost from the

Pattern:					
Benefactors:	{N,S}	{N,E,S}	{N,E,S,W}	{S,W}	{N,S,W}
Wastage:	0	1	2	0	1
Pattern:					
Benefactors:	{S,W}	{N,S}	{E,S,W}	{E,S}	{E,W}
Wastage:	0	0	1	0	0
Pattern:					
Benefactors:	{S,W}	{S,W}	{S,W}	{N,E,W}	{S}
Wastage:	0	0	0	1	-1

Table 8.3: Possible patterns around a live cell showing token benefactors and any wastage.

theoretical maximum density by looking at the amount of “wastage” produced by the patterns in the currently labeled cells.

To achieve this, we reformulate the objective function in the Maximum Density Still Life Problem as follows. For each cell c , let $P(c)$ be the 3×3 pattern around that cell. Note that if c is on the edge of the $n \times n$ region, the dead cells beyond the edge are also included in this pattern. Let $w(P)$ be the wastage for each 3×3 pattern as listed in Tables 8.1 and 8.3, with the exception that the last two patterns in 8.3 both have $w(P) = 0$. Define $w(c)$ for each cell c as follows. If c is within the $n \times n$ region, then $w(c) = w(P(c))$. If c is in the row immediately beyond the $n \times n$ region and shares an edge with it (there are $4n$ such cells), then $w(c) = 1$ if the cell in the $n \times n$ region with which it shares an edge is dead, and $w(c) = 0$ otherwise. For all other c , let $w(c) = 0$. Let $W = \sum w(c)$ over all cells.

Theorem 8.2.2. *Wastage and live cells are related by*

$$\text{live_cells} = \frac{n^2}{2} + n - \frac{W}{4} \quad (8.1)$$

Proof. We adapt the proof for the infinite board to the $n \times n$ region. Let us assign 2 tokens to each cell within the $n \times n$ region, and 1 token to each of the $4n$ cells in the row immediately beyond the edge of the $n \times n$ region. Now, for each dead cell within the $n \times n$ region, partition the token among its live orthogonal neighbors as before. For each dead cell in the row immediately beyond the $n \times n$ region, give its 1 token to the cell in the $n \times n$ region with which it shares an edge. Again, since the last two 3×3 patterns listed above must occur in pairs, we transfer an extra 1 token from one to the other. Note also that the second last pattern of Table 8.3 cannot appear on the South border (which would mean that the last pattern appeared outside the shape) since it is not stable in this position. Clearly, after the transfers, all live cells once again have ≥ 4 tokens, and wastage for the 3×3 patterns centered around cells within the $n \times n$ region remain the same. However, since we are in the finite case, we also have wastage for the cells which are in the row immediately beyond the edge of the $n \times n$ region. These dead cells always give 1 token to the neighboring cell which is in the $n \times n$ region. If that cell is live, the token is received. If that cell is dead, that 1 token is wasted. The reformulation above counts all these wastage as follows. The total number of tokens that are used is $2n^2$ from the cells within the $n \times n$ region and $4n$ from the $4n$ cells in the row immediately beyond the edge, for a total of $2n^2 + 4n$. Now, 4 times live cells will be equal to the total token minus all the token wasted, and hence we end up with Equation (8.1). $\square$

We can trivially derive some upper bounds on the number of live cells using this equation. Clearly $W \geq 0$ and, thus, we have: $\text{live cells} \leq \lfloor \frac{n^2}{2} + n \rfloor$. Also, by the still life constraints, there cannot be three consecutive live cells along the edge of the $n \times n$ region. Hence, there is always at least 1 wastage per 3 cells along the edge and we can improve the bound to: $\text{live cells} \leq \lfloor \frac{n^2}{2} + n - \lfloor \frac{1}{3}n \rfloor \rfloor$. While this bound is very close to the optimal value for small n , it differs from the true optimum by $O(n)$ and will diverge from the optimum for large n .

8.3 Solving Small n with Complete Search

The power of a branch and bound algorithm is hugely dependent on how strong a bound we can prove on the objective at each node in the search tree. The stronger the bound we can prove, the earlier we can prune off failed subtrees. The naive Still Life model based on counting the number of live cells is horribly weak, because the upper bound on the number of live cells that propagation can prove is usually very weak and search generally does not fail until the board is at least half filled.

Remodeling the Still Life Problem in terms of minimizing wastage instead of maximizing live cells allows us to propagate much stronger bounds on the objective, as it is easy to tell how much space has already been wasted in the parts of the board that are labeled. Let `sl_waste` be a width 10 table which specifies the wastage value of each 3×3 pattern satisfying the still life constraints. For example, the entries corresponding to the first and second patterns in Table 8.3 would be $(0, 1, 0, 0, 1, 0, 0, 1, 0, 0)$ and $(0, 1, 0, 0, 1, 0, 1, 0, 0, 1)$. A single table constraint using `sl_waste` will be sufficient to enforce the still life constraints and set the value of the wastage variable. We propose the following simple model:

```
int: n; % instance parameter

array [0..n+1,0..n+1] of var 0..1: x; % cell live/dead status
array [0..n+1,0..n+1] of var 0..2: w; % cell wastage
var 0..2*n*n+4*n: total_wastage;
var 0..n*n: live_cells;

% still life and wastage constraints in n by n region
constraint forall (i,j in 1..n) (
  table(sl_waste, [x[i-1,j-1], x[i,j-1], x[i+1,j-1], x[i-1,j],
    x[i,j], x[i+1,j], x[i-1,j+1], x[i,j+1], x[i+1,j+1], w[i,j]]));

% boundary conditions
constraint forall (i in 0..n+1) (
```

```
x[i,0] = 0 /\ x[0,i] = 0 /\ x[i,n+1] = 0 /\ x[n+1,i] = 0);
```

```
constraint forall (i in 1..n-2) (
sum (j in i..i+2) (x[1,j]) <= 2 /\
sum (j in i..i+2) (x[n,j]) <= 2 /\
sum (j in i..i+2) (x[j,1]) <= 2 /\
sum (j in i..i+2) (x[j,n]) <= 2
);
```

```
% wastage constraints for boundary
```

```
constraint forall (i in 1..n) (w[i,0] = 1 - x[i,1]);
constraint forall (i in 1..n) (w[0,i] = 1 - x[1,i]);
constraint forall (i in 1..n) (w[i,n+1] = 1 - x[i,n]);
constraint forall (i in 1..n) (w[n+1,i] = 1 - x[n,i]);
```

```
% objective function
```

```
constraint total_wastage = sum (i,j in 0..n+1) (w[i,j]);
constraint live_cells = (2*n*n+4*n - total_wastage)/4;
```

```
solve maximize live_cells;
```

This basic model is capable of counting the wastage in the labeled parts of the board and using it to enforce an upper bound on the number of live cells. However, it is also important to get a good lower bound on the wastage that must occur in the parts of the board which are not yet labeled. The simplest bound we can get is that there must be at least 1 wastage per 3 cells along any unlabeled edge. We can implement this by adding a few lines and modifying the constraint on total wastage:

```
% wastage per 3 edge cells along edge i
array [1..4,1..n/3] of var 1..3: ew;
```

```
constraint forall (i in 1..n/3) (
```

```

ew[1,i] = sum (j in 3*i-2..3*i) (w[0,j]) /\
ew[2,i] = sum (j in 3*i-2..3*i) (w[j,0]) /\
ew[3,i] = sum (j in 3*i-2..3*i) (w[n+1,j]) /\
ew[4,i] = sum (j in 3*i-2..3*i) (w[j,n+1])
);

constraint total_wastage = sum (i,j in 1..n) (w[i,j]) +
sum (i in 1..4, j in 1.. n/3) (ew[i,j]) +
sum (i in n/3*3+1..n) (w[0,i] + w[i,0] + w[n+1,i] + w[i,n+1]);

```

In this modified model, the wastage of groups of three consecutive edge cells are summed together into variables $ew[i,j]$ before being added to the objective. Since each $ew[i,j]$ variable has a lower bound of 1, this facilitates the “at least 1 wastage per 3 edge cells” lookahead rule. However, this model is still a bit too weak for the bigger instances, and we use a more advanced lookahead based on relaxations. We defer detailed discussion of this to Section 8.4.

The search strategy is also important. We use a labeling strategy where we label from the boundary of the board inwards. We first label the first 3 rows of each edge and each 8×8 corner. Thereafter, we label one row in at a time in concentric squares. The reason for this labeling strategy will become much clearer in view of the insights discussed in Section 8.4. Basically, most wastage that is forced to occur by the constraints occurs along the boundary of the $n \times n$ region, and thus labeling those cells first increases the bound on the objective the quickest, allowing us to detect suboptimal assignments earlier.

Finally, we must note that a certain feature of the solver we used is critical for solving this problem effectively. One major problem with any search strategy for the Still Life problem is that it is possible to make a “mistake” in labeling that makes the subtree unsatisfiable, but propagation may not be able to notice this until many decision levels later. A normal CP solver will take an exponential amount of nodes to backtrack to this mistake and fix it. The solver we use however, is the Lazy Clause Generation solver CHUFFED, which supports conflict analysis and backjumping. Thus

n	opt.	time	n	opt.	time	n	opt.	time
21	232	7.0	31	497	121.3	41	864	89.2
22	253	82.4	32	531	86.9	42	907	25696
23	276	5.1	33	563	54.2	43	949	402.3
24	302	35.4	34	598	2.0	44	993	103.3
25	326	0.4	35	633	155.2	45	1039	218.4
26	353	116.3	36	668	165.1	46	1085	403.2
27	379	98.9	37	706	161.2	47	1132	200.2
28	407	48.8	38	744	193.0	48	1181	65.3
29	437	76.6	39	782	312.2	49	1229	456.9
30	467	240.8	40	824	17.6	50	1280	585.1

Table 8.4: Optimum number of live cells in the Maximum Density Still Life Problem found by complete search.

CHUFFED can analyze conflicts and immediately backjump to the mistake and fix it without having to waste an exponential amount of time searching in the failed subtree.

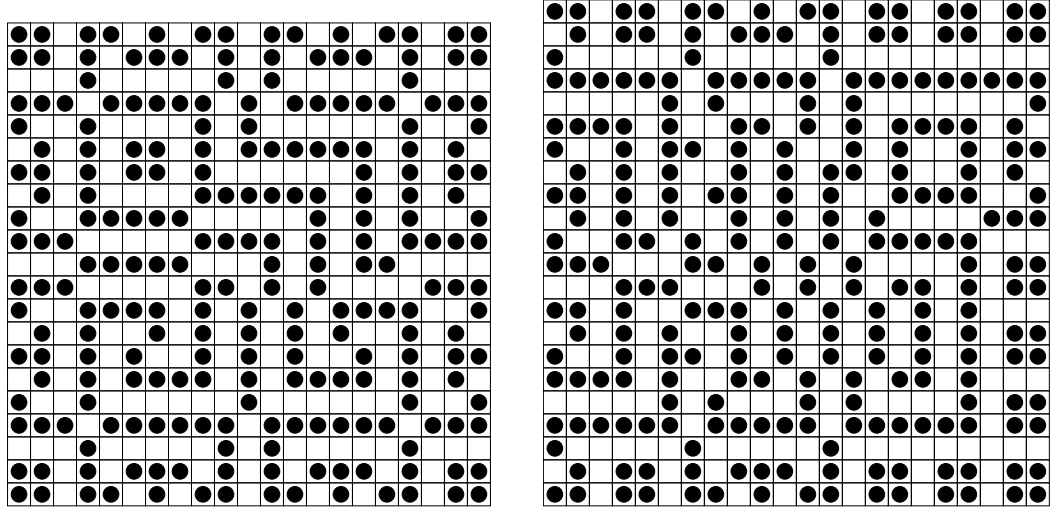
In Table 8.4, we show the results for $21 \leq n \leq 50$ ($n \leq 20$ have previously been solved). Clearly, we are already able to solve instances which are much larger than the previous state of the art methods. Optimal solutions for $n = 21$ and $n = 22$ are shown in Figure 8.1. We can solve instances somewhat larger than $n = 50$ using complete search, but the run time grows very quickly. Instead, we use better methods to tackle larger n in the next few sections.

8.4 Upper Bounds for Large n

To solve the problem for larger n , we need more mathematical insight into the problem. We make the following conjecture:

Conjecture 8.4.1. *For sufficiently large n , all forced wastage is caused by the still life constraints within k rows from the edge of the $n \times n$ region, where k is some small, fixed constant.*

The conjecture is inspired by the following two facts. Firstly, in the optimal solutions for $n \leq 20$ that were found by previous methods, it was often the case that wastage only appeared in the corners or within the first 3 rows from the edge.

Figure 8.1: Optimal solutions for $n = 21$ and $n = 22$.

Secondly, we already know that there exist wastage free labellings in the infinite case, thus there is nothing inherent in the still life constraints which force wastage. Instead, it would appear that it is the boundary conditions in the finite case which is forcing the extra wastage to occur, hence the conjecture.

If the conjecture holds, then it should be possible to relax the Still Life problem onto just the boundary variables (variables within k rows from edge) and still derive the same wastage lower bound. This has the advantage that: 1) such a relaxed problem has far fewer variables ($O(nk)$ instead of $O(n^2)$) and thus a much smaller search space, 2) such a relaxed problem has low pathwidth ($O(k)$ instead of $O(n)$) and there exist various techniques that can take advantage of this to reduce the search space even further to $O(n2^{2k})$, e.g., caching, nogood learning, dynamic programming, variable elimination.

Theorem III.2.2 tells us that any lower bound we prove for the wastage in the relaxed problem is also a valid lower bound for the original problem. However, this bound may or may not be the optimal bound for the original problem. If we relaxed the problem too much, then the bound we derive from the relaxed problem will be weaker than the optimal bound for the original problem. Now, if Conjecture 8.4.1 holds, then there should exist some small k such that the bound from relaxing the

problem onto a width k boundary should still be optimal for the original problem. We need to find this k .

The complexity of solving the relaxation is $O(n2^{2k})$, so choosing k too large will make the problem intractable. We wish to find the smallest k such that the relaxation is sufficient to prove the optimal bounds for the original problem. We performed a series of experiments to try to guess what this minimal k is. We define the edge still life problem $edge(n, k)$ as follows:

```
int: n, k; % instance parameters

array [0..n+1,0..k+1] of var 0..1: x; % cell live/dead status
array [1..n,0..k] of var 0..2: w; % cell wastage

% still life and wastage constraints in n by k region
constraint forall (i in 1..n, j in 1..k) (
  table(wastage, [x[i-1,j-1], x[i,j-1], x[i+1,j-1], x[i-1,j],
x[i,j], x[i+1,j], x[i-1,j+1], x[i,j+1], x[i+1,j+1], w[i,j]]));

% boundary conditions
constraint forall (i in 0..n+1) (x[i,0] = 0);

constraint forall (i in 1..n-2) (
  sum (j in i..i+2) (x[1,j]) <= 2);

% wastage constraints for boundary
constraint forall (i in 1..n) (w[i,0] = 1 - x[i,1]);

% objective function
solve minimize sum (i in 1..n, j in 0..k) (w[i,j]);
```

Let $m(n, k)$ be the minimum wastage for problem $edge(n, k)$. For each k , we calculate the edge wastage ratio $r(k)$ as: $r(k) = \lim_{n \rightarrow \infty} m(n, k)/n$. These limits

k	$r(k)$ (exact)	$r(k)$ (decimal)	castles	blocks
1-2	$1/3$	0.333	0	1
3-11	$4/11$	0.364	1	1
12	$7/19$	0.368	2	1
≥ 13	$10/27$	0.370	3	1

Table 8.5: Ratio of wastage to edge cells in an optimal edge for different edge width, and number of patterns that appear in the first three rows.

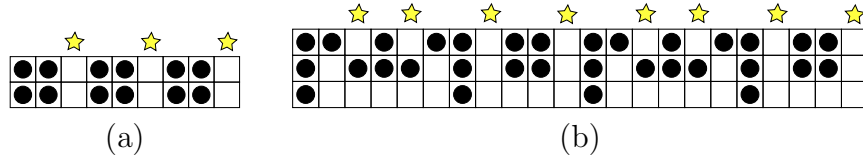


Figure 8.2: Optimal edge patterns for a width 2 and width 3 boundary relaxation. Wastage is highlighted with a star.

exists and can be calculated in constant time using dynamic programming, because the optimal edge patterns all become periodic for sufficiently large n for any k . See Table 8.5 for the values of $r(k)$ and Figure 8.2 for a sample of the optimal edge patterns.

First, we note that $r(1) = r(2) = 1/3 = 0.333$ gives us the trivial edge wastage bound discussed in Section 8.2. The optimal edge pattern for $k = 2$ is unique and periodic and is shown in Figure 8.2(a). However, this optimal width 2 edge cannot be extended to a width 3 edge without introducing additional wastage. For a width 3 edge, $r(3) = 4/11 = 0.364$ and the optimal edge pattern is once again unique and periodic and is shown in Figure 8.2(b). We will refer to the pattern shown in the first 8 columns of Figure 8.2(b) the “castle” pattern and the next 3 columns of Figure 8.2(b) the “block” pattern. The castle pattern has length 8 with 3 wastage and the block pattern has length 3 with 1 wastage. As k increases, $r(k)$ continues to increase slightly. At $k = 13$, we have $r(13) = 10/27 = 0.370$. The optimal edge is no longer unique or periodic, however, all of them are identical within the first 3 rows up to translational symmetry, and are composed of three castle patterns followed by a block pattern, which gives $3 * 3 + 1 = 10$ wastage per $3 * 8 + 3 = 27$ cells. All

of the optimal edge patterns only have wastage in the row beyond the edge, which is an interesting fact to note. Beyond, $k = 13$, the value of $r(k)$ plateaus, and we stopped our experimentation at $k = 20$. We show later in Section 8.7 that there exists arbitrarily thick edge patterns which achieve exactly an edge wastage ratio of $r(13) = 10/27$. So in fact, $r(k) = 10/27$ for all $k \geq 13$.

What the edge wastage ratio results show is that the still life constraints up to a depth of 13 are capable of forcing wastage to occur. We should not relax these constraints away, as that will degrade the bound that we can prove using the relaxed problem. Thus we need to look at an edge relaxation of at least width 13. However, the fact that $r(k)$ plateaus after $k = 13$ also suggests that width 13 is sufficient, i.e. the still life constraints beyond a depth of 13 can have no effect on forcing edge wastage, and we can relax them away without changing the bound. This turns out to be correct, as our results later show.

8.5 Solving the Boundary Relaxation

We wish to solve the Still Life problem, relaxed onto a width 13 boundary. The technique we choose to use is bounded dynamic programming [117]. We discuss the choice of this technique and show how it can be applied. First of all, it is necessary to choose one of the methods that can exploit the low pathwidth of the relaxed problem. Of these, dynamic programming is by far the easiest to implement. However, we use a variant of dynamic programming called bounded dynamic programming [117]. The idea behind bounded dynamic programming is that for some kinds of dynamic programs, instead of calculating the exact value for every subproblem, for many subproblems, it is actually sufficient to prove a bound on its value.

Example 8.5.1. Consider the recursion: $a(n) = \min(a(n-1) + 2, a(n-2) + 5)$. Suppose we want to calculate the value of $a(n)$. Suppose we have already calculated that $a(n-1) = 6$. Now, we need to find some information on $a(n-2)$. However, we note that one of the terms in the $\min$: $a(n-1) + 2 = 8$, is already known, therefore if it is known that $a(n-2) \geq 3$, then its exact value is irrelevant as far as the value of $a(n)$ is concerned. We only want to know: is $a(n-2) \geq 3$, or if it is < 3 , then

what is its exact value? Thus we only wish to answer a bounded query on the value of $a(n - 2)$, where we do not care what its exact value is if it is above or below a certain range.

On certain types of problems, bounded dynamic programming can be exponentially faster than normal dynamic programming, as there are a lot of subproblems whose exact values are irrelevant and all we need to do is to prove a certain bound on their value. Proving a weak bound is often exponentially faster than finding the exact value. In the case of the Still Life problem, there are many, many ways to label the cells, and most of them lead to large amounts of wastage. Therefore, we often just need to prove that a subproblem is sufficiently bad (has at least a certain amount of wastage), rather than calculate exactly how bad it is. Bounded dynamic programming is a rigorous way to do this.

We illustrate how dynamic programming can be applied with a simplified example. Consider a more constrained version of $edge(n, k)$ where we have additional boundary conditions which fix the first 2 and last 2 columns. We denote this problem by $edge(n, k, s_1, s_2, e_1, e_2)$ where the model has the additional lines:

```
% instance parameters
int: s1, s2, e1, e2;

% additional boundary conditions
constraint s1 = sum (i in 0..k-1) (x[1,i] * 2^i);
constraint s2 = sum (i in 0..k-1) (x[2,i] * 2^i);
constraint e1 = sum (i in 0..k-1) (x[n,i] * 2^i);
constraint e2 = sum (i in 0..k-1) (x[n-1,i] * 2^i);
```

As can be seen, we are denoting the value of a column of k cells by a single number in $\{0, \dots, 2^k - 1\}$, where the value of the cells are being interpreted as the binary digits of this number. We define the Boolean function $s(a, b, c)$ where $a, b, c \in \{0, \dots, 2^k - 1\}$ as *true* if three consecutive columns labeled as a , b , and c in that order do not violate the still life constraints, and *false* otherwise. We define the integer function $w(a, b, c)$

where $a, b, c \in \{0, \dots, 2^k - 1\}$ as the wastage in the central column if three consecutive columns are labeled as a , b , and c in that order.

Let $m(n, k, s_1, s_2, e_1, e_2)$ be the minimum wastage for problem $edge(n, k, s_1, s_2, e_1, e_2)$. Then the following recursive formulas hold: $\forall n \geq 3, s_1, s_2, e_1, e_2 \in \{0, \dots, 2^k - 1\}$, $m(n, k, s_1, s_2, e_1, e_2) = \min\{m(n-1, k, s_1, s_2, e_2, e_3) + w(e_1, e_2, e_3) \mid e_3 \in \{0, \dots, 2^k - 1\}, s(e_1, e_2, e_3)\}$.

Let's consider why. Consider any solution of $edge(n, k, s_1, s_2, e_1, e_2)$. The $(n-3)$ th column must have been fixed to something. Let it be denoted by e_3 . To satisfy the still life constraints, we must have $s(e_1, e_2, e_3)$ be true. For each such solution, $w(e_1, e_2, e_3)$ gives us the wastage in the $n-2$ th column. If we project this solution onto the first $n-1$ columns, then we clearly have a solution for $edge(n-1, k, s_1, s_2, e_2, e_3)$. Hence the minimum wastage among all such solutions is $m(n-1, k, s_1, s_2, e_2, e_3) + w(e_1, e_2, e_3)$. Taking the minimum of these over all possible e_3 gives $m(n, k, s_1, s_2, e_1, e_2)$, and hence the recursive formula.

Such a set of recursive formulas can be solved using dynamic programming in $O(n)$. We can use precalculated values of $m(n, k, s_1, s_2, e_1, e_2)$ as a wastage lookahead in the complete search method described in Section 8.3. Given a partially filled edge where the last two columns are given by s_1 and s_2 , we can look up $\min_{i,j} m(n, k, s_1, s_2, i, j)$ to get a lower bound on the wastage in the remaining cells of the edge. This will give a bound of approximately 10/27 wastage per remaining edge cell instead of the trivial 1/3 wastage per remaining edge cell we had before. Although the difference is small, it is quite crucial for solving the larger n with complete search.

Now, it turns out that for any k , for sufficiently large n , the values of $m(n, k, s_1, s_2, e_1, e_2)$ become periodic. This is formalized by the following theorem.

Theorem 8.5.2. *If for some constants M, p, q , we have: $\forall s_1, s_2, e_1, e_2, m(M + p, k, s_1, s_2, e_1, e_2) = m(M, k, s_1, s_2, e_1, e_2) + q$, then $\forall n \geq M, \forall s_1, s_2, e_1, e_2, m(n + p, k, s_1, s_2, e_1, e_2) = m(n, k, s_1, s_2, e_1, e_2) + q$. That is, once $n \geq M$, the values of m simply increase by q every p cells.*

Proof. The proof is by induction. The base case of $n = M$ is true by assumption. Suppose it is true for $n = t$. Consider $n = t + 1$. We have:

$$\begin{aligned}
& m(t+1+p, k, s_1, s_2, e_1, e_2) \\
= & \min\{m(t+p, k, s_1, s_2, e_2, e_3) + w(e_1, e_2, e_3) \mid e_3 \in \{0, \dots, 2^k - 1\}, s(e_1, e_2, e_3)\} \\
= & q + \min\{m(t, k, s_1, s_2, e_2, e_3) + w(e_1, e_2, e_3) \mid e_3 \in \{0, \dots, 2^k - 1\}, s(e_1, e_2, e_3)\} \\
= & q + m(t+1, k, s_1, s_2, e_1, e_2)
\end{aligned}$$

□

Theorem 8.5.2 tells us that if we ever reach an n where the m values become periodic, then it remains periodic for all larger n . This always happens due to the translational symmetry in the Still Life Problem. For $k = 13$, the values typically become periodic by the time we reach $n = 200$. Once this happens, we can calculate closed form equations for $m(n, k, s_1, s_2, e_1, e_2)$ for all larger n . Thus, we can calculate the values of $m(n, k, s_1, s_2, e_1, e_2)$ for all n in constant time.

In the relaxed Still Life Problem, we also need to deal with the corners. However, it is easy to see that the same kind of recursive formulation into a dynamic program is possible. Thus the relaxed problem can be completely solved for all n using bounded dynamic programming very efficiently. Table 8.6 shows the wastage lower bounds and the corresponding live cell upper bounds we derive for $21 \leq n \leq 56$. We note several things. Firstly, all the bounds calculated by bounded dynamic programming on the relaxed problem are consistent with the optimal live cell values calculated by complete search in Section 8.3. Secondly, the bounds on the relaxed problem are often the optimal bounds for the original problem (the only exceptions being $n = 24, 26, 28, 38$). This is consistent with Conjecture 8.4.1, which stated that for sufficiently large n , the relaxed problem should have the same bound as the original. It would appear that the “sufficiently large n ” in the conjecture is $n > 38$.

For $n \geq 61$, the wastage lower bounds becomes periodic and is given by the equations in Figure 8.3. The corresponding live cell upper bound is given by the equations in Figure 8.4. We shall see in the next two sections that these live cell upper bounds are in fact achievable, so the equations in Figure 8.4 actually give us the optimal number of live cells for $n \geq 61$.

n	waste lb.	live ub.	n	waste lb.	live ub.	n	waste lb.	live ub.
20	40	210	34	56	598	48	76	1181
21	36	232	35	59	632	49	80	1229
22	42	253	36	62	668	50	80	1280
23	43	276	37	60	706	51	80	1331
24	40	302*	38	64	744*	52	85	1382
25	46	326	39	67	782	53	84	1436
26	44	353*	40	64	824	54	87	1490
27	48	379	41	68	864	55	89	1545
28	51	407*	42	68	907	56	88	1602
29	48	437	43	71	949	57	91	1658
30	53	466	44	73	993	58	92	1717
31	55	497	45	72	1039	59	92	1776
32	53	530	46	76	1085	60	97	1835
33	56	563	47	78	1132	61	96	1897

Table 8.6: Lower bounds on wastage and corresponding upper bounds on live cells as calculated by bounded dynamic programming on a problem relaxation. Instances where the bound on the relaxed problem differed from the original problem are indicated with an asterisk.

$$wastage \geq \begin{cases} \lfloor 40/27 * n + 5 \rfloor, & n \equiv 5, 13, 21, 24, 32, 35, 40, 43, 48, 51 \pmod{54} \\ \lfloor 40/27 * n + 6 \rfloor, & n \equiv 0, 2, 7, 8, 10, 14, 15, 16, 18, 22, 23, 26, \\ & 29, 30, 34, 37, 38, 42, 45, 46, 50, 53 \pmod{54} \\ \lfloor 40/27 * n + 7 \rfloor, & n \equiv 1, 3, 4, 9, 11, 12, 17, 19, 20, 25, 27, 28, \\ & 31, 36, 39, 41, 44, 47, 49, 52 \pmod{54} \\ \lfloor 40/27 * n + 8 \rfloor, & n \equiv 6, 33 \pmod{54} \end{cases}$$

Figure 8.3: Closed form equations for the wastage lower bound for $n \geq 61$.

8.6 Lower Bounds for Large n

Lower bounds on live cells can be proved by finding actual solutions to the problem. If the number of live cells in the solution found coincides with the upper bound proved in Section 8.5, then we know that it is an optimal solution and we have solved the

$$livecells \leq \begin{cases} \lfloor n^2/2 + 17/27 * n - 2 \rfloor & n \equiv 0, 1, 3, 8, 9, 11, 16, 17, 19, 25, 27, \\ & 31, 33, 39, 41, 47, 49 \pmod{54} \\ \lfloor n^2/2 + 17/27 * n - 1 \rfloor & n \equiv 2, 4, 5, 6, 7, 10, 12, 13, 14, 15, 18, \\ & 20, 21, 22, 23, 24, 26, 28, 29, 30, \\ & 32, 34, 35, 36, 37, 38, 40, 42, 43, \\ & 44, 45, 46, 48, 50, 51, 52, 53 \pmod{54} \end{cases}$$

Figure 8.4: Closed form equations for the live cell upper bound for $n \geq 61$.

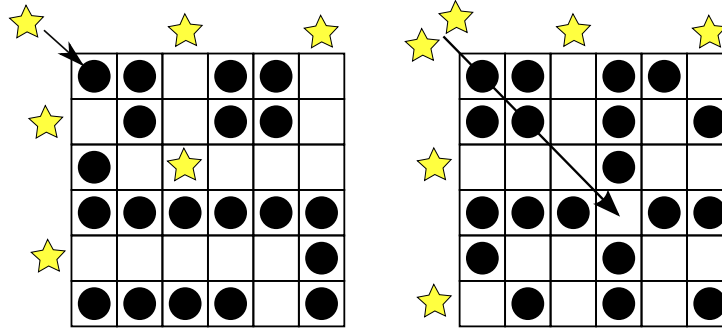


Figure 8.5: The two best ways to fill a corner (up to symmetry). Wastage is highlighted with a star. Note that there are two units of wastage in the cell in the 4th column and 4th row in the second pattern.

problem for that n . However, finding optimal solutions is very hard, because the default search space of the Still Life Problem is extremely large. The model given in Section 8.3 was good enough for us to solve up to around $n = 50$ using complete search. However, to go beyond that size, we need something that reduces our search space much further. We make the following conjecture:

Conjecture 8.6.1. *For sufficiently large n , there always exists optimal solutions of the following form: wastage only exists at the four 4×4 corners of the board, or in the one row beyond the edge of the board.*

Conjecture 8.6.1 is supported by the experiments described in Section 8.4. We reasoned that for sufficiently large n , the edges should follow one of the optimal edge patterns, and we know that none of them have any wastage other than in the one row beyond the edge of the board. Similarly, there are only two ways to label a corner with minimum wastage (see Figure 8.5) and both of them only have wastage within the 4×4 corners. And finally, the center of the board should be wastage free, because by Conjecture 8.4.1, the constraints in the center of the board do not force any wastage, and any wastage there would simply make the solution suboptimal.

Assuming that Conjecture 8.6.1 holds, we can look only for solutions of this special form. Note that searching only for solutions of this special form gives an incomplete search on the Still Life Problem. However, incomplete search is perfectly sufficient for proving lower bounds on the number of live cells, since the solution itself is the

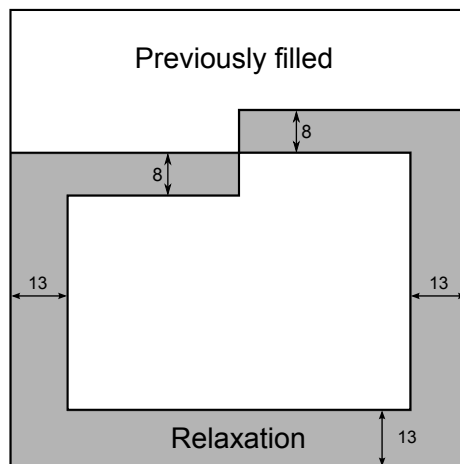


Figure 8.6: Dynamic relaxation lookahead for still life search.

proof. To solve for all n , we must be able to find an optimal solution for every single n . We use two further techniques to reduce the search space: 1) dynamic relaxations as lookahead, and 2) a customized limited discrepancy search.

In Section 8.4, we described how we can perform a relaxation onto the boundary of the board in order to derive a lower bound on the wastage. We can do the same thing during search. Our search strategy is to label the board 8 rows at a time from top to bottom. Conjecture 8.4.1 tells us that for sufficiently large n , wastage is only forced by the boundary constraints. In a subproblem however, the boundary also includes the values of the cells we have labeled. If we relax the subproblem onto the boundary of the unlabeled region, we should be able to derive a very strong lower bound on the wastage in the unlabeled region of the board. Thus, at each search node, we relax the remaining problem onto: the unlabeled parts of the width 13 boundary, and the 8 unlabeled rows below the last row we labeled (see Figure 8.6).

We note several things. Firstly, the set of variables involved in the relaxation is different at each search depth. Secondly, the boundary conditions of the relaxation are different even for two nodes at the same search depth if their cells in the last two rows are labeled differently. Thus each of these relaxed subproblems are different and have to be solved separately to derive the wastage lower bound. Now, as before, we can solve each of these in $O(n)$ using dynamic programming. However, we can

do better by noting that the relaxed problem at any node is very similar to the relaxed problem at its parent node, differing only by one column of 8 variables. By appropriately caching the solutions to the relaxed subproblem at the parent node, we can solve the relaxed problem at each node in $O(1)$. The cost is still non-trivial, however, the wastage lower bounds derived by these relaxations are very strong and provide a large amount of pruning, so it is well worth it.

One might wonder why we use a different search order than that described in Section 8.3, where we label the board from outside in rather than from top to bottom. This is because the relaxation lookahead already tells us how much wastage there has to be in the unlabeled parts of the boundary, thus it is unnecessary to actually label it to force the wastage lower bound up. It is better not to label the boundary yet, as there are exponentially many ways to do it, but most of them are probably bad. It is better to fix the variables which are close to those variables already fixed so that we can detect inconsistencies earlier, hence the top to bottom strategy.

We also use the relaxation lookahead as a branching heuristic. When faced with the choice of labeling the next column of 8 variables as any one of the 256 possible values, our relaxation lookahead is able to tell us how much the wastage lower bound will increase by, given each of those choices. We order the choices according to how low the wastage lower bound given by the lookahead is. This is far superior to a naive, greedy branching heuristic which orders the choices based on the wastage in only the labeled part of the board, as it is often the case that greedily minimizing wastage locally will cause much more wastage further on. Our lookahead is capable of seeing the wastage caused further on and will not pick such locally optimal but globally suboptimal choices.

We use a modified version of limited discrepancy search [68]. Firstly, rather than defining a discrepancy as any choice which is not the first choice given by the branching heuristic, we define it as the amount that the wastage lower bound given by the relaxation lookahead would increase by if we made this decision. This means that there can be ties, i.e. multiple choices which are all equally good according to the lookahead. In such cases, we randomly tie-break between them. Secondly, we add randomized restarts to the search. At randomized points in time, the search will

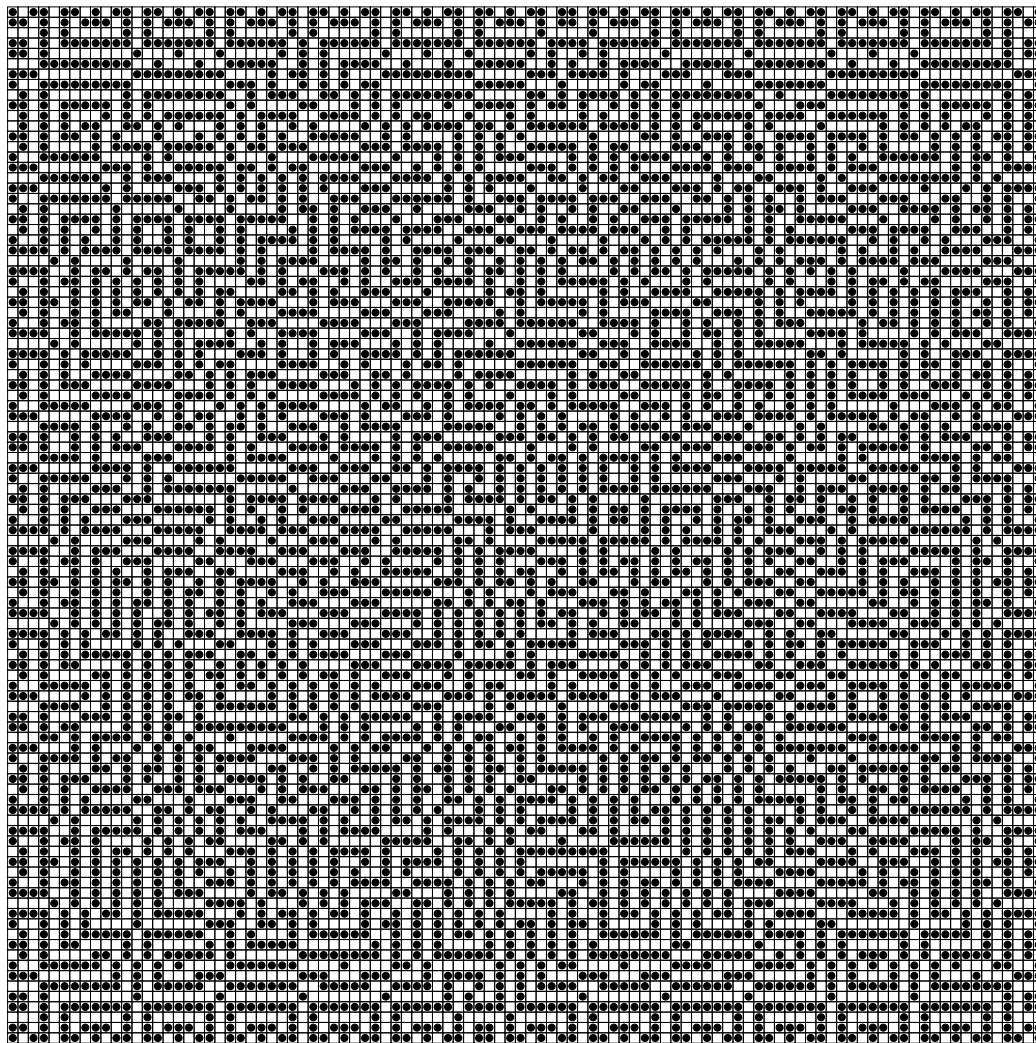


Figure 8.7: Optimal solution for $n = 100$.

backtrack by a random number of rows. This is important, because despite our lookahead, there still exist vast subtrees where no optimal solutions can be found, and a complete search will take an exponential number of nodes to backtrack out of them. The combination of random restarts, plus random tie-breaking between equally good choices, is very effective. Using these techniques, we are able to find optimal solutions to instances as large as $n = 200$ in several hours on average. An optimal solution for $n = 100$ is shown in Figure 8.7.

8.7 Constructing Optimal Solutions for Arbitrarily Large n

Through our experimentation and analysis, it became clear that the Still Life Problem is actually fairly well behaved for sufficiently large n . We have the following properties: 1) the wastage lower bound is periodic in n , 2) there exists optimal periodic edge patterns which achieves this wastage lower bound, 3) it is “easy” to fill in the center of the board without any wastage. Combining these facts together, it seems possible that with a bit of work, we can construct closed form optimal solutions for arbitrarily large n .

In Section 8.5, we already worked out wastage lower bounds for all n . If we can construct solutions that achieve these wastage lower bounds for all n , then we are done. This can be done by solving instances of the Still Life Problem under additional periodic constraints, so that parts of the solution can be tiled indefinitely to produce arbitrarily large optimal solutions (see Figure 8.8). The initial solution is broken up by two horizontal and two vertical cuts into: 4 corner pieces, 4 edge pieces, and 1 center piece. The edge pieces and center pieces must satisfy periodic constraints so that they can be tiled and still satisfy the still life constraints. Furthermore, there are strong restrictions on the amount of wastage that can occur in these tiled pieces. We know that for sufficiently large n , the wastage lower bounds from Section 8.4 has a wastage to edge cell ratio of exactly $10/27$. Thus to hit this lower bound, our 4 periodic edge pieces must have precisely this wastage ratio. Also, the periodic center piece must be completely wastage free.

Now, for the periodic edge pieces to have precisely a wastage to edge cell ratio of $10/27$, their period must be a multiple of 27. Unfortunately, if the center piece is 27×27 , then it is impossible for it to be wastage free, since it has an odd number of cells. Thus the minimum period we can use is $2 \times 27 = 54$. The aim then is to solve an instance with $n = n'$ under additional period 54 constraints and wastage constraints on the edge and center pieces. If an optimal solution can be found, then it can be tiled to create optimal solutions for $n = n' + k \times 54$ for any $k \in \mathbb{N}$. Clearly, to solve for all n , this has to be done for each value of $n' \bmod 54$, so there are 54 cases. We

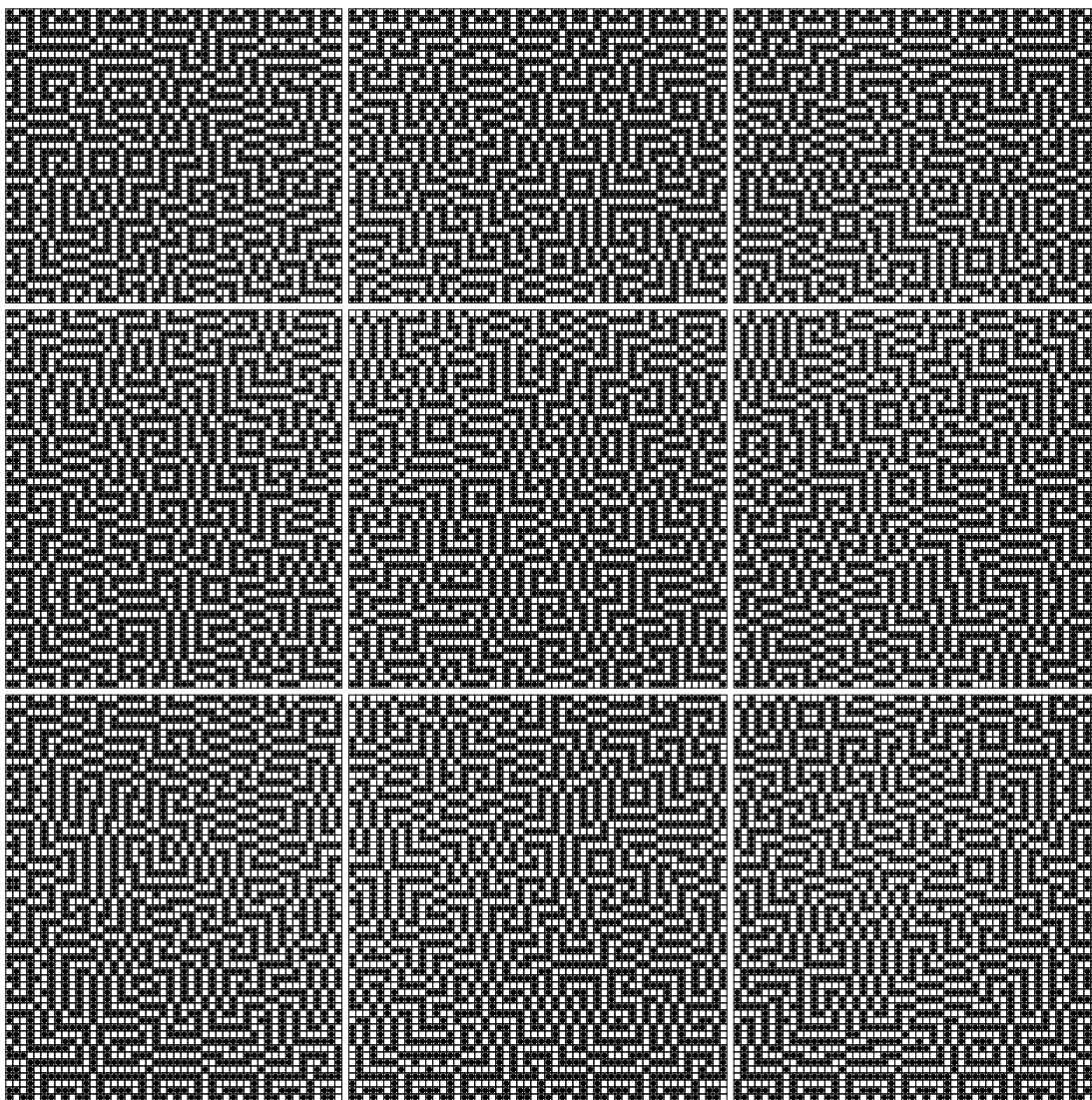


Figure 8.8: A periodic optimal solution where the center sections can be tiled indefinitely to produce optimal solutions for $n = 100 + k * 54$.

found that n' had to be around 150 or higher before the Still Life problem became sufficiently well behaved that the periodic version was solvable. So we had to solve 54 instances of size 150+, one for each modulus class mod 54, under the additional periodic constraints and wastage constraints.

Rather than directly solving such problems from scratch, we decided to utilize the

solutions that we had already found, and try to extend them into periodic solutions by splicing in a periodic section. We first take an optimal solution for $n = n' - 54$. We cut it into two pieces vertically at some point. We then move the two pieces apart by 54 cells and try to fill up the gap with a periodic 54 section which satisfies the additional wastage constraints. This periodic 54 section will end up being two of the periodic edge pieces. Secondly, we take this new solution and cut it horizontally at some point. We then move the two pieces apart by 54 cells and do the same again. This new section will end up being the other two periodic edge pieces plus the center piece. Now we end up with an optimal solution for $n = n'$ which satisfies the periodic constraints and the wastage constraints. Here is a MiniZinc model for the first step. The second step is analogous.

```
int: n; % instance parameter

% four columns covering the splice point from sol of n = n' - 54
array [1..4,1..n] of 0..1: s;

array [1..54,0..n+1] of var 0..1: x; % cell live/dead status

% still life and wastage constraints in 54 by n region
constraint forall (i in 2..53, j in 1..n) (
  table(sl_waste, [x[i-1,j-1], x[i,j-1], x[i+1,j-1], x[i-1,j],
    x[i,j], x[i+1,j], x[i-1,j+1], x[i,j+1], x[i+1,j+1], 0]));

% boundary conditions
constraint forall (i in 0..n+1) (
  x[i,0] = 0 /\ x[i,n+1] = 0);

constraint forall (i in 1..52) (
  sum (j in i..i+2) (x[j,1]) <= 2 /\
  sum (j in i..i+2) (x[j,n]) <= 2
```



```

);

constraint forall (i in 1..n) (
x[1,i] = s[3,i] /\ x[2,i] = s[4,i] /\
x[n-1,i] = s[1,i] /\ x[n,i] = s[2,i]);

% wastage constraints for boundary

constraint sum (i in 1..54) (x[i,1]) = 34;
constraint sum (i in 1..54) (x[i,n]) = 34;

solve maximize satisfy;

```

This splicing does not always succeed as the constraints are very, very strong. Whether it is satisfiable or not depends on the initial solution and the point at which we make the cut. In particular, the cuts must be made at a point where both edges have already transitioned into the optimal edge pattern of 3 castles per 1 block periodic pattern. If we could not solve a particular instance after a reasonable time, we tried a different cut point or tried it using a different solution of $n = n' - 54$. The success rate was around 80%, so most of them succeeded on the first try. After approximately 3000 hours of computation, we were able to find periodic solutions for all 54 cases, and thus the Still Life Problem was solved for all large n . The Still Life shown in Figure 8.8 is one such periodic solution where the center sections can be tiled indefinitely to produce optimal solutions for $n = 100 + k * 54$.

8.8 Conclusion

We have solved the Maximum Density Still Life Problem for all n by combining deep mathematical insights into the problem with clever application of remodeling, lazy clause generation, bounded dynamic programming, relaxations, and custom search. The complete solution consists of four parts: 1) complete search which can solve $n \leq$

50, 2) bounded dynamic programming with relaxation to prove optimal live cell upper bounds for $n > 50$, 3) incomplete search for special form solutions which can prove optimal live cell lower bounds for around $50 < n \leq 200$, 4) incomplete search to find optimal periodic solutions which can be tiled to construct arbitrarily large solutions that prove the optimal live cell lower bounds for around $n > 200$. The optimal values for $n \leq 50$ are given in Table 8.4, and the optimal values for $n \geq 51$ are given by Equation 8.4. Optimal solutions for small and medium n and periodic optimal solutions for large n can be found at www.csse.unimelb.edu.au/~pjs/still-life/. The total time taken to completely solve the Maximum Density Still Life Problem for all n was approximately 3000 hours.

Part IV

Parallelization

Parallelization is an obvious way to try to speed up constraint solving, especially in view of the recent proliferation of multi-core systems. There are a number of ways to parallelize CP solvers. The simplest and most popular is search parallelization, where different parts of the search tree are searched by different processors (e.g., [114, 138, 105]). Another possibility is to parallelize the propagation engine by having different processors execute different propagators (e.g., [132, 167]).

The basic search algorithm described in Figure 2.2 is easy to parallelize. Subproblems arising from different parts of the search tree are solved independently, thus they can be partitioned out to different processors to be solved. Communication between processors is only required when a new solution has been found, or if some form of dynamic load balancing is required. Previous work on parallel search has concentrated mainly on load balancing [130, 138], or on developing an appropriate architecture for parallel search [138, 105].

IV.1 Contributions

In Chapter 9, we improve the state of the art SAT solver MiniSat [39] by modifying its data structures to be more cache-aware. Cache contention can be a serious problem when parallelizing SAT or CP solvers. Although the number of cores in modern processors is growing rapidly, the amount of cache available to these cores do not appear to be growing at the same rate. In most multi-core processor designs, L2 and L3 caches are shared between multiple cores, with the result that each core has a lot less L2 and L3 cache on average than in older single processor designs. This is fine if the cores are using the same set of data. However, in SAT solver and Lazy Clause Generation solvers, the memory access patterns are quite different in different parts of the search tree, hence different cores tend to require different parts of the data at any one time. This leads to serious cache contention issues that severely limit the amount of speedup achievable on multi-core systems. Our data structure improvements are able to speed up the sequential version of MiniSat by around 80%, and speed up a parallel version of MiniSat by around 140%. These improvements carry over to Lazy Clause Generation solvers, which have SAT solvers embedded in them.

In Chapter 10, we address an important theoretical issue which has not been properly addressed in the literature. Many analyses of parallel search and load balancing schemes implicitly or explicitly assume that the total amount of work is fixed (e.g. [89]). This assumption is indeed true for unsatisfiable instances with a fixed search tree. However, it is not true for satisfiable satisfaction problems or for optimization problems. In such problems, finding a (good) solution quickly can dramatically reduce the total amount of work required. Much work in CP is concerned with developing strong branching heuristics which can shape and order the branches of the search tree so that (good) solutions are found early on in the search. However, many parallelization schemes, e.g., splitting the search tree as close to the root as possible [138], focus on work granularity and completely ignore the branching heuristic when partitioning the work. This can result in many of the processors searching in unfruitful parts of the search tree. We analyzed this problem and develop a new work stealing scheme called *confidence based work stealing*, which partitions the work based on how strong the branching heuristic is at each node. The new parallel algorithm produced near linear or super linear speedup on all the problems we tested.

Chapter 9

Cache Conscious Data Structures for Boolean Satisfiability Solvers

9.1 Introduction

Boolean Satisfiability (SAT) solvers find a wide range of application in areas such as electronic design automation and artificial intelligence. A tremendous amount of research has been undertaken to find ways to speed up SAT solvers. Algorithmic improvements in terms of decision heuristics and learning schemes have made modern SAT solvers orders of magnitude faster than their predecessors. Besides the algorithmic improvements, however, the engineering and design of data structures and the core propagation algorithm has also led to substantial increases in speed [109]. A typical modern SAT solver spends some 70-90% of its time performing Boolean constraint propagation, hence optimization of this inner propagation loop is of great importance.

SAT solving often produces highly non-local memory access patterns. Variable and clause reordering schemes have been investigated in the past [4] to try to increase locality of reference, but the non-locality is inherent in SAT solving and can never be completely eliminated.

Memory issues are even more important in parallel SAT solving and can dramatically affect the speed of a parallel SAT solver [44]. Most current multi-core systems contain L2 or L3 caches that are shared between multiple cores. When more than one thread is used, cache contention occurs and the effective cache capacity for each thread is reduced compared to the sequential case. Cache coherency caused by false sharing can also become an issue. If the private data of different threads are mixed together within the same cache line of memory (as could be the case, for example, if each thread used malloc to allocate memory for small chunks of private data like clauses), then any of those threads writing to their private data would force all the other threads to fetch that cache line from memory again, causing an unnecessary cache miss. Main memory access on a multi-core machine is also typically slower than on single processor machines and does not scale well with the number of cores. This causes a huge problem for memory intensive algorithms such as SAT solving. These issues makes it imperative for us to design data structures and algorithms that are as cache efficient as possible.

In this chapter, we modify MiniSAT 2.0 [39], a very well engineered SAT solver that came first in the industrial category of the SAT 2005 and SAT 2006 competitions, to utilize new data structures and algorithms that significantly improve its caching properties and thus its core propagation speed. We also demonstrate our improvements using our parallel solver PMiniSat, which is a simple parallelization of MiniSat 2.0.

The layout of this chapter is as follows. In Section 9.2, we describe in detail the data structures and algorithms currently employed in MiniSat 2.0. In Section 9.3, we describe the data structures and algorithmic modifications we made. In Section 9.4, we present an experimental evaluation of our changes. In Section 9.5, we discuss related work. Finally in Section 9.6 we conclude.

9.2 Boolean Constraint Propagation

As described in Chapter 2, the Boolean Satisfiability problem is a special case of the Constraint Satisfaction problem where all variables are Boolean, and all constraints are in the form of disjunctive clauses, i.e., constraints of form: $l_1 \vee \dots \vee l_k$ where each literal l_i is either a variable x or its negation $\neg x$.

State of the art SAT solvers are based on the DPLL algorithm [35], which interleave decision making with constraint propagation. Due to the specialization to clausal constraints, SAT solvers can use more specialized propagation algorithms than their CP counterparts. Modern SAT solvers utilize a two-watched literal scheme [109] for their core propagation engine. In the two-watched literal scheme, only two of the literals in any clause is being “watched” at any time. Whenever one of those watched literals is assigned a value and becomes “false” in the clause, the clause is examined to see if it is capable of causing a propagation. The two-watched literal scheme represents a vast improvement over earlier counter-based schemes used in older SAT solvers, significantly cutting down on the number of memory accesses and the amount of processing required [99].

In the two watched literal scheme, a “watched list” is created for each literal in the problem (both positive and negative literals). Each clause is attached to the watched

lists of the two literals in it that are being watched. Hence the pointer to each clause exists in exactly two of the watched lists at any time. Whenever a variable becomes assigned, the watched list of the literal that became “false” is examined, and each of the clauses pointed to by the pointers in the watched list is examined to determine if something needs to be done.

When a clause is examined, four different things can happen:

- (a) One of the literals in the clause is found to have value “true”. In this case, the clause is already satisfied and nothing needs to be done.
- (b) A literal other than the other watched literal is found to be unassigned. In this case, the watched literal that just became “false” is replaced by this new unassigned literal as the watched literal, and the clause is attached to the watched list of the new watched literal.
- (c) No other unassigned literal can be found, and the other watched literal is unassigned. In this case, the other watched literal is forced to have value “true” and we have a propagation.
- (d) no other unassigned literal can be found, and the other watched literal already has value “false”. In this case we have a conflict.

Pseudo-code for Boolean constraint propagation with watch literals is shown in Figure 9.1. The code takes an assignment A and a decision literal d . The queue Q of literals to be processed is a FIFO queue implemented by PUSH, POP, EMPTYQ and ISEMPY. The set $watch[l]$ is the set of clauses in T watched on literal l . $watch1[C]$ and $watch2[C]$ are the two watched literals of clause C . In certain places, we treat A and C as sets of literals. We can see the four cases defined above in the **if elseif elseif else** structure, although the case (a) is split into two since the first condition is common and cheap to check.

9.2.1 Boolean constraint propagation in MiniSat 2.0

MiniSat 2.0 utilizes a two-watched literal scheme. This is implemented in MiniSat 2.0 through a vector of “watched lists”, each of which is a vector of pointers pointing

```

bcp( $A, d$ )
   $A := A \cup \{d\}$ 
   $Q := \text{PUSH}(\text{EMPTYQ}, d)$ 
  while  $\neg \text{ISEMPTY}(Q)$ 
     $(Q, l) := \text{POP}(Q)$ 
    foreach  $C \in \text{watch}[\neg l]$ 
      if (a)  $\text{watch1}[C] \in A \vee \text{watch2}[C] \in A$ 
        continue
      elseif (a)  $\exists l' \in C \cap A$ 
         $\text{watch}[\text{watch1}[C]] := \text{watch}[\text{watch1}[C]] - \{C\}$ 
         $\text{watch1}[C] := l'$ 
         $\text{watch}[l'] := \text{watch}[l'] \cup \{C\}$ 
      elseif (b)  $\exists l' \in C, \neg l' \notin A, l' \neq \text{watch1}[C], l' \neq \text{watch2}[C]$ 
         $\text{watch}[\neg l] := \text{watch}[\neg l] - \{C\}$ 
         $\text{watch}[l'] := \text{watch}[l'] \cup \{C\}$ 
        if  $\neg l = \text{watch1}[C]$ 
           $\text{watch1}[C] := l'$ 
        else  $\text{watch2}[C] := l'$ 
      elseif (c)  $\exists l' \in C, \neg l' \notin A$ 
         $\text{watch}[\text{watch1}[C]] := \text{watch}[\text{watch1}[C]] - \{C\}$ 
         $\text{watch1}[C] := l'$ 
         $\text{watch}[l'] := \text{watch}[l'] \cup \{C\}$ 
         $A := A \cup \{l'\}$ 
         $Q := \text{PUSH}(Q, l')$ 
      else (d)
        return false
    return true

```

Figure 9.1: Boolean constraint propagation of the decision literal d given a current assignment A .

to the bodies of the clauses. Some excerpts of the MiniSat 2.0 data structure code together with their memory layout is shown in Figure 9.2.

The clauses have a special data structure in MiniSat 2.0. A clause consists of an initial 32 bits (`size_etc`) that contain the size of the clause as well as a few bit flags, followed by another 32 bits (`extra`) that contain the “activity” of the clause. This is followed by the literals of the clause itself, each of which take 32 bits. The clause is

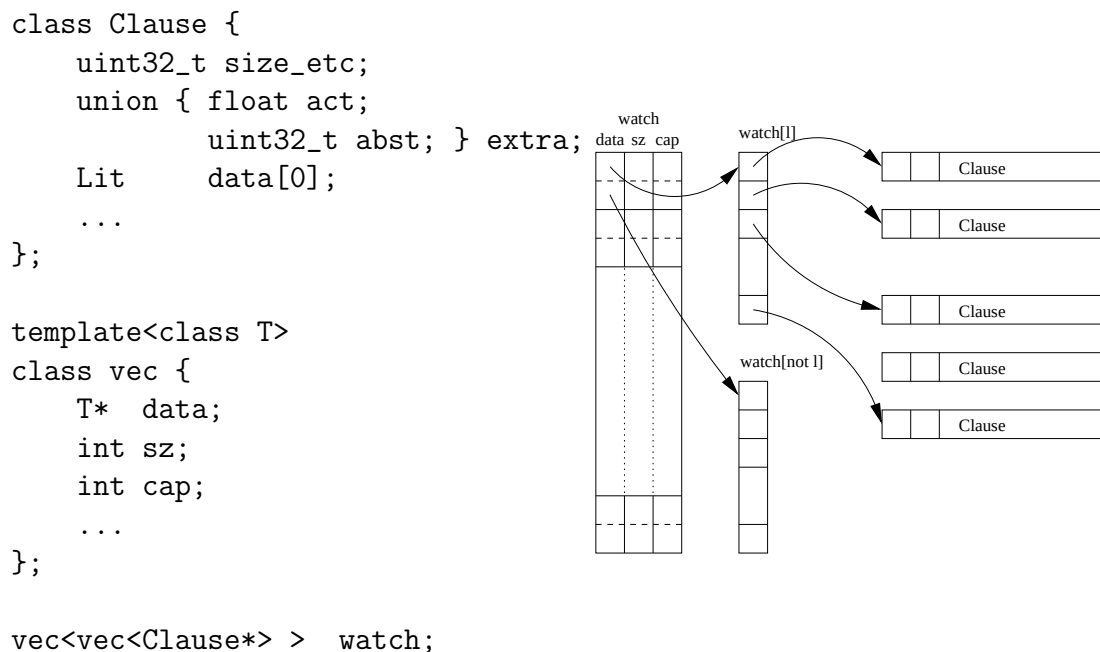


Figure 9.2: Data structure used in MiniSat 2.0

produced by mallocing the exact amount of memory needed to store the clause.

9.3 Improving the Cache Behavior

9.3.1 Rationale for changes

The two largest sources of memory consumption in MiniSat 2.0 are the watched lists, and the clauses. They are also the largest sources of cache misses. SAT solving typically exhibit highly non-local memory access patterns, as variable assignments can potentially require any clause to be examined, and clauses can potentially cause any variable to become fixed. Thus at any time, a very wide number of clauses and watched lists may potentially need to be examined. This makes caching highly ineffective as there is very little locality of reference. For typical industrial instances where the number of clauses could number in the hundreds of thousands, cache misses can cause a substantial slowdown in the core propagation engine. It has been found that more cache aware SAT solvers like Chaff and Berkmin are faster than their

predecessors by up to 3 times due to improved cache performance alone [166].

It may seem that with such awareness of cache issues and with so much work having been done to improve the core propagation engine of SAT solvers, there would be little if any improvements left that can be made. However, that turns out not to be the case. Using Cachegrind, a cache simulator in the Valgrind tool suite, we were able to identify and measure various sources of cache misses. The core of a SAT solver is the inner loop of the propagation engine where a clause is examined to see if it can produce a propagation. This loop is run hundreds of millions of times and constitutes the bulk of the processing in a SAT solver. By running Cachegrind on 10 typical medium sized industrial instances, we were able to find out how cache misses were affecting this inner loop. It was found that the SAT solver spent an average of around 80 instructions, 3.2 L1 misses and 0.64 L2 misses per inner loop on the instances tested (more details are provided in the results section). Current computers have L1 miss penalties of the order of 10-15 cycles, and L2 miss penalties of the order of 100-200 cycles, thus these figures show that a very substantial portion of the run time, perhaps 60-70%, is spent simply waiting for data to be read in. Having confirmed that cache misses are indeed a major problem, we targeted each source of cache misses we found in order to design data structures that could reduce them.

In the following sections, we describe each of our modifications in turn. Our changes are aimed at two things. Firstly, to increase locality of reference by using data structures that can pack the data closely together, and second, to eliminate as much data lookup as possible.

9.3.2 Watched list data structure modification

The data structure currently employed by MiniSat 2.0 to store the watched lists is a “vector” data structure. A vector as implemented in MiniSat 2.0 consists of both a head and a body. The head contains the size of the vector, the amount of memory malloced for the body, and a pointer to the body. The body contains the actual elements.

Cache simulations of MiniSat 2.0 on a set of 119 medium sized industrial instance

(see Appendix 1.1) revealed that reading in the watched lists during Boolean constraint propagation causes a significant number of cache misses. The cache misses can be divided up into two groups: misses from reading the head of the watched list vector, and misses from reading the body of the watched list containing the actual data. Interestingly, the number of cache misses from reading the head of the vectors is close to the number of cache misses from reading the body of the vectors even though the head actually contains no real information at all. This is clearly a source of inefficiency.

Further examination shows the cause of the problem. It turns out that in typical industrial instances, watched lists are often very short. The average length of the watched lists depends on the ratio between the number of clauses and the number of variables. This ratio often lies somewhere between 3-6 for the original problem depending on the problem class, and is largely independent of problem size. As more learnt clauses are created, the ratio will increase. But for a large problem, the number of learnt clauses never exceeds the number of original clauses by much. As a result, watched lists often only contain a small number of elements. For example, among the 119 problems we collected statistics for, 105 of them had over 75% of their watched lists having 16 elements or less, with most problems having around 85-90% of their watched lists being short. Further details can be found in Figure 9.3.

This small number of data elements means that most of the time all of the data in the watched list can be stored in just one cache line of memory. Normally, the cost of reading the vector head would be amortized over the cost of reading in the body of the vector. But since the watched list bodies often only require 1 cache line to be pulled in to read all of its data, the memory read required to read in the head becomes relatively expensive. Reading the head and the body separately basically causes 2 memory reads instead of one and doubles the number of reads required to read in the data and hence doubles the number of cache misses.

The vector data structure was probably chosen because the length of a watched list can change, and so any fixed sized data structure would not work. However, this doubling of the number of memory reads is inefficient. The modification we propose is to use custom vector data structures that can store the first few elements of the

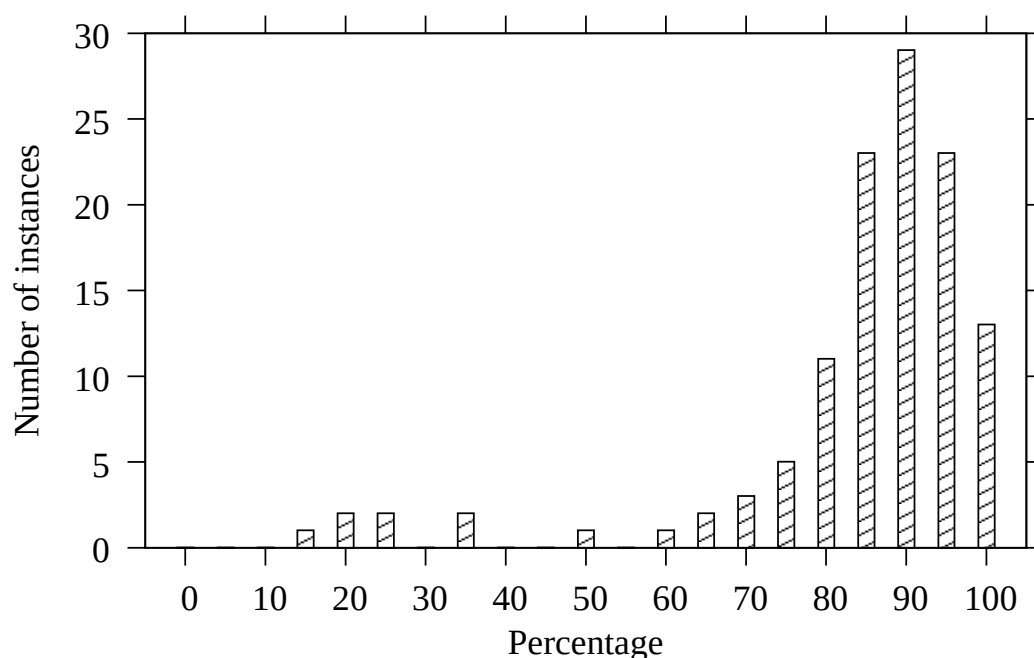


Figure 9.3: The histogram over the 119 tested instances of the percentage of short watched lists (length ≤ 16) encountered during the inner loop for Boolean constraint propagation.

watched list in the head part of the vector, and have the rest stored in the body as usual. Thus for a 64 byte cache line for example, we store the usual vector head information in the first 16 bytes, and use the remaining 48 bytes for storing actual data. This gives enough space to store 12 elements in the watched list head. The C++ definition and a diagram of the data structure is shown in Figure 9.4. Since watched lists are usually short, the data would often fit completely within those 48 bytes and the vector body does not need to be examined. Thus we can read the entire watched list by reading in only 1 cache line of memory. This modification basically halves the number of memory reads and cache misses caused by reading in the watched lists. However, some extra processing is required to both read and write to such a data structure.

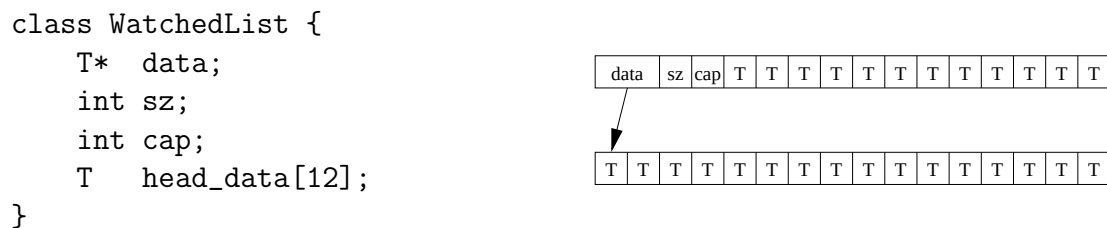


Figure 9.4: Watched list data structure

9.3.3 Clause data structure modification

Examination of typical propagation behavior also revealed some interesting statistics. It was found that clauses are often not fully examined during propagation, rather, only the first few literals are examined. Both case (a) and case (b) as described in Section 9.2 can result in the clause not having to be fully examined, as a true or unassigned literal can be found within the first few literals of the clause. Figure 9.5 shows the statistics on the distribution of how many literals of the clause are examined during each propagation. As can be seen, the probability that a clause has to be examined beyond a certain length drops very rapidly after the first few literals. In a very large proportion of the instances we investigated, some 50-90% of clause examinations terminate after just examining the first literal, and some 85-100% of clause examinations terminate within the first 4 literals.

In the original MiniSat, whenever a clause is examined, an entire cache line of data is likely to be pulled in. Since we often only actually need to read the first few literals of the clause, e.g 16 bytes out of a 64 byte cache line, pulling in an entire cache line consisting of the rest of the clause is wasteful and will tend to flush out other useful data. Furthermore, MiniSat 2.0 used a simple malloc to allocate memory for clauses, and thus we have no guarantee that clauses do not cross cache line boundaries or that there is no wasted space between clauses, etc.

The modification we propose is to split up clauses into a head part and a tail part. The head part consists of the first few literals of the clause, which is the part most likely to be examined, and the tail part consists of the rest of the literals. Since only the head part is likely to be examined in any examination of the clause, the head parts of clauses often have better locality of reference with each other than with their

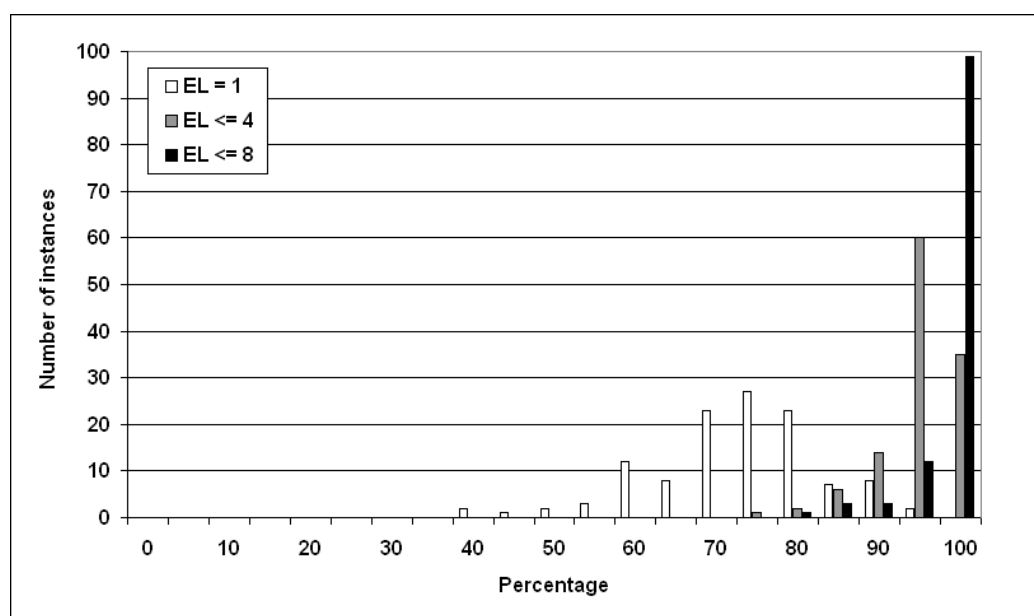


Figure 9.5: The histogram over the 119 tested instances of the percentage of clauses where $\leq k$ literals were examined in the inner loop for Boolean constraint propagation. The histogram shows the results for $k = 1$ (only first literal examined), $k = 4$ and $k = 8$.

own tail parts. Furthermore, since we know their exact size, we can manage their memory allocation manually and ensure that they are well packed along cache line boundaries.

The optimal choice for the length of the head part varies depending on the characteristics of the problem instance. It is necessary for a whole number of the clause heads to fit into each cache line, thus the available choice for the clause head sizes would for example 4, 8, 16, etc literals. If a new watch is often found within the first 4 literals in the problem instance, a choice of 4 would be best. If the instance often requires you to go beyond the first 4 literals to find a new watched literal, then a choice of 8 may be better, etc. However, in general, 4 literals for the head seems to be a good choice, as the statistics shows that in most problems, some 85-100% of clause examinations terminate within the first 4 literals. Furthermore, since most of the original clauses are of length 2 or 3, picking 4 literals for the head will allow us

to pack all of the original clauses very tightly together with little wasted space. Our experiments confirm that this is indeed the optimal length when this modification is implemented alone. We will later show, however, that some of our other very effective modifications alter the statistics so significantly that a choice of 4 literals for the head is no longer the optimum when those other modifications are made. This will be discussed in a later section.

We implement our split clause data structure by having a vector of clause heads, and a vector of clause bodies. If a clause has length 4 or less, we simply store those 4 literals in the clause head. If the clause has length greater than 4, we store the first 3 literals in the head and store an index to the rest of the clause in the 4th spot. The body of the clause is also split up into chunks, forming a linked list where each node fits exactly in one cache line. Having fixed size chunks for both clause heads and clause bodies allows us to easily free and reuse blocks of memory that have already been allocated. There is little if any penalty for splitting up the body into chunks, as the body chunks are rarely examined, and when the end of the cache line is reached, a new cache line would have to be read in in any case even if the clause was stored in a contiguous block of memory. A diagram of our data structure is shown in Figure 9.6.

A side benefit of this scheme is that the clauses can now be referred to by a 32 bit index rather than by a pointer, and on 64 bit systems, this cuts the size of the watched lists by half, further improving cache performance.

In the parallel context, since both the clause packing and the watched list modification pack the bulk of the data belonging to each thread into large contiguous arrays that belong to the thread, cache contention caused by false sharing is almost completely eliminated.

9.3.4 First literal

Perhaps the most significant statistic we collected is that, as noted above, in a surprisingly large proportion of time, only one literal in the clause is ever examined. It is often the case that the first literal in the clause is already assigned a value of true and thus the clause examination can immediately terminate. This occurred somewhere

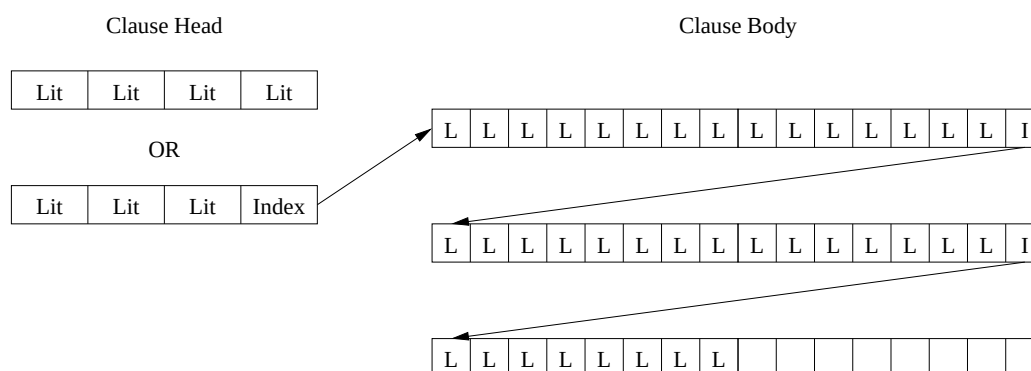


Figure 9.6: Clause data structures

between 50-90% of the time on most of the instances we examined. This is due to the way that the literals in the clause are reshuffled by the propagation algorithm. i.e., literals found to have value true are moved to the first spot, and literals that are made “true” by Boolean constraint propagation on that clause are moved to the first spot. This interesting statistic means that in a significant proportion of the time, a clause is read from memory only to read the value of that first literal. This is tremendously inefficient as each such read would pull in for example a 64 byte cache line just to read 4 bytes of data.

The modification we make is to store the first literal of the clause in the watched list itself along with the index to the clause. Then when the watched list is examined to perform propagation, that literal is first checked to see if it is in fact already true, in which case the clause does not have to be read in at all. And if the literal is not true, then the index is used to read in the clause as per usual. This modification effectively eliminates the memory read required to pull in the clause some 50-90% of the time. The cost of this modification is that the size of the watched lists are now doubled. However, the locality of reference is far better on the watched lists than among randomly located clauses in memory so this modification substantially improves the caching properties of the solver.

9.3.5 Binary clauses

An almost trivial extension to the previous modification is to handle binary clauses specially. Since we are already storing one literal of the clause in the watched list itself, it is quite easy to have each of the literals in the binary clause store the other literal in its watched list. We can simply mark the clause index with a bit flag to indicate whether the clause is binary or not, and if it is it can be handled specially. Since knowing the other literal in the clause means that you know everything about the clause, the clause itself never needs to be read in to perform propagation.

The cache benefit of this modification is to some extent negated by the previous modification as the previous improvement has already made it unnecessary to read in a large proportion of clauses. However, experiments show that a significant enough number of binary clauses lookups get through the previous improvement to make this modification worthwhile.

9.3.6 Ternary clauses

A somewhat more non-trivial change is to handle ternary clauses specially as well. Since the first literal modification in Subsection 9.3.4 requires that we store both a 4 byte clause index and a 4 byte first literal in the watched list for each clause, there is already enough space for us to store two literals, and thus we can store all of the information contained in ternary clauses in the watched lists themselves. The different variations of the watched list element are shown in Figure 9.7. This change complicates certain other aspects of the solver but is certainly possible. The benefit of this modification is less clear, as although we would eliminate all lookups of ternary clauses and thus have fewer cache misses, each of those clauses would now be attached to three watched lists instead of two, and thus the processing required to propagate them may increase by as much as 50%. Experiments seem to indicate that there is some benefit however.

As mentioned in Subsection 9.3.3, the optimal choice of the clause head length depends on the statistics of the problem. For the unmodified solver, a length of 4 is optimal. However, once the first literal modification, the binary clause handling

Normal Clause	First Literal	Clause Index
Binary Clause	Other Literal	Unused
Ternary Clause	1st Other Literal	2nd Other Literal

Figure 9.7: Watched list element data structure

and the ternary clause handling in Subsections 9.3.4, 9.3.5 and 9.3.6 are implemented as well, the statistics of the problem changes significantly. All binary and ternary clause lookups are now eliminated, as well as all lookups where the first literal is already true. This represents a significant proportion of the cases where the clause examination terminated within the first 4 literals. Thus once these modifications are made it can no longer be said that most remaining clause lookups terminate within the first few literals. The new optimum was found experimentally to be 16 for our set of instances, which coincides with the size of a cache line on our machine, although the difference in speedup is only around 5-10%.

9.3.7 Packing assignments

Besides the above major modifications, there are also some trivial packing of data that can be done. For example, in the original MiniSat 2.0, a single byte is used to represent the current value of each variable. Because of the very non-local way in which variables are examined, it will often be the case that when the algorithm examines one variables value, an entire cache line of assignment data is pulled in, the rest of which will never be used. Since 2 bits is in fact sufficient to store a variables assignment, it is possible to pack the value of 4 variables into each byte. Such packing means that each cache line contains more assignments and there is a greater chance that some of the other assignments pulled in with the cache line have some use.

9.3.8 Expected benefits

Most of the above changes trade more processing for reduced cache misses. Thus they are not guaranteed to improve performance. A benefit would only be gained if

the underlying assumptions for the modification hold, i.e., the source of cache misses being tackled is a major source of slow down, and the statistical assumptions behind the modification hold for the particular instance you are trying to solve. We expect this to be true for all reasonably large instances, e.g., $> 50,000$ variables. For small, instances however, cache misses are not an issue and the changes may well cause a slow down.

Another important point to note is that the effects of the modifications do not simply add up. In some cases there are synergies such that the total effect of the modifications is greater than the sum. In others, the effect of one modification is replicated by another to some extent and the total effect is less than the sum.

9.4 Experiments

9.4.1 Sequential solving

In the first set of experiments we test the effectiveness of our modifications in the context of sequential SAT solving using modified versions of MiniSat 2.0. An exhaustive test of each combination of data structure modifications is too time consuming. We test each modification in turn (except for the ternary clause handling as it is difficult to implement alone), all of them together, and certain combinations that produce interesting synergies.

The tests were performed on Xeon Pro 2.4GHz processors with 32kb L1 cache, 4Mb L2 cache, 64 byte cache line and 16-way set associativity. The solver was compiled using gcc 3.4.5 using -O3 optimization. 24 medium sized industrial instances from SAT competition 2007 [134] were used (See Appendix 1.2). These instances come from 9 different problem classes. The original version was run 25 times with different random seeds and the modified versions were run 20 times with different random seeds. This brings the standard error down to around 5%. The instruction count and cache miss counts were collected using the Valgrind cache simulator. For the cache simulation, only 10 instances were tested with 1 run each, so these numbers are merely indicative. The results are shown in Table 9.1.

Table 9.1: Cache conscious optimizations for sequential SAT solving. Comparison of various combinations of optimizations: WL — Watched list modification (9.3.2), PC(n) — Clause modification with length n clause head, n = 16 if labeled as just PC (9.3.3), FL — Store first literal in watched list (9.3.4), BC — Handle binary clauses specially (9.3.5), TC — Handle ternary clauses specially (9.3.6) PA — Pack assigns (9.3.7). The results show the (geometric mean) runtime for the suite, the speedup compared to MiniSat 2.0, I/Loop stands for average number of instructions per inner loop of the propagation engine, L1M/Loop and L2M/Loop stands for the average number of L1 and L2 misses per inner loop of the propagation engine.

24 medium sized industrial instances (Appendix 1.2)

Modifications	Runtime (s)	Speedup	I/Loop	L1M/Loop	L2M/Loop
MiniSat 2.0	59.05	—	80.7	3.21	0.64
WL	62.17	0.95	97.7	3.02	0.53
BC	62.07	0.96	93.7	3.30	0.69
PA	59.17	1.00	99.7	2.89	0.62
PC	64.39	0.92	97.7	3.14	0.65
PC(8)	54.85	1.08	97.9	3.05	0.48
PC(4)	49.78	1.19	98.3	2.95	0.35
FL	47.91	1.25	83.7	2.58	0.39
PA+PC(4)	45.51	1.30	99.3	2.63	0.34
WL+PC+FL	38.96	1.53	104.5	2.44	0.29
WL+PC+FL+BC	35.14	1.70	99.3	2.23	0.24
WL+PC+FL+BC+TC	33.16	1.80	89.8	1.78	0.16

22 hard industrial instances (Appendix 1.3)

Modifications	Runtime (s)	Speedup
MiniSat 2.0	603.23	—
WL+PC+FL+BC+TC	318.62	1.89

151 industrial instances from SAT Race 2008 (Appendix 1.4)

Modifications	Runtime (s)	Speedup
MiniSat 2.0	63.41	—
WL+PC+FL+BC+TC	41.48	1.53

To test that our improvements generalize, we ran the final version with all modifications on two other sets of instances. One set consists of 22 hard instances taking up to several thousands seconds to solve. These were run without any timeout to test the effect of the cache improvements when the problem is difficult and there is a high

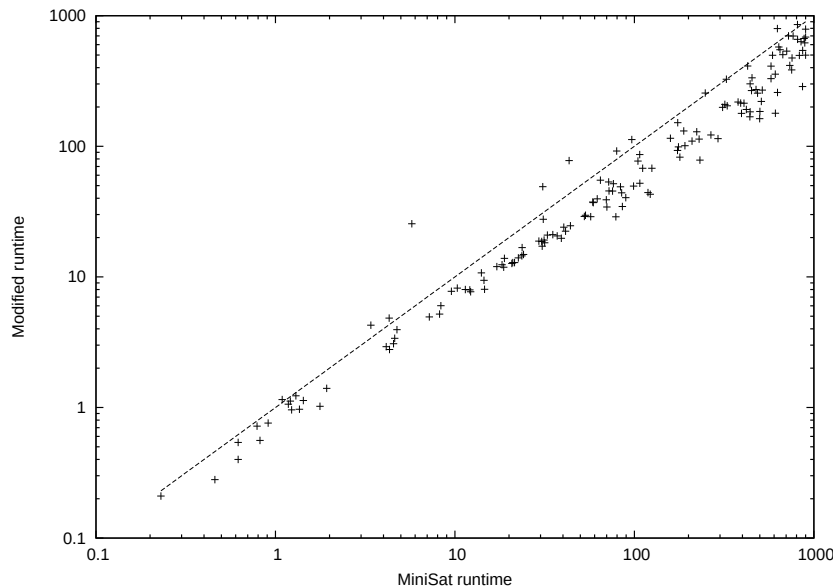


Figure 9.8: Runtime comparisons on 151 SAT Race instances

proportion of learnt clauses. The second set consists of the 200 industrial instances used in SAT Race 2008. These contain a wide range of problem classes and problem difficulty, including smaller instances for which our cache improvements are not effective. With a timeout of 900 seconds, only 151 of them can be solved consistently. Figure 9.8 shows a scatter plot comparing the runtime of the modified version with the original version on those 151 instances.

As the figures in Table 9.1 show, the modifications all increased the amount of processing required, but most of them compensated for that with reduced cache misses, leading to substantial speedups. Clause packing, when implemented alone, is most effective for a clause head length of 4, which is what we expect given the large number of binary and ternary clauses in the original problems. Clause packing is able to reduce L1 cache misses by around 10% and L2 cache misses by around 45%, resulting in a speedup of around 19%. The first literal modification also produce a significant speedup, reducing L1 cache misses by around 20% and L2 cache misses by around 40%, giving a speedup of around 25%. The case for the watched list modification, binary clause modification and assignments packing is less clear. The watched list

modification is able to reduce L2 cache misses by around 20%, but the extra processing required has apparently outweighed the benefits. A more efficient implementation of the modification may yield a better speedup. The binary clause modification yielded no speedup whatsoever. In fact, it seems slightly inferior to the original. However, it produces some speedup when coupled with the first literal modification (extra 16%), as the data structure changes used in the first literal modification allow the binary clause modification to be implemented with little extra overhead. Assignments packing also yielded no speedup when implemented alone. However, as a comparison of PA+PC(4) with PC(4) shows, assignments packing can produce a non-negligible speedup (extra 11%) when coupled with other modifications. The best combination of modifications WL+PC+FL+BC+TC was able to reduce L1 misses by around 45% and reduce L2 misses by around 75%, leading to a speedup of 80%.

The average speedup gained from the best combination of modifications is around 80% on the initial 24 instances we examined. There is a large amount of individual variation in the speedups of the instances, with the speedup ranging between 19% to as high as 178%. The average speedup on the 22 hard instances is 89%. Thus the result seems to generalize to hard instances with a higher proportion of learnt clauses. The speedup on the 151 solvable SAT Race instances is only 53%. However, around 40% of those are smaller instances where our cache improvements are not supposed to be effective.

In summary, it can be said that the clause packing, first literal modification, and the special binary clause handling each increase the propagation speed by around 15-25%, and the ternary clause handling increases it by another 10%. The changes caused by the watched list modification and the assignments packing are too inconsistent to be considered a real improvement.

9.4.2 Parallel solving

In the second set of experiments we test the effectiveness of our modifications in the context of parallel SAT solving using the parallel SAT solver PMiniSat. Two versions of PMiniSat were made, one with and one without the data structure modifications.

Table 9.2: Cache conscious optimizations for parallel SAT solving. We compare a parallelized version of MiniSat 2.0, PMiniSat, using the original data structures and the combination of optimizations detailed in Table 9.1. The results are the geometric mean of 20 runs.

Threads	Original		WL+PC+FL+BC+TC		
	Runtime (s)	Efficiency	Runtime (s)	Efficiency	Speedup
1	44.03	—	28.30	—	1.56
2	26.94	0.82	15.19	0.93	1.77
3	21.75	0.67	12.70	0.74	1.71
4	19.06	0.58	10.52	0.67	1.81
5	19.90	0.44	10.15	0.56	1.96
6	21.42	0.34	10.70	0.44	2.00
7	23.41	0.27	10.31	0.39	2.27
8	26.44	0.21	11.03	0.32	2.40

The tests were performed on an 8 core Mac with 2x Harpertown Quad core Xeon 2.8GHz processors. Each core has a 32kb L1 cache, and each pair of cores share a 6Mb L2 cache with a 64 byte cache line and 24-way set associativity. The solver was compiled using gcc 4.0.1 using -O3 optimization. We run both versions of the solver with 1 to 8 threads. Again the tests are done on the 24 medium sized industrial instances (Appendix 1.2). Each instance was run 20 times with different random seeds. The results are given in Table 9.2.

The sequential (1 thread) speedup due to the modifications is slightly lower on the Mac than on the Xeon Pro. This is to be expected as the solver has access to a full 6Mb of L2 cache on the Mac compared to only 4Mb on the Xeon Pro, thus cache misses are less of an issue and the cache improvements have a smaller effect. The speedups produced by the modifications clearly increase with increasing number of threads, from 56% speedup for 1 thread up to 140% for 8 threads.

The dramatic drop in efficiency as more threads are used is somewhat surprising at first glance, as it is known from other experiments that PMiniSat is highly efficient algorithmically, achieving an efficiency of 103% at 4 threads and 87% at 8 threads on these 24 instances in terms of the total number of executions of the inner loop

over all threads. The communication cost in PMiniSat is also negligible. Further investigation shows that the relatively low efficiency of PMiniSat on this multi-core machine (with or without cache improvements) can be directly attributed to memory costs.

Our cache simulation numbers show that the average rate of L2 cache misses on the unmodified sequential solver was of the order of 10 million misses per second. With a write-back cache policy, and a 64 byte cache line size, this translates into a fairly massive 1.28 Gb/s of traffic between the processor and the main memory. Thus even though the entire clause database of a medium sized industrial problem may only require 20-50 Mb of memory, the highly non-local memory access pattern forces an enormous amount of memory transfer to occur. The situation is far worse for our parallel solver, as the memory bandwidth requirement will be multiplied by the number of threads. Furthermore, issues such as cache contention, shared data and speculative prefetching by the Harpertown processors will increase the amount of memory transfer that each thread needs even further. In the sequential solver, L2 misses were already consuming some 50-60% of the cpu time. From our numbers, it appears that in the parallel solver, this proportion grows to around 75% for 4 threads and around 90% for 8 threads. Thus when the parallel solver is run with many threads, the limiting factor is not processing power, but the ability of the main memory to handle an extremely large number of memory accesses. On the 8-core Mac that the experiments were run on, the main memory is clearly not fast enough to satisfy all 8 cores.

To illustrate this, memory stress tests using a memory access pattern similar to a SAT solver were run on the 8 core Mac. In the stress tests, 64 Mb of contiguous memory is allocated to each thread. Each thread then reads a random location within its block and increments the value. This loop is performed a billion times. As can be seen in table 9.3, the efficiency drops dramatically with increasing number of threads and the trend is very similar to what is observed on the parallel SAT solver.

In the sequential context, our modifications were able to substantially reduce cache misses. In the parallel context, they provide even more speedup because of the other memory issues that occur. When multiple threads share an L2 cache, capacity

Table 9.3: Parallel memory stress test.

Threads	Runtime (s)	Efficiency
1	29.2	—
2	22.6	0.65
4	18.2	0.40
8	16.8	0.22

contention occurs. Since the vast majority of cache misses in SAT solving are capacity misses, when two threads share the same cache, the effective L2 cache size for each of them is roughly halved. On the Harpertown processors, each thread would effectively have 3Mb or less of L2 cache instead of 6Mb for the sequential case. Thus L2 misses are far higher and our cache improvements produce more speedup. Main memory contention is clearly a very serious source of inefficiency for parallel SAT solvers. Our cache improvements reduce L2 misses, which translates into less main memory accesses and thus less contention, once again producing more speedup. Although our numbers show that it would be impractical to run our solver with more than 4 or 5 threads on this particular machine, the speedups illustrate very well how much more effective our modifications are in the parallel context, as our modifications produce a far greater speedup of 140% with 8 threads than the 56% with 1 thread.

PMiniSat came 2nd in both the sequential and parallel track of SAT Race 2008 [135], showing that the cache improvements described in this paper can indeed make a solver highly competitive with other state of the art SAT solvers.

9.5 Related Work

The inefficiency of having the clauses reside in non-contiguous, randomly allocated memory locations has been noted before, and array style clause memory management have been attempted. For example, in Chaff [109], a large chunk of contiguous memory is initially allocated for clauses. Memory from this pool is then allocated to clauses as needed. However, in their implementation, the memory is simply allocated

from the end of the pool, and they do not attempt to reuse freed chunks of memory caused by deleted clauses until an infrequent monolithic garbage collection step compactifies the remaining clauses. This somewhat reduces the effectiveness of using an array for the clauses, as clauses are no longer in contiguous parts of memory due to there being deleted clauses in between. Our implementation allocates fixed sized chunks of memory to clauses, thus it is easy for us to keep a stack of freed chunks that can be reused immediately. This makes better use of memory and also produces better locality of reference.

Our first literal modification bears some resemblance to the Watched Literal Reference List (WLRL) used in [96], however, their WLRL data structure is motivated by different considerations. In their parallel solver MiraXT, clauses are read-only and are shared among multiple threads, thus in order for watched literal propagation to occur, each thread has to remember the two literals in the clause that are attached to their watched lists. This data is stored in an extra data structure called the WLRL. Similar to our first literal modification, examining the WLRL can allow the clause examination to terminate without examining the original clause. However, the authors do not examine the cache behavior of this design or whether it can produce any speedups.

The clause data structures used in CMUSAT [79] is also similar to our first literal modification and is motivated by the same reasons. However, instead of storing the first literal in the watched list itself as we do, they store the two watched literals of each clause in a separate array, essentially creating an abridged version of each clause. This is very similar to the WLRL described above. When a clause is examined, their algorithm first examines these two watched literals to see if they are true. If they are, then the main clause is not examined. It seems to us however, that such a design may not decrease cache misses by much. The examination of the two watched literals, which is stored in a separate array from the watched list, is very likely to cause a cache miss as well, as it has the same highly non-local access pattern that the clauses suffer from. And in the case that the main clause has to be examined, their design may end up causing two misses instead of one. Our modification is superior, as storing the first literal in the watched list itself provides far better locality of reference.

The special handling of binary clauses is not a new idea. In fact, MiniSat 1.13 had a similar implementation that was later removed in MiniSat 2.0 for readability reasons. Special handling for ternary clauses have also been considered before, for example [133] discusses ways in which ternary clause could be handled specially by packing the literals together. Their packing scheme involves packing the 3 literals into only 64 bits. This can substantially reduce the memory footprint when a large majority of clauses are of length 3. However, their implementation limits each literal to 20 bits, with 1 bit reserved for a flag. This means that a maximum of 524,288 variables are allowed. Such a restriction may have been reasonable at the time, but is no longer really reasonable given the sizes of the problems that can be solved today. Our changes do not modify the length of a literal, but do utilize two of its bits for special flags. Thus we retain a limit of 536,870,912 variables, which ought to be sufficient for the foreseeable future.

A data structure that tackles cache misses in a different way is presented in [15]. In their solver PicoSAT, the watched lists are stored as a linked list rather than as a vector. A list head points to the first clause being watched, and each clause contains a link to the next clause on the watched list. This redistributes the memory needed to store the watched list into the clauses themselves. Cache misses from reading watched lists should be substantially reduced. However, this data structure is incompatible with the first literal modification presented in this chapter. Their data structure requires that all clauses be examined in order to find the next clause to be examined, however, the purpose of the first literal modification is to avoid having to examine any clauses in the first place. They also handle binary clauses specially by storing them in a separate stack. Their results show that their linked list based solver with special binary clause handling was $\sim 15\%$ faster than the naive vector based data structure as used in MiniSAT. However, this would seem to be inferior to our modifications, as even just the first literal modification alone gives a $\sim 25\%$ speedup.

A number of cache related optimizations are discussed in [72], which was published after the work in this chapter was initially published. Some of these optimizations are very similar to those contained in this chapter, e.g., compacting the data structures used for storing assignments, using a slab allocator to manage memory. They also

discuss a number of optimizations which are already known, such as maintaining the watch list lazily and moving clause literals into the “clause header” (both already implemented in MiniSat 2.0). The novelty in this paper is the use of prefetching during propagation to avoid cache misses, which reportedly gave $\sim 20\%$ speedup when implemented in MiniSat 2.0.

9.6 Conclusion

In this chapter, we have described a series of data structure and algorithmic modifications that were able to increase the core propagation speed of MiniSat 2.0 by some 80% on a wide range of medium sized industrial instances. An even greater speedup of around 140% was obtained when the modifications were incorporated into the parallel SAT solver PMiniSat running with 8 threads. Cache simulations were used to find the main sources of cache misses in MiniSat 2.0. Propagation statistics were then collected that guided us towards the design of more cache conscious data structures and algorithms. Three of our modifications produced unambiguous speedups of the order of 15-25% each in the sequential context. Our experiments indicated that memory costs dominate execution in the parallel solver, and hence cache improvements are even more important. Our modifications are relatively easy to implement and can easily be incorporated into other state of the art SAT solvers, most of which also utilize the same two watched propagation as MiniSat 2.0.

Chapter 10

Confidence Based Work Stealing

10.1 Introduction

One important criteria for the efficient parallelization of CP solvers is load balancing, i.e., distributing the work so that none of the processors go idle due to a lack of work. Architectures for parallel search in constraint programming typically use *work stealing* for dynamic load balancing, see for example [105, 114, 138, 130]. In work stealing schemes, whenever a processor runs out of work, it would “steal” some workload from a processor that still has work remaining.

Analysis of work stealing schemes often assume that the amount of work to be done is fixed and is independent of the work stealing scheme (e.g., [89]). While this is true for certain kinds of problems (finding all solutions, proving unsatisfiability), it is not true for others (finding a first solution, finding an optimal solution). Such analyses fail to take into account the fact that the place from which work is stolen in the search tree affects the search strategy, and can have a dramatic effect on the efficiency of the search.

Many approaches choose to steal from as close to the root of the search tree as possible, e.g., [138], as this tends to give the greatest granularity of work and minimizes the overhead of work stealing. However, this is not always the best strategy in terms of search order.

Effect of work stealing. Let us consider a relatively simple framework for parallel search. One thread begins with ownership of the entire search tree. When a thread finishes searching the subtree it was responsible for, it will steal an unexplored part of the search tree (a *job*) from its current owner. This continues until a solution is found or the entire search tree has been searched.

Example 10.1.1. The first problem we will consider is a simple model for small instances of the Traveling Salesman Problem, similar to [22]. In our experiments, with 8 threads, stealing left and low (as deep in the tree as possible) requires visiting a number of nodes equal to the sequential algorithm, while stealing high (near the root) requires visiting $\sim 30\%$ more nodes on average (see Table 10.1).

The explanation is simple. In one example instance, the sequential algorithm finds

the optimal solution after 47 seconds of CPU time, after which it spends another ~ 300 seconds proving that no better solution exists. When the parallel algorithm is stealing left and low, all threads work towards finding that leftmost optimal solution, and the optimal solution is found in 47 seconds of CPU time as before (wall clock time $47/8 \approx 6$ seconds). After this, the search takes another 300 seconds of CPU time to conclude. Thus we have perfect linear speedup both in finding the optimal solution, and in proving that no better solution exist.

However, when the parallel algorithm is stealing high, only one thread actually explores the leftmost part of the search tree and works towards that leftmost optimal solution. The other seven threads explore other parts of the search tree, unfruitfully in this case. This time, the optimal solution is found in 47 seconds of *wall clock* time (376 seconds of CPU time!). The algorithm then spends another 200 seconds of CPU time proving optimality. That is, there is no speedup whatsoever for finding the optimal solution, but linear speedup for proving optimality. Since the optimal solution has been found so much later (376 seconds CPU time instead of 47 seconds), the threads search without the pruning benefits of the optimal solution. Hence, the number of nodes searched drastically increases, leading to a great loss of efficiency. Clearly, this effect gets worse as the number of threads increases. $\square$

It may appear from this example that stealing left and low would be efficient for all problems. However, such a strategy can produce at best linear speedup.

Example 10.1.2. Consider the n -Queens problem. The search tree is very deep and a top level mistake in the branching will not be recovered from for hours.

Stealing low solves an instance within the time limit iff sequential depth first search solves it within the time limit. This is the case when a solution is in the very leftmost part of the search tree (only 4 instances out of 100, see Table 10.2).

Stealing high, in contrast, allows many areas of the search tree to be explored, so a poor choice at the root of the search tree is not as important. Stealing high results in solving 100 out of 100 instances tested. This is clearly far more robust than stealing low, producing greatly super-linear speedup. $\square$

Veron *et al* [151] claim that linear and super linear speedups can be expected for branch and bound problems, but they fail to note that finding the optimal solution does not parallelize trivially as shown by Example 10.1.1. Rao and Kumar [125] (and others) show that super linear speedup ought to be consistently attainable for finding the first or the optimal solution for certain types of problems. Their analysis is valid if the search tree is random (i.e., the solutions are randomly distributed), but is not valid in systems where a branching heuristic orders the branches based on their likelihood of yielding a solution. The presence of such a branching heuristic makes linear speedup in finding solutions non-trivial. Gendron and Crainic [57] describe the issue and provide a description of how the issue is handled in several systems. In general, the solutions utilize some kind of best-first criterion to guide how the problem is split up (see e.g., [123, 107]).

Contributions. We give a quantitative analysis of how different work stealing strategies affect the total amount of work performed and explain the relationship between branching heuristic strength and optimal search strategy. We introduce *confidence-based work stealing* as an adaptive algorithm that, when provided with a user-defined *confidence*, will steal work in a near optimal manner. The confidence is the estimated ratio of solution densities between the subtrees at each node. This chapter shows that confidence-based work stealing leads to very good algorithmic efficiencies, that is, not many more, and sometimes much less, nodes are explored than for sequential DFS (Depth First Search).

Although the analysis is done in the context of parallel search for constraint programming, the analysis is actually about the relationship between branching heuristic strength and the optimal search order created by that branching heuristic. Thus the analysis actually applies to all complete tree search algorithms whether sequential or parallel. It shows that when the assumptions about branching heuristic strength that lie behind standard sequential algorithms such as DFS, Interleaved Depth First Search (IDFS) [103], Limited Discrepancy Search (LDS) [68] or Depth-bounded Discrepancy Search (DDS) [153] are given to the algorithm as *confidence* estimates, the algorithm produces the exact same search patterns used in those algorithms. Thus the analysis

and algorithm provides a framework which explains/unifies/produces all those standard search strategies. In contrast to the standard sequential algorithms which are based on rather simplistic assumptions about how branching heuristic strength varies in different parts of the search tree, our algorithm can adapt to branching heuristic strength on a node by node basis, potentially producing search patterns that are vastly superior to the standard ones. The algorithm is also fully parallel and thus we have in effect parallelised DFS, IDFS, LDS and DDS.

Confidence-based search shares the idea to explore more promising parts of the search tree first with other approaches such as impact-based search [126], using solution counting [164] and constraint-level advice [147] for guiding search. However, there are two key differences. First, confidence-based search uses standard branchings heuristics that define the shape of the search tree augmented by confidence. This makes the addition of confidence-based search to an existing constraint programming solver straightforward and allows us to reuse existing constraint models with user-defined branchings. Second, confidence-based search is designed to work in parallel.

10.2 Analysis of Work Allocation

In this section we show quantitatively that the strength of the branching heuristic determines the optimal place to steal work from. We will concentrate on the case of solving a satisfaction problem. The case for optimization is related since it is basically just a series of satisfaction problems.

Preliminary definitions. The basic algorithm for a CP solver is given in Figure 2.2. At each search node, we perform propagation until we reach a fixed point. If the problem is not trivially failed or solved, we call a function $branch(C, D')$ which divide the problem up into more constrained subproblems. The function $branch(C, D')$ returns an ordered set of decision constraints $\{b_1, \dots, b_k\}$. We try to solve each of the subproblems $(V, D', C \wedge b_i)$ in turn. The function $branch(C, D')$ implements what we call the branching heuristic, and defines a *search tree*. Each node where we have to branch becomes an internal node, and each node where propagation solved the

subproblem becomes a leaf node. Leaf nodes can either be failure nodes or solution nodes. We define the *solution density* of a search tree T with k nodes and l solution nodes as l/k .

Optimal split for binary nodes. For simplicity, assume that visiting each node in the search tree has roughly equal cost. Assuming an oracle that provides accurate information on solution density, work stealing from nodes whose subtrees have the highest solution densities will be optimal.

In practice however, the solution density estimates will not be perfect:

1. Any estimate of the solution density of a subtree will have a very high error, with a substantial chance that the solution density is actually zero.
2. The real solution densities, and hence the errors in the estimate, are highly correlated between nearby subtrees, as they share decision constraints from higher up in the tree.
3. The solution density estimate of a subtree should decrease as nodes in that subtree are examined without finding a solution. The causes are as follows:
 - (a) As the most fruitful parts of the subtree are searched, the average solution density of the remaining nodes decrease.
 - (b) The correlation between solution densities between nearby subtrees mean that the more nodes have failed in that subtree, the more likely the remaining nodes are to fail as well.

We have to take these issues into account when utilizing solutions densities to determine where to steal work. Given the actual solution density probability distribution for the two branches, we can calculate the expected number of nodes searched to find a solution. We derive the expression for a simple case. Suppose the solution density probability distribution is uniform, that is, it has equal probability of being any value between 0 and S where S is the solution density estimate. Let A and B be the solution density estimates for the left and right branch respectively, and assume

a proportion p and $(1 - p)$ of the processing power is sent down the left and right branch respectively. Then the expected number of nodes to be searched is given by the hybrid function:

$$f(A, B, p) = \begin{cases} \frac{1}{pA}(2 + \ln(\frac{pA}{(1-p)B})) & \text{for } pA > (1-p)B \\ \frac{1}{(1-p)B}(2 + \ln(\frac{(1-p)B}{pA})) & \text{otherwise} \end{cases} \quad (10.1)$$

The shape of this function does not depend on the absolute values of A and B (which only serve to scale the function), but on their ratio, thus the shape is fixed for any fixed value of $r = A/(A + B)$.

The value of p which minimizes the function for given value of r is well approximated by the straight line $p = r$. In fact, the value of the f function at $p = r$ is no more than 2% higher than the true minimum for any r over the range of $0.1 \leq r < 0.9$. For simplicity, we will make this approximation from now on. This means that it is near optimal to divide the amount of processing power according to the ratio of the solution density estimate for the two branches. For example, if $r = 0.9$, which means that A is 9 times as high as B , then it is near optimal to send 0.9 of our processing power down the left branch and 0.1 of our processing power down the right.

Branching confidence. Define the *confidence* of a branching heuristic at each node as the ratio $r = A/(A + B)$. The branching heuristic can be considered *strong* when $r \rightarrow 1$, that is the solution density estimate of the left branch is far greater than for the right branch. In other words, the heuristic is really good at shaping the search tree so that solutions are near the left. In this case, our analysis shows that since r is close to 1, we should allocate almost all our processing power to the left branch every time. This is equivalent to stealing work for search as left and as low as possible. The branching heuristic is *weak* when $r \approx 0.5$, that is, the solution density estimate of the left branch and right branch are similar because the branching heuristic has no insight into where the solutions are. In this case, our analysis shows that since $r = 0.5$, the processing power should be distributed evenly between left and right branches at each node. This is equivalent to stealing work for search as high as possible.

10.3 Adaptive Work Stealing

Our analysis shows that the optimal work stealing strategy depends on the strength of the branching heuristic. Since we have a quantitative understanding of how optimal work stealing is related to branching heuristic strength, we can design a search algorithm that can automatically adapt and produce “optimal” search patterns when given some indication of the strength of the branching heuristic by the problem model. In this section, we flesh out the theory and discuss the implementation details of the algorithm.

10.4 Dynamically Updating Solution Density Estimates

Now we examine how solution density estimates should be updated during search as more information becomes available.

First we need to relate the solution density estimate of a subtree with root (C, D) with the solution density estimate of its child subtrees (the subtrees rooted at its child states $(C \wedge c_i, \text{solv}(C \wedge c_i, D))$). Consider an n -ary node. Let the subtree have solution density estimate S . Let the child subtree at the i -th branch have solution density estimate A_i and have size (number of nodes) k_i . If S and A_i are estimates of average solution density, then clearly: $S = \sum_{i=1}^n A_i k_i / \sum_{i=1}^n k_i$, i.e., the average solution density of the subtree is the weighted average of the solution densities of its child subtrees.

Uncorrelated subtrees. Assuming no correlation between the solution densities of subtrees, we have that if the first j child subtrees have been searched unsuccessfully, then the updated solution density estimate is $S = \sum_{i=j+1}^n A_i k_i / \sum_{i=j+1}^n k_i$. Assuming that k_i are all approximately equal, then the expression simplifies to:

$$S = \sum_{i=j+1}^n A_i / (n - j)$$

For example, suppose $A_1 = 0.3, A_2 = 0.2, A_3 = 0.1$, then initially, $S = (0.3 + 0.2 + 0.1)/3 = 0.2$. After branch 1 is searched, we have $S = (0.2 + 0.1)/2 = 0.15$, and after branch 2 is searched, we have $S = (0.1)/1 = 0.1$. This has the effect of reducing S as the branches with the highest values of A_i are searched, as the average of the remaining branches will decrease.

Correlated subtrees. Now we consider the case where there are correlations between the solution density estimates of the child subtrees. The correlation is likely since all of the nodes in a subtree share the constraint C of the parent state. Since the correlation is difficult to model we pick a simple generic model. Suppose the solution density estimates for each child subtree is given by $A_i = \rho A'_i$, where ρ represents the effect on the solution density due to the constraint added at the parent node, and A'_i represents the effect on the solution density due to constraints added within branch i . Then ρ is a common factor in the solution density estimates for each branch and represents the correlation between them. We have that:

$$S = \frac{\sum_{i=1}^n A_i k_i}{\sum_{i=1}^n k_i} = \rho \frac{\sum_{i=1}^n A'_i k_i}{\sum_{i=1}^n k_i}.$$

Suppose that when j out of n of the branches have been searched without finding a solution, the value of ρ is updated to $\rho^{\frac{n-j}{n}}$. This models the idea that the more branches have failed, the more likely it is that the constraint C added at the parent node has already made solutions unlikely or impossible. Then, after j branches have been searched, we have: $S = \rho^{\frac{n-j}{n}} \sum_{i=j+1}^n A'_i k_i / \sum_{i=j+1}^n k_i$. Assuming that all k_i are approximately equal again, the expression simplifies to: $S = \rho^{\frac{n-j}{n}} \sum_{i=j+1}^n A'_i / (n-j) = \frac{\rho}{n} \sum_{i=j+1}^n A'_i = \sum_{i=j+1}^n A_i / n$. Equivalently, we can write it as:

$$S = \frac{1}{n} \sum_{i=1}^n A_i \text{ and } A_i = 0 \text{ for } 1 \leq i \leq j \quad (10.2)$$

where we update A_i to 0 when branch i fails. The formula can be recursively applied to update the solution density estimates of any node in the tree given a change in solution density estimate in one of its subtrees.

Using confidence. In all of our results, the actual values of the solution densities are not required. We can formulate everything using *confidence*, the ratio between the solution densities of the different branches at each node. In terms of confidence, when a subtree is searched and fails, the confidence values should be updated as follows:

Let r_i be the confidence value of the node i levels above the root of the failed subtree and r'_i be the updated confidence value. Let $\bar{r}_i = r_i, \bar{r}'_i = r'_i$ if the failed subtree is in the left branch of the node i levels above the root of the failed subtree and $\bar{r}_i = 1 - r_i, \bar{r}'_i = 1 - r'_i$ otherwise. Then:

$$\bar{r}'_i = (\bar{r}_i - \prod_{k=1}^i \bar{r}_k) / (1 - \prod_{k=1}^i \bar{r}_k) \quad (10.3)$$

10.5 Confidence Model

Given a confidence at each node, we now know how to steal work “optimally” and how to update confidence values as search proceeds. But how do we get an initial confidence at each node. Ideally, the problem modeler, with expert knowledge about the problem and the branching heuristic can develop a solution density heuristic that gives a confidence value at each node. However, this may not always happen, perhaps due to a lack of time or expertise. We can simplify things by using general confidence models. For example, we could assume that the confidence takes on an equal value *conf* for all nodes. This is sufficient to model general ideas like: the heuristic is strong or the heuristic is weak. Or we could have a confidence model that assigns $r = 0.5$ to the top d levels and $r = 0.99$ for the rest. This can model ideas like the heuristic is weak for the first d levels, but very strong after that, much like the assumptions used in DDS [153].

10.6 Algorithm

Given that we have a confidence value at each node, our confidence-based search algorithm will work as follows. The number of threads down each branch of a node is updated as the search progresses. When search for a subtree is finished, the confidence

values of all nodes above the finished subtree are updated as described by Equation (10.3) above.

When work stealing is required, we start at the root of the tree. At each node we use the number of threads down each branch, the confidence value, to determine which branch to take. Given the number of threads down each branch is currently a and b respectively then if $|(a + 1)/(a + b + 1) - r| \leq |a/(a + b + 1) - r|$ we go left, otherwise right (i.e., which move would split the work closer to the confidence value). We continue this process until we find an unexplored node, at which point we steal the subtree with that unexplored node as root.

There is an exception to this. Although we may sometimes want to steal as low as possible, we cannot steal too low, as then the granularity would become too small and communication costs will dominate the runtime. Thus we dynamically determine a granularity bound under which threads are not allowed to steal, e.g., 15 levels above the average fail depth. If the work stealing algorithm guides the work stealing to the level of the granularity bound, then the last unexplored node above the granularity bound is stolen instead. The granularity bound is dynamically adjusted to maintain a minimum average subtree size so that work stealing does not occur more often than a certain threshold.

Since the confidence values are constantly updated, the optimal places to search next changes as search progresses. In order for our algorithm to adapt quickly, we do not require a thread to finish the entire subtree it has stolen before stealing again. Instead, after a given *restart* time has passed, the thread returns the unexplored parts of its subtree to a master coordinating work stealing and steals work again from the top. The work frontier is stored at the master and the work is resumed when work stealing guides a thread to the area again (similar to the idea used in interleaving DFS [103]).

Example with reasonably high confidence. Suppose we know that the branching heuristic is reasonably strong, but not perfect. We may use $conf = 0.8$.

Suppose we have 8 threads. Initially, all confidence values are 0.8. When the 8 threads attempt to steal work at the root, thread 1 will go down the left hand side.

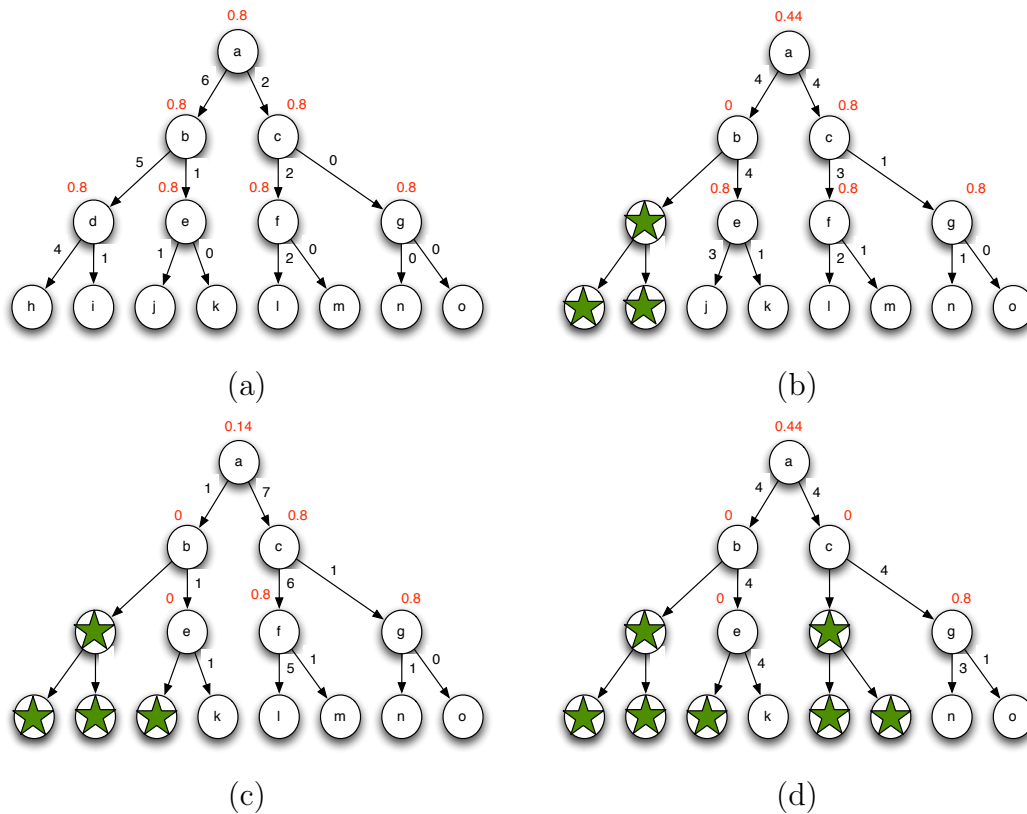


Figure 10.1: Example with reasonably high confidence

Thread 2 will go down the left hand side as well. Thread 3 will go down the right hand side. Thread 4 will go down the left hand side, etc, until 6 threads are down left and 2 threads are down right. At node **b**, we will have 5 threads down the left and 1 thread down the right, and so on. The work stealing has strongly favored sending threads towards the left side of each node because of the reasonably high confidence values of 0.8. The result is shown in Figure 10.1(a).

Suppose as search progresses the subtree starting at node **d** finishes without producing a solution. Then we need to update the confidence values. Using Equation (10.3), the confidence value at node **b** becomes 0, and at node **a** 0.44. Now when the threads steal work from the root, the situation has changed. Since one of the most fruitful parts of the left branch has been completely searched without producing a solution, it has become much less likely that there is a solution down the left branch. The updated confidence value reflects this. Now the threads will be distributed such

that 4 threads are down the left branch and 4 threads are down the right branch, as shown in Figure 10.1(b).

Next, perhaps the subtree starting at node *j* finishes. The confidence value at node *e* then becomes 0, the confidence value at node *b* remains 0 and the confidence value at node *a* becomes 0.14. The vast majority of the fruitful places in the left branch has been exhausted without finding a solution, and the confidence value at the root has been updated to strongly favor the right branch. The threads will now be distributed such that 7 threads go down the right and 1 thread goes down the left, as shown in Figure 10.1(c).

Next, suppose the subtree starting at node *f* finishes. The confidence value at node *c* becomes 0 and the confidence value at node *a* becomes 0.44. Since the most fruitful part of the right branch has also failed, the confidence value now swings back to favor the left branch more, as shown in Figure 10.1(d). This kind of confidence updating and redistribution of threads will continue, distributing the threads according to the current best solution density estimates. In our explanation here, for simplicity we only updated the confidence values very infrequently. In the actual implementation, confidence values are updated after every job is finished and thus occur much more frequently and in much smaller sized chunks.

Example with low confidence. For the second example, suppose we know that the heuristic is very bad and is basically random. We may use $conf = 0.5$, i.e., the initial solution density estimates down the left and right branch are equal.

Suppose we have 4 threads. Initially, all the confidence values are 0.5. The threads will distribute as shown in Figure 10.2(a). This distributes the threads as far away from each other as possible which is exactly what we want. However, if the search tree is deep, and the first few decisions that the each threads makes are wrong, all threads may still get stuck and never find a solution.

This is where the *restart* limit kicks in. After a certain time threshold is reached, the threads abandon their current search and begin work stealing from the root again. Since the confidence values are updated when they abandon their current job, they take a different path when they next steal work. For example, if the thread down

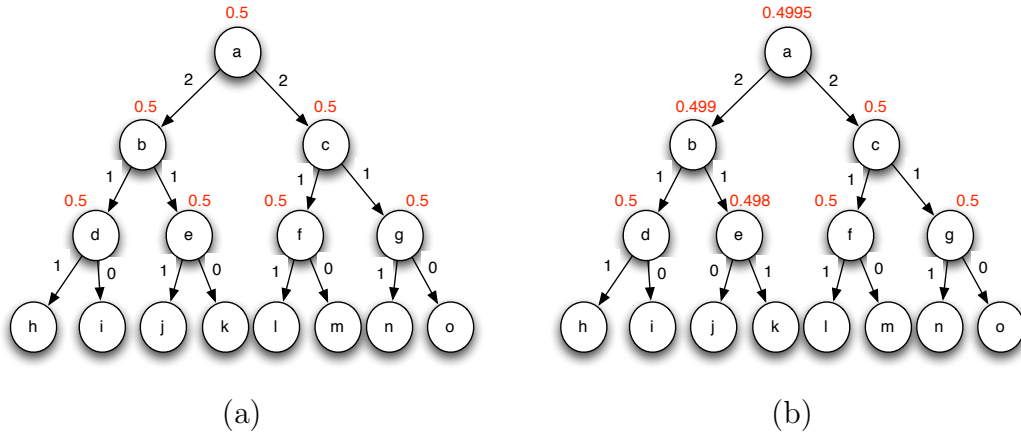


Figure 10.2: Example with low confidence

node **e** abandons after having finished a subtree with root node at depth 10, then the confidence at node **e** becomes 0.498, the confidence at node **b** becomes 0.499, and the confidence at node **a** becomes 0.4995. Then when a thread steals work from the root, it will again go left, then right. When it gets to node **e** however, the confidence value is 0.498 and there are no threads down either branch, thus it will go right at this node instead of left like last time. The result is shown in Figure 10.2(b). The updated confidence value has guided the thread to an unexplored part of the search tree that is as different from those already searched as possible. This always happens because solution density estimates are decremented whenever a part of a subtree is found to have failed, so the confidence will always be updated to favor the unexplored parts of the search tree.

Emulating standard search patterns. As some other examples, we briefly mention what confidence models lead to some standard search patterns. DFS: $conf = 1$, $restart = \infty$. IDFS: $conf = 1$, $restart = 1000$. LDS: $conf = 1 - \epsilon$, $restart = 1$ node. DDS: $conf = 0.5$ if $depth < d$, $1 - \epsilon$ if $depth \geq d$, $restart = 1$ node.

10.7 Experiments

Confidence-based work stealing is implemented using Gecode 3.0.2 [56] with an additional patch to avoid memory management contention during parallel execution. The benchmarks are run on a Mac with 2×2.8 GHz Xeon Quad Core E5462 processors with 4Gb of memory. However, due to memory limitations, we could not run the large instances of n-Queens or Knights on this machine. We run those two benchmarks on a Dell PowerEdge 6850 with 4×3.0 GHz Xeon Dual Core Pro 7120 CPUs with 32Gb of memory. 8 threads are used for the parallel search algorithm. We use a time limit of 20 min CPU time (2.5 min wall clock time for 8 threads), a restart time of 5 seconds, and a dynamic granularity bound that adjusts itself to try to steal no more than once every 0.5 seconds. We collected the following data: wall clock runtime, CPU utilization, communication overhead, number of steals, total number of nodes searched and number of nodes explored to find the optimal solution.

Optimization problems. In our first set of experiments we examine the efficiency of our algorithm for three optimization problems from Gecode’s example problems. The problems are: Traveling Salesman Problem (TSP), Golomb-Ruler and Queens-Armies. A description of these problems can be found at [56]. We use the given search heuristic (in the Gecode example file) for each, except for TSP where we try both a strong heuristic based on maximizing cost reduction and a weak heuristic that just picks variables and values in order. For TSP, we randomly generated many instances of an appropriate size for benchmarking. Only the size 12 and size 13 instances of Golomb Ruler, and only the size 9 and size 10 instances of Queen-Armies, are of an appropriate size for benchmarking. We use the simple confidence model with $conf = 1, 0.66$ and 0.5 . The results are given in Table 10.1.

It is clear that in all of our problems, runtime is essentially proportional to the number of nodes searched, and it is highly correlated to the amount of time taken to find the optimal solution. The quicker the optimal solution is found, the fewer the nodes searched and the lower the total runtime. The communication cost, which

Table 10.1: Experimental results for optimization problems with simple confidence model. The results show: wall clock runtime in seconds (Runtime), speedup relative to the sequential version (Speedup), and runtime efficiency (RunE) which is Speedup/8, CPU utilization (CPU%), communication overhead (Comm%), number of steals (Steals), total number of nodes explored (Nodes), the algorithmic efficiency (AlgE) the total number of nodes explored in the parallel version versus the sequential version, the number of nodes explored to find the optimal solution (Onodes), and the solution finding efficiency (SFE) the total number of nodes explored in the parallel version to find the optimal versus the sequential version. Values for Runtime, CPU%, Comm%, Steals, Nodes, and Onodes are the geometric mean of the instances solved by all 4 versions.

TSP with strong heuristic, 100 instances (Mac)										
<i>conf</i>	Runtime	Speedup	RunE	CPU%	Comm%	Steals	Nodes	AlgE	Onodes	SFE
Seq	313.3	—	—	100.0%	0.0%	—	5422k	—	1572k	—
1	38.2	7.25	0.91	96.5%	0.4%	708	5357k	1.01	1589k	0.99
0.66	47.2	5.88	0.74	93.7%	0.1%	319	6657k	0.81	5130k	0.31
0.5	48.0	5.77	0.72	92.9%	0.1%	467	6747k	0.80	5275k	0.30

TSP with weak heuristic, 100 instances (Mac)										
<i>conf</i>	Runtime	Speedup	RunE	CPU%	Comm%	Steals	Nodes	AlgE	Onodes	SFE
Seq	347.8	—	—	99.0%	0.0%	—	7.22M	—	1.15M	—
1	46.7	7.45	0.93	99.4%	0.6%	1044	6.96M	1.04	1.09M	1.06
0.66	45.8	7.60	0.95	96.9%	0.1%	379	7.02M	1.03	1.10M	1.05
0.5	41.6	8.36	1.05	97.5%	0.1%	304	6.36M	1.14	0.96M	1.20

Golomb Ruler, 2 instances (n = 12, 13) (Mac)										
<i>conf</i>	Runtime	Speedup	RunE	CPU%	Comm%	Steals	Nodes	AlgE	Onodes	SFE
Seq	562	—	—	100.0%	0.0%	—	9.71M	—	1.07M	—
1	69.0	8.15	1.02	99.3%	0.2%	572	8.96M	1.08	0.81M	1.33
0.66	59.0	9.54	1.19	99.3%	0.1%	346	7.58M	1.28	0.49M	2.21
0.5	65.2	8.63	1.08	99.3%	0.1%	259	8.42M	1.15	0.66M	1.63

Queen Armies, 2 instances (n = 9, 10) (Mac)										
<i>conf</i>	Runtime	Speedup	RunE	CPU%	Comm%	Steals	Nodes	AlgE	Onodes	SFE
Seq	602	—	—	100.0%	0.0%	—	13.6M	—	845k	—
1	87.1	6.91	0.86	99.3%	0.7%	1521	14.5M	0.94	1878k	0.45
0.66	86.3	6.98	0.87	98.8%	0.2%	1143	14.5M	0.96	2687k	0.31
0.5	86.0	7.00	0.87	99.5%	0.2%	983	14.5M	0.95	2816k	0.30

includes all work stealing and synchronization overheads, is less than 1% for all problems.

The strong heuristic in TSP is quite strong. Using $conf = 1$ achieves near perfect algorithmic efficiency, where algorithm efficiency is defined as the total number of nodes searched in the parallel algorithm vs the sequential algorithm. Other values of $conf$ clearly cause an algorithmic slowdown. The optimal solution is found on average 3.2 and 3.3 times slower for $conf = 0.66$ and 0.5 respectively, resulting in an algorithmic efficiency of 0.81 and 0.80 respectively. The opposite is true when the weak heuristic is used. Using $conf = 1$ or 0.66 allows us to find the leftmost optimal solution in approximately the same number of nodes as the sequential algorithm, but using $conf = 0.5$ to reflect that the heuristic is weak allows the algorithm to find the optimal solution even faster, producing an algorithmic efficiency of 1.14.

The branching heuristic in Golomb Ruler is a greedy heuristic that selects the minimum possible value for the variable at each stage. This is a reasonable heuristic but by no means perfect. It turns out that for Golomb Ruler 12 and 13, the optimal solution does not lie directly in the left-most branch, and a certain degree of non-greediness leads to super-linear solution finding efficiencies.

The results for Queens-Armies show little difference depending on confidence. Clearly the heuristic is better than random at finding an optimal solution, and solution finding efficiency degrades slightly as we ignore the heuristic. But the overall nodes searched are almost identical for all confidence values, as the work required for the proof of optimality make up the bulk of the run time, and the proof of optimality parallelizes trivially regardless of confidence.

Satisfaction problems. In our second set of experiments we examine the efficiency of our algorithm for three satisfaction problems from Gecode's examples [56]. The problems are: n -Queens, Knights, and Perfect-Square.

The sequential version solved very few instances of n -Queens and Knights. Furthermore, all those solves are extremely fast (< 3 sec) and are caused by the search engine finding a solution at the very leftmost part of the search tree. Most of the time spent in those runs is from moving down to the leaf of the search tree rather than actual search and is not parallelizable, thus comparison of the statistics for the parallel vs sequential algorithms on those instances is not meaningful as there is very

Table 10.2: Experimental results for satisfaction problems with simple confidence model

<i>n</i> -Queens, 100 instances (n = 1500, 1520, ..., 3480)									
<i>conf</i>	Solved	Runtime	Speedup	RunE	CPU%	Comm%	Steals	Nodes	AlgE
Seq	4	2.9	—	—	99.9%	0.0%	—	1859	—
1	4	10.4	—	—	99.0%	86.6%	2	1845	—
0.66	29	18.0	—	—	81.6%	0.3%	9	15108	—
0.5	100	2.9	—	—	65.5%	1.6%	8	14484	—

Knights, 40 instances (n = 20, 22, ..., 98)									
<i>conf</i>	Solved	Runtime	Speedup	RunE	CPU%	Comm%	Steals	Nodes	AlgE
Seq	7	0.22	—	—	99.9%	0.0%	—	1212	—
1	7	0.26	—	—	68.1%	59.7%	2	1150	—
0.66	13	0.50	—	—	48.0%	4.7%	8	8734	—
0.5	21	0.66	—	—	35.2%	6.0%	8	8549	—

Perfect-Square, 100 instances									
<i>conf</i>	Solved	Runtime	Speedup	RunE	CPU%	Comm%	Steals	Nodes	AlgE
Seq	15	483.1	—	—	99.9%	0.0%	—	213k	—
1	13	72.3	6.68	0.83	98.0%	19.1%	419	216k	0.99
0.66	14	71.2	6.78	0.85	86.4%	2.9%	397	218k	0.98
0.5	82	8.9	54.02	6.75	89.0%	4.8%	21	32k	6.64

little work to parallelize. The number of instances solved is a more interesting statistic and is a better means of comparison. The parallel algorithm beats the sequential algorithm by an extremely large margin in terms of the number of instances solved.

The *n*-Queens and Knights problems both have very deep subtrees. Once the sequential algorithm fails to find a solution in the leftmost subtree, it will often end up stuck forever. Modeling the fact that the branching heuristic is very weak at the top by using *conf* = 0.5 clearly produce a super linear speedup. The parallel algorithm solves 100 out of 100 instances of *n*-Queens, compared to 4 out of 100 instances for the sequential algorithm or the parallel algorithm with *conf* = 1. The speedup cannot be measured as the sequential algorithm does not terminate for days when it fails to find a solution quickly. Similarly the parallel algorithm with *conf* = 0.5 solved 21 instances of Knights compared to 7 for the sequential and the parallel version with *conf* = 1.

The branching heuristic for the Perfect Square problem is better than random, but is still terribly weak. Using *conf* = 0.5 to model this once again produces super

Table 10.3: Experimental results showing Nodes and algorithmic efficiency (AlgE) using accurate confidence values, where we follow the confidence value to degree α .

	Seq		$\alpha = 1$	$\alpha = 0.5$	$\alpha = 0$	$\alpha = -0.5$	$\alpha = -1$
Golomb-Ruler 12	5.31M	—	2.24M 2.37	3.48M 1.53	4.27M 1.24	10.8M 0.49	10.6M 0.50
Golomb-Ruler 13	71.0M	—	53.2M 1.34	57.6M 1.23	61.9M 1.15	74.8M 0.95	111M 0.64

linear speedup, solving 82 instances out of 100 compared to 15 out of 100 for the sequential algorithm. We can compare run times for this problem as the sequential version solved a fair number of instances and those solves actually require some work (483 sec on average). The speedup in this case is 54 using 8 threads.

Using accurate confidence values. So far, we have tested the efficiency of our algorithm using simple confidence models where the confidence value is the same for all nodes. This does not really illustrate the algorithm's full potential. We expect that the algorithm should perform even better when node-specific confidence values are provided. In our third set of experiments, we examine the efficiency of our algorithm when node specific confidence values are provided.

Due to our lack of domain knowledge, we will not attempt to write a highly accurate confidence heuristic. Rather, we will simulate one by first performing an initial full search of the search tree to find all solutions, then produce confidence estimates for the top few levels of the search tree using several strategies like, follow the measured solution density exactly, follow it approximately, ignore it, go against it, etc, to see what effect this has on runtime. Let α quantify how closely we follow the measured confidence value and let $conf$ be the measured confidence value. Then we use the following formula for our confidence estimate: $conf' = \alpha \times conf + (1 - \alpha) \times 0.5$. If $\alpha = 1$, then we follow it exactly. If $\alpha = -1$, we go against it completely, etc. We use the Golomb-Ruler problem for our experiment as the full search tree is small enough to enumerate completely. The results are shown in Table 10.3.

The results show that using confidence values that are even a little biased towards the real value is sufficient to produce super linear speedup. And not surprisingly, going against the real value will result in substantial slowdowns.

10.8 Conclusion

By analyzing work stealing schemes using a model based on solution density, we were able to quantitatively relate the strength of the branching heuristic with the optimal place to steal work from. This leads to an adaptive work stealing algorithm that can utilize confidence estimates to automatically produce “optimal” work stealing patterns. The algorithm produced near perfect or better than perfect algorithmic efficiency on all the problems we tested. In particular, by adapting to a steal high, interleaving search pattern, it is capable of producing super linear speedup on several problem classes. The real efficiency is lower than the algorithmic efficiency due to hardware effects, but is still quite good at a speedup of at least 7 at 8 threads. Communication costs are negligible on all problems even at 8 threads.

Chapter 11

Conclusion

In this thesis, we have presented important contributions to a number of areas in Constraint Programming, including nogood learning, symmetry breaking, relaxations, dominance and parallelization.

Nogood learning is undoubtedly one of the most important techniques for improving combinatorial optimization. Its application in the specialized field of Boolean Satisfiability led to solvers which are orders of magnitudes faster than their predecessors and to massive industrial adoption. Similar gains are now being seen when nogood learning techniques are applied to CP solvers. In Chapter 3, we presented a new nogood learning technique based on constraint projection which can be used to exploit subproblem dominances that arise when different search paths lead to equivalent subproblems. Our new method can provide orders of magnitude speedup on appropriate problems. It is competitive with Lazy Clause Generation and can be exponentially stronger on some problems. An important avenue of future work is to understand more fully the strengths and weaknesses of these different nogood learning techniques so that we can combine their strengths together.

Symmetry breaking techniques are critical for solving problems with symmetries efficiently. In Chapter 4, we presented a new symmetry breaking technique called SBDS-1UIP, which uses symmetric versions of the 1UIP nogoods derived by Lazy Clause Generation solvers to prune symmetric parts of the search space. We showed that SBDS-1UIP can exploit at least as many symmetries as the original SBDS method, and that it can exploit types of symmetries that no previous general symmetry breaking technique is capable of exploiting. In particular, SBDS-1UIP can exploit any symmetry that holds in the sub-component of the problem which caused the failure. This is a fundamental step forward in symmetry breaking, as SBDS-1UIP can be strictly stronger than complete methods like SBDS or SBDD. An interesting avenue of future work would be to explore what other kinds of symmetries can be exploited using different types of nogoods, e.g., the nogoods derived using constraint projection as described in Chapter 3.

Partially broken symmetries frequently occur in industrial problems. Such almost symmetries can often still be exploited for significant speedups. Although there has been some work on almost symmetries, most of it is theoretical and there exists

only one previous implementation of a general method. In Chapter 5, we formalized almost symmetries and presented two new general methods for exploiting almost symmetries. The first is to treat almost symmetries as conditional symmetries and exploit them by posting conditional symmetry breaking constraints. The second is to modify SDBS-1UIP to handle almost symmetries. Both techniques are capable of producing orders of magnitude speedups on appropriate problems. This work is an important contribution to the still immature, but promising field of almost symmetry breaking.

Dominance techniques are another important way to improve combinatorial optimization. Identifying dominance relations allow us to prune off parts of the search tree which are suboptimal, and can often lead to large speedups. In Chapter 6, we identified 4 different dominance relations for the Minimization of Open Stacks Problem (MOSP), which when exploited, provide some 5-6 orders of magnitude speedup. In Chapter 7, we identified 2 dominance relations for the Talent Scheduling Problem which provides ~ 2 times speedup. Dominance is a generalization of symmetry. However, unlike symmetry, it is not well studied in the literature, despite its huge potential for improving combinatorial optimization. An important avenue of future work would be to develop a fuller understanding of dominance and to develop generic dominance methods that can be applied across a wide range of problems.

Relaxation techniques can provide very significant speedups on appropriate problems. In Chapter 6, we identified an effective relaxation to the Minimization of Open Stacks Problem that allows us to speed up the proof of optimality on the hardest instances by some 3-4 orders of magnitude. In Chapter 7, we presented a lower bound calculation for the Talent Scheduling Problem based on relaxations which provides around 1 order of magnitude speedup. In Chapter 8, we tackled the extremely hard Maximum Density Still Life Problem. The previous state of the art could only solve up to $n = 20$. By using an appropriate relaxation technique, we were able to change the asymptotic complexity of the problem from $O(2^{n^2})$ to $O(n)$. This relaxation technique, combined with remodeling, bounded dynamic programming and customized search, allowed us to completely solved the Maximum Density Still Life Problem for all n . An interesting avenue of future work would be to see if these relaxation

techniques can be generalized and applied to other problems as well.

Parallelization is another way to improve combinatorial optimization. In Chapter 9, we identified cache contention as a serious bottleneck that severely limits the amount of speedup achievable by parallel SAT and LCG solvers on multi-core systems. We alter the data structures used in the state of the art SAT solver MiniSat [39] to be more cache aware, leading to a sequential SAT solver that is some 80% faster on average and a parallel SAT solver that is some 140% faster on average.

In Chapter 10, we examined the interaction between branching heuristics and dynamic load balancing schemes used in parallel solvers. Many commonly used load balancing schemes focus on work granularity and completely ignore the branching heuristic, causing many of the processors to search in unfruitful parts of the search tree. We analyze this issue and develop a new work stealing scheme which partitions work based on how strong the branching heuristic is at each node. This analysis addresses an important theoretical issue that has largely been ignored in the literature, and is critical for the efficient parallelization of CP solvers. Our new parallel algorithm produced near linear or super linear speedup on all the problems we tested. An interesting avenue of future work would be to find a way to automate the process of estimating the strength of the branching heuristic at each node, as our current method requires the problem modeler to provide this information.

In this thesis, we have presented many new and powerful techniques for improving combinatorial optimization. The application of these techniques should provide significant speedups on a wide range of combinatorial optimization problems, translating into time, resource and monetary savings for industry.

Bibliography

- [1] R. Adelson, J. Norman, and G. Laporte. A Dynamic Programming Formulation with Diverse Applications. *Operational Research Quarterly*, 27(1):119–121, 1976.
- [2] A. Aho and J. Hopcroft. *Design & Analysis Of Computer Algorithms*. Pearson Education India, 1974.
- [3] T. A. Aldowaisan. A new heuristic and dominance relations for no-wait flowshops with setups. *Computers & OR*, 28(6):563–584, 2001.
- [4] F. A. Aloul, I. L. Markov, and K. A. Sakallah. FORCE: a fast and easy-to-implement variable-ordering heuristic. In *Proceedings of the 13th ACM Great Lakes Symposium on VLSI*, pages 116–119. ACM, 2003.
- [5] C. Ambühl, M. Mastrolilli, and O. Svensson. Inapproximability Results for Sparsest Cut, Optimal Linear Arrangement, and Precedence Constrained Scheduling. In *Proceedings of the 48th Annual IEEE Symposium on Foundations of Computer Science*, pages 329–337. IEEE Computer Society, 2007.
- [6] A. Appleby. Murmurhash 2.0, 2009.
- [7] D. Baatar, N. Boland, S. Brand, and P. J. Stuckey. Minimum Cardinality Matrix Decomposition into Consecutive-Ones Matrices: CP and IP Approaches. In P. V. Hentenryck and L. A. Wolsey, editors, *Proceedings of the 4th International Conference on Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*, volume 4510 of *Lecture Notes in Computer Science*, pages 1–15. Springer, 2007.

- [8] R. Backofen and S. Will. Excluding Symmetries in Constraint-Based Search. In J. Jaffar, editor, *Proceedings of the 5th International Conference on Principles and Practice of Constraint Programming*, volume 1713 of *Lecture Notes in Computer Science*, pages 73–87. Springer, 1999.
- [9] J. C. Becceneri, H. H. Yanasse, and N. Y. Soma. A method for solving the minimization of the maximum number of open stacks problem within a cutting process. *Computers & OR*, 31(14):2315–2332, 2004.
- [10] N. Beldiceanu, M. Carlsson, and J. Rampon. Global constraint catalog. *SICS Research Report*, 2005. <http://www.emn.fr/z-info/sdemasse/gccat/>.
- [11] R. Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- [12] T. Benoist, F. Laburthe, and B. Rottembourg. Lagrange Relaxation and Constraint Programming Collaborative Schemes for Travelling Tournament Problems. In *Proceedings of the 3rd International Workshop on Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*. Wye College, 2001.
- [13] C. Bessière, E. Hebrard, B. Hnich, and T. Walsh. The Complexity of Global Constraints. In D. L. McGuinness and G. Ferguson, editors, *Proceedings of the 19th National Conference on Artificial Intelligence and the 16th Conference on Innovative Applications of Artificial Intelligence*, pages 112–117. AAAI Press / The MIT Press, 2004.
- [14] C. Bessière and J.-C. Régin. MAC and Combined Heuristics: Two Reasons to Forsake FC (and CBJ?) on Hard Problems. In E. C. Freuder, editor, *Proceedings of the 2nd International Conference on Principles and Practice of Constraint Programming*, volume 1118 of *Lecture Notes in Computer Science*, pages 61–75. Springer, 1996.
- [15] A. Biere. PicoSAT Essentials. *JSAT*, 4(2-4):75–97, 2008.

- [16] S. Bistarelli, U. Montanari, and F. Rossi. Constraint Solving over Semirings. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence*, pages 624–630. Morgan Kaufmann, 1995.
- [17] J. R. Bitner and E. M. Reingold. Backtrack Programming Techniques. *Commun. ACM*, 18(11):651–656, 1975.
- [18] R. Bosch. Integer Programming and Conway’s Game of Life. *SIAM Review*, 41(3):596–604, 1999.
- [19] R. Bosch and M. Trick. Constraint Programming and Hybrid Formulations for Three Life Designs. *Annals of OR*, 130(1-4):41–56, 2004.
- [20] F. Boussemart, F. Hemery, C. Lecoutre, and L. Sais. Boosting Systematic Search by Weighting Constraints. In R. L. de Mántaras and L. Saitta, editors, *Proceedings of the 16th European Conference on Artificial Intelligence*, pages 146–150. IOS Press, 2004.
- [21] H. Cambazard and B. O’Sullivan. Propagating the Bin Packing Constraint Using Linear Programming. In D. Cohen, editor, *Proceedings of the 16th International Conference on Principles and Practice of Constraint Programming*, volume 6308 of *Lecture Notes in Computer Science*, pages 129–136. Springer, 2010.
- [22] Y. Caseau and F. Laburthe. Solving Small TSPs with Constraints. In *Proceedings of the 14th International Conference on Logic Programming*, pages 316–330. MIT Press, 1997.
- [23] T. C. E. Cheng, J. E. Diamond, and B. M. T. Lin. Optimal Scheduling in Film Production to Minimize Talent Hold Cost. *Journal of Optimization Theory and Applications*, 79(3):479–482, 1993.
- [24] G. Chu, M. Garcia de la Banda, C. Mears, and P. J. Stuckey. Symmetries and Lazy Clause Generation. In *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (to appear)*, 2011.

- [25] G. Chu, M. Garcia de la Banda, and P. J. Stuckey. Automatically Exploiting Subproblem Equivalence in Constraint Programming. In A. Lodi, M. Milano, and P. Toth, editors, *Proceedings of the 7th International Conference on Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*, volume 6140 of *Lecture Notes in Computer Science*, pages 71–86. Springer, 2010.
- [26] G. Chu, A. Harwood, and P. J. Stuckey. Cache Conscious Data Structures for Boolean Satisfiability Solvers. *JSAT*, 6(1-3):99–120, 2009.
- [27] G. Chu, C. Schulte, and P. J. Stuckey. Confidence-Based Work Stealing in Parallel Constraint Programming. In I. P. Gent, editor, *Proceedings of the 15th International Conference on Principles and Practice of Constraint Programming*, volume 5732 of *Lecture Notes in Computer Science*, pages 226–241. Springer, 2009.
- [28] G. Chu and P. J. Stuckey. Minimizing the Maximum Number of Open Stacks by Customer Search. In I. P. Gent, editor, *Proceedings of the 15th International Conference on Principles and Practice of Constraint Programming*, volume 5732 of *Lecture Notes in Computer Science*, pages 242–257. Springer, 2009.
- [29] G. Chu, P. J. Stuckey, and M. Garcia de la Banda. Using Relaxations in Maximum Density Still Life. In I. P. Gent, editor, *Proceedings of the 15th International Conference on Principles and Practice of Constraint Programming*, volume 5732 of *Lecture Notes in Computer Science*, pages 258–273. Springer, 2009.
- [30] F. Clautiaux, J. Carlier, and A. Moukrim. A new exact method for the two-dimensional orthogonal packing problem. *European Journal of Operational Research*, 183(3):1196–1211, 2007.
- [31] D. A. Cohen, P. Jeavons, C. Jefferson, K. E. Petrie, and B. M. Smith. Symmetry Definitions for Constraint Satisfaction Problems. In P. van Beek, editor, *Proceedings of the 11th International Conference on Principles and Practice of*

- Constraint Programming*, volume 3709 of *Lecture Notes in Computer Science*, pages 17–31. Springer, 2005.
- [32] J. M. Crawford, M. L. Ginsberg, E. M. Luks, and A. Roy. Symmetry-Breaking Predicates for Search Problems. In *Proceedings of the 5th International Conference on Principles of Knowledge Representation and Reasoning*, pages 148–159. Morgan Kaufmann, 1996.
- [33] CSPLib. www.csplib.org. Accessed on 9 July 2011.
- [34] R. Dakin. A tree-search algorithm for mixed integer programming problems. *The Computer Journal*, 8(3):250, 1965.
- [35] M. Davis, G. Logemann, and D. W. Loveland. A machine program for theorem-proving. *Commun. ACM*, 5(7):394–397, 1962.
- [36] R. Dechter. Enhancement Schemes for Constraint Processing: Backjumping, Learning, and Cutset Decomposition. *Artif. Intell.*, 41(3):273–312, 1990.
- [37] R. Dechter and I. Meiri. Experimental Evaluation of Preprocessing Techniques in Constraint Satisfaction Problems. In *Proceedings of the 11th International Joint Conference on Artificial Intelligence*, pages 271–277. Morgan Kaufmann, 1989.
- [38] A. Donaldson. Partial Symmetry in Model Checking. In *SymNet Workshop on Almost-Symmetry in Search*, pages 17–21, 2005. http://www.allydonaldson.co.uk/edited_volumes/SymNet2005.pdf.
- [39] N. Eén and N. Sörensson. An Extensible SAT-solver. In E. Giunchiglia and A. Tacchella, editors, *Proceedings of the 6th International Conference on Theory and Applications of Satisfiability Testing*, volume 2919 of *Lecture Notes in Computer Science*, pages 502–518. Springer, 2003.
- [40] N. Elkies. The still-life density problem and its generalizations. *Voronoi’s Impact on Modern Science: Book I*, pages 228–253, 1998. arXiv:math/9905194v1.

- [41] H. Everett III. Generalized Lagrange multiplier method for solving problems of optimum allocation of resources. *Operations Research*, pages 399–417, 1963.
- [42] E. Faggioli and C. Bentivoglio. Heuristic and exact methods for a cutting sequencing problem. *European Journal of Operational Research*, 110(3):564–575, 1998.
- [43] T. Fahle, S. Schamberger, and M. Sellmann. Symmetry Breaking. In T. Walsh, editor, *Proceedings of the 7th International Conference on Principles and Practice of Constraint Programming*, volume 2239 of *Lecture Notes in Computer Science*, pages 93–107. Springer, 2001.
- [44] Y. Feldman, N. Dershowitz, and Z. Hanna. Parallel Multithreaded Satisfiability Solver: Design and Implementation. *Electr. Notes Theor. Comput. Sci.*, 128(3):75–90, 2005.
- [45] T. Feydy and P. J. Stuckey. Lazy Clause Generation Reengineered. In I. P. Gent, editor, *Proceedings of the 15th International Conference on Principles and Practice of Constraint Programming*, volume 5732 of *Lecture Notes in Computer Science*, pages 352–366. Springer, 2009.
- [46] M. Fisher. The Lagrangian relaxation method for solving integer programming problems. *Management science*, 27(1):1–18, 1981.
- [47] P. Flener, A. M. Frisch, B. Hnich, Z. Kiziltan, I. Miguel, J. Pearson, and T. Walsh. Breaking Row and Column Symmetries in Matrix Models. In P. V. Hentenryck, editor, *Proceedings of the 8th International Conference on Principles and Practice of Constraint Programming*, volume 2470 of *Lecture Notes in Computer Science*, pages 462–476. Springer, 2002.
- [48] E. C. Freuder. Eliminating Interchangeable Values in Constraint Satisfaction Problems. In *Proceedings of the 9th National Conference on Artificial Intelligence*, pages 227–233. AAAI Press / The MIT Press, 1991.

- [49] E. C. Freuder and R. J. Wallace. Partial Constraint Satisfaction. *Artif. Intell.*, 58(1-3):21–70, 1992.
- [50] A. M. Frisch, C. Jefferson, and I. Miguel. Constraints for Breaking More Row and Column Symmetries. In F. Rossi, editor, *Proceedings of the 9th International Conference on Principles and Practice of Constraint Programming*, volume 2833 of *Lecture Notes in Computer Science*, pages 318–332. Springer, 2003.
- [51] A. S. Fukunaga and R. E. Korf. Bin Completion Algorithms for Multicon-tainer Packing, Knapsack, and Covering Problems. *J. Artif. Intell. Res. (JAIR)*, 28:393–429, 2007.
- [52] M. Garcia de la Banda, K. Marriott, R. Rafeh, and M. Wallace. The Modelling Language Zinc. In F. Benhamou, editor, *Proceedings of the 12th International Conference on Principles and Practice of Constraint Programming*, volume 4204 of *Lecture Notes in Computer Science*, pages 700–705. Springer, 2006.
- [53] M. Garcia de la Banda and P. J. Stuckey. Dynamic Programming to Minimize the Maximum Number of Open Stacks. *INFORMS Journal on Computing*, 19(4):607–617, 2007.
- [54] M. Garcia de la Banda, P. J. Stuckey, and G. Chu. Solving Talent Scheduling with Dynamic Programming. *INFORMS Journal on Computing*, 23(1):120–137, 2011.
- [55] M. R. Garey, D. S. Johnson, and L. J. Stockmeyer. Some Simplified NP-Complete Graph Problems. *Theor. Comput. Sci.*, 1(3):237–267, 1976.
- [56] Gecode Team. Gecode: Generic Constraint Development Environment. <http://www.gecode.org>, 2006. Accessed on 9 July 2011.
- [57] B. Gendron and T. G. Crainic. Parallel Branch-and-Bound Algorithms: Survey and Synthesis. *Operations Research*, 42(6):1042–1066, November-December 1994.

- [58] I. P. Gent, W. Harvey, and T. Kelsey. Groups and constraints: Symmetry breaking during search. In P. V. Hentenryck, editor, *Proceedings of the 8th International Conference on Principles and Practice of Constraint Programming*, volume 2470 of *Lecture Notes in Computer Science*, pages 415–430. Springer, 2002.
- [59] I. P. Gent, W. Harvey, T. Kelsey, and S. Linton. Generic sbdd using computational group theory. In F. Rossi, editor, *Proceedings of the 9th International Conference on Principles and Practice of Constraint Programming*, volume 2833 of *Lecture Notes in Computer Science*, pages 333–347. Springer, 2003.
- [60] I. P. Gent, C. Jefferson, T. Kelsey, I. Lynce, I. Miguel, P. Nightingale, B. M. Smith, and A. Tarim. Search in the patience game 'Black Hole'. *AI Commun.*, 20(3):211–226, 2007.
- [61] I. P. Gent, T. Kelsey, S. Linton, I. McDonald, I. Miguel, and B. M. Smith. Conditional Symmetry Breaking. In P. van Beek, editor, *Proceedings of the 11th International Conference on Principles and Practice of Constraint Programming*, volume 3709 of *Lecture Notes in Computer Science*, pages 256–270. Springer, 2005.
- [62] I. P. Gent and B. M. Smith. Symmetry Breaking in Constraint Programming. In W. Horn, editor, *Proceedings of the 14th European Conference on Artificial Intelligence*, pages 599–603. IOS Press, 2000.
- [63] C. Gomes. Complete randomized backtrack search. *Constraint and integer programming: Toward a unified methodology*, pages 233–283, 2003.
- [64] C. Gomes and D. B. Shmoys. Completing Quasigroups or Latin Squares: A structured Graph Colouring Problem. In D. S. Johnson, A. Mehrotra, and M. Trick, editors, *Proceedings of the Computational Symposium on Graph Colouring and Extensions*, pages 22–39, 2002.

- [65] P. Gregory. Almost-Symmetry in Planning. In *SymNet Workshop on Almost-Symmetry in Search*, pages 14–16, 2005. http://www.allydonaldson.co.uk/edited_volumes/SymNet2005.pdf.
- [66] R. M. Haralick and G. L. Elliott. Increasing Tree Search Efficiency for Constraint Satisfaction Problems. *Artif. Intell.*, 14(3):263–313, 1980.
- [67] W. Harvey. Symmetric Relaxation Techniques for Constraint Programming. In *SymNet Workshop on Almost-Symmetry in Search*, pages 50–59, 2005. http://www.allydonaldson.co.uk/edited_volumes/SymNet2005.pdf.
- [68] W. D. Harvey and M. L. Ginsberg. Limited Discrepancy Search. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence*, pages 607–615. Morgan Kaufmann, 1995.
- [69] S. Heipcke. Comparing constraint programming and mathematical programming approaches to discrete optimisation-the change problem. *The Journal of the Operational Research Society*, 50(6):581–595, 1999.
- [70] M. Held and R. Karp. The traveling-salesman problem and minimum spanning trees. *Operations Research*, 18(6):1138–1162, 1970.
- [71] B. Hnich, Z. Kiziltan, and T. Walsh. Modelling a Balanced Academic Curriculum Problem. In *Proceedings of the 4th International Workshop on Integration of AI and OR techniques in Constraint Programming for Combinatorial Optimization Problems*, pages 121–131, 2002.
- [72] S. Hölldobler, N. Manthey, and A. Saptawijaya. Improving Resource-Unaware SAT Solvers. In C. G. Fermüller and A. Voronkov, editors, *Proceedings of the 17th International Conference on Logic for Programming, Artificial Intelligence, and Reasoning*, volume 6397 of *Lecture Notes in Computer Science*, pages 519–534. Springer, 2010.
- [73] J. N. Hooker. Logic-Based Methods for Optimization. In A. Borning, editor, *Proceedings of the 2nd International Workshop on Principles and Practice of*

- Constraint Programming*, volume 874 of *Lecture Notes in Computer Science*, pages 336–349. Springer, 1994.
- [74] J. N. Hooker and M. A. O. Lama. Mixed Logical-linear Programming. *Discrete Applied Mathematics*, 96-97:395–442, 1999.
- [75] J. N. Hooker and H. Yan. A Relaxation of the Cumulative Constraint. In P. V. Hentenryck, editor, *Proceedings of the 8th International Conference on Principles and Practice of Constraint Programming*, volume 2470 of *Lecture Notes in Computer Science*, pages 686–690. Springer, 2002.
- [76] E. Horowitz and S. Sahni. *Fundamentals of computer algorithms*, volume 146. Computer Science Press Rockville, MD, 1978.
- [77] S. W. Hur and J. Lillis. Relaxation and clustering in a local search framework: application to linear placement. In *Proceedings of the 36th Conference on Design Automation*, pages 360–366. ACM, 1999.
- [78] T. Ibaraki. The Power of Dominance Relations in Branch-and-Bound Algorithms. *J. ACM*, 24(2):264–279, 1977.
- [79] H. Jain and E. Clarke. Sat solver descriptions: CMUSAT-base and CMUSAT. *System description for the SAT competition*, 2007. <http://www.satcompetition.org/2007/cmusat-solvers.pdf>.
- [80] V. Jain and I. E. Grossmann. Algorithms for Hybrid MILP/CP Models for a Class of Optimization Problems. *INFORMS Journal on Computing*, 13(4):258–276, 2001.
- [81] N. Karmarkar. A new polynomial-time algorithm for linear programming. *Combinatorica*, 4(4):373–396, 1984.
- [82] R. M. Karp. Mapping the genome: some combinatorial problems arising in molecular biology. In *Proceedings of the 25th Annual ACM Symposium on Theory of Computing*, pages 278–285. ACM, 1993.

- [83] R. M. Karp. Reducibility among combinatorial problems. *50 Years of Integer Programming 1958-2008*, pages 219–241, 2010.
- [84] G. Katsirelos and F. Bacchus. Generalized NoGoods in CSPs. In M. M. Veloso and S. Kambhampati, editors, *Proceedings of the 20th National Conference on Artificial Intelligence and the 17th Conference on Innovative Applications of Artificial Intelligence*, pages 390–396. AAAI Press / The MIT Press, 2005.
- [85] N. G. Kinnersley. The vertex separation number of a graph equals its path-width. *Inf. Process. Lett.*, 42(6):345–350, 1992.
- [86] M. Kitching and F. Bacchus. Symmetric Component Caching. In M. M. Veloso, editor, *Proceedings of the 20th International Joint Conference on Artificial Intelligence*, pages 118–124, 2007.
- [87] W. H. Kohler and K. Steiglitz. Characterization and Theoretical Comparison of Branch-and-Bound Algorithms for Permutation Problems. *J. ACM*, 21(1):140–156, 1974.
- [88] Y. Koren and D. Harel. A Multi-scale Algorithm for the Linear Arrangement Problem. In L. Kucera, editor, *Proceedings of the 28th International Workshop on Graph-Theoretic Concepts in Computer Science*, volume 2573 of *Lecture Notes in Computer Science*, pages 296–309. Springer, 2002.
- [89] V. Kumar and V. Rao. Parallel depth first search. Part II. analysis. *International Journal of Parallel Programming*, 16(6):501–519, 1987.
- [90] J. Larrosa and R. Dechter. Boosting Search with Variable Elimination in Constraint Optimization and Constraint Satisfaction Problems. *Constraints*, 8(3):303–326, 2003.
- [91] J. Larrosa, E. Morancho, and D. Niso. On the Practical use of Variable Elimination in Constraint Optimization Problems: ‘Still-life’ as a Case Study. *J. Artif. Intell. Res. (JAIR)*, 23:421–440, 2005.

- [92] Y. C. Law, J. H.-M. Lee, T. Walsh, and J. Y. K. Yip. Breaking Symmetry of Interchangeable Variables and Values. In C. Bessiere, editor, *Proceedings of the 13th International Conference on Principles and Practice of Constraint Programming*, volume 4741 of *Lecture Notes in Computer Science*, pages 423–437. Springer, 2007.
- [93] E. Lawler and D. Wood. Branch-and-bound methods: A survey. *Operations research*, 14(4):699–719, 1966.
- [94] C. Lecoutre, L. Sais, S. Tabary, and V. Vidal. Recording and Minimizing Nogoods from Restarts. *JSAT*, 1(3-4):147–167, 2007.
- [95] C. Lemaréchal. Lagrangian Relaxation. In M. Jünger and D. Naddef, editors, *Computational Combinatorial Optimization, Optimal or Provably Near-Optimal Solutions*, volume 2241 of *Lecture Notes in Computer Science*, pages 112–156. Springer, 2001.
- [96] M. D. T. Lewis, T. Schubert, and B. Becker. Multithreaded SAT Solving. In *Proceedings of the 12th Conference on Asia South Pacific Design Automation*, pages 926–931. IEEE, 2007.
- [97] A. Linhares and H. H. Yanasse. Connections between cutting-pattern sequencing, VLSI design, and flexible machines. *Computers & OR*, 29(12):1759–1772, 2002.
- [98] E. M. Luks and A. Roy. The Complexity of Symmetry-Breaking Formulas. *Ann. Math. Artif. Intell.*, 41(1):19–45, 2004.
- [99] I. Lynce and J. P. M. Silva. Efficient data structures for backtrack search SAT solvers. *Ann. Math. Artif. Intell.*, 43(1):137–152, 2005.
- [100] R. Marinescu and R. Dechter. AND/OR Branch-and-Bound for Graphical Models. In L. P. Kaelbling and A. Saffiotti, editors, *Proceedings of the 19th International Joint Conference on Artificial Intelligence*, pages 224–229. Professional Book Center, 2005.

- [101] R. Martin. The Challenge of Exploiting Weak Symmetries. In B. Hnich, M. Carlsson, F. Fages, and F. Rossi, editors, *Proceedings of the International Workshop on Constraint Solving and Constraint Logic Programming*, volume 3978 of *Lecture Notes in Computer Science*, pages 149–163. Springer, 2005.
- [102] C. Mears. *Automatic Symmetry Detection and Dynamic Symmetry Breaking for Constraint Programming*. PhD thesis, Clayton School of Information Technology, Monash University, 2010.
- [103] P. Meseguer. Interleaved Depth-First Search. In M. E. Pollack, editor, *Proceedings of the 15th International Joint Conference on Artificial Intelligence*, pages 1382–1387. Morgan Kaufmann, 1997.
- [104] P. Meseguer and C. Torras. Exploiting symmetries within constraint satisfaction search. *Artif. Intell.*, 129(1-2):133–163, 2001.
- [105] L. Michel, A. See, and P. V. Hentenryck. Parallelizing Constraint Programs Transparently. In C. Bessiere, editor, *Proceedings of the 13th International Conference on Principles and Practice of Constraint Programming*, volume 4741 of *Lecture Notes in Computer Science*, pages 514–528. Springer, 2007.
- [106] MiniZinc Challenge 2010 Results. <http://www.g12.csse.unimelb.edu.au/minizinc/challenge2010/results2010.html>. Accessed on 9 July 2011.
- [107] J. Mohan. *Performance of Parallel Programs: Model and Analyses*. PhD thesis, Carnegie-Mellon University, Pittsburgh, PA, USA, 1984.
- [108] J. Monette, P. Schaus, S. Zampelli, Y. Deville, and P. Dupont. A CP Approach to the Balanced Academic Curriculum Problem. In *Seventh International Workshop on Symmetry and Constraint Satisfaction Problems*, volume 7, 2007. http://www.info.ucl.ac.be/~yde/Papers/SymCon2007_bacp.pdf.
- [109] M. W. Moskewicz, C. F. Madigan, Y. Zhao, L. Zhang, and S. Malik. Chaff: Engineering an Efficient SAT Solver. In *Proceedings of the 38th Design Automation Conference*, pages 530–535. ACM, 2001.

- [110] J. Nelder and R. Mead. A simplex method for function minimization. *The Computer Journal*, 7(4):308, 1965.
- [111] G. Nemhauser and L. Wolsey. *Integer and combinatorial optimization*. Wiley-Interscience Series in Discrete Mathematics and Optimization, 1999.
- [112] N. Nethercote, P. J. Stuckey, R. Becket, S. Brand, G. J. Duck, and G. Tack. MiniZinc: Towards a Standard CP Modelling Language. In C. Bessiere, editor, *Proceedings of the 13th International Conference on Principles and Practice of Constraint Programming*, volume 4741 of *Lecture Notes in Computer Science*, pages 529–543. Springer, 2007.
- [113] O. Ohrimenko, P. J. Stuckey, and M. Codish. Propagation = Lazy Clause Generation. In C. Bessiere, editor, *Proceedings of the 13th International Conference on Principles and Practice of Constraint Programming*, volume 4741 of *Lecture Notes in Computer Science*, pages 544–558. Springer, 2007.
- [114] L. Perron. Search Procedures and Parallelism in Constraint Programming. In J. Jaffar, editor, *Proceedings of the 5th International Conference on Principles and Practice of Constraint Programming*, volume 1713 of *Lecture Notes in Computer Science*, pages 346–360. Springer, 1999.
- [115] S. Prestwich. Increasing Solution Density by Dominated Relaxation. *Modelling and Reformulating Constraint Satisfaction Problems, Fourth International Workshop*, 2005. <http://homes.ieu.edu.tr/bhnich/mod-proc.pdf>.
- [116] S. Prestwich and J. Beck. Exploiting dominance in three symmetric problems. In *Fourth International Workshop on Symmetry and Constraint Satisfaction Problems*, pages 63–70, 2004. <http://zeynep.web.cs.unibo.it/SymCon04/SymCon04.pdf>.
- [117] J. Puchinger and P. J. Stuckey. Automating branch-and-bound for dynamic programs. In R. Glück and O. de Moor, editors, *Proceedings of the 2008 ACM SIGPLAN Symposium on Partial Evaluation and Semantics-based Program Manipulation*, pages 81–89. ACM, 2008.

- [118] J. F. Puget. On the Satisfiability of Symmetrical Constrained Satisfaction Problems. In H. J. Komorowski and Z. W. Ras, editors, *Proceedings of the 7th International Symposium on Methodologies for Intelligent Systems*, volume 689 of *Lecture Notes in Computer Science*, pages 350–361. Springer, 1993.
- [119] J. F. Puget. Breaking All Value Symmetries in Surjection Problems. In P. van Beek, editor, *Proceedings of the 11th International Conference on Principles and Practice of Constraint Programming*, volume 3709 of *Lecture Notes in Computer Science*, pages 490–504. Springer, 2005.
- [120] J. F. Puget. Breaking symmetries in all different problems. In L. P. Kaelbling and A. Saffiotti, editors, *Proceedings of the 19th International Joint Conference on Artificial Intelligence*, pages 272–277. Professional Book Center, 2005.
- [121] P. W. Purdom, Jr. Search Rearrangement Backtracking and Polynomial Average Time. *Artif. Intell.*, 21(1-2):117–133, 1983.
- [122] C. G. Quimper and T. Walsh. Global Grammar Constraints. In F. Benhamou, editor, *Proceedings of the 12th International Conference on Principles and Practice of Constraint Programming*, volume 4204 of *Lecture Notes in Computer Science*, pages 751–755. Springer, 2006.
- [123] M. J. Quinn. Analysis and Implementation of Branch-and Bound Algorithms on a Hypercube Multicomputer. *IEEE Trans. Computers*, 39(3):384–387, 1990.
- [124] A. Ramani and I. L. Markov. Automatically Exploiting Symmetries in Constraint Programming. In B. Faltings, A. Petcu, F. Fages, and F. Rossi, editors, *Proceedings of the International Workshop on Constraint Solving and Constraint Logic Programming*, volume 3419 of *Lecture Notes in Computer Science*, pages 98–112. Springer, 2004.
- [125] V. N. Rao and V. Kumar. Superlinear Speedup in Parallel State-Space Search. In K. V. Nori and S. Kumar, editors, *Proceedings of the 8th Conference on Foundations of Software Technology and Theoretical Computer Science*, volume 338 of *Lecture Notes in Computer Science*, pages 161–174. Springer, 1988.

- [126] P. Refalo. Impact-Based Search Strategies for Constraint Programming. In M. Wallace, editor, *Proceedings of the 10th International Conference on Principles and Practice of Constraint Programming*, volume 3258 of *Lecture Notes in Computer Science*, pages 557–571. Springer, 2004.
- [127] J. C. Régin. A Filtering Algorithm for Constraints of Difference in CSPs. In *Proceedings of the 12th National Conference on Artificial Intelligence*, pages 362–367. AAAI Press, 1994.
- [128] J. C. Régin. Generalized Arc Consistency for Global Cardinality Constraint. In *Proceedings of the 13th National Conference on Artificial Intelligence and 8th Innovative Applications of Artificial Intelligence Conference*, pages 209–215. AAAI Press / The MIT Press, 1996.
- [129] R. Rodosek, M. Wallace, and M. Hajian. A new approach to integrating mixed integer programming and constraint logic programming. *Annals of Operations research*, 86:63–87, 1999.
- [130] C. C. Rolf and K. Kuchcinski. Load-balancing methods for parallel and distributed constraint solving. In *Proceedings of the 2008 IEEE International Conference on Cluster Computing*, pages 304–309. IEEE, 2008.
- [131] D. J. Rose. Triangulated graphs and the elimination process. *J. Math. Appl.*, 32:597–609, 1970.
- [132] A. Ruiz-Andino, L. Araujo, F. Sáenz, and J. J. Ruz. Parallel Execution Models for Constraint Programming over Finite Domains. In G. Nadathur, editor, *Proceedings of the International Conference on Principles and Practice of Declarative Programming*, volume 1702 of *Lecture Notes in Computer Science*, pages 134–151. Springer, 1999.
- [133] L. Ryan. *Efficient algorithms for clause-learning SAT solvers*. PhD thesis, Simon Fraser University, 2004. <http://www.cs.sfu.ca/~mitchell/papers/ryan-thesis.ps>.

- [134] SAT 2007 Competition. <http://www.satcompetition.org/2007/>. Accessed on 9 July 2011.
- [135] SAT Race 2008. <http://baldur.iti.uka.de/sat-race-2008/>. Accessed on 9 July 2011.
- [136] T. Schiex and G. Verfaillie. Nogood Recording for Static and Dynamic Constraint Satisfaction Problems. In *Proceedings of the 5th International Conference on Tools with Artificial Intelligence*, pages 48–55. IEEE Computer Society, 1993.
- [137] A. Schrijver. *Theory of linear and integer programming*, volume 11. Wiley New York, 1986.
- [138] C. Schulte. Parallel search made simple. In *Proceedings of TRICS: Techniques for Implementing Constraint programming Systems, a post-conference workshop of CP*, pages 41–57, 2000. https://www.ps.uni-saarland.de/Publications/documents/TRICS_00.pdf.
- [139] C. Schulte and P. J. Stuckey. Efficient constraint propagation engines. *ACM Trans. Program. Lang. Syst.*, 31(1), 2008.
- [140] M. Sellmann and T. Fahle. Constraint Programming Based Lagrangian Relaxation for the Automatic Recording Problem. *Annals of OR*, 118(1-4):17–33, 2003.
- [141] P. Shaw and P. Laborie. A Constraint Programming Approach to the Min-Stack Problem. In *Constraint Modelling Challenge 2005* [146].
- [142] J. P. M. Silva and K. A. Sakallah. GRASP: A Search Algorithm for Propositional Satisfiability. *IEEE Trans. Computers*, 48(5):506–521, 1999.
- [143] B. M. Smith. Reducing symmetry in a combinatorial design problem. Technical report, School of Computer Studies, University of Leeds, 2001. http://www.lirmm.fr/~bessiere/ws_ijcai01/smith.ps.gz.

- [144] B. M. Smith. Constraint Programming in Practice: Scheduling a Rehearsal. Technical report, St Andrews University, 2003. <http://compeng.hud.ac.uk/staff/scombms/Papers/apes-67-2003.pdf>.
- [145] B. M. Smith. Caching Search States in Permutation Problems. In P. van Beek, editor, *Proceedings of the 11th International Conference on Principles and Practice of Constraint Programming*, volume 3709 of *Lecture Notes in Computer Science*, pages 637–651. Springer, 2005.
- [146] B. M. Smith and I. Gent. Constraint Modelling Challenge Report 2005. <http://www.cs.st-andrews.ac.uk/~ipg/challenge/ModelChallenge05.pdf>.
- [147] R. Szymanek and B. O’Sullivan. Guiding Search Using Constraint-Level Advice. In G. Brewka, S. Coradeschi, A. Perini, and P. Traverso, editors, *Proceedings of the 17th European Conference on Artificial Intelligence*, volume 141 of *Frontiers in Artificial Intelligence and Applications*, pages 158–162. IOS Press, 2006.
- [148] E. S. Thorsteinsson and G. Ottosson. Linear Relaxations and Reduced-Cost Based Propagation of Continuous Variable Subscripts. *Annals of OR*, 115(1-4):15–29, 2002.
- [149] W. J. van Hoeve, G. Pesant, L.-M. Rousseau, and A. Sabharwal. Revisiting the Sequence Constraint. In F. Benhamou, editor, *Proceedings of the 12th International Conference on Principles and Practice of Constraint Programming*, volume 4204 of *Lecture Notes in Computer Science*, pages 620–634. Springer, 2006.
- [150] G. Verfaillie, M. Lemaître, and T. Schiex. Russian Doll Search for Solving Constraint Optimization Problems. In *Proceedings of the 13th National Conference on Artificial Intelligence and 8th Innovative Applications of Artificial Intelligence Conference*, pages 181–187. AAAI Press / The MIT Press, 1996.
- [151] A. Véron, K. Schuerman, M. Reeve, and L.-L. Li. Why and How in the Elip-Sys OR-parallel CLP System. In A. Bode, M. Reeve, and G. Wolf, editors,

- Proceedings of the 5th International Conference on Parallel Architectures and Languages Europe*, volume 694 of *Lecture Notes in Computer Science*, pages 291–303. Springer, 1993.
- [152] P. Vilím. Edge Finding Filtering Algorithm for Discrete Cumulative Resources in $O(kn \log n)$. In I. P. Gent, editor, *Proceedings of the 15th International Conference on Principles and Practice of Constraint Programming*, volume 5732 of *Lecture Notes in Computer Science*, pages 802–816. Springer, 2009.
- [153] T. Walsh. Depth-bounded Discrepancy Search. In M. E. Pollack, editor, *Proceedings of the 15th International Joint Conference on Artificial Intelligence*, pages 1388–1395. Morgan Kaufmann, 1997.
- [154] H. P. Williams and H. Yan. Representations of the all-different Predicate of Constraint Satisfaction in Integer Programming. *INFORMS Journal on Computing*, 13(2):96–103, 2001.
- [155] N. Wilson and K. Petrie. Using Customer Elimination Orderings to Minimise the Maximum Number of Open Stacks. In *Constraint Modelling Challenge 2005* [146].
- [156] L. Wolsey. Integer programming. *IIE Transactions*, 32:273–285, 2000.
- [157] H. H. Yanasse. On a pattern sequencing problem to minimize the maximum number of open stacks. *European Journal of Operational Research*, 100(3):454–463, 1997.
- [158] H. H. Yannasse. A note on generating solutions of a pattern sequencing problem to minimize the maximum number of open orders. *Technical Report LAC-002/98, INPE, São José dos Campos, SP, Brazil*, 1998.
- [159] C. F. Yu and B. W. Wah. Learning Dominance Relations in Combinatorial Search Problems. *IEEE Trans. Software Eng.*, 14(8):1155–1175, 1988.
- [160] B. Yuen. Heuristics for sequencing cutting patterns. *European Journal of Operational Research*, 55:183–190, 1991.

- [161] B. Yuen. Improved heuristics for sequencing cutting patterns. *European Journal of Operational Research*, 87:57–64, 1995.
- [162] B. Yuen and K. Richardson. Establishing the optimality of sequencing heuristics for cutting stock problems. *European Journal of Operational Research*, 84(3):590–598, 1995.
- [163] R. Zabih and D. A. McAllester. A Rearrangement Search Strategy for Determining Propositional Satisfiability. In *Proceedings of the 7th National Conference on Artificial Intelligence*, pages 155–160. AAAI Press / The MIT Press, 1988.
- [164] A. Zanarini and G. Pesant. Solution Counting Algorithms for Constraint-Centered Search Heuristics. In C. Bessiere, editor, *Proceedings of the 13th International Conference on Principles and Practice of Constraint Programming*, volume 4741 of *Lecture Notes in Computer Science*, pages 743–757. Springer, 2007.
- [165] L. Zhang, C. F. Madigan, M. W. Moskewicz, and S. Malik. Efficient Conflict Driven Learning in Boolean Satisfiability Solver. In *Proceedings of the International Conference on Computer-Aided Design*, pages 279–285. ACM, 2001.
- [166] L. Zhang and S. Malik. Cache Performance of SAT Solvers: a Case Study for Efficient Implementation of Algorithms. In E. Giunchiglia and A. Tacchella, editors, *Proceedings of the 6th International Conference on Theory and Applications of Satisfiability Testing*, volume 2919 of *Lecture Notes in Computer Science*, pages 287–298. Springer, 2003.
- [167] Y. Zhang and A. Mackworth. Parallel and distributed finite constraint satisfaction: Complexity, algorithms and experiments. *Machine Intelligence and Pattern Recognition*, 14:305–305, 1994.

Appendix A

Appendix

1.1 119 industrial SAT instances

These examples were taken from the SAT Competition and SAT Race instances available at

<http://www.satcompetition.org/> and

<http://www-sr.informatik.uni-tuebingen.de/sat-race-2008/downloads.html>.

1.2 24 medium sized industrial instances

AProVE07-04.cnf

AProVE07-06.cnf

AProVE07-15.cnf

AProVE07-22.cnf

blocks-4-ipc5-h21-unknown.cnf

clauses-4.cnf

cube-9-h10-unsat.cnf

cube-9-h11-sat.cnf

dated-10-15-u.cnf

dated-5-11-u.cnf

manol-pipe-c10nidw_s.cnf

manol-pipe-c6nidw_i.cnf
mizh-md5-47-3.cnf
mizh-sha0-35-2.cnf
mizh-sha0-35-4.cnf
mizh-sha0-35-5.cnf
total-10-11-u.cnf
total-5-11-u.cnf
total-5-13-u.cnf
uts-105-ipc5-h26-unsat.cnf
uts-105-ipc5-h27-unknown.cnf
uts-106-ipc5-h28-unknown.cnf
uts-106-ipc5-h29-unknown.cnf
uts-106-ipc5-h30-unknown.cnf

1.3 22 hard industrial instances

AProVE07-02.cnf
AProVE07-03.cnf
AProVE07-06.cnf
AProVE07-08.cnf
AProVE07-09.cnf
AProVE07-16.cnf
AProVE07-21.cnf
AProVE07-27.cnf
blocks-4-ipc5-h22-unknown.cnf
dated-10-11-u.cnf
dated-10-13-u.cnf
dated-5-15-u.cnf
dated-5-17-u.cnf
manol-pipe-c10nidw_s.cnf
manol-pipe-c7bidw_i.cnf
manol-pipe-c7nidw.cnf
manol-pipe-f9b.cnf

manol-pipe-g10bidw.cnf
total-10-13-u.cnf
uts-l06-ipc5-h30-unknown.cnf
uts-l06-ipc5-h31-unknown.cnf
uts-l06-ipc5-h32-unknown.cnf

1.4 200 industrial instances from SAT Race 2008

aloul-chnl11-13
aloul-sr06-chnl10-13 *
anbul-dated-5-15-u *
anbul-part-10-13-s
anbul-part-10-15-s
babic-dspam-vc1080
babic-dspam-vc949
babic-dspam-vc973
cmu-bmc-barrel6
cmu-bmc-longmult13 *
cmu-bmc-longmult15 *
een-pico-prop00-75
een-pico-prop05-75 *
een-pico-sr06-pr02-02 *
een-tipb-sr06-par1
een-tipb-sr06-tc6b
een-tip-sat-nusmv-t5.B
een-tip-sat-texas-tp-5e
een-tip-sat-vis-eisen
fuhs-aprove-15 *
fuhs-aprove-16 *
golddb-heqc-alu4mul *
golddb-heqc-dalumul *
golddb-heqc-desmul *
golddb-heqc-frg1mul

golddb-heqc-i8mul *
golddb-heqc-k2mul
golddb-heqc-rotmul *
golddb-heqc-x1mul
grieu-vmpc-27 *
grieu-vmpc-31
grieu-vmpc-s05-05s *
grieu-vmpc-s05-24s *
grieu-vmpc-s05-27r *
hoons-vbmc-lucky7 *
hoons-vbmc-s04-05 *
hoons-vbmc-s04-06 *
hoons-vbmc-s04-07 *
hoons-vbmc-s04-07 *
ibm-2002-04r-k80 *
ibm-2002-11r1-k45 *
ibm-2002-18r-k90 *
ibm-2002-20r-k75 *
ibm-2002-22r-k60
ibm-2002-22r-k75 *
ibm-2002-22r-k80 *
ibm-2002-23r-k90
ibm-2002-24r3-k100 *
ibm-2002-25r-k10 *
ibm-2002-29r-k75 *
ibm-2002-30r-k85
ibm-2002-31_1r3-k30 *
ibm-2004-01-k90 *
ibm-2004-1_11-k80 *
ibm-2004-23-k100 *
ibm-2004-23-k80 *
ibm-2004-29-k25 *
ibm-2004-29-k55 *

ibm-2004-3_02_3-k95 *
jarvi-eq-atree-9 *
manol-pipe-c10b *
manol-pipe-c10bi_s *
manol-pipe-c10id_s *
manol-pipe-c10ni_s *
manol-pipe-c10nid_i
manol-pipe-c10nid_s *
manol-pipe-c10nidw
manol-pipe-c6b_i *
manol-pipe-c6bid_i *
manol-pipe-c6bidw_i *
manol-pipe-c6id *
manol-pipe-c6idw_s *
manol-pipe-c6n *
manol-pipe-c6ni_s *
manol-pipe-c6nid_s *
manol-pipe-c6nidw *
manol-pipe-c7_i *
manol-pipe-c7_i *
manol-pipe-c7idw *
manol-pipe-c7idw_s *
manol-pipe-c8_i *
manol-pipe-c8b_i *
manol-pipe-c8b_i *
manol-pipe-c8n *
manol-pipe-c8nidw *
manol-pipe-c9n_i *
manol-pipe-cha05-113 *
manol-pipe-cha05-143 *
manol-pipe-f6b *
manol-pipe-f6i *
manol-pipe-f6n *

manol-pipe-f6nid *
manol-pipe-f7idw *
manol-pipe-f7nidw *
manol-pipe-f9b *
manol-pipe-g10bid_i
manol-pipe-g10idw *
manol-pipe-g10ni *
manol-pipe-g10nid *
manol-pipe-g6bid *
manol-pipe-g7n *
manol-pipe-g8b *
manol-pipe-g8bidw *
manol-pipe-g8n *
manol-pipe-g8nidw *
marijn-philips *
maris-s03-gripper11
miza-sr06-md5-47-03 *
miza-sr06-md5-48-01 *
miza-sr06-sha0-35-03 *
mizh-md5-47-3 *
mizh-md5-47-4 *
mizh-md5-47-5 *
mizh-md5-48-2 *
mizh-md5-48-5 *
mizh-sha0-35-3 *
mizh-sha0-35-4 *
mizh-sha0-36-1 *
mizh-sha0-36-3 *
mizh-sha0-36-4 *
narain-vpn-clauses-10
narain-vpn-clauses-8
narai-vpn-10s *
narai-vpn-sat05-02s *

narai-vpn-sat05-07 *
palac-sn7-ipc5-h16 *
palac-uts-l06-ipc5-h34 *
post-c32s-col400-16 *
post-c32s-gcdm16-22 *
post-c32s-gcdm16-23 *
post-c32s-ss-8
post-cbmc-aes-d-r1
post-cbmc-aes-d-r2
post-cbmc-aes-ee-r2
post-cbmc-aes-ee-r3
post-cbmc-aes-ele *
post-cbmc-zfcp-2.8-u2
schup-l2s-abp4-1-k31 *
schup-l2s-bc56s-1-k391
schup-l2s-motst-2-k315 *
schup-l2s-s04-abp4 *
schup-l2s-s04s2-09 *
schup-l2s-s04-valves
simon-mixed-s02bis-01 *
simon-mixed-s02bis-03 *
simon-mixed-s02bis-05 *
simon-mixed-s02bis-05 *
simon-s02b-r4b1k1.1 *
simon-s02b-r4b1k1.2 *
simon-s02-f2clk-50 *
simon-s03-fifo8-400 *
simon-s03-w08-15 *
stric-bmc-ibm-10 *
stric-bmc-ibm-12 *
vange-col-abb313GPIA-9-c
vange-color-inc-54 *
velev-dlx-uns-1.0-05

velev-engi-uns-1.0-4nd *
velev-eng-uns-1.0-04 *
velev-eng-uns-1.0-04a *
velev-fvp-sat-3.0-07 *
velev-fvp-sat-3.0-12 *
velev-fvp-sat-3.0-b18 *
velev-live-2.0-02 *
velev-live-sat-1.0-01 *
velev-live-sat-1.0-03 *
velev-live-sat-1.0-05 *
velev-npe-1.0-02 *
velev-npe-1.0-03 *
velev-npe-1.0-9dlx-b71 *
velev-pipe-1.0-08 *
velev-pipe-1.0-09 *
velev-pipe-1.1-03 *
velev-pipe-1.1-05 *
velev-pipe-oun-1.0-04
velev-pipe-oun-1.1-05
velev-pipe-sat-1.0-01 *
velev-pipe-sat-1.0-03 *
velev-pipe-sat-1.1-01 *
velev-pipe-sat-1.1-03 *
velev-pipe-sat-1.1-09 *
velev-pipe-uns-1.0-08
velev-pipe-uns-1.0-14 *
velev-pipe-uns-1.1-03 *
velev-pipe-uns-1.1-05
velev-pipe-uns-1.1-07
velev-pipe-uns-1.1-08
velev-sss-1.0-05 *
velev-sss-1.0-cl
velev-vliw-sat-2.0-02 *

velev-vliw-sat-2.0-04 *
velev-vliw-sat-4.0-03 *
velev-vliw-sat-4.0-b4 *
velev-vliw-sat-4.0-b8 *
velev-vliw-uns-2.0-02
velev-vliw-uns-2.0-iq1
velev-vliw-uns-2.0-iq2
velev-vliw-uns-2.0-uq5
velev-vliw-uns-4.0-9
velev-vliw-uns-4.0-9-i1

* instances that were consistently solvable within the 900 second timeout and used to produce Figure 9.8.

Note: there were files with the exact same name but different content in the different sets of instances used for SAT Race 2008. The duplicate filenames are not an error.